دانشکدهٔ مهندسی کامپیوتر — دانشگاه صنعتی شریف

جزوهٔ کامل
ساختار و زبان کامپیوتر

Computer Structure and Language (CSL)

بر اساس اسلایدهای درس **دکتر حمید سربازی‌آزاد**

نیم‌سال اول ۱۴۰۵–۱۴۰۶

شامل توضیح کامل مطالب، همهٔ مثال‌ها و حل کامل تمرین‌ها

ساختار فون‌نویمان و طبقه‌بندی فلین

حافظه و قالب دستور

ثبات‌ها و روش‌های آدرس‌دهی

ماشین‌های ۰، ۱، ۲ و ۳ آدرسه

اسمبلر، پیونددهنده و بارکننده

نمایش داده‌ها و ممیز شناور

آدرس‌دهی نسبی، قطعه‌ای و صفحه‌ای

ماشین IBM 360: قالب‌ها و پایه-جابه‌جایی

دستورهای RR و RX، کد شرط و ماسک پرش

برنامه‌نویسی 360: حلقه، شرط و زیربرنامه

جلسه‌های ۱ تا ۸ درس

پیش‌گفتار

این جزوه بر اساس اسلایدهای هشت جلسهٔ نخست درس **ساختار و زبان کامپیوتر** نوشته شده است. اسلایدهای درس معمولاً فقط سرتیترها، شکل‌ها و کدها را دارند؛ در این جزوه تلاش شده است هر اسلاید با توضیح کامل همراه شود، منطق هر مثال قدم‌به‌قدم باز شود و برای **همهٔ تمرین‌ها و تکلیف‌های مطرح‌شده در اسلایدها** حل کامل (برنامهٔ اسمبلی، ترجمه به کد ماشین و ردگیری اجرا) آورده شود.

چهار قاعده در نوشتن این جزوه رعایت شده است: (۱) هیچ چیز بدیهی فرض نشده و هر مفهوم از پایه آموزش داده شده است؛ (۲) همه چیز **گام‌به‌گام** و بدون پرش توضیح داده شده است؛ (۳) همهٔ مفاهیم اسلایدها کامل پوشش داده شده‌اند؛ (۴) برای مفهوم‌های کلیدی کادر **«شهود»** آمده است که با یک تصویر ذهنی یا تشبیه نشان می‌دهد **چرا** آن مفهوم این‌طور طراحی شده است.

> **دربارهٔ درستی کدهای ماشین**
>
> همهٔ برنامه‌هایی که در این جزوه به کد ماشین ترجمه شده‌اند (چه برنامه‌های اسلاید و چه حل تکلیف‌ها) با یک شبیه‌ساز کوچک برای همان ماشین فرضی اجرا و آزموده شده‌اند؛ برای نمونه برنامهٔ مرتب‌سازی Gnome ماشین ۷، جمع ۶۳ ثبات در ماشین ۵ و جمع ۱۰۰ عدد اول (حاصل: 24133). برای فصل‌های ۶ تا ۸ یک **اسمبلر و شبیه‌ساز IBM 360** نوشته شده است: همهٔ ۱۰۱ مثال اسلایدهای جلسهٔ ۶ تا ۸ (کد ماشین، نتیجه و کد شرط) و ۲۱ برنامهٔ کامل فصل‌ها با آن بررسی شده‌اند و جدول‌های «Loc / Object code» مستقیماً از خروجی همین اسمبلر ساخته شده‌اند.

ساختار جزوه

فصل	موضوع	جلسهٔ درس
۱	مقدمه: ساختار فون‌نویمان، طبقه‌بندی فلین، ابررایانه‌ها، معرفی درس	جلسهٔ ۱
۲	حافظهٔ اصلی، قالب دستورالعمل، مسیرهای داخلی و زبان اسمبلی (ماشین‌های ۱ و ۲)	جلسهٔ ۲
۳	ثبات‌ها، روش‌های آدرس‌دهی، اسمبلر/پیونددهنده/بارکننده، ماشین‌های تک‌آدرسه و پشته‌ای، نمادگذاری لهستانی (ماشین‌های ۳ تا ۶)	جلسهٔ ۳
۴	نمایش داده‌ها: مبناها، اعداد علامت‌دار، نمایش افزونی، ممیز ثابت و شناور، IEEE 754، کدهای دهدهی و کاراکتری	جلسهٔ ۴
۵	آدرس‌دهی نسبی، قطعه‌ای و صفحه‌ای، مرتب‌سازی و اعداد اول در ماشین ۷، پشتیبانی چندکاربره	جلسهٔ ۵
۶	ماشین IBM 360: ثبات‌ها و PSW، هم‌ترازی، آدرس‌دهی پایه-جابه‌جایی، شش قالب دستور، ساختار برنامه، BALR، DS/DC/EQU و USING، برنامهٔ STOCK	جلسهٔ ۶

فصل	موضوع	جلسهٔ درس
۷	دستورهای RR و کد شرط: LR، LCR، LNR، LPR، AR، SR، MR، DR، ALR، SLR، NR، OR، XR، CR، CLR، BCTR	جلسهٔ ۷
۸	دستورهای RX، LA، BCT، پرش شرطی با ماسک (BC، BCR)، ترجمهٔ if/while/for، زیربرنامه و برنامهٔ کامل	جلسهٔ ۸
پیوست	خلاصهٔ ماشین‌های نمونه (و IBM 360)، فهرست برنامه‌ها و واژه‌نامهٔ انگلیسی–فارسی	—

قراردادهای نوشتاری

- عددهای مبنای ۱۶ با پسوند `h` (مثل `100h`) یا زیرنویس $_{16}$(…) نوشته می‌شوند؛ کدهای ماشین همیشه به صورت شانزده‌شانزدهی‌اند.
- M_{addr} یعنی «خانهٔ حافظه با آدرس addr» و (M$_{addr}$) یعنی «محتوای آن خانه». علامت ← یعنی انتقال/نوشتن.
- کادرهای رنگی: **تعریف**، **مثال و حل**، **نکته**، **توجه** و **تمرین / تکلیف**، 💡 **شهود** (تصویر ذهنی و «چرا») و **روش گام‌به‌گام**.
- در فصل‌های ۶ تا ۸ قرارداد کد شرط (CC) همان قرارداد اسلایدهای درس است؛ هرجا سخت‌افزار واقعی 360 فرق دارد، فقط در یک «نکتهٔ تکمیلی» به آن اشاره شده است.
- هرجا در اسلاید اشتباه تایپی یا محاسباتی دیده شد، در کادر «توجه» شکل درست آن آمده است.

فهرست مطالب

فصل ۱: مقدمه: ساختار کامپیوتر، طبقه‌بندی فلین و ابررایانه‌ها ۶

۱-۱ ساختار فون‌نویمان (کامپیوتر با برنامهٔ ذخیره‌شده) ۶

۱-۲ طبقه‌بندی فلین (Flynn's Taxonomy) ۸

۱-۳ ابررایانه‌ها (Supercomputers) ۱۱

۱-۴ معرفی درس ساختار و زبان کامپیوتر ۱۶

فصل ۲: حافظهٔ اصلی، قالب دستورالعمل و زبان اسمبلی ۱۸

۲-۱ سیستم کامپیوتری ۱۸

۲-۲ حافظهٔ اصلی (Main Memory) ۱۸

۲-۳ قالب دستورالعمل (Instruction Format) ۲۶

۲-۴ مسیرهای داخلی ماشین ۲۸

۲-۵ زبان اسمبلی (Assembly Language) ۳۰

۲-۶ ماشین نمونهٔ ۲ ..... ۳۱

فصل ۳: ثبات‌ها، روش‌های آدرس‌دهی، اسمبلر، پیونددهنده و بارکننده ۳۶

۳-۱ ثبات‌ها (Registers) ۳۶

۳-۲ روش‌های آدرس‌دهی: مستقیم و غیرمستقیم ۴۱

۳-۳ ماشین نمونهٔ ۳ ..... ۴۳

۳-۴ روند اجرای برنامهٔ اسمبلی: اسمبلر، پیونددهنده و بارکننده ۴۸
۳-۵ آدرس‌دهی بی‌واسطه (Immediate) ۵۱
۳-۶ ماشین نمونهٔ ۴ ۵۲
۳-۷ آدرس‌دهی شاخص‌دار (Indexed) ۵۵
۳-۸ ماشین نمونهٔ ۵ ۵۵
۳-۹ آدرس‌دهی ضمنی (Implied / Inherent) ۵۹
۳-۱۰ ماشین تک‌آدرسه (ماشین انباره‌ای) ۶۰
۳-۱۱ ماشین صفرآدرسه (ماشین پشته‌ای) ۶۳
۳-۱۲ نمادگذاری لهستانی و ماشین‌های پشته‌ای ۶۵
۳-۱۳ ماشین نمونهٔ ۶ (ماشین پشته‌ای) ۶۸
۳-۱۴ تبدیل عبارت میانوندی به پسوندی ۷۳

فصل ۴: نمایش داده‌ها (Data Representation) ۷۷

۴-۱ دسته‌بندی اطلاعات ۷۷
۴-۲ سیستم اعداد دودویی ۷۷
۴-۳ سیستم‌های عددی موضعی و تبدیل مبنا ۷۸
۴-۴ اعداد منفی ۸۱
۴-۵ نمایش افزونی (Excess Representation) ۸۸
۴-۶ اعداد حقیقی: ممیز ثابت ۹۰
۴-۷ اعداد حقیقی: ممیز شناور ۹۰
۴-۸ استاندارد IEEE 754 ۹۴
۴-۹ اعداد دهدهی و کدهای دهدهی ۹۶
۴-۱۰ کدهای کاراکتری ۱۰۰

فصل ۵: آدرس‌دهی نسبی، قطعه‌ای و صفحه‌ای ۱۰۲

۵-۱ آدرس‌دهی نسبی (Relative) ۱۰۲
۵-۲ ماشین نمونهٔ ۷ ۱۰۳
۵-۳ آدرس‌دهی قطعه‌ای (Segment) ۱۱۰
۵-۴ آدرس‌دهی صفحه‌ای (Page) ۱۱۳
۵-۵ پشتیبانی چندکاربره با آدرس‌دهی قطعه‌ای ۱۱۴

فصل ۶: آشنایی با IBM System/360: ساختار، آدرس‌دهی پایه-جابه‌جایی و قالب دستورها ۱۱۷

۶-۱ چرا IBM 360؟ تاریخچه و جایگاه ۱۱۷
۶-۲ ساختار کلی ماشین ۱۱۸
۶-۳ ثبات‌های داخلی ۱۲۰
۶-۴ هم‌ترازی داده‌ها در حافظه (Alignment) ۱۲۲
۶-۵ آدرس‌دهی پایه-جابه‌جایی (Base-Displacement) ۱۲۴
۶-۶ شش قالب دستور ۱۲۷
۶-۷ مثال‌های کدگذاری هر قالب (همهٔ مثال‌های اسلاید) ۱۳۰
۶-۸ ساختار برنامهٔ اسمبلی 360 و نمادگذاری ۱۳۲
۶-۹ اعداد، ثابت‌ها و رهنمودهای DS، DC و EQU ۱۳۳
۶-۱۰ BALR و USING: تعیین ثبات پایه ۱۳۶
۶-۱۱ برنامهٔ نمونهٔ اسلاید: موجودی انبار (STOCK) — اسمبل و اجرای گام‌به‌گام ۱۳۸

۶-۱۲ تمرین‌های حل‌شدهٔ فصل ۶ ۱۴۰

فصل ۷: دستورهای RR (ثبات-ثبات) و کد شرط ۱۴۳

۷-۱ مرور قالب RR و روش کدگذاری ۱۴۳
۷-۲ کد شرط (Condition Code — CC) ۱۴۴
۷-۳ دستورهای بارگذاری ثبات: LCR، LNR، LPR ۱۴۵
۷-۴ جمع و تفریق علامت‌دار: AR و SR ۱۴۶
۷-۵ ضرب: MR و جفت ثبات زوج-فرد ۱۴۷
۷-۶ تقسیم: DR ۱۴۸
۷-۷ جمع و تفریق منطقی (بدون علامت): ALR و SLR ۱۴۹
۷-۸ دستورهای منطقی بیتی: NR، OR، XR ۱۵۰
۷-۹ مقایسه: CR و CLR ۱۵۱
۷-۱۰ BCTR: کم کردن و پرش (Branch on Count Register) ۱۵۲
۷-۱۱ زیبایی طراحی کدهای عمل ۱۵۳
۷-۱۲ جدول خلاصهٔ دستورهای RR ۱۵۴
۷-۱۳ تمرین‌های حل‌شدهٔ فصل ۷ ۱۵۴

فصل ۸: دستورهای RX (حافظه)، پرش‌ها و برنامه‌نویسی 360 ۱۵۸

۸-۱ چرا دستورهای RX؟ ۱۵۸
۸-۲ شش شکل نوشتن عملوند دوم ۱۵۹
۸-۳ بارگذاری و ذخیره: L و ST ۱۶۰
۸-۴ حساب با حافظه: A، S، M، D ۱۶۱
۸-۵ حساب منطقی با حافظه: AL و SL ۱۶۳
۸-۶ مقایسه با حافظه: C و CL ۱۶۳
۸-۷ منطق بیتی با حافظه: N، O، X ۱۶۳
۸-۸ LA: محاسبهٔ نشانی (Load Address) — کد 41 ۱۶۴
۸-۹ BCT: حلقهٔ شمارشی (Branch on Count) — کد 46 ۱۶۶
۸-۱۰ پرش شرطی با ماسک: BC — کد 47 ۱۶۶
۸-۱۱ BCR: پرش شرطی به نشانی داخل ثبات — کد 07 ۱۶۹
۸-۱۲ ترجمهٔ ساختارهای سطح بالا به اسمبلی 360 ۱۷۱
۸-۱۳ زیربرنامه: BALR و BR ۱۷۲
۸-۱۴ برنامه‌های کامل حل‌شده ۱۷۳
۸-۱۵ تمرین‌های حل‌شدهٔ فصل ۸ ۱۸۴

پیوست الف: خلاصهٔ ماشین‌های نمونه ۱۹۰

الف-۱ فهرست برنامه‌های حل‌شده در جزوه ۱۹۱

پیوست ب: واژه‌نامهٔ انگلیسی – فارسی ۱۹۲

فصل ۱

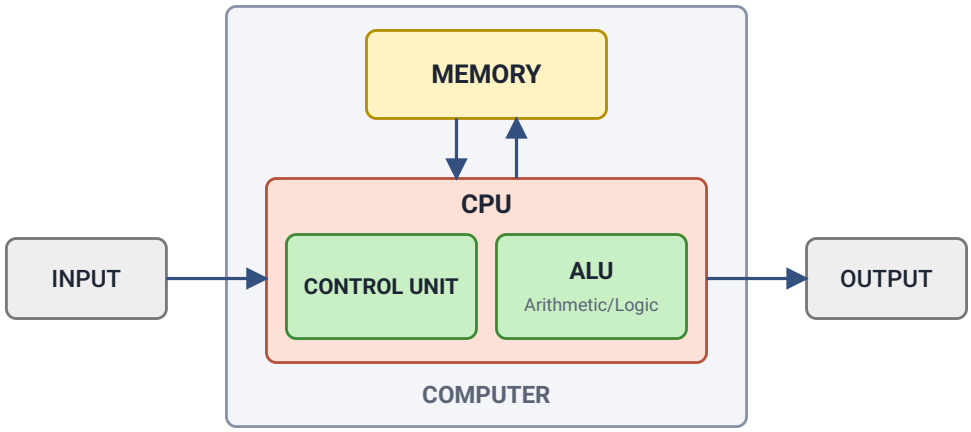
مقدمه: ساختار کامپیوتر، طبقه‌بندی فلین و ابررایانه‌ها

در این فصل (جلسهٔ اول درس) با **مدل پایهٔ کامپیوتر** یعنی ساختار «برنامهٔ ذخیره‌شده» یا فون‌نویمان آشنا می‌شویم، سپس **طبقه‌بندی فلین** را که ساختارهای کامپیوتری را بر اساس جریان دستور و جریان داده دسته‌بندی می‌کند می‌بینیم و با چند **ابررایانهٔ** مشهور تاریخ آشنا می‌شویم. در پایان، اهداف، سرفصل‌ها، منابع و بارم‌بندی درس آمده است.

> 💡 **شهود: دو سؤال، چهار کلاس درس**
>
> فلین فقط دو سؤال می‌پرسد: «چند **جریان دستور** هم‌زمان داریم؟» و «چند **جریان داده**؟». یک کلاس درس را تصور کنید: **SISD** یک معلم و یک شاگرد است. **SIMD** یک معلم است که یک فرمان را برای همهٔ شاگردها می‌خواند و هر کس آن را روی دفتر خودش انجام می‌دهد (پردازندهٔ برداری، GPU). **MISD** چند ممتحن است که یک برگهٔ امتحانی را جداگانه تصحیح می‌کنند (برای اطمینان؛ کمیاب). **MIMD** شاگردانی هستند که هر کدام مسئلهٔ خودش را مستقل حل می‌کند (پردازنده‌های چندهسته‌ای و خوشه‌ها).

۱-۱ ساختار فون‌نویمان (کامپیوتر با برنامهٔ ذخیره‌شده)

تقریباً همهٔ کامپیوترهای امروزی بر پایهٔ ایده‌ای ساخته شده‌اند که **جان فون‌نویمان** در سال ۱۹۴۵ (در گزارش طراحی کامپیوتر EDVAC) مطرح کرد. به این مدل، **مدل برنامهٔ ذخیره‌شده** (Stored-program model) یا **مدل فون‌نویمان** می‌گویند.



شکل: ساختار کامپیوتر با برنامهٔ ذخیره‌شده (مدل فون‌نویمان)

تعریف: مدل برنامهٔ ذخیره‌شده

در این مدل، **برنامه (دستورالعمل‌ها) و داده‌ها هر دو در یک حافظه ذخیره می‌شوند**. پردازنده دستورها را یکی‌یکی از حافظه می‌خواند، رمزگشایی می‌کند و اجرا می‌کند. چون برنامه هم مثل داده در حافظه است، برای اجرای یک برنامهٔ دیگر کافی است برنامهٔ جدید را در حافظه بارگذاری کنیم؛ نیازی به سیم‌کشی دوبارهٔ سخت‌افزار (کاری که در کامپیوترهای اولیه مثل ENIAC لازم بود) نیست.

اجزای اصلی این ساختار عبارت‌اند از:

- **حافظه (Memory)**: آرایه‌ای از خانه‌ها که هر کدام آدرس یکتا دارد و دستورها و داده‌ها را نگه می‌دارد.
- **واحد پردازش مرکزی (CPU)** که خود دو بخش دارد:
  - **واحد کنترل (CU: Control Unit)**: دستور بعدی را از حافظه واکشی (fetch) می‌کند، آن را رمزگشایی (decode) می‌کند و با فرستادن سیگنال‌های کنترلی، بقیهٔ اجزا را برای اجرای آن هدایت می‌کند.
  - **واحد محاسبه و منطق (ALU: Arithmetic/Logic Unit)**: عملیات حسابی (جمع، تفریق، …) و منطقی (AND، OR، XOR، …) را روی داده‌ها انجام می‌دهد.
- **ورودی (Input)** و **خروجی (Output)**: دستگاه‌هایی که داده را از بیرون به کامپیوتر می‌آورند یا نتیجه را به بیرون می‌فرستند.

💡 **شهود: آشپزخانه با یک دفترچه**

فرض کنید حافظه یک دفترچه است که هم **دستور پخت** (برنامه) و هم **فهرست مواد** (داده) در آن نوشته شده و CPU آشپزی است که خط‌به‌خط دستور پخت را می‌خواند و اجرا می‌کند. ایدهٔ «برنامهٔ ذخیره‌شده» یعنی برای پختن غذای دیگر فقط **محتوای دفترچه را عوض می‌کنیم**، نه سیم‌کشی آشپزخانه را (کاری که در ENIAC لازم بود). چون برنامه هم داده است، یک برنامه (مثلاً اسمبلر یا کامپایلر) می‌تواند برنامهٔ دیگری را بنویسد. هزینه‌اش: آشپز و دفترچه فقط **یک راه ارتباطی** دارند و دستور و داده باید نوبتی از آن رد شوند؛ این همان «گلوگاه فون‌نویمان» است.

۱-۱-۱ چرخهٔ واکشی–اجرا

کار پردازنده در مدل فون‌نویمان را می‌توان به صورت یک حلقهٔ بی‌پایان خلاصه کرد:

1. **واکشی (Fetch)**: خواندن دستوری که آدرس آن در ثبات شمارندهٔ برنامه (PC) است و قرار دادن آن در ثبات دستور (IR)؛ سپس افزایش PC تا به دستور بعدی اشاره کند.
2. **رمزگشایی (Decode)**: تشخیص نوع عمل (از روی کد عمل) و تعیین عملوندها.
3. **اجرا (Execute)**: خواندن عملوندها، انجام عمل در ALU و نوشتن نتیجه.
4. بازگشت به مرحلهٔ ۱.

با جزئیات این چرخه و ثبات‌های PC و IR در فصل‌های ۲ و ۳ بیشتر آشنا می‌شویم.

نکته: گلوگاه فون‌نویمان

در این مدل، همهٔ دستورها و داده‌ها باید از **یک مسیر** میان پردازنده و حافظه عبور کنند و پردازنده در هر لحظه یک دستور را اجرا می‌کند. به این محدودیت «گلوگاه فون‌نویمان» (von Neumann bottleneck) می‌گویند. همان‌طور که در اسلاید آمده، مدل فون‌نویمان ساختار پایه است و **برای بهبود کارایی آن از ساختارهای دیگر** (مثل خط لوله، پردازش موازی، چند پردازنده‌ای و ...) استفاده می‌شود؛ طبقه‌بندی فلین دقیقاً همین ساختارها را دسته‌بندی می‌کند.

💡 **شهود: PC یعنی نشانک کتاب**

CPU هیچ درکی از «کل برنامه» ندارد؛ فقط یک حلقهٔ ساده را میلیاردها بار تکرار می‌کند: خطی را که **نشانک (PC)** رویش است بخوان، نشانک را یک خط جلو ببر، کاری را که خط گفته انجام بده. دستور پرش فقط یعنی «نشانک را دستی جای دیگری بگذار». پس هر رفتار پیچیده‌ای (حلقه، تابع، شرط) در نهایت فقط جابه‌جا کردن همین نشانک است.

۱-۲ طبقه‌بندی فلین (Flynn's Taxonomy)

مایکل فلین (Michael J. Flynn, 1934–2025) در سال ۱۹۶۶ طبقه‌بندی‌ای برای ساختار کامپیوترها ارائه کرد که بر دو مفهوم استوار است:

تعریف: جریان دستور و جریان داده

جریان دستور (Instruction Stream, IS): دنبالهٔ دستورهایی که پردازنده واکشی و اجرا می‌کند.

جریان داده (Data Stream, DS): دنبالهٔ داده‌هایی که توسط یک جریان دستور خوانده و پردازش می‌شوند.

بر اساس اینکه هر یک از این دو جریان «تکی» باشد یا «چندگانه»، چهار دسته به دست می‌آید:

	یک جریان دستور (Single IS)	چند جریان دستور (Multiple IS)
یک جریان داده (Single DS)	SISD	MISD
چند جریان داده (Multiple DS)	SIMD	MIMD

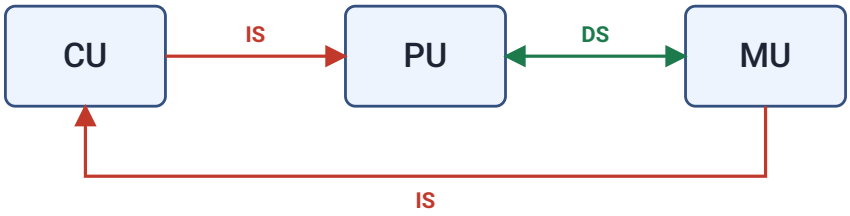
جدول: چهار دستهٔ فلین. سه دستهٔ سبز «ساختارهای پیشرفته» (Advanced Structures) هستند.

- **SISD**: Single Instruction stream, Single Data stream
- **SIMD**: Single Instruction stream, Multiple Data streams
- **MISD**: Multiple Instruction streams, Single Data stream
- **MIMD**: Multiple Instruction streams, Multiple Data streams

در شکل‌های زیر از این علامت‌ها استفاده شده است: **CU** واحد کنترل، **PU** واحد پردازش (Processing Unit)، **MU** واحد حافظه (Memory Unit)، **IS** جریان دستور و **DS** جریان داده.

۱-۲-۱ SISD — همان ساختار فون‌نویمان

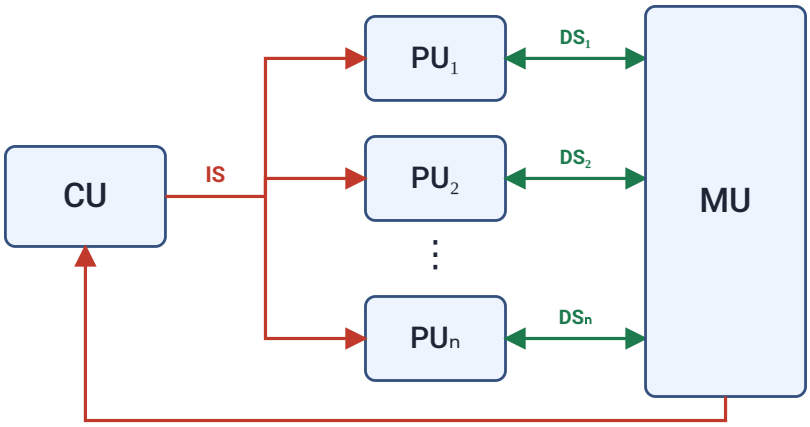
یک واحد کنترل، دستورها را یکی‌یکی از حافظه می‌گیرد (IS از MU به CU) و به یک واحد پردازش می‌دهد (IS از CU به PU). واحد پردازش هم روی یک جریان داده کار می‌کند (DS بین PU و MU). این **همان کامپیوتر پایهٔ فون‌نویمان** است؛ مثل پردازنده‌های تک‌هسته‌ای قدیمی.



SISD: یک جریان دستور و یک جریان داده (همان ساختار پایهٔ فون‌نویمان)

۱-۲-۲ SIMD — یک دستور، چند داده

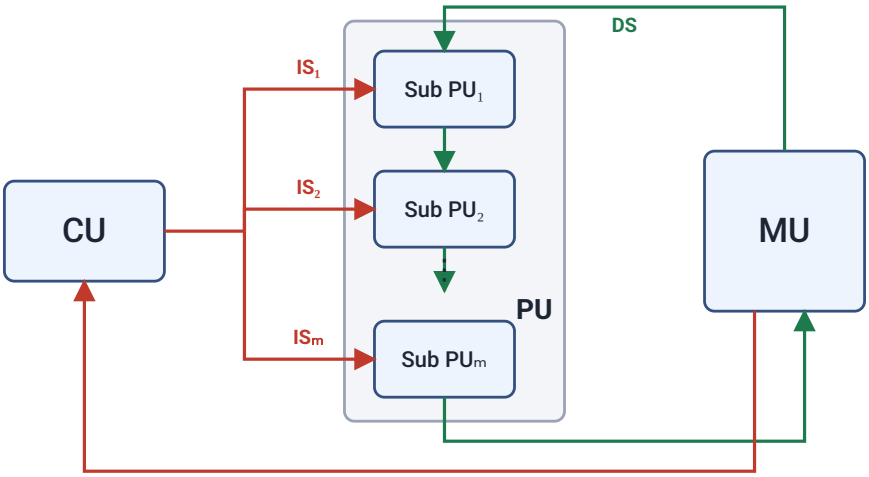
یک واحد کنترل وجود دارد که **یک دستور را هم‌زمان برای n واحد پردازش پخش می‌کند**؛ هر واحد پردازش آن دستور را روی دادهٔ خودش (DS_i) اجرا می‌کند. برای مثال با یک دستور «جمع برداری» می‌توان ۶۴ جمع را هم‌زمان روی ۶۴ جفت عدد انجام داد. **بیشتر ابررایانه‌های اولیه** (مثل Cray-1 که یک ماشین برداری بود) از این ساختار استفاده می‌کردند. امروزه هم واحدهای برداری پردازنده‌ها (مثل SSE و AVX) و پردازنده‌های گرافیکی (GPU) بر همین ایده استوارند.



SIMD: یک جریان دستور، چند جریان داده (پردازنده‌های برداری و بیشتر ابررایانه‌های اولیه)

۱-۲-۳ MISD — چند دستور، یک داده

چند جریان دستور روی **یک جریان داده** کار می‌کنند: داده از یک زیرواحد پردازشی به زیرواحد بعدی می‌رود و هر زیرواحد، عمل متفاوتی (دستور خودش، IS_i) را روی آن انجام می‌دهد. دربارهٔ این دسته اختلاف نظر وجود دارد: **بعضی معماران کامپیوتر معتقدند ماشین MISD واقعی وجود ندارد**؛ اما بعضی دیگر **کامپیوترهای خط لوله‌ای (Pipelined) و آرایه‌های سیستولیک (Systolic arrays)** را از نوع MISD می‌دانند.

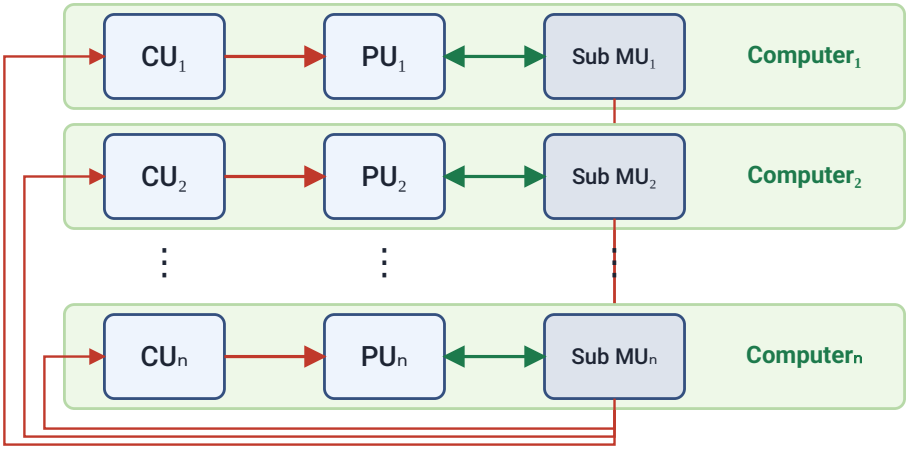


MISD: چند جریان دستور روی یک جریان داده (ساختارهای خط لوله‌ای / آرایه‌های سیستولیک)

> **نکته**
>
> طبق اسلاید، **ساختارهای خط لوله‌ای مقرون‌به‌صرفه‌ترین (most cost-effective) ساختارها هستند**؛ چون با تقسیم کار به چند مرحله و کار هم‌زمان مراحل روی داده‌های پشت‌سرهم، بدون تکرار کل سخت‌افزار، توان عملیاتی چند برابر می‌شود.

۴-۲-۱ MIMD — چند دستور، چند داده

چند کامپیوتر مستقل (هر کدام با CU، PU و بخشی از حافظهٔ خودش) هم‌زمان برنامه‌های جداگانه را روی داده‌های جداگانه اجرا می‌کنند. چنین سیستمی را **چندکامپیوتری (Multicomputer)** می‌نامند. **تقریباً همهٔ ابررایانه‌های فعلی از این ساختار استفاده می‌کنند**؛ پردازنده‌های چندهسته‌ای و خوشه‌های سرور هم نمونه‌های MIMD هستند.



MIMD: چند جریان دستور و چند جریان داده (چندکامپیوتری؛ تقریباً همهٔ ابررایانه‌های امروزی)

مثال ۱-۱: دسته‌بندی سیستم‌ها

هر یک از سیستم‌های زیر در کدام دستهٔ فلین قرار می‌گیرد؟

1. یک پردازندهٔ تک‌هسته‌ای ساده که برنامه را دستور به دستور اجرا می‌کند.
2. یک GPU که یک دستور را هم‌زمان روی هزاران پیکسل اجرا می‌کند.
3. خوشه‌ای از ۱۰۰ سرور که هر کدام بخش جداگانه‌ای از یک شبیه‌سازی را اجرا می‌کنند.
4. یک خط لولهٔ پنج‌مرحله‌ای که هر داده از پنج مرحلهٔ پردازشی متفاوت عبور می‌کند.
5. ابررایانهٔ Earth Simulator (۶۴۰ گره، هر گره ۸ پردازندهٔ برداری).

پاسخ

(۱) SISD. (۲) SIMD. (۳) MIMD. (۴) از دید معمارانی که خط لوله را MISD می‌دانند، (۵) MISD. ترکیبی است: هر پردازندهٔ برداری SIMD است و گره‌ها مستقل کار می‌کنند؛ به همین دلیل در اسلاید «Multi-SIMD» نامیده شده است.

۱-۳ ابررایانه‌ها (Supercomputers)

استفاده از **ساختارهای پیشرفتهٔ کامپیوتری در دسته‌های MISD ،SIMD و MIMD** در کنار **پیشرفت فناوری مدارهای مجتمع**، در هفت دههٔ گذشته عطش ساخت کامپیوترهای بسیار قدرتمند (ابررایانه‌ها) را زنده نگه داشته است.

ابررایانه‌ها **هر ۶ ماه یک بار** (ژوئن و نوامبر) ارزیابی و در فهرست www.top500.org رتبه‌بندی می‌شوند. معیار رتبه‌بندی **توان محاسباتی بر حسب FLOPS** است.

تعریف: FLOPS

FLOPS مخفف FLoating-point Operations Per Second یعنی «تعداد عملیات ممیز شناور در ثانیه» است. در فهرست TOP500 دو عدد گزارش می‌شود: $\mathbf{R_{max}}$ (کارایی اندازه‌گیری‌شده روی محک LINPACK) و $\mathbf{R_{peak}}$ (کارایی بیشینهٔ نظری).

پیشوند	نماد	مقدار	رابطه
kiloflops	KFLOPS	10^3	
megaflops	MFLOPS	10^6	
gigaflops	GFLOPS	10^9	
teraflops	TFLOPS	10^{12}	
petaflops	PFLOPS	10^{15}	
exaflops	EFLOPS	10^{18}	$1\ \text{EF} = 10^3\ \text{PF} = 10^6\ \text{TF} = 10^9\ \text{GF} = 10^{12}\ \text{MF} = 10^{15}\ \text{KF} = 10^{18}\ \text{FLOPS}$

۱-۳-۱ چند سیستم موفق در تاریخ ابررایانه‌ها

Cray-1 (ماشین SIMD)

- سال ۱۹۷۵؛ توان ۱۶۰ MFLOPS؛ **ماشین برداری** (Vector machine)؛ ۸ مگابایت حافظهٔ اصلی.

Cray T3E (ماشین MIMD، تا ۳ ترافلاپس)

تحویل در سال ۱۹۹۵. مشخصات:

ریزپردازندهٔ هر عنصر پردازشی (PE)	Alpha 21164A (EV5.6)، RISC سوپراسکالر ۴ راهه، ۲ عمل ممیز شناور در هر سیکل، قالب ممیز شناور ۳۲ و ۶۴ بیتی IEEE
حافظهٔ محلی	۲۵۶ یا ۵۱۲ مگابایت
کارایی بیشینه	۱۳۵۰ MFLOPS برای هر PE
بسته‌بندی	۸ PE در هر ماژول، خنک‌سازی مایع
تعداد PE در سیستم	۴۰ تا ۲۰۴۸، با گام‌های ۸ تایی
فناوری	DRAM با ۶۴ مگابایت و زمان دسترسی ۵۰ نانوثانیه
معماری	هم‌دوس از نظر حافظهٔ نهان (cache coherent)، حافظهٔ فیزیکی توزیع‌شده و قابل آدرس‌دهی سراسری
کل حافظه	۱۰ گیگابایت تا ۱ ترابایت
شبکهٔ اتصال	توپولوژی چنبرهٔ (torus) سه‌بعدی دوطرفه

Earth Simulator (ماشین Multi-SIMD)

- عملیاتی از اواخر ۲۰۰۲؛ حاصل ۵ سال طراحی و پیاده‌سازی؛ توانی معادل ۱۵ ابررایانهٔ برتر آمریکا در سال ۲۰۰۲.
- ۶۴۰ گره؛ **۸ پردازندهٔ برداری در هر گره** (در مجموع ۵۱۲۰ پردازنده)؛ ۸ GFLOPS برای هر پردازنده، در مجموع **۴۰ TFLOPS** (40×10^{12}).
- ۱۶ گیگابایت حافظه در هر گره (در مجموع ۱۰ ترابایت)؛ ۲۸۰۰ کیلومتر کابل؛ ۳۲۰ کابینت (هر کدام ۲ گره)؛ هزینه: ۳۵۰ میلیون دلار.

IBM BlueGene (ماشین MIMD)

- تحویل: ۲۰۰۴ تا ۲۰۰۷؛ کارایی بیشینه: **۳۶۰ TFLOPS**؛ توپولوژی: چنبرهٔ سه‌بعدی؛ تعداد گره‌ها: ۶۵۵۳۶؛ هر گره: ۲ پردازنده که هر کدام یک واحد ممیز شناور دوگانه دارد؛ هزینه: ۱۳۰ میلیون دلار.
- ساختار سلسله‌مراتبی سیستم (عدد اول کارایی یک پردازنده و عدد دوم کارایی بیشینه را نشان می‌دهد):

سطح	اجزا	کارایی	حافظه
تراشه (Chip)	۲ پردازنده	2.8/5.6 GF/s	4 MB

سطح	اجزا	کارایی	حافظه
کارت محاسباتی	۲ تراشه (1×2×1)	5.6/11.2 GF/s	0.5 GB DDR
بُرد گره (Node Board)	۳۲ تراشه (4×4×2)، ۱۶ کارت	90/180 GF/s	8 GB DDR
کابینت	۳۲ بُرد گره (8×8×16)	2.9/5.7 TF/s	256 GB DDR
سیستم	۶۴ کابینت (64×32×32)	180/360 TF/s	16 TB DDR

Cray X1 (ماشین عظیم SIMD–MIMD)

این سیستم (که در اسلاید «تکمیل‌نشده» ذکر شده) ترکیبی از دو خط محصول کِری بود:

- از **Cray PVP** (پردازنده‌های برداری): پردازنده‌های تکی بسیار قوی، پهنای باند حافظهٔ بسیار زیاد، محاسبه با گام غیرواحد (non-unit stride)، ویژگی‌های خاص مجموعه‌دستور؛ که مجموعه‌دستور و ریزمعماری آن مدرن شد.
- از **Cray T3E**: حافظهٔ اشتراکی توزیع‌شده، شبکهٔ مقیاس‌پذیر با پهنای باند زیاد، امکانات بهینهٔ ارتباط و همگام‌سازی؛ که با پردازنده‌های سفارشی بهبود یافت.
- هر گره: ۴ پردازندهٔ چندجریانی (MSP) هر کدام ۱۲.۸ GFLOPS (در مجموع ۵۱ GFLOPS و ۲۰۰ GB/s)، حافظهٔ اشتراکی محلی با پهنای باند زیاد (۱۲۸ کانال Direct Rambus)، ۳۲ پیوند شبکه و ۴ پیوند ورودی/خروجی. هر گره یک «خوشهٔ حافظهٔ اشتراکی» است.
- سیستم **NUMA** (دسترسی غیریکنواخت به حافظه) و قابل گسترش تا ۱۰۲۴ گره که از طریق شبکهٔ اتصال به هم وصل می‌شوند. نقشهٔ راه کِری رسیدن به «پتافلاپس پایدار» در بازهٔ ۲۰۱۰ تا ۲۰۱۵ بود.

IBM Roadrunner (ماشین عظیم MIMD)

- ابررایانهٔ IBM که «قواعد شناخته‌شدهٔ ابررایانه‌ها را تغییر داد». پروژه در ۲۰۰۶ در آزمایشگاه ملی لوس‌آلاموس آغاز و در ۲۰۰۸ تحویل شد؛ **نخستین ابررایانهٔ پتافلاپسی جهان**.
- کاربرد: تضمین ایمنی و قابلیت اطمینان ذخایر سلاح‌های هسته‌ای آمریکا؛ همچنین پژوهش در نجوم، انرژی، ژنوم انسان، تغییر اقلیم و کاربردهای نظامی.
- از پردازنده‌های چندهسته‌ای **Cell** (طراحی سونی، توشیبا و IBM) استفاده کرد و خط جدید «**ابررایانه‌های کم‌هزینه**» را بنا نهاد.
- هزینه: ۱۳۳ میلیون دلار؛ ۱۲۹۶۰ پردازندهٔ Cell (هر کدام ۹ هسته)؛ کارایی بیشینه ۱.۳۳ PFLOPS (1.33×10^{15})؛ کارایی پایدار ۱ PFLOPS؛ توان مصرفی ۳.۹ مگاوات؛ ۲۹۶ رَک در ۵۱۱ متر مربع؛ وزن ۲۲۷ تن.

LineShine (ماشین عظیم MIMD، ۲۰۲۶)

- هزینه: حدود **یک میلیارد دلار**؛ ۱۳,۷۸۹,۴۴۰ هستهٔ پردازندهٔ LX2؛ کارایی بیشینه حدود **۲.۷۳۵ اگزافلاپس** (۵۲ GFLOPS بر وات در FP64)؛ حافظه: ۱۳ پتابایت؛ توان: ۴۲.۲ مگاوات؛ سال: ۲۰۲۶.

۱-۳-۲ ده ابررایانهٔ برتر جهان (فهرست ژوئن ۲۰۲۶)

رتبه	سیستم	کشور	هسته‌ها	R_{max} (PF)	R_{peak} (PF)	توان (kW)
1	LineShine — LingKun LX2 304C 1.55GHz, LingQi, Kylin OS (NSCS)	*	13,789,440	2,198.40	2,735.82	42,220
2	El Capitan — HPE Cray EX255a, AMD EPYC 24C, MI300A	آمریکا	11,340,000	1,809.00	2,821.10	29,685
3	Frontier — HPE Cray EX235a, AMD EPYC 64C, MI250X	آمریکا	9,066,176	1,353.00	2,055.72	24,607
4	Aurora — HPE Cray EX, Intel Xeon CPU Max 9470	آمریکا	9,264,128	1,012.00	1,980.01	38,698
5	JUPITER Booster — BullSequana XH3000, GH200	آلمان	4,801,344	1,000.00	1,226.28	15,794
6	Eagle — Microsoft NDv5, Xeon 8480C, H100	آمریکا	2,073,600	561.20	846.84	—
7	HPC6 — HPE Cray EX235a, AMD EPYC 64C, MI250X	ایتالیا	3,143,520	477.90	606.97	8,461
8	Fugaku — A64FX 48C 2.2GHz, Tofu interconnect D	ژاپن	7,630,848	442.01	537.21	29,899
9	Alps — HPE Cray EX254n, Grace 72C, GH200	سوئیس	2,121,600	434.90	574.84	7,124
10	LUMI — HPE Cray EX235a, AMD EPYC 64C, MI250X	فنلاند	2,752,704	379.70	531.51	7,107

* در اسلاید، محل سیستم «National Supercomputing Centre in Shenzhen (NSCS)» و کشور آن «United States» درج شده است؛ مرکز نام‌برده در شنژن چین قرار دارد.

۱-۳-۳ ابررایانه در برابر مغز انسان

در سال ۲۰۱۴ پژوهشگران ژاپنی با **K-Computer** (با توان ۱۰ پتافلاپس، چهارمین ابررایانهٔ سریع جهان در آن زمان) محاسبات مربوط به **فقط یک ثانیه** از فعالیت بخشی از مغز را در **۴۰ دقیقه** انجام دادند! مقایسهٔ LineShine با مغز انسان:

معیار	LineShine	مغز انسان	برنده
توان محاسباتی	exaFLOPS 2.7~	petaFLOPS 1.2~ (برآورد)	ابررایانه
حجم	m³ 2000~	1400 cm³ (~0.001 m³)	مغز
توان مصرفی	MW 42	W 20	مغز

معیار	LineShine	مغز انسان	برنده
قیمت	~1000 میلیون دلار (هواپیمای A380: ۴۴۶ میلیون)	؟	مغز!

تمرین‌های حل‌شدهٔ بخش ابررایانه‌ها

تمرین ۱. LineShine (بر اساس R_{peak}) چند برابر Cray-1 سریع‌تر است؟

حل

$2.735\times10^{18} / (160\times10^{6}) \approx 1.71\times10^{10}$؛ یعنی حدود ۱۷ میلیارد برابر در ۵۱ سال.

تمرین ۲. عدد «۵۲ GFLOPS بر وات» برای LineShine را بررسی کنید.

حل

بر اساس R_{max}: $2198.40\times10^{15} / (42{,}220\times10^{3}\ \text{W}) \approx 5.21\times10^{10}\ \text{FLOPS/W} \approx 52\ \text{GFLOPS/W}$. پس این عدد بر اساس کارایی اندازه‌گیری‌شده است. اگر R_{peak} را در نظر بگیریم حدود ۶۴.۸ GFLOPS/W می‌شود.

تمرین ۳. بازدهی انرژی مغز را با LineShine مقایسه کنید.

حل

مغز: $60{,}000\ \text{GFLOPS/W} = 6\times10^{13}\ \text{FLOPS/W}$. نسبت: $1.2\times10^{15} / 20 = 6\times10^{13}$؛ یعنی مغز $60000/52 \approx 1150$ برابر کم‌مصرف‌تر است.

تمرین ۴. کار K-Computer (۴۰ دقیقه با ۱۰ PFLOPS) را LineShine با سرعت R_{max} در چه زمانی انجام می‌دهد؟

حل

حجم کار: $10\times10^{15} \times 2400\ \text{s} = 2.4\times10^{19}$ عمل. زمان: $2.4\times10^{19} / 2.1984\times10^{18} \approx 10.9\ \text{s}$. پس حتی سریع‌ترین ابررایانهٔ ۲۰۲۶ هم برای شبیه‌سازی یک ثانیه از بخشی از مغز حدود ۱۱ ثانیه وقت لازم دارد.

تمرین ۵. اعداد Earth Simulator و BlueGene را بررسی کنید.

حل

Earth Simulator: $640 \times 8 = 5120$ پردازنده؛ $5120 \times 8\ \text{GFLOPS} = 40{,}960\ \text{GFLOPS} \approx 40\ \text{TFLOPS}$؛ حافظه $640 \times 16\ \text{GB} = 10{,}240\ \text{GB} \approx 10\ \text{TB}$.

BlueGene: $64 \times 32 \times 32 = 65{,}536$ تراشه (گره)؛ $65{,}536 \times 5.6\ \text{GF/s} \approx 367\ \text{TF/s}$ که همان حدود ۳۶۰ TFLOPS است.

تمرین ۶. کارایی بیشینهٔ هر پردازندهٔ Cell در Roadrunner و بازدهی انرژی آن چقدر است؟

حل

$1.33\times10^{15} / 12960 \approx 102.6\ \text{GFLOPS}$ برای هر پردازنده. بازدهی: $3.9\times10^{6} \approx 256\ \text{MFLOPS/W}$. 10^{15}.

LineShine با ۵۲ GFLOPS/W حدود ۲۰۰ برابر بهتر است.

۱-۴ معرفی درس ساختار و زبان کامپیوتر

۱-۴-۱ هدف درس

در این درس **بخش‌های عملیاتی مختلف یک کامپیوتر و تعامل آن‌ها برای اجرای برنامه‌ای که به زبان ماشین نوشته شده است** را یاد می‌گیریم. همچنین **برنامه‌نویسی اسمبلی** و مراحلی را که یک برنامهٔ سطح بالا طی می‌کند تا سرانجام روی یک ماشین مشخص اجرا شود، فرا می‌گیریم.

۱-۴-۲ سرفصل‌ها

#	موضوع	سهم
۱	ساختار و زبان کامپیوتر و برنامه‌نویسی اسمبلی (مفاهیم عمومی)	۲۵٪
۲	ساختار و زبان اسمبلی IBM 360/370	۲۰٪
۳	ساختار و برنامه‌نویسی اسمبلی Intel 8086/88	۲۵٪
۴	ساختار و زبان اسمبلی RISC-V	۱۰٪
۵	ساختار و برنامه‌نویسی اسمبلی GPU	۱۵٪

۱-۴-۳ منابع

1. A. S. Tanenbaum, *Structured Computer Organization*, Prentice-Hall.
2. W. Rudd, *Assembly Language Programming and the IBM 360/370 Computers*, Prentice-Hall.
3. Y.-C. Liu, G. Gibson, *Microcomputer Systems: the 8086/8088 Family: Architecture, Programming, and Design*, Prentice-Hall.
4. P. A. Carter, *PC Assembly Language*, eBook, 2019 (pacman128.github.io/static/pcasm-book.pdf).
5. Intel Corp., *Intel 64 and IA-32 Architectures Software Developer's Manual*.
6. A. Waterman, K. Asanovic, (J. Hauser), *The RISC-V Instruction Set Manual*, Vol. I (II), UC Berkeley.
7. Nvidia, *CUDA Binary Utilities*, Release 12.2.
8. Nvidia, *PTX ISA*, Release 8.2.

۱-۴-۴ بارم‌بندی و زمان‌بندی

مورد	سهم	زمان
میان‌ترم ۱	۲۰٪	پنج‌شنبه ۱۴ آبان ۱۴۰۵، ساعت ۱۴

مورد	سهم	زمان
میان‌ترم ۲	۱۵٪	پنج‌شنبه ۱۹ آذر ۱۴۰۵، ساعت ۱۴
میان‌ترم ۳	۱۵٪	پنج‌شنبه ۱۰ دی ۱۴۰۵، ساعت ۱۴
پایان‌ترم	۲۰٪	چهارشنبه ۷ بهمن ۱۴۰۵، ساعت ۹
پروژه	۱۵٪	یکشنبه ۱۱ بهمن ۱۴۰۵
تکالیف	۱۵٪	آخر هفته (جمعه شب) برای تکالیفی که یکشنبه/سه‌شنبه داده می‌شوند

۱-۴-۵ گروه درس

- مدرس: **دکتر حمید سربازی‌آزاد** — azad@sharif.edu / azad@ipm.ir — تلفن ۶۶۱۶۶۶۲۲ — اتاق ۶۲۱
- دستیار ارشد: **عطیه غیبی‌فطرت** — atiye.gheibi@gmail.com — تلفن ۶۶۱۶۶۶۷۲ — آزمایشگاه HPCAN (اتاق ۷۰۳)

خلاصهٔ فصل ۱

- مدل فون‌نویمان: برنامه و داده در یک حافظه؛ CPU = CU + ALU؛ اجرای ترتیبی دستورها با چرخهٔ واکشی–رمزگشایی–اجرا.
- طبقه‌بندی فلین بر اساس تعداد جریان‌های دستور و داده: SISD (فون‌نویمان)، SIMD (برداری/GPU)، MISD (خط لوله و سیستولیک، محل اختلاف)، MIMD (چندکامپیوتری، ابررایانه‌های امروزی).
- ابررایانه‌ها هر ۶ ماه در TOP500 بر حسب FLOPS رتبه‌بندی می‌شوند؛ از Cray-1 با ۱۶۰ (۱۹۷۵) MFLOPS تا LineShine با حدود ۲.۷ (۲۰۲۶) EFLOPS.

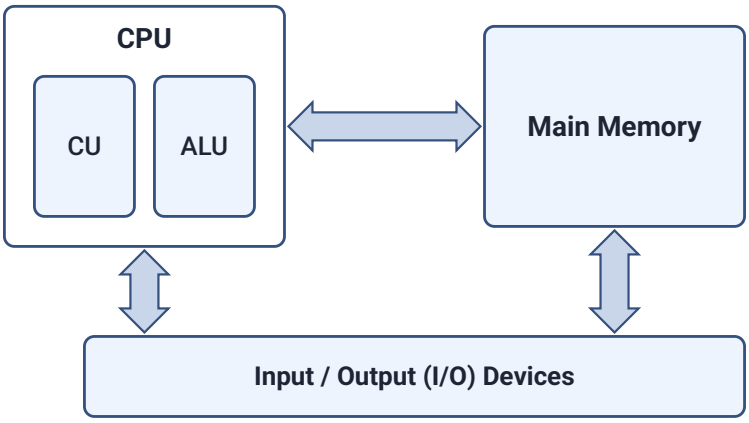
فصل ۲

حافظهٔ اصلی، قالب دستورالعمل و زبان اسمبلی

در این فصل (جلسهٔ دوم) می‌بینیم که **حافظهٔ اصلی** از دید برنامه‌نویس زبان ماشین چگونه است و پردازنده چطور با کمک ثبات‌های **MAR** و **MBR** به آن دسترسی پیدا می‌کند. سپس مفاهیم **واحد آدرس‌پذیر**، **کلمه**، **پهنای دسترسی**، **هم‌ترازی و ترتیب بایت‌ها (Endianness)** را بررسی می‌کنیم. در ادامه با **قالب دستورالعمل**، **کد عمل**، **مسیرهای داخلی ماشین** و سرانجام **زبان اسمبلی** آشنا می‌شویم و نخستین برنامه‌ها را برای ماشین‌های نمونهٔ ۱ و ۲ می‌نویسیم.

۲-۱ سیستم کامپیوتری

در ساختار معروف کامپیوتر با برنامهٔ ذخیره‌شدهٔ جان فون‌نویمان (مدل فون‌نویمان)، **پردازنده (CPU) به سیستم حافظه و دستگاه‌های ورودی/خروجی (I/O) متصل است**. پردازنده از دو بخش واحد کنترل (CU) و واحد محاسبه و منطق (ALU) تشکیل شده است.



شکل: اجزای اصلی یک سیستم کامپیوتری در مدل فون‌نویمان

۲-۲ حافظهٔ اصلی (Main Memory)

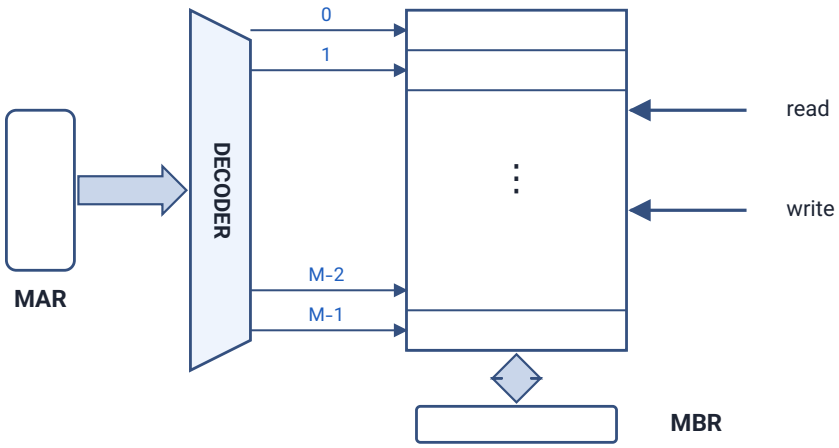
۲-۲-۱ حافظه از دید برنامه‌نویس

از دید برنامه‌نویس زبان ماشین، **حافظهٔ اصلی آرایه‌ای از خانه‌ها (locations) است که از ۰ تا M-1 آدرس‌دهی شده‌اند**؛ M اندازهٔ حافظه (تعداد خانه‌ها) است. هر خانه یک آدرس یکتا دارد و می‌تواند یک مقدار دودویی با طول ثابت را نگه دارد.

۲-۲-۲ دسترسی به حافظه: MAR، MBR و سیگنال‌های Read/Write

همهٔ دسترسی‌ها به حافظه از طریق **دو ثبات** و **دو سیگنال کنترلی** انجام می‌شوند:

- **MAR** (Memory Address Register، ثبات آدرس حافظه): آدرس خانه‌ای که باید خوانده یا نوشته شود در MAR قرار می‌گیرد. خروجی MAR به یک **رمزگشا (Decoder)** می‌رود که از میان M خط، دقیقاً خط مربوط به آن خانه را فعال می‌کند.
- **MBR** یا **MDR** (Memory Buffer/Data Register، ثبات دادهٔ حافظه): داده‌ای که با حافظه مبادله می‌شود (داده‌ای که قرار است نوشته شود یا داده‌ای که خوانده شده است) در MBR قرار دارد.
- **Read** و **Write**: سیگنال‌هایی که نوع عمل را مشخص می‌کنند.



شکل: دسترسی به حافظه از طریق MAR، رمزگشا (Decoder) و MBR

پرسش اسلاید: طول MAR و MBR چند بیت است؟

پاسخ

- **طول MAR** باید آن‌قدر باشد که بتواند آدرس هر خانه را نگه دارد. چون آدرس‌ها از ۰ تا M-1 هستند: $L_{MAR} = \lceil \log_2 M \rceil$ بیت.
- **طول MBR** برابر تعداد بیت‌هایی است که در هر دسترسی به حافظه خوانده یا نوشته می‌شود (پهنای دسترسی)؛ اگر چیز دیگری گفته نشده باشد، برابر طول هر خانهٔ حافظه است.

مثلاً برای حافظهٔ 16×16 بیتی (۱۶ خانهٔ ۱۶ بیتی): $L_{MAR} = \log_2 16 = 4$ بیت و $L_{MBR} = 16$ بیت. برای حافظه‌ای با 2^{32} بایت که در هر دسترسی ۳۲ بیت جابه‌جا می‌کند: $L_{MAR} = 32$ و $L_{MBR} = 32$.

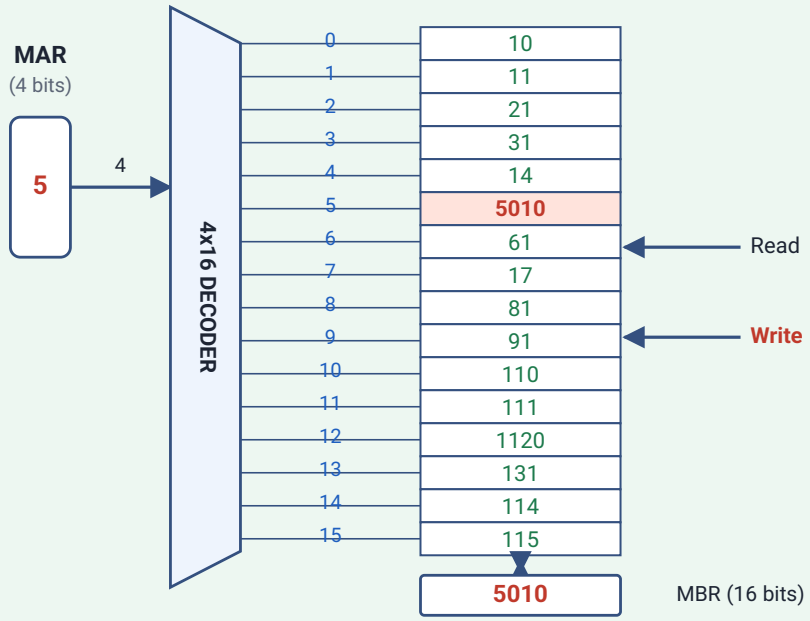
مثال ۲-۱: نوشتن در حافظه

در یک حافظهٔ 16×16 بیتی، مراحل نوشتن عدد 5010 در آدرس 5 را نشان دهید.

حل (عمل نوشتن)

1. عدد 5 (آدرس) در MAR قرار می‌گیرد. رمزگشای 16×4 خط شمارهٔ 5 را فعال می‌کند.
2. عدد 5010 (داده) در MBR قرار می‌گیرد.

3. سیگنال **Write** فعال می‌شود و داده در خانهٔ 5 نوشته می‌شود (محتوای قبلی این خانه از بین می‌رود).



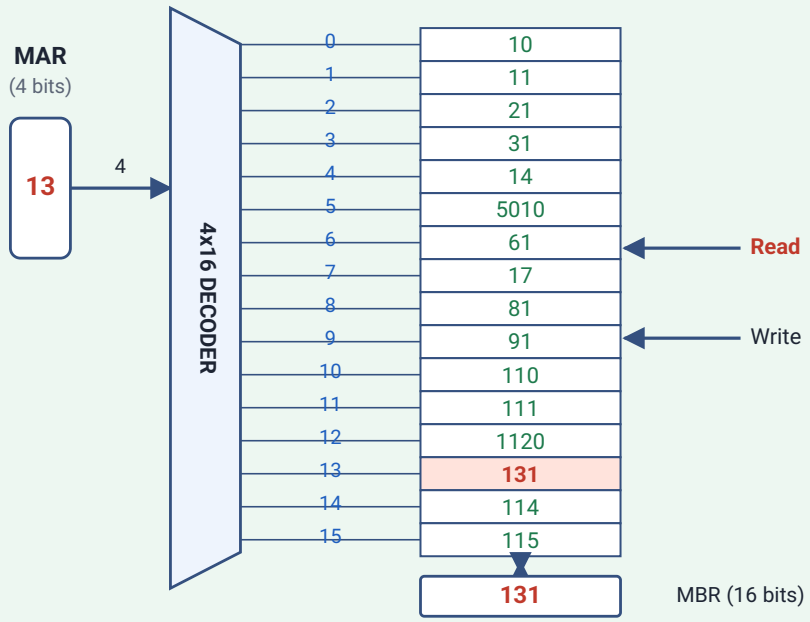
حافظه پس از عمل نوشتن: $(M_5) = 5010$

مثال ۲-۲: خواندن از حافظه

در همان حافظه، مراحل خواندن از آدرس 13 را نشان دهید.

حل (عمل خواندن)

1. عدد 13 در MAR قرار می‌گیرد؛ رمزگشا خط 13 را فعال می‌کند.
2. سیگنال **Read** فعال می‌شود و محتوای خانهٔ 13 یعنی 131 در MBR قرار می‌گیرد (محتوای خانه تغییر نمی‌کند).



حافظه پس از عمل خواندن: (MBR) = 131

💡 شهود: MAR برگهٔ درخواست، MBR سینی تحویل

حافظه مثل مخزن یک کتابخانه است که CPU اجازهٔ ورود به آن را ندارد. برای گرفتن یک کتاب، شمارهٔ قفسه را روی **برگهٔ درخواست (MAR)** می‌نویسد و سیگنال **Read** می‌دهد؛ کتابدار کتاب را در **سینی تحویل (MBR)** می‌گذارد. برای نوشتن، کتاب را در سینی می‌گذارد، شمارهٔ قفسه را در MAR می‌نویسد و سیگنال **Write** می‌دهد. به همین دلیل طول MAR تعیین می‌کند چند قفسه قابل نام بردن است (2^n خانه) و طول MBR اندازهٔ هر بار تحویل (یک کلمه) را.

۳-۲-۲ نمادگذاری

قرارداد نمادگذاری

- M_{addr}: خانهٔ حافظه با آدرس addr. مثلاً M_{1000} یعنی خانهٔ شمارهٔ 1000 حافظهٔ اصلی.
- (M_{addr}): **محتوای** آن خانه. پرانتز دور نام هر خانهٔ حافظه یا ثبات (مثل MAR یا MBR) یعنی محتوای آن. مثلاً (M_{1000}) یعنی داده‌ای که در خانهٔ 1000 ذخیره شده است.
- A ← B: مقدار B در A نوشته شود.

مثال ۲-۳: کار با نمادگذاری

محتوای یک حافظهٔ ۱۶ خانه‌ای (آدرس‌های 0 تا 15) به ترتیب چنین است:

آدرس	0	1	2	3	4	5	6	7
محتوا	10	11	21	31	14	1010	61	17

آدرس	8	9	10	11	12	13	14	15
محتوا	81	91	110	111	1120	131	114	115

تحلیل عبارت‌ها

- $(M_9) = 91$: محتوای خانهٔ 9.
- $M_{(M_0)}$ یعنی خانه‌ای که آدرسش برابر محتوای خانهٔ 0 است. چون $(M_0) = 10$، این عبارت همان M_{10} (خانهٔ شمارهٔ 10) است.
- $(M_{(M_1)})$: چون $(M_1) = 11$، این عبارت برابر $(M_{11}) = 111$ است.
- فرمان $(M_{12}) \leftarrow M_{10}$ یعنی محتوای خانهٔ 12 در خانهٔ 10 کپی شود؛ پس پس از اجرا $(M_{10}) = 1120$.
- فرمان $(M_{(MAR)}) \leftarrow MBR$ یعنی محتوای خانه‌ای که آدرسش در MAR است به MBR منتقل شود؛ این دقیقاً همان **عمل خواندن حافظه** است.

مثال ۲-۴ (پرسش اسلاید)

با همان محتوای اولیهٔ حافظه، پس از اجرای فرمان $(M_{(M_4)}) + (M_0) \leftarrow M_{(M_0)}$ چه اتفاقی می‌افتد؟

حل

- مقصد: $(M_0) = 10$ پس مقصد خانهٔ M_{10} است.
- عملوند اول: $(M_0) = 10$.
- عملوند دوم: $(M_4) = 14$ پس $(M_{14}) = 114$ = $(M_{(M_4)})$.

نتیجه: $10 + 114 \leftarrow M_{10}$، یعنی **$(M_{10}) = 124$**.

تمرین ۲-۱ (حل‌شده)

با محتوای اولیهٔ همان حافظه، اثر هر فرمان را (به‌طور جداگانه) بنویسید: (الف) $(M_{(M_0)}) - (M_{(M_4)}) \leftarrow M_4$ (ب) $(M_{(MAR)}) \leftarrow MBR$; $(M_0) \leftarrow MAR$ (ج) $(M_{14}) - (M_{15}) \leftarrow M_{(M_4)}$

حل

(الف) $(M_{14}) - (M_{10}) = 114 - 110 = 4$، پس $(M_4) = 4$. (جالب است که پس از آن، $M_{(M_4)}$ به خود M_4 اشاره می‌کند.)

(ب) $(MAR) = 10$ و سپس با یک عمل خواندن $(M_{10}) = 110$ = (MBR).

(ج) مقصد M_{14} است و مقدار $115 - 114 = 1$ در آن نوشته می‌شود: $(M_{14}) = 1$.

۲-۲-۴ ویژگی‌های حافظه

تعریف‌ها

- **واحد آدرس‌پذیر (Addressable unit)**: کوچک‌ترین گروه بیت‌ها که در حافظه آدرس یکتا دارد. در بیشتر ماشین‌ها **۸ بیت (یک بایت)** است.
- **پهنای/طول دسترسی (Access width/length)**: تعداد بیت‌هایی که در **یک** دسترسی به حافظه خوانده یا نوشته می‌شود. از ماشینی به ماشین دیگر فرق می‌کند.
- **کلمه (Word)**: تعداد بیت‌های عملوندهایی که CPU هنگام اجرای یک دستور ماشین پردازش می‌کند. در یک ماشین n بیتی، کلمه n بیتی است. در پردازنده‌های نسل اول n = 8 یا ۱۶ بود؛ در بیشتر ماشین‌های فعلی n = 32 است، اما ماشین‌های ۶۴ بیتی و بزرگ‌تر هم وجود دارند.

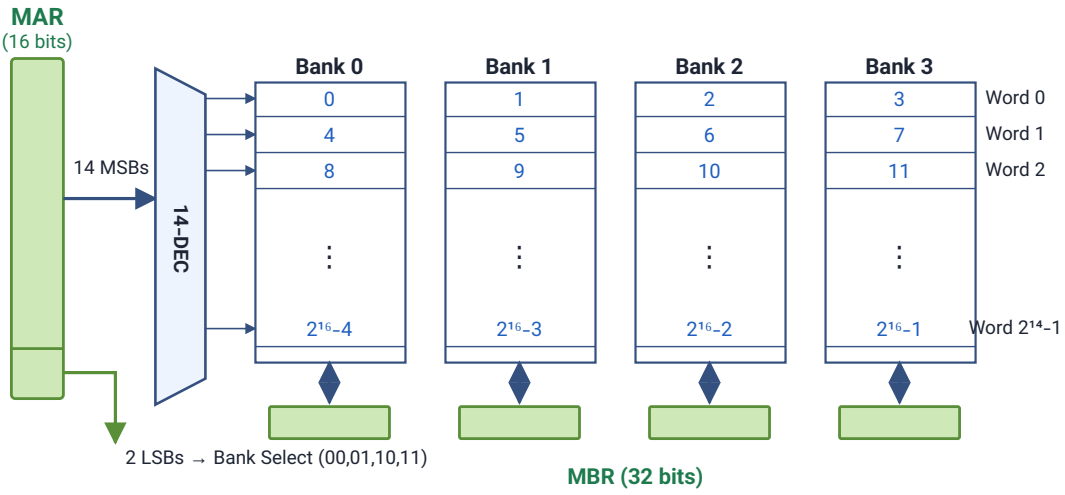
معمولاً: **اندازهٔ واحد آدرس‌پذیر < اندازهٔ کلمه < پهنای دسترسی. اگر به‌صراحت چیزی گفته نشده باشد، فرض می‌کنیم هر سه برابرند.**

۲-۲-۵ نمونهٔ یک معماری حافظهٔ اصلی (بانک‌بندی‌شده)

حافظه‌ای با مشخصات زیر را در نظر بگیرید:

- واحد آدرس‌پذیر = ۸ بیت، اندازهٔ کلمه = ۳۲ بیت، طول دسترسی = ۳۲ بیت؛
- ظرفیت حافظه = 2^{16} واحد آدرس‌پذیر = 2^{14} کلمه.

برای اینکه بتوان در یک دسترسی ۳۲ بیت (۴ بایت) را جابه‌جا کرد، حافظه به **۴ بانک** تقسیم شده است: بایت با آدرس a در بانک a mod 4 و در سطر (کلمهٔ) ⌊a/4⌋ قرار دارد. MAR ۱۶ بیتی است: **۱۴ بیت پرارزش** وارد یک رمزگشای ۱۴ به 2^{14} می‌شوند و شمارهٔ سطر را انتخاب می‌کنند و **۲ بیت کم‌ارزش** بانک را انتخاب می‌کنند (00 بانک 0، 01 بانک 1، 10 بانک 2، 11 بانک 3). MBR هم ۳۲ بیتی است و از چهار بخش ۸ بیتی (هر کدام متصل به یک بانک) تشکیل شده است.



شکل: حافظهٔ بایت‌آدرس‌پذیر با ۴ بانک؛ عدد داخل هر خانه آدرس آن بایت است

مثال ۲-۵: پیدا کردن بانک و سطر

بایت با آدرس `1235h` در کدام بانک و کدام سطر قرار دارد؟

حل

1235h = 0001 0010 0011 01|01. دو بیت کم‌ارزش 01 ⇐ **بانک ۱**. ۱۴ بیت پرارزش `048Dh` = 00 0100 1000 1101 = 1165 ⇐ **سطر (کلمهٔ) 1165**. (بررسی: 4×1165 + 1 = 4661 = 1235h).

۲-۲-۶ هم‌ترازی کلمه (Word Alignment)

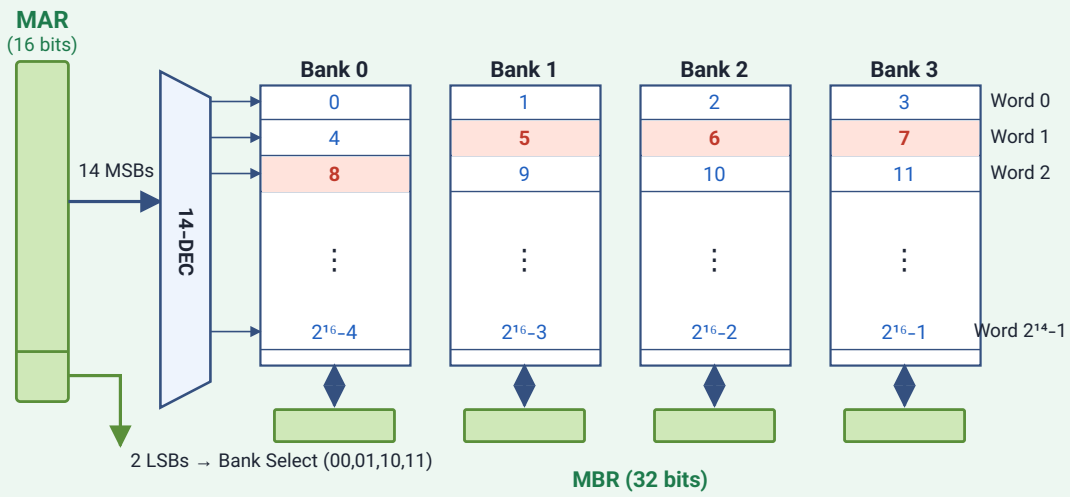
- **معماری حافظهٔ هم‌تراز (Aligned)**: آدرس هر کلمه مضربی از طول آن است. در حافظهٔ بالا یعنی آدرس‌های 0، 4، 8، 12، ... . در این حالت **هر کلمه در یک دسترسی خوانده/نوشته می‌شود** ☺ (هر چهار بایت در یک سطر هستند).
- **معماری حافظهٔ ناهم‌تراز (Unaligned)**: آدرس کلمه می‌تواند هر عددی باشد (0، 1، 2، 3، ...). در این حالت **ممکن است خواندن/نوشتن یک کلمه به دو دسترسی نیاز داشته باشد** ☹.

مثال ۲-۶: کلمهٔ ناهم‌تراز

در حافظهٔ ناهم‌تراز بالا، خواندن کلمهٔ ۳۲ بیتی که از آدرس 5 شروع می‌شود چند دسترسی لازم دارد؟

حل

این کلمه بایت‌های 5، 6، 7 و 8 را می‌گیرد. بایت‌های 5، 6 و 7 در سطر 1 (بانک‌های 1 تا 3) و بایت 8 در سطر 2 (بانک 0) است. چون رمزگشا در هر دسترسی فقط یک سطر را فعال می‌کند، **دو دسترسی** لازم است: یک بار سطر 1 و یک بار سطر 2؛ سپس سخت‌افزار باید چهار بایت را کنار هم بچیند.



کلمهٔ ناهم‌تراز در آدرس 5 (خانه‌های رنگی) در دو سطر پخش شده است

تمرین ۲-۲ (حل‌شده)

در همین حافظه (ناهم‌تراز)، (الف) یک کلمهٔ ۳۲ بیتی از چه آدرس‌هایی شروع شود تا در یک دسترسی خوانده شود؟ (ب) یک دادهٔ ۱۶ بیتی چطور؟ (ج) در مجموع چند درصد از آدرس‌های شروع ممکن، برای دادهٔ ۱۶ بیتی دو دسترسی لازم دارند؟

حل

(الف) فقط آدرس‌های مضرب ۴ ($a \bmod 4 = 0$). (ب) دادهٔ ۲ بایتی تا وقتی از مرز سطر عبور نکند در یک دسترسی خوانده می‌شود؛ یعنی $a \bmod 4 \in \{0, 1, 2\}$. (ج) فقط وقتی $a \bmod 4 = 3$ باشد دو دسترسی لازم است؛ یعنی ۲۵٪ آدرس‌ها.

💡 **شهود: هر ردیف قفسه یک کلمه**

حافظهٔ بانک‌بندی‌شده را ردیف‌هایی از قفسه‌های ۴ خانه‌ای تصور کنید که در هر دسترسی **یک ردیف کامل** برداشته می‌شود. کلمه‌ای که از ابتدای یک ردیف شروع شود (نشانی مضرب ۴) با یک دسترسی خوانده می‌شود. کلمه‌ای که وسط ردیف شروع شود نیمی در این ردیف و نیمی در ردیف بعد است: دو دسترسی و جابه‌جایی بایت‌ها لازم دارد. برای همین بعضی ماشین‌ها (مثل IBM 360 در فصل ۶) نشانی ناهم‌تراز را اصلاً نمی‌پذیرند.

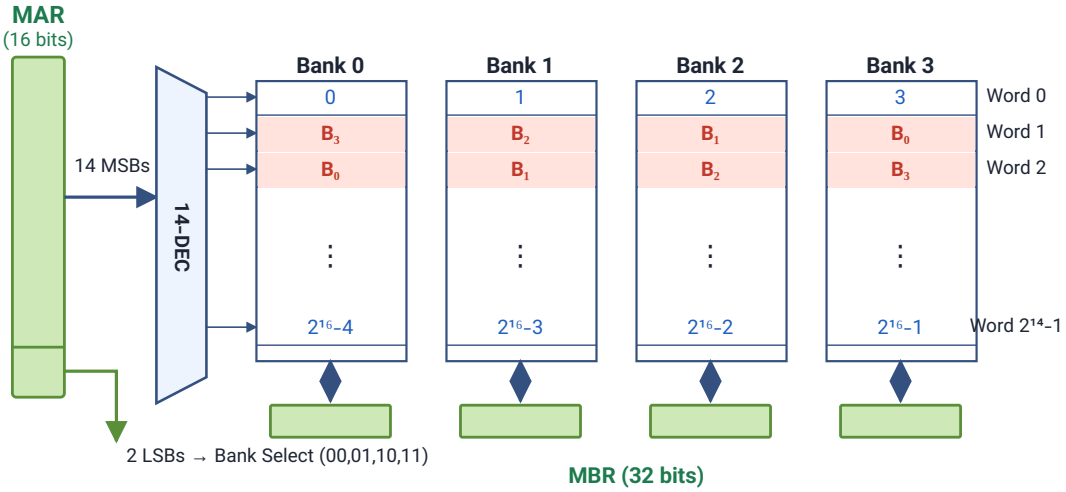
۲-۲-۷ ترتیب بایت‌ها (Endianness)

تعریف

فرض کنید یک کلمهٔ ۴ بایتی $B_3B_2B_1B_0$ داریم (B_3 پرارزش‌ترین بایت است).

- **Big Endian**: کلمه از **پرارزش‌ترین** واحد آدرس‌پذیر به سمت کم‌ارزش‌ترین، در آدرس‌های متوالی ذخیره می‌شود (بایت پرارزش در آدرس کوچک‌تر).
- **Little Endian**: کلمه از **کم‌ارزش‌ترین** واحد آدرس‌پذیر به سمت پرارزش‌ترین ذخیره می‌شود (بایت کم‌ارزش در آدرس کوچک‌تر).

در مثال اسلاید، کلمهٔ $B_3B_2B_1B_0$ یک بار به صورت Big Endian در آدرس `0004h` (سطر 1) و یک بار به صورت Little Endian در آدرس `0008h` (سطر 2) ذخیره شده است:



Big Endian در آدرس 0004h (سطر ۱) و Little Endian در آدرس 0008h (سطر ۲)

مثال ۲-۷

عدد ۳۲ بیتی `12345678h` را در آدرس `100h` به دو روش ذخیره کنید.

حل

آدرس	100h	101h	102h	103h
Big Endian	12	34	56	78
Little Endian	78	56	34	12

برای مثال، خانواده‌های IBM 360/370 و ماشین‌های نمونهٔ این درس Big Endian هستند، در حالی که پردازنده‌های Intel x86 (از جمله 8086/88) و RISC-V از Little Endian استفاده می‌کنند.

💡 شهود: مثل نوشتن تاریخ

تاریخ را می‌توان «۱۴۰۵/۰۸/۲۰» (اول بخش بزرگ‌تر: سال) یا «۲۰/۰۸/۱۴۰۵» (اول بخش کوچک‌تر: روز) نوشت؛ هر دو درست‌اند، فقط قرارداد فرق دارد. **Big Endian** اول بایت پرارزش را در نشانی کوچک‌تر می‌گذارد (همان ترتیب خواندن ما) و **Little Endian** اول بایت کم‌ارزش را. تا وقتی کلمه را یکجا بخوانیم فرقی دیده نمی‌شود؛ فرق فقط وقتی پیدا می‌شود که بایت‌های تک‌تک را نگاه کنیم (خواندن محتوای حافظه، انتقال داده بین دو ماشین). مزیت Little Endian: جمع از بایت کم‌ارزش (نشانی کوچک‌تر) شروع می‌شود و یک عدد با هر طولی از همان نشانی شروع می‌شود.

۲-۳ قالب دستورالعمل (Instruction Format)

یک جملهٔ امری در زبان‌های انسانی این قالب را دارد: **فاعل + فعل + مفعول(ها)**. مثال: "Hasan call Mike." یا "Zahra eat

your food."

در زبان ماشین هم قالب مشابهی داریم؛ با این تفاوت که **در کامپیوتر تنها فاعل، خود CPU است**، پس نیازی به ذکر آن نیست. بنابراین یک دستور ماشین این شکل را دارد:

فعل + مفعول(ها) یا **عمل (Operation) + عملوند(ها) (Operands)**

عمل با یک الگوی بیتی به نام **کد عمل (Operation Code, OpCode)** کد می‌شود و **مفعول‌ها** آدرس محل‌هایی هستند که عملوندها در آن‌ها قرار دارند (به صورت الگوی دودویی). بنابراین قالب دستور در یک **ماشین n آدرسه** چنین است:

Opcode	Operand 1	.....	Operand n
? bits	? bits		? bits

قالب کلی دستور در ماشین n آدرسه

> 💡 **شهود: دستور یک فرم با بودجهٔ ثابت بیت است**
>
> هر دستور مثل یک فرم چندخانه‌ای است: خانهٔ **کد عمل** فعل جمله است («جمع کن») و خانه‌های **عملوند** مفعول‌ها («این را با آن»). اگر خانه‌ای k بیت داشته باشد، 2^k انتخاب ممکن دارد. چون طول کل دستور محدود است، طراح باید بین «تعداد دستورها»، «اندازهٔ نشانی‌ها» و «تعداد عملوندها» مصالحه کند: بیت بیشتر برای یکی یعنی بیت کمتر برای بقیه. تقریباً همهٔ سؤال‌های «طول فیلد را حساب کنید» در امتحان همین حساب بودجه است.

۱-۳-۲ ماشین نمونهٔ ۱: یک ماشین دوآدرسه

- اندازهٔ حافظه: ۱۲۸ واحد ۱۶ بیتی.
- عملیات: جمع **Add** (کد 00)، تفریق **Subtract** (کد 01)، انتقال **Move** (کد 10) و بدون عمل **No-operation** (کد 11).

Opcode	Operand 1	Operand 2
2 bits	7 bits	7 bits

قالب دستور ماشین ۱ (در مجموع ۱۶ بیت = یک کلمه)

چرا این طول‌ها؟ چهار عمل داریم، پس $\lceil \log_2 4 \rceil = 2$ بیت برای کد عمل کافی است. حافظه ۱۲۸ خانه دارد، پس هر آدرس $\log_2 128 = 7$ بیت لازم دارد.

مثال ۲-۸: رمزگشایی یک دستور ماشین

دستور ماشین `01 0001111 0001010` چه کاری انجام می‌دهد؟

حل

کد عمل 01 یعنی **تفریق**؛ عملوند اول 0001111 = 15 و عملوند دوم 0001010 = 10. یعنی عملوندها M_{15} و M_{10} هستند و دستور به شکل نمادین چنین است:

$$M_{15} \leftarrow (M_{15}) - (M_{10});$$

تمرین ۲-۳ (حل‌شده)

در ماشین ۱: (الف) دستور $M_3 \leftarrow (M_3) + (M_{100})$ را به کد ماشین (دودویی و شانزده‌شانزدهی) بنویسید. (ب) کد ماشین `A1B5h` را رمزگشایی کنید.

حل

(الف) کد جمع 00، 3 = 0000011 و 100 = 1100100؛ پس `01E4h` = 0000 0001 1110 0100 = 00 0000011 1100100.

(ب) `A1B5h` = 1010 0001 1011 0101 ⇐ کد عمل 10 (Move)، عملوند اول 1000011 = 67، عملوند دوم 0110101 = 53؛ یعنی $M_{67} \leftarrow (M_{53})$.

۲-۳-۲ کد عمل چه اطلاعاتی می‌تواند بدهد؟

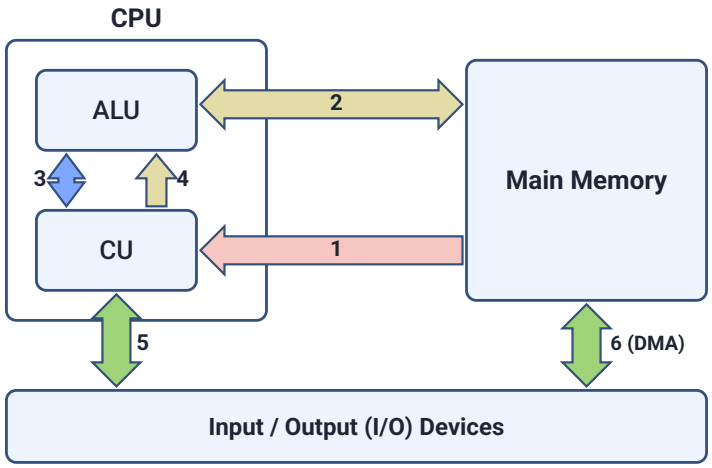
1. **نوع عمل**: جمع، تفریق، … (add، sub، …).
2. **تعداد عملوندها** (اگر ماشین چند قالب دستور داشته باشد).
3. **طول دستور** (اگر ماشین چند قالب دستور داشته باشد)؛ واحد کنترل از روی کد عمل می‌فهمد چند بایت دیگر باید واکشی کند.
4. **نوع عملوندها**: صحیح، حقیقی، علامت‌دار، … .
5. **طول عملوندها**: ۸ بیت، ۱۶ بیت، n بایت.
6. **روش آدرس‌دهی عملوندها**: مستقیم، غیرمستقیم، … (در فصل ۳ بررسی می‌شود).

۲-۴ مسیرهای داخلی ماشین

برای اجرای دستورها، داده و فرمان‌ها از مسیرهای مختلفی میان اجزای ماشین عبور می‌کنند:

مسیر	کاربرد	توضیح
۱	واکشی دستورها	از حافظهٔ اصلی به CU (ثبات دستور)

مسیر	کاربرد	توضیح
۲	دسترسی به داده	میان حافظه و ALU (خواندن عملوند و نوشتن نتیجه)
۳	ارتباط فرمان/وضعیت	CU فرمان عمل (مثلاً «تفریق کن») را به ALU می‌فرستد و ALU وضعیت (صفر بودن، منفی بودن، سرریز، …) را به CU برمی‌گرداند
۴	دسترسی ویژه به داده	دادهٔ بی‌واسطه (immediate) که داخل خود دستور است، از CU به ALU می‌رود
۵	دادهٔ I/O از طریق CPU	تبادل داده میان CPU و دستگاه‌های ورودی/خروجی
۶	دادهٔ I/O با DMA	دسترسی مستقیم دستگاه‌های I/O به حافظه بدون دخالت CPU (Direct Memory Access)



شکل: مسیرهای داخلی ماشین

مثال ۲-۹: مسیرهای استفاده‌شده در اجرای یک دستور

در ماشین ۱، برای واکشی و اجرای دستور `0001010 0001111 01` از کدام مسیرها استفاده می‌شود؟

حل

1. مسیر ۱ برای واکشی دستور `0001010 0001111 01` به CU.
2. مسیر ۲ برای آوردن داده (M_{15}) به ALU.
3. مسیر ۲ برای آوردن داده (M_{10}) به ALU.
4. مسیر ۳ برای فرستادن فرمان تفریق (کد 01) به ALU.
5. مسیر ۲ برای نوشتن نتیجه در M_{15}.

۲-۵ زبان اسمبلی (Assembly Language)

نوشتن، خواندن و فهمیدن برنامه‌هایی که از دستورهای ماشین به شکل دودویی تشکیل شده‌اند **مبهم و پیچیده** است. به همین دلیل از **نمایش نمادین** دستورها استفاده می‌کنیم که به آن **دستور اسمبلی** می‌گویند. مثلاً دستور ماشین `01 0001111 0001010` در ماشین ۱ را می‌توان به شکل نمادین زیر نوشت:

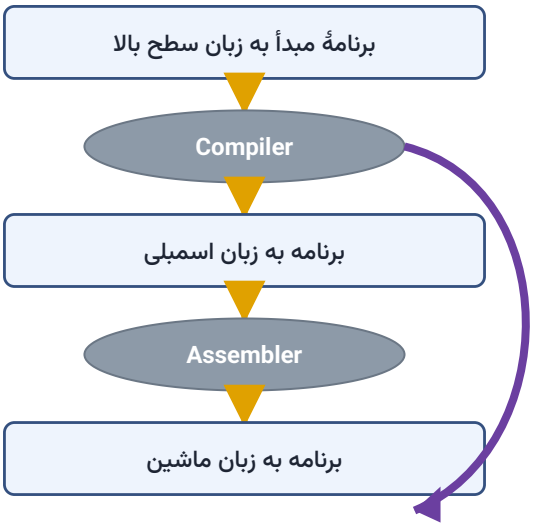
```
sub num1,num2
```

که در آن `num1` و `num2` معادل‌های نمادین عملوندهای M_{15} و M_{10} هستند و `sub` معادل نمادین (**نیمونیک، Mnemonic**) کد عمل 01 است.

> 💡 **شهود: اسمبلی یعنی دفترچهٔ تلفن برای کدهای ماشین**
>
> به‌جای حفظ کردن شمارهٔ تلفن‌ها، نام افراد را می‌نویسیم و دفترچه شماره را پیدا می‌کند. اسمبلی همین کار را با کدهای ماشین می‌کند: `ADD` به‌جای 0011 و `LOOP` به‌جای نشانی 0040. ترجمه **یک‌به‌یک** است (هر سطر اسمبلی ⇐ یک دستور ماشین)، پس هیچ قدرتی اضافه نمی‌شود؛ فقط خواندن، نوشتن و به‌خصوص **تغییر** برنامه آسان می‌شود، چون با اضافه کردن یک سطر، اسمبلر همهٔ نشانی‌ها را خودش دوباره حساب می‌کند.

۲-۵-۱ روند اجرای برنامه

برنامهٔ مبدأ به زبان سطح بالا (مثل C یا Pascal) توسط **کامپایلر** به برنامهٔ اسمبلی ترجمه می‌شود و **اسمبلر** برنامهٔ اسمبلی را به زبان ماشین تبدیل می‌کند. کامپایلر می‌تواند مستقیماً هم کد ماشین تولید کند (پیکان بنفش).



شکل: روند تبدیل برنامه تا زبان ماشین (کامپایلر می‌تواند مستقیماً کد ماشین هم تولید کند)

۲-۵-۲ مزایای زبان اسمبلی

دانستن زبان اسمبلی این دانش را می‌دهد:

- رابط برنامه‌ها با سیستم‌عامل، پردازنده و BIOS (سیستم پایهٔ ورودی/خروجی)؛

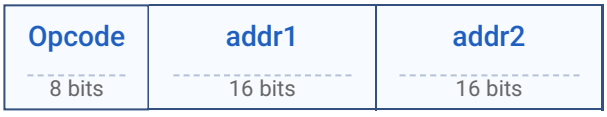
- نحوهٔ نمایش داده‌ها در حافظه و دیگر دستگاه‌های بیرونی؛
- اینکه پردازندهٔ X چگونه به دستورها دسترسی پیدا می‌کند و آن‌ها را اجرا می‌کند؛
- اینکه دستورها در پردازندهٔ X چگونه به داده دسترسی دارند و آن را پردازش می‌کنند؛
- اینکه برنامه‌ای که روی X اجرا می‌شود چگونه به دستگاه‌های بیرونی دسترسی پیدا می‌کند.

مزایای استفاده از زبان اسمبلی:

- حافظه و زمان اجرای کمتری لازم دارد؛
- انجام کارهای پیچیدهٔ وابسته به سخت‌افزار را آسان‌تر می‌کند؛
- برای کارهای حساس به زمان (time-critical) مناسب است؛
- بهترین گزینه برای نوشتن روال‌های خدمت وقفه (ISR) و دیگر برنامه‌های مقیم در حافظه است؛
- **برای شکستن قفل نرم‌افزارها استفاده می‌شود؛**
- **برای نوشتن ویروس‌ها، کرم‌ها و ... استفاده می‌شود.**

۲-۶ ماشین نمونهٔ ۲

یک ماشین دوآدرسه با حافظهٔ اصلی 2^{16} واحد آدرس‌پذیر (هر کدام ۸ بیت)، یعنی ۶۴ کیلوبایت. اندازهٔ کلمه ۱۶ بیت (۲ واحد آدرس‌پذیر) است. قالب دستور:



قالب دستور ماشین ۲ — طول هر دستور ۴۰ بیت = ۵ بایت

Opcode	Mnemonic	Operation
`00000000`	`mov addr1,addr2`	$M_{addr1} \leftarrow (M_{addr2})$
`00000001`	`add addr1,addr2`	$M_{addr1} \leftarrow (M_{addr1}) + (M_{addr2})$
`00000010`	`sub addr1,addr2`	$M_{addr1} \leftarrow (M_{addr1}) - (M_{addr2})$
`00000011`	`and addr1,addr2`	$M_{addr1} \leftarrow (M_{addr1}) \wedge (M_{addr2})$
`00000100`	`or addr1,addr2`	$M_{addr1} \leftarrow (M_{addr1}) \vee (M_{addr2})$
`00000101`	`xor addr1,addr2`	$M_{addr1} \leftarrow (M_{addr1}) \oplus (M_{addr2})$

پرسش اسلاید: طول ثبات‌های MAR و MBR چقدر است؟

پاسخ

حافظه 2^{16} خانه دارد ⇐ $L_{MAR} = 16$ **بیت**. چون عملوندها کلمه‌های ۱۶ بیتی‌اند و فرض می‌کنیم پهنای دسترسی برابر کلمه است ⇐ $L_{MBR} = 16$ **بیت**. (توجه کنید که واحد آدرس‌پذیر ۸ بیت است، اما هر دسترسی یک کلمهٔ ۱۶ بیتی را جابه‌جا می‌کند.)

۱-۶-۲ رهنمودهای اسمبلر (Directives)

در برنامه‌های اسمبلی علاوه بر دستورهای ماشین، **رهنمود** هم وجود دارد. رهنمودها به کد ماشین ترجمه نمی‌شوند و فقط به اسمبلر می‌گویند چه کند:

- `org X` : از این به بعد کد یا داده را از آدرس X قرار بده (origin).
- `name: dw v` : یک کلمه (define word) با مقدار اولیهٔ v تعریف کن و آدرس آن را با برچسب name بشناس. `dw 10 dup(?)` یعنی ۱۰ کلمه با مقدار نامعلوم.
- `end` : پایان برنامهٔ مبدأ.
- هر چیزی که پس از `;` بیاید توضیح (comment) است.

مثال ۲-۱۰ (تمرین اسلاید): جمع ۷ جملهٔ اول دنبالهٔ فیبوناچی

یک برنامهٔ اسمبلی برای ماشین ۲ بنویسید که مجموع ۷ جملهٔ اول دنبالهٔ فیبوناچی (0، 1، 1، 2، 3، 5، 8، 13، …) را حساب کند؛ سپس آن را به کد ماشین ترجمه کنید.

ایدهٔ حل

ماشین ۲ دستور پرش ندارد، پس نمی‌توانیم حلقه بنویسیم و باید دستورها را پشت‌سرهم تکرار کنیم. دو متغیر `t1` و `t2` دو جملهٔ آخر را نگه می‌دارند (ابتدا 0 و 1) و `sum` مجموع جمله‌ها را (ابتدا 1 = 1 + 0). در هر مرحله جملهٔ بعدی را با جمع دو جملهٔ قبلی می‌سازیم و به sum اضافه می‌کنیم. `t1` و `t2` به نوبت جملهٔ جدید را نگه می‌دارند، پس نیازی به جابه‌جا کردن مقدارها نیست.

برنامه و ترجمهٔ آن: هر دستور ۵ بایت است، پس آدرس دستورها ۵ تا ۵ تا زیاد می‌شود (10Ah، 105h، 100h، …). متغیرها از آدرس 200h و هر کدام ۲ بایتی‌اند: `sum` = 0204h، `t2` = 0202h، `t1` = 0200h. کد ماشین `add t1,t2` یعنی کد عمل `01`، addr1 = `0202` و addr2 = `0204`.

Address	Machine Code	Label	Instr.	Operands	توضیح
			`org`	`100h`	شروع برنامه از آدرس 100h
`0100`	`01 0202 0204`		**add**	`t1,t2`	جملهٔ ۳ در t1 (t1 = 0+1 = 1)
`0105`	`01 0200 0202`		**add**	`sum,t1`	افزودن جملهٔ ۳ به sum
`010A`	`01 0204 0202`		**add**	`t2,t1`	جملهٔ ۴ در t2 (t2 = 1+1 = 2)

Address	Machine Code	Label	Instr.	Operands	توضیح
010F	01 0200 0204		add	sum,t2	افزودن جملهٔ ۴ به sum
0114	01 0202 0204		add	t1,t2	جملهٔ ۵ در t1 (t1 = 1+2 = 3)
0119	01 0200 0202		add	sum,t1	افزودن جملهٔ ۵ به sum
011E	01 0204 0202		add	t2,t1	جملهٔ ۶ در t2 (t2 = 2+3 = 5)
0123	01 0200 0204		add	sum,t2	افزودن جملهٔ ۶ به sum
0128	01 0202 0204		add	t1,t2	جملهٔ ۷ در t1 (t1 = 3+5 = 8)
012D	01 0200 0202		add	sum,t1	افزودن جملهٔ ۷ به sum
			org	200h	شروع متغیرها از آدرس 200h
0200	0001	sum:	dw	1	متغیر sum (مجموع دو جملهٔ اول: 1+0)
0202	0000	t1:	dw	0	متغیر t1
0204	0001	t2:	dw	1	متغیر t2
			end		پایان برنامه

ردگیری اجرا:

بعد از دستور	t1	t2	sum
(ابتدا)	0	1	1
add t1,t2 / add sum,t1	1	1	2
add t2,t1 / add sum,t2	1	2	4
add t1,t2 / add sum,t1	3	2	7
add t2,t1 / add sum,t2	3	5	12
add t1,t2 / add sum,t1	8	5	**20**

نتیجه: 20 = 8+5+3+2+1+1+0 ✓. اندازهٔ کد: ۱۰ دستور × ۵ بایت = ۵۰ بایت (100h تا 131h).

تمرین ۲-۴ (حل‌شده): جابه‌جایی دو متغیر بدون متغیر کمکی

در ماشین ۲ برنامه‌ای بنویسید که محتوای دو متغیر `x` و `y` را بدون استفاده از متغیر سوم جابه‌جا کند و آن را به کد ماشین ترجمه کنید (برنامه از 100h و داده‌ها از 200h).

حل

از خاصیت $a \oplus a = 0$ و $a \oplus 0 = a$ استفاده می‌کنیم:

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	

Address	Machine Code	Label	Instr.	Operands	توضیح
0100	05 0200 0202		xor	x,y	x = x⊕y
0105	05 0202 0200		xor	y,x	y = y⊕(x⊕y) = x
010A	05 0200 0202		xor	x,y	x = (x⊕y)⊕x = y
			org	200h	
0200	1234	x:	dw	1234h	مقدار نمونه
0202	ABCD	y:	dw	0ABCDh	مقدار نمونه
			end		

ردگیری با مقدارهای نمونه: پس از دستور اول x = 1234h ⊕ ABCDh = B9F9h؛ پس از دستور دوم y = ABCDh ⊕ B9F9h = 1234h؛ پس از دستور سوم x = B9F9h ⊕ 1234h = ABCDh. ✓

تمرین ۲-۵ (حل‌شده)

در ماشین ۲ مقدار r = 2(a + b) - c را حساب کنید (برنامه از 100h؛ متغیرهای a، b، c و r از 200h).

حل

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	
0100	00 0206 0200		mov	r,a	r = a
0105	01 0206 0202		add	r,b	r = a+b
010A	01 0206 0206		add	r,r	r = 2(a+b)
010F	02 0206 0204		sub	r,c	r = 2(a+b)-c
			org	200h	
0200	????	a:	dw	?	
0202	????	b:	dw	?	
0204	????	c:	dw	?	
0206	????	r:	dw	?	
			end		

خلاصهٔ فصل ۲

- حافظه آرایه‌ای از M خانه است؛ دسترسی با MAR (آدرس، $\lceil \log_2 M \rceil$ بیت)، MBR (داده) و سیگنال‌های Read/Write.
- M_a خانه و (M_a) محتوای آن است؛ $M_{(M_a)}$ یعنی محتوای a به عنوان آدرس به کار رود.
- واحد آدرس‌پذیر، کلمه و پهنای دسترسی؛ کلمهٔ ناهم‌تراز ممکن است دو دسترسی بخواهد؛ Big Endian (بایت پرارزش در آدرس کم) و Little Endian.
- دستور ماشین = کد عمل + عملوندها؛ طول کد عمل از تعداد عمل‌ها و طول هر آدرس از اندازهٔ حافظه به دست می‌آید.
- زبان اسمبلی نمایش نمادین زبان ماشین است (نیمونیک + برچسب)؛ کامپایلر ← اسمبلر ← کد ماشین.

فصل ۳

ثبات‌ها، روش‌های آدرس‌دهی، اسمبلر، پیونددهنده و بارکننده

این فصل (جلسهٔ سوم) مفصل‌ترین بخش جزوه است. ابتدا انواع **ثبات‌ها** را از نظر ساختار، پیاده‌سازی، آدرس‌پذیری و کاربرد بررسی می‌کنیم. سپس **روش‌های آدرس‌دهی** (مستقیم، غیرمستقیم، بی‌واسطه، شاخص‌دار و ضمنی) را همراه با **ماشین‌های نمونهٔ ۳، ۴ و ۵** می‌بینیم و برای هر کدام برنامه می‌نویسیم و به کد ماشین ترجمه می‌کنیم. روند **اسمبلر، پیونددهنده (Linker) و بارکننده (Loader)** را توضیح می‌دهیم و در پایان با **ماشین تک‌آدرسه (انباره‌ای)**، **ماشین صفرآدرسه (پشته‌ای)**، ماشین نمونهٔ ۶ و **نمادگذاری لهستانی** آشنا می‌شویم.

۳-۱ ثبات‌ها (Registers)

تعریف

یک **ثبات n بیتی** از **n فلیپ‌فلاپ** ساخته شده است که **به صورت گروهی کنترل می‌شوند** (کلاک و سیگنال‌های کنترلی مشترک دارند) و با هم یک مقدار n بیتی را نگه می‌دارند.

بر اساس در دسترس بودن خروجی فلیپ‌فلاپ‌ها و روش دسترسی به آن‌ها، ثبات‌ها چهار نوع‌اند:

- **PIPO**: ورود موازی، خروج موازی (Parallel In, Parallel Out)
- **SIPO**: ورود سریال، خروج موازی (Serial In, Parallel Out)
- **PISO**: ورود موازی، خروج سریال (Parallel In, Serial Out)
- **SISO**: ورود سریال، خروج سریال (Serial In, Serial Out)

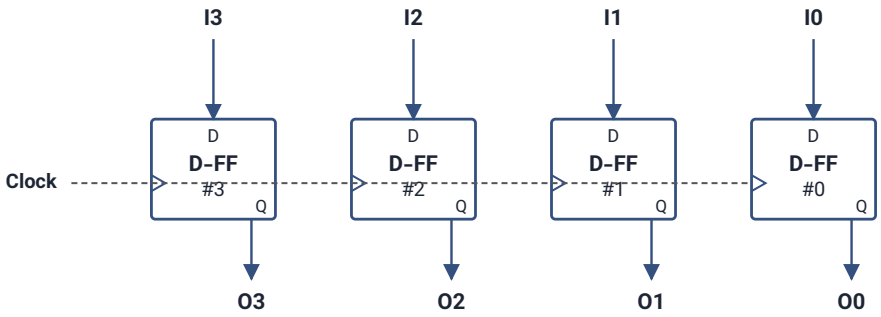
💡 شهود: ثبات‌ها میز کار، حافظه قفسه

ثبات‌ها داخل خود CPU و کنار ALU هستند؛ مثل میز کار که کوچک ولی دم دست است، در برابر حافظه که بزرگ ولی دور است. دو دلیل باعث می‌شود تعداد ثبات‌ها کم باشد: (۱) هر ثبات باید در دستور با چند بیت نام برده شود (۱۶ ثبات ⇐ ۴ بیت) و (۲) ثبات‌های زیاد مدار را بزرگ و کند می‌کنند. هنر برنامه‌نویسی اسمبلی تا حد زیادی همین است که داده‌های پرمصرف را روی میز نگه داریم.

۳-۱-۱ ثبات PIPO

ورودی D هر فلیپ‌فلاپ مستقیماً به یکی از خطوط ورودی موازی (I_0…I_3) وصل است و خروجی Q هر فلیپ‌فلاپ یکی از خطوط خروجی موازی (O_0…O_3) است. با هر لبهٔ کلاک، هر چهار بیت هم‌زمان بار می‌شوند و هم‌زمان قابل خواندن‌اند.

ثبات‌های معمولی پردازنده (مثل ثبات‌های همه‌منظوره) از این نوع‌اند.



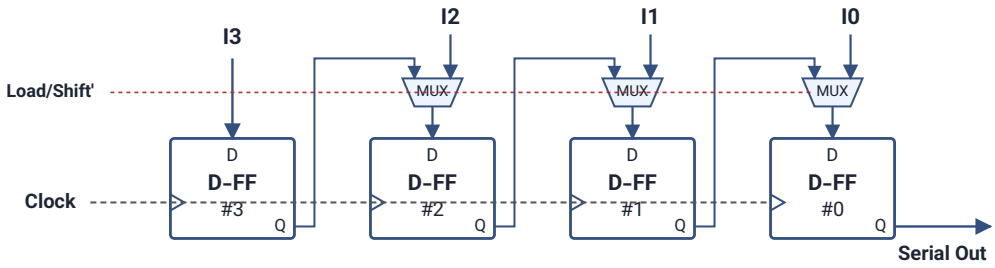
ثبات ۴ بیتی PIPO (ورود موازی، خروج موازی)

۳-۱-۲ ثبات PISO

جلوی ورودی هر فلیپ‌فلاپ (به‌جز اولی) یک **مالتی‌پلکسر ۲ به ۱** قرار دارد که با سیگنال Load/Shift' کنترل می‌شود:

- وقتی Load = 1: ورودی هر فلیپ‌فلاپ، بیت موازی متناظر (I_i) است؛ یعنی بار موازی.
- وقتی Load/Shift' = 0 (انتقال): ورودی هر فلیپ‌فلاپ خروجی فلیپ‌فلاپ سمت چپ آن است؛ با هر کلاک محتوا یک بیت به راست منتقل می‌شود و بیت‌ها یکی‌یکی از خروجی فلیپ‌فلاپ ۰ (Serial Output) خارج می‌شوند.

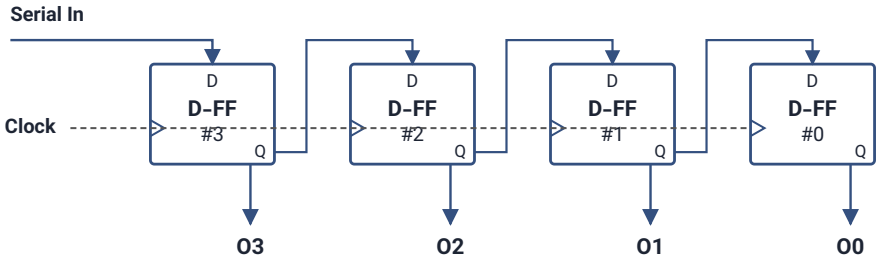
کاربرد: تبدیل دادهٔ موازی به سریال، مثلاً برای ارسال روی یک سیم (درگاه سریال).



ثبات ۴ بیتی PISO (ورود موازی، خروج سریال) — با مالتی‌پلکسر Load/Shift

۳-۱-۳ ثبات SIPO

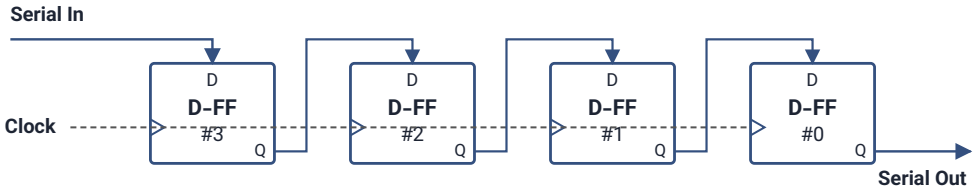
داده بیت‌به‌بیت از ورودی سریال وارد فلیپ‌فلاپ ۳ می‌شود و با هر کلاک، محتوای هر فلیپ‌فلاپ به فلیپ‌فلاپ سمت راست می‌رود (ثبات انتقالی). پس از n کلاک، n بیت دریافت‌شده به صورت موازی روی خروجی‌ها در دسترس است. کاربرد: تبدیل سریال به موازی (گیرندهٔ سریال).



ثبات ۴ بیتی SIPO (ورود سریال، خروج موازی) — ثبات انتقالی

۴-۱-۳ ثبات SISO

مانند SIPO است، اما فقط خروجی آخرین فلیپ‌فلاپ در دسترس است. هر بیت ورودی پس از n کلاک به خروجی می‌رسد؛ بنابراین می‌تواند به عنوان **خط تأخیر** n کلاکی به کار رود.



ثبات ۴ بیتی SISO (ورود سریال، خروج سریال)

مثال ۳-۱: کار ثبات‌های انتقالی

(الف) دنبالهٔ بیت‌های 1, 0, 1, 1 (اول ۱) به ورودی سریال یک SIPO چهاربیتی (با محتوای اولیهٔ صفر) داده می‌شود. محتوای ثبات پس از هر کلاک چیست؟ (ب) عدد 1011 در یک PISO بار می‌شود. ترتیب بیت‌های خروجی سریال چیست؟

حل

(الف) ترتیب بیت‌ها $Q_3Q_2Q_1Q_0$:

کلاک	ورودی	Q3 Q2 Q1 Q0
0	—	0 0 0 0
1	1	1 0 0 0
2	0	0 1 0 0
3	1	1 0 1 0
4	1	1 1 0 1

پس از ۴ کلاک، اولین بیت ارسالی در Q_0 و آخرین بیت در Q_3 است.

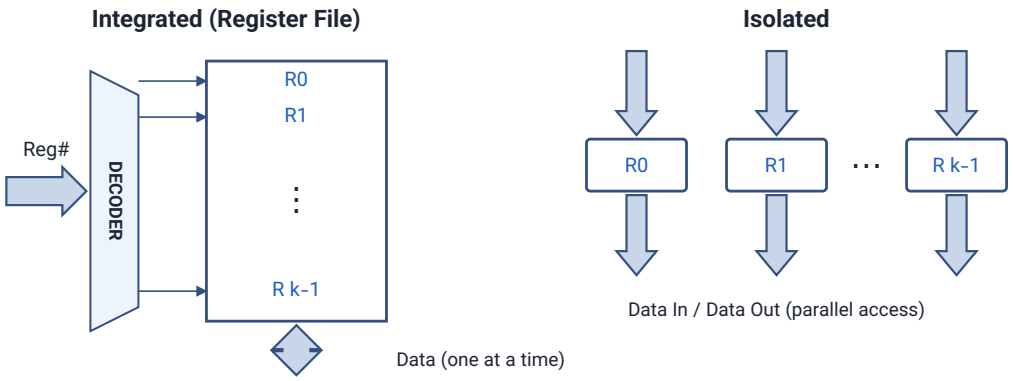
(ب) پس از بار شدن 1011 = $Q_3Q_2Q_1Q_0$، خروجی سریال Q_0 است. در کلاک‌های بعدی (انتقال) به ترتیب Q_1، Q_2 و Q_3 به خروجی می‌رسند؛ پس دنبالهٔ خروجی 1, 1, 0, 1 است (اول کم‌ارزش‌ترین بیت).

💡 **شهود: جادهٔ یک‌بانده و چندبانده**

ورود و خروج **موازی** مثل جادهٔ n بانده است: همهٔ بیت‌ها با هم در یک پالس ساعت رد می‌شوند. ورود و خروج **سریال** مثل جادهٔ یک‌بانده است: بیت‌ها پشت سر هم، یکی در هر پالس. SIPO داده‌ای را که از یک سیم (مثلاً درگاه سریال) بیت‌به‌بیت می‌رسد جمع می‌کند و یکجا تحویل می‌دهد؛ PISO برعکس، یک کلمه را می‌گیرد و روی یک سیم بیت‌به‌بیت می‌فرستد. سریال سیم کمتری لازم دارد و موازی سریع‌تر است.

۳-۱-۵ ثبات‌ها از نظر پیاده‌سازی

- **یکپارچه (Integrated) یا فایل ثبات (Register File)**: مانند یک حافظهٔ کوچک است که هر خانهٔ آن یک ثبات است. همهٔ ثبات‌ها از طریق گذرگاه آدرس و دادهٔ داخلی (شمارهٔ ثبات وارد یک رمزگشا می‌شود) دسترسی می‌شوند ⇐ **در هر لحظه فقط به یک ثبات می‌توان دسترسی داشت.**
- **مجزا (Isolated)**: هر ثبات خطوط ورودی و خروجی جداگانه دارد ⇐ **دسترسی موازی (هم‌زمان) به ثبات‌ها** ممکن است.



شکل: ثبات‌های یکپارچه (Register File) در برابر ثبات‌های مجزا (Isolated)

۳-۱-۶ ثبات‌ها از نظر آدرس‌پذیری و کاربرد

- **آدرس‌پذیر (Addressable)**: برنامه‌نویس اسمبلی می‌تواند در دستورها به آن‌ها اشاره کند؛ مثل R0، R1، ...، R15 در خانوادهٔ IBM 360، یا AX، BX، CX، DX، ... در پردازندهٔ 8086/88.
- **غیرآدرس‌پذیر (Non-addressable)**: برنامه‌نویس نمی‌تواند مستقیماً به آن‌ها اشاره کند، اما ماشین از آن‌ها استفاده می‌کند؛ مثل PC، MBR، MAR و IR.

از نظر کاربرد، ثبات‌های آدرس‌پذیر دو دسته‌اند:

- **همه‌منظوره (General purpose)**: برای همهٔ کاربردهایی که مجموعه‌دستور ماشین تعریف کرده است قابل استفاده‌اند؛ مثل ثبات‌های 0 تا 15 در IBM 360/370.
- **خاص‌منظوره (Special purpose)**: بعضی دستورها از آن‌ها برای کار خاصی استفاده می‌کنند؛ مثل CX (برای شمارش در حلقه) یا DX (برای ضرب، تقسیم و دستورهای I/O) در خانوادهٔ 8086/88.

۳-۱-۷ ثبات‌های داخلی ماشین

ثبات‌های غیرآدرس‌پذیر اصلی عبارت‌اند از:

- **MAR** (ثبات آدرس حافظه): آدرس خانهٔ حافظه را در طول یک سیکل حافظه نگه می‌دارد.
- **MBR** (ثبات بافر حافظه): داده‌ای را که باید از/در حافظه خوانده/نوشته شود نگه می‌دارد.
- **IR** (Instruction Register، ثبات دستور): دستوری را که از حافظه به CU واکشی شده نگه می‌دارد.
- **PC** (Program Counter، شمارندهٔ برنامه): آدرس دستور بعدی را که باید از حافظه واکشی و اجرا شود نگه می‌دارد.

ثبات‌های آدرس‌پذیر شامل ثبات‌های همه‌منظوره و خاص‌منظوره هستند. **تعداد و کاربرد این ثبات‌ها از ماشینی به ماشین دیگر فرق می‌کند.**

نکته: طول ثبات‌های داخلی (قاعدهٔ کلی برای همهٔ ماشین‌های نمونه)

- $L_{MAR} = \lceil \log_2(\text{تعداد واحدهای آدرس‌پذیر}) \rceil$ و $L_{PC} = L_{MAR}$ (چون PC هم آدرس حافظه را نگه می‌دارد).
- L_{MBR} = پهنای دسترسی (معمولاً برابر طول کلمه).
- L_{IR} = طول **بلندترین** قالب دستور (IR باید بتواند هر دستوری را نگه دارد).
- طول ثبات‌های همه‌منظوره و انباره = طول کلمه.

مثال ۳-۲: چرخهٔ واکشی با ثبات‌های داخلی

مراحل واکشی یک دستور ۵ بایتی را به زبان انتقال ثبات بنویسید (فرض: هر دسترسی یک واحد آدرس‌پذیر می‌خواند).

حل

```
MAR ← (PC);  MBR ← (M(MAR));  IR[39:32] ← (MBR);  PC ← (PC) + 1   ; کد عمل
...   (همین کار برای ۴ بایت بعدی و بخش‌های بعدی IR)..
PC ← (آدرس دستور بعد (یعنی ۵ واحد جلوتر از ابتدای این دستور
```

سپس CU کد عمل موجود در IR را رمزگشایی می‌کند، آدرس‌های عملوند را از IR برمی‌دارد و با همان روش MAR/MBR عملوندها را می‌خواند و نتیجه را می‌نویسد. دستورهای پرش، مقدار جدیدی در PC می‌نویسند.

۳-۲ روش‌های آدرس‌دهی: مستقیم و غیرمستقیم

روش آدرس‌دهی (Addressing mode) تعیین می‌کند که CPU چگونه از روی فیلدهای دستور، محل عملوند را پیدا کند. در این فصل و فصل ۵ هشت روش را می‌بینیم: مستقیم، غیرمستقیم، بی‌واسطه، شاخص‌دار، ضمنی، نسبی، قطعه‌ای و صفحه‌ای.

۳-۲-۱ ۱. آدرس‌دهی مستقیم (Direct)

آدرس عملوند در قالب دستور (IF) داده می‌شود. دو حالت دارد:

- **ثباتی مستقیم**: شمارهٔ ثبات در IF آمده است.
- **حافظه‌ای مستقیم**: آدرس حافظه در IF آمده است.

مثال‌ها

- `mov ax, array` (در 8086/88): `ax` آدرس نمادین ثبات ax است که در IF آمده و `array` هم آدرس نمادینی در IF است که مستقیماً یک کلمه در حافظهٔ اصلی را آدرس می‌دهد. (ax ← محتوای خانهٔ array)
- `L 2,NUM` (در خانوادهٔ IBM 360): 2 یعنی ثبات شمارهٔ ۲ و NUM آدرس نمادین یک کلمه در حافظهٔ اصلی است. ($R2 \leftarrow (M_{NUM})$)

تقریباً همهٔ ماشین‌ها آدرس‌دهی مستقیم دارند.

۲-۲-۳ ۲. آدرس‌دهی غیرمستقیم (Indirect)

آدرسی که در IF آمده، **ثبات یا خانه‌ای از حافظه را نشان می‌دهد که آدرس عملوند (در حافظه) را در خود دارد**. دو حالت دارد: **ثباتی غیرمستقیم** و **حافظه‌ای غیرمستقیم**.

مثال‌ها

- `L 2,0(3)` (در IBM 360): ثبات ۲ با کلمه‌ای از حافظه بار می‌شود که **آدرس آن در ثبات ۳** است: $R2 \leftarrow (M_{(R3)})$.
- `call far ptr [p1]` (در 8086/88): روالی فراخوانی می‌شود که **آدرسش در کلمه‌ای در خانهٔ p1** ذخیره شده است. p1 آدرس نمادین اشاره‌گری است که در IF آمده؛ یعنی (M_{p1}) = آدرس روال.

تقریباً همهٔ ماشین‌ها آدرس‌دهی غیرمستقیم دارند.

سطوح غیرمستقیم: آدرس‌دهی غیرمستقیم می‌تواند ۱، ۲ یا چند سطحی باشد:

- یک سطحی: $M_{(M_{addr})}$ آدرس واقعی عملوند است.
- دو سطحی: $M_{(M_{(M_{addr})})}$ آدرس واقعی عملوند است؛ و به همین ترتیب.

بیشتر ماشین‌هایی که آدرس‌دهی غیرمستقیم دارند از یک سطح استفاده می‌کنند؛ تعداد بسیار کمی دو سطح دارند.

مثال ۳-۳: مقایسهٔ مستقیم و غیرمستقیم

فرض کنید $(M_{200}) = 300$، $(M_{300}) = 400$، $(M_{400}) = 7$ و $(R3) = 300$. در دستوری که فیلد آدرس آن 200 است (یا ثبات R3 را نام برده است)، عملوند در هر روش چیست؟

حل

روش	آدرس مؤثر	عملوند
حافظه‌ای مستقیم (addr = 200)	200	300
حافظه‌ای غیرمستقیم یک سطحی	$(M_{200}) = 300$	400
حافظه‌ای غیرمستقیم دو سطحی	$(M_{300}) = 400$	7
ثباتی مستقیم (R3)	—	300
ثباتی غیرمستقیم ((R3))	300	400

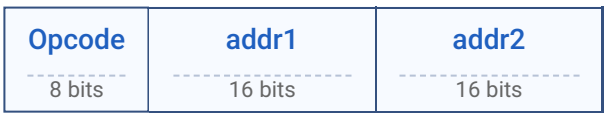
💡 **شهود: یادداشتی که می‌گوید کتاب کجاست**

مستقیم یعنی «کتاب در قفسهٔ ۵۰ است». غیرمستقیم یعنی «در قفسهٔ ۵۰ یادداشتی است که می‌گوید کتاب کجاست» (همان **اشاره‌گر**). مزیت بزرگ: برای رفتن سراغ کتاب بعدی لازم نیست خود دستور را عوض کنیم؛ فقط عدد روی یادداشت را یکی زیاد می‌کنیم. برای همین پیمایش آرایه بدون «برنامهٔ خودتغییردهنده» ممکن می‌شود. هزینه: یک دسترسی اضافه به حافظه برای خواندن یادداشت.

۳-۳ ماشین نمونهٔ ۳

یک ماشین دوآدرسه با مشخصات زیر:

- حافظهٔ اصلی: 2^{16} واحد آدرس‌پذیر (هر کدام ۸ بیت)؛
- اندازهٔ کلمه: ۱۶ بیت، **ناهم‌تراز**، **Big Endian**؛
- روش‌های آدرس‌دهی: حافظه‌ای مستقیم و **حافظه‌ای غیرمستقیم**؛
- **دستورهای پرش شرطی** برای ساختن حلقه؛
- قالب دستور:



قالب دستور ماشین ۳ (۴۰ بیت = ۵ بایت)

Opcode	Mnemonic	Operation
00000000	`mov addr1,addr2`	$M_{addr1} \leftarrow (M_{addr2})$
00000001	`mov addr1,(addr2)`	$M_{addr1} \leftarrow (M_{(M_{addr2})})$
00000010	`mov (addr1),addr2`	$M_{(M_{addr1})} \leftarrow (M_{addr2})$
00000011	`add addr1,addr2`	$M_{addr1} \leftarrow (M_{addr1}) + (M_{addr2})$
00000100	`sub addr1,addr2`	$M_{addr1} \leftarrow (M_{addr1}) - (M_{addr2})$
00000101	`add addr1,(addr2)`	$M_{addr1} \leftarrow (M_{addr1}) + (M_{(M_{addr2})})$
00000110	`sub addr1,(addr2)`	$M_{addr1} \leftarrow (M_{addr1}) - (M_{(M_{addr2})})$
10000000	`jnz addr1,addr2`	if $(M_{addr1}) \neq 0$ then PC ← addr2
10000001	`jz addr1,addr2`	if $(M_{addr1}) = 0$ then PC ← addr2
10000010	`jneg addr1,addr2`	if $(M_{addr1}) < 0$ then PC ← addr2
10000011	`jpos addr1,addr2`	if $(M_{addr1}) > 0$ then PC ← addr2

در دستورهای پرش، addr1 آدرس متغیری است که شرط روی آن بررسی می‌شود و addr2 آدرس مقصد پرش است (آدرس دستوری که در صورت برقراری شرط، در PC قرار می‌گیرد).

پرسش اسلاید: طول ثبات‌های ماشین ۳

پاسخ

$L_{MAR} = 16$ بیت (2^{16} واحد آدرس‌پذیر)؛ $L_{MBR} = 16$ بیت (کلمه)؛ $L_{IR} = 40$ بیت (طول دستور)؛ $L_{PC} = 16$ بیت.

در اسلاید برای این ماشین چهار تمرین مطرح شده است: (۱) برنامه‌ای برای محاسبهٔ مجموع ۱۰۰ جملهٔ اول دنبالهٔ فیبوناچی متناوب و (۲) ترجمهٔ آن؛ (۳) برنامه‌ای برای جمع عناصر یک آرایهٔ ۱۰۰ کلمه‌ای و (۴) ترجمهٔ آن. در ادامه همه را کامل حل می‌کنیم.

۱-۳-۳ برنامهٔ ۱: مجموع ۱۰۰ جملهٔ اول دنبالهٔ فیبوناچی متناوب

دنبالهٔ موردنظر … 0, -1, 1, -2, 3, -5, 8, -13, 21, -34, است؛ یعنی $a_i = (-1)^i F_i$. جمله‌های با اندیس زوج مثبت و با اندیس فرد منفی‌اند.

ایدهٔ حل

دو متغیر `curr` و `next` دو جملهٔ متوالی دنبالهٔ فیبوناچی معمولی را نگه می‌دارند (ابتدا $F_0 = 0$ و $F_1 = 1$). در هر دور حلقه، **دو جمله** را پردازش می‌کنیم: `curr` (اندیس زوج) را به sum **اضافه** و `next` (اندیس فرد) را از sum **کم** می‌کنیم. سپس دو جملهٔ بعدی را می‌سازیم: curr ← curr + next و next ← next + curr (با curr جدید). چون هر دور دو جمله را حساب می‌کند، حلقه **۵۰ بار** اجرا می‌شود. ماشین دستور بی‌واسطه ندارد، پس ثابت 1 را در متغیر `one` نگه می‌داریم.

Address	Machine Code	Label	Instr.	Operands	توضیح
			`org`	`0`	
`0000`	`03 0022 001E`	`loop:`	**add**	`sum,curr`	افزودن جملهٔ فعلی (زوج) به sum
`0005`	`04 0022 0020`		**sub**	`sum,next`	کم کردن جملهٔ بعدی (فرد) از sum
`000A`	`03 001E 0020`		**add**	`curr,next`	ساختن جملهٔ بعدی در curr
`000F`	`03 0020 001E`		**add**	`next,curr`	ساختن جملهٔ بعد از آن در next
`0014`	`04 0024 0026`		**sub**	`count,one`	یکی کم کردن از شمارنده
`0019`	`80 0024 0000`		**jnz**	`count,loop`	اگر صفر نشد، به loop برو
`001E`	`0000`	`curr:`	**dw**	`0`	جملهٔ فعلی
`0020`	`0001`	`next:`	**dw**	`1`	جملهٔ بعدی
`0022`	`0000`	`sum:`	**dw**	`0`	مجموع
`0024`	`0032`	`count:`	**dw**	`50`	شمارندهٔ ۵۰ دور (32h)
`0026`	`0001`	`one:`	**dw**	`1`	ثابت ۱
			`end`		پایان برنامه

نحوهٔ ترجمه: برنامه از آدرس 0 شروع می‌شود و هر دستور ۵ بایت است، پس دستورها در 0000، 0005، 000A، 000F، 0014 و 0019 هستند و داده‌ها از 1Eh = 30 = 6×5 شروع می‌شوند (هر متغیر ۲ بایت). مثلاً `add sum,curr` ⇐ کد عمل add یعنی `03`، آدرس sum یعنی `0022` و آدرس curr یعنی `001E`. در `jnz count,loop`، کد عمل `80` (10000000)، addr1 = آدرس count و addr2 = آدرس loop (0000) است.

دور	sum پس از add	sum پس از sub	curr	next	count	جمله‌های جمع‌شده
ابتدا	—	0	0	1	50	
1	0	-1	1	2	49	0, -1
2	0	-2	3	5	48	1, -2
3	1	-4	8	13	47	3, -5
…	count = 0 پس از ۵۰ دور، ۱۰۰ جمله جمع شده و است و حلقه تمام می‌شود.					

نکتهٔ تکمیلی: سرریز

مجموع واقعی این ۱۰۰ جمله با فرمول $\Sigma_{i=0..n}(-1)^i F_i = (-1)^n F_{n-1} - 1$ برابر $-F_{98} - 1 \approx -1.35\times10^{20}$ است که بسیار بیشتر از گسترهٔ یک کلمهٔ ۱۶ بیتی (−32768 تا 32767) است. پس در عمل جمع‌ها سرریز می‌کنند و ماشین فقط حاصل به پیمانهٔ 2^{16} را نگه می‌دارد (با شبیه‌سازی: −18114). هدف این تمرین، تمرین نوشتن حلقه است؛ اما همیشه به گسترهٔ اعداد توجه کنید.

۲-۳-۳ برنامهٔ ۲: جمع عناصر یک آرایهٔ ۱۰۰ کلمه‌ای (با آدرس‌دهی غیرمستقیم)

ایدهٔ حل

متغیر `arrptr` یک **اشاره‌گر** است که ابتدا آدرس اولین عنصر آرایه را دارد. با دستور غیرمستقیم `add sum,(arrptr)` عنصری که arrptr به آن اشاره می‌کند به sum اضافه می‌شود. سپس arrptr را ۲ واحد جلو می‌بریم (هر کلمه ۲ بایت است). برای شمارش، به‌جای ثابت 1 از همان ثابت 2 استفاده می‌کنیم: `count` را با 200 شروع می‌کنیم و هر دور ۲ تا کم می‌کنیم؛ پس حلقه دقیقاً 200/2 = 100 بار اجرا می‌شود و به یک ثابت کمتر نیاز داریم.

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	شروع برنامه از 100h
0100	05 0116 0114	loop:	add	sum,(arrptr)	افزودن عنصری که arrptr به آن اشاره می‌کند
0105	03 0114 011A		add	arrptr,two	اشاره‌گر به عنصر بعد
010A	04 0118 011A		sub	count,two	دو تا کم کردن از شمارنده
010F	80 0118 0100		jnz	count,loop	اگر صفر نشد، به loop برو
0114	011C	arrptr:	dw	array	اشاره‌گر به عناصر آرایه
0116	0000	sum:	dw	0	مجموع
0118	00C8	count:	dw	200	شمارنده (۱۰۰ دور)
011A	0002	two:	dw	2	ثابت ۲
011C	???? ???? …	array:	dw	100 dup(?)	آرایهٔ ۱۰۰ کلمه‌ای (۲۰۰ بایت)
			end		پایان برنامه

توجه کنید که مقدار اولیهٔ `arrptr` خودِ آدرس `array` یعنی `011C` است. در دور k ام (k از ۰)، arrptr برابر 011Ch + 2k است و پس از ۱۰۰ دور به 011Ch + 200 = 01E4h می‌رسد.

پرسش اسلاید: پس از آخرین دور حلقه چه اتفاقی می‌افتد؟

وقتی count صفر شود، `jnz` پرش نمی‌کند و PC به آدرس بعدی یعنی `0114h` می‌رود. اما آنجا **داده** است، نه دستور! CPU هیچ راهی برای تشخیص داده از دستور ندارد و بایت‌های

… ?? ?? 02 00 C8 00 00 00 1C 01 (با مقدارهای اولیه؛ در عمل مقدارهای پس از اجرا) را به عنوان دستور واکشی و اجرا می‌کند: مثلاً 01 را کد عمل mov addr1,(addr2) می‌گیرد و … . نتیجه غیرقابل پیش‌بینی است و ممکن است داده‌ها یا حتی خود برنامه خراب شوند.

راه‌حل: در پایان باید کنترل را به سیستم‌عامل برگرداند. ماشین ۳ پرش غیرشرطی ندارد، اما می‌توان با یک پرش شرطی که شرطش همیشه برقرار است همین کار را کرد؛ مثلاً پس از jnz بنویسیم jnz two,OS_return ؛ چون 0 ≠ 2 = (two)، همیشه به آدرس بازگشت سیستم‌عامل می‌پرد.

۳-۳-۳ برنامهٔ ۳: جمع آرایه بدون آدرس‌دهی غیرمستقیم (کد خودتغییرده)

پرسش اسلاید: **فرض کنید آدرس‌دهی غیرمستقیم نداریم** (دستورهای با کد 01، 02، 05 و 06 حذف شده‌اند). اکنون برنامهٔ جمع عناصر آرایهٔ ۱۰۰ کلمه‌ای را بنویسید.

ایدهٔ حل: تغییر دادن خود دستور

در دستور add sum,array آدرس عنصر آرایه (array) در فیلد addr2 دستور قرار دارد. اگر بعد از هر دور، **خود این فیلد** را ۲ واحد زیاد کنیم، دور بعد همین دستور عنصر بعدی را جمع می‌کند. دستور از آدرس loop شروع می‌شود: بایت loop کد عمل، بایت‌های loop+1 و loop+2 فیلد addr1 و بایت‌های **loop+3 و loop+4 فیلد addr2** هستند. چون ماشین ناهم‌تراز و Big Endian است، کلمهٔ ۱۶ بیتی در آدرس loop+3 دقیقاً همان فیلد addr2 است؛ پس add loop+3,two آدرس عملوند دستور اول را ۲ واحد جلو می‌برد.

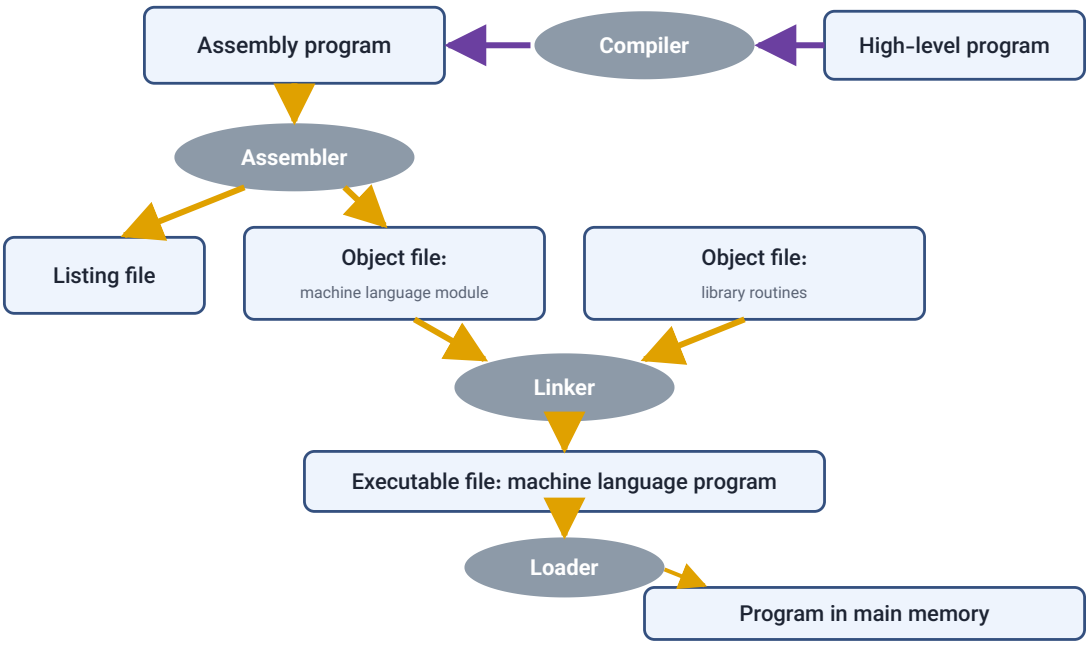
Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	
0100	03 0114 011A	loop:	add	sum,array	این دستور در هر دور تغییر می‌کند
0105	03 0103 0118		add	loop+3,two	addr2 دستور بالا را ۲ واحد زیاد کن
010A	04 0116 0118		sub	count,two	شمارنده
010F	80 0116 0100		jnz	count,loop	تکرار
0114	0000	sum:	dw	0	
0116	00C8	count:	dw	200	
0118	0002	two:	dw	2	
011A	???? …	array:	dw	100 dup(?)	
			end		

ردگیری: در دور اول دستور خانهٔ 0100 به شکل 03 0114 011A است (جمع عنصر 0)؛ پس از آن add loop+3,two بایت‌های 0103–0104 را از 011A به 011C تغییر می‌دهد و دستور به 03 0114 011C تبدیل می‌شود (جمع عنصر 1)؛ … و در پایان فیلد addr2 به 01E2h رسیده است.

مزایا و معایب تغییر کد (Code alteration)

- تغییر کد برای **رمزنگاری** یا **طراحی قفل نرم‌افزاری** مفید است (تحلیل برنامه را سخت می‌کند).
- اما از دید **مهندسی نرم‌افزار** بد است (خوانایی، اشکال‌زدایی و نگهداری دشوار می‌شود).
- همچنین اگر با دقت مدیریت نشود، وقتی یک تکه کد باید **بیش از یک بار** اجرا شود **نتیجهٔ اشتباه** می‌دهد. در برنامهٔ بالا اگر برنامه دوباره اجرا شود، دستور اول با addr2 = 01E2h شروع می‌کند، نه 011Ah! راه‌حل: در ابتدای برنامه فیلد را دوباره مقداردهی کنیم (مثلاً `mov loop+3,init` که `init: dw array`) و sum و count را هم دوباره مقداردهی کنیم.

۳-۴ روند اجرای برنامهٔ اسمبلی: اسمبلر، پیونددهنده و بارکننده



شکل: روند اجرای یک برنامهٔ اسمبلی: اسمبلر ← پیونددهنده ← بارکننده

برنامهٔ سطح بالا ابتدا توسط کامپایلر به اسمبلی ترجمه می‌شود (یا برنامه از اول به اسمبلی نوشته شده است). **اسمبلر** آن را به یک **فایل شیء** (ماژول زبان ماشین) و یک **فایل فهرست** (listing) تبدیل می‌کند. **پیونددهنده** فایل شیء برنامه را با فایل‌های شیء روال‌های کتابخانه‌ای ترکیب می‌کند و **فایل اجرایی** می‌سازد. سرانجام **بارکننده** فایل اجرایی را در حافظهٔ اصلی قرار می‌دهد و اجرا می‌کند.

۳-۴-۱ اسمبلر (Assembler)

اسمبلر زبان اسمبلی را به **فایل‌های شیء** (object files) تبدیل می‌کند. همچنین **فایل فهرست** می‌سازد که متنی خوانا (کد ماشین هر خط کنار خود دستور، مثل جدول‌های این جزوه) است. فایل شیء ترکیبی از دستورهای ماشین، داده‌ها و اطلاعات لازم برای قرار دادن درست دستورها و داده‌ها در حافظه است.

بیشتر اسمبلرها دوگذره (two-pass) کار می‌کنند:

- **گذر ۱:** همهٔ خط‌های برنامهٔ مبدأ را می‌خواند و (با یک شمارندهٔ مکان، Location Counter) آدرس همهٔ برچسب‌ها را در **جدول نمادها** (Symbol Table) ثبت می‌کند.
- **گذر ۲:** با استفاده از اطلاعات جدول نمادها، کد ماشین واقعی هر خط برنامه را تولید می‌کند.

چرا دو گذر؟ به دلیل **ارجاع رو به جلو** (forward reference): در برنامهٔ ۲ ماشین ۳، دستور اول (`add sum,(arrptr)`) از برچسب‌هایی استفاده می‌کند که بعداً تعریف شده‌اند؛ اسمبلر هنگام رسیدن به این خط هنوز آدرس آن‌ها را نمی‌داند.

مثال ۳-۴: گذرهای اسمبلر برای برنامهٔ ۲ ماشین ۳

گذر ۱ (شمارندهٔ مکان و جدول نمادها)

خط	قبل LC	اندازه	نماد ثبت‌شده
org 100h	—	—	LC ← 0100
loop: add …	0100	5	loop = 0100
add / sub / jnz	0105 / 010A / 010F	هر کدام 5	
arrptr: dw	0114	2	arrptr = 0114
sum: dw	0116	2	sum = 0116
count: dw	0118	2	count = 0118
two: dw	011A	2	two = 011A
array: dw 100 dup	011C	200	array = 011C

گذر ۲: برای هر دستور، کد عمل از جدول کدهای عمل و آدرس‌ها از جدول نمادها خوانده می‌شود؛ مثلاً `add sum,(arrptr)` ⇐ `05 0116 0114`؛ برای `arrptr: dw array` هم مقدار `011C` نوشته می‌شود.

💡 شهود: چرا دو گذر؟

در کتابی که ارجاع «ر.ک. صفحهٔ ؟» به فصل‌های بعدی دارد، نمی‌توان در یک بار خواندن همهٔ شماره‌صفحه‌ها را نوشت. اسمبلر هم همین مشکل را با **ارجاع جلورو** (پرش به برچسبی که پایین‌تر تعریف شده) دارد. پس در **گذر اول** فقط می‌شمارد هر سطر در چه نشانی قرار می‌گیرد و جدول نمادها (فهرست برچسب‌ها و نشانی‌ها) را می‌سازد؛ در **گذر دوم** با جدول کامل، کد ماشین هر سطر را می‌نویسد.

۳-۴-۲ قالب فایل شیء

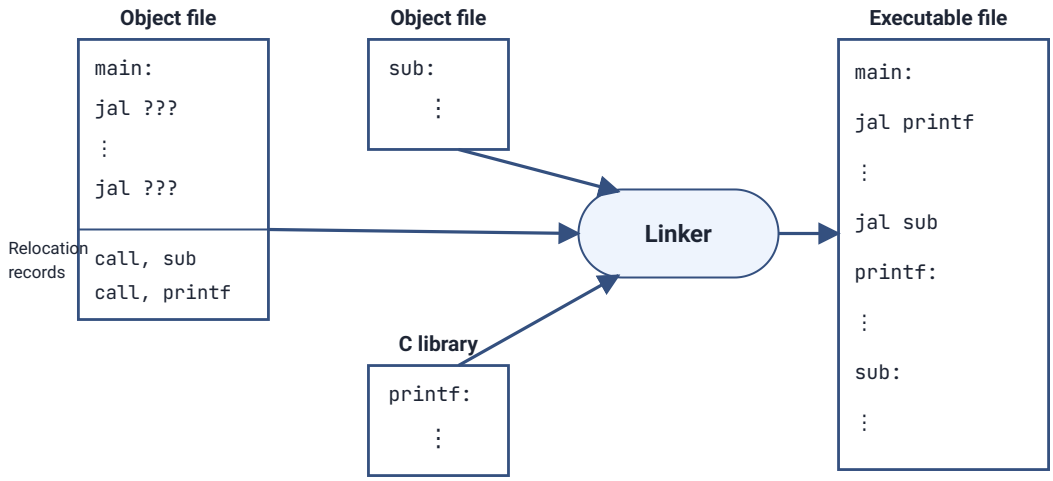
بخش	محتوا
سرآیند فایل شیء (header)	اندازه و محل بخش‌های دیگر فایل را توصیف می‌کند.

بخش	محتوا
بخش متن (text segment)	دستورهای ماشین.
بخش داده (data segment)	نمایش دودویی داده‌های فایل اسمبلی.
اطلاعات جابه‌جایی (relocation information)	دستورها و داده‌هایی را مشخص می‌کند که به **آدرس‌های مطلق** وابسته‌اند (اگر برنامه جای دیگری بار شود باید اصلاح شوند).
جدول نمادها (symbol table)	آدرس برچسب‌های خارجی (قابل استفاده در ماژول‌های دیگر) و فهرست **ارجاع‌های حل‌نشده** (برچسب‌هایی که در ماژول‌های دیگر تعریف شده‌اند).
اطلاعات اشکال‌زدایی (debugging info)	ارتباط کد ماشین با خط‌های برنامهٔ مبدأ و نام متغیرها برای اشکال‌زدا.

۳-۴-۳ پیونددهنده (Linker)

ابزاری است که فایل‌های شیء تولیدشده با کامپایل/اسمبل جداگانه را با هم ادغام می‌کند و یک **فایل اجرایی** می‌سازد. سه وظیفه دارد:

1. برنامه را جست‌وجو می‌کند تا **روال‌های کتابخانه‌ای** مورد استفاده (مثل `printf()` ، `scanf()` ، روال‌های ریاضی و ...) را پیدا کند.
2. **محل حافظه‌ای** را که کد هر ماژول اشغال خواهد کرد تعیین می‌کند و با اصلاح ارجاع‌های مطلق، دستورهای آن را **جابه‌جا (relocate)** می‌کند.
3. **ارجاع‌های میان فایل‌ها** را حل می‌کند.



شکل: کار پیونددهنده — ترکیب فایل‌های شیء و رفع ارجاع‌های حل‌نشده

در شکل، ماژول main دو دستور `jal ???` (پرش به زیرروال) دارد که مقصدشان هنوز معلوم نیست؛ رکوردهای جابه‌جایی می‌گویند این‌ها به `sub` (در فایل شیء دیگر) و `printf` (در کتابخانهٔ C) ارجاع دارند. پیونددهنده همهٔ ماژول‌ها را پشت‌سرهم می‌چیند و آدرس واقعی را جایگزین `???` می‌کند.

۳-۴-۴ بارکننده (Loader)

بخشی از سیستم‌عامل است که فایل اجرایی موجود روی دیسک را به حافظه می‌آورد و اجرای آن را آغاز می‌کند. مراحل:

1. خواندن سرآیند فایل اجرایی برای تعیین اندازهٔ بخش‌های متن و داده؛
2. ساختن یک فضای آدرس جدید برای برنامه (توسط سیستم‌عامل)؛
3. کپی کردن دستورها و داده‌ها در فضای آدرس تخصیص‌یافته؛
4. کپی کردن آرگومان‌های ارسالی به برنامه روی پشته؛
5. مقداردهی اولیهٔ ثبات‌های ماشین از جمله اشاره‌گر پشته؛
6. پرش به یک روال آغازین (startup routine) که آرگومان‌های برنامه را از پشته به ثبات‌ها کپی می‌کند و با بار کردن PC، روال اصلی (main) برنامه را فراخوانی می‌کند.

> 💡 **شهود: پیونددهنده صحاف است، بارکننده قفسه‌چین**
>
> پیونددهنده مثل صحافی است که فصل‌های چند نویسنده را یک کتاب می‌کند و ارجاع‌های «فصل ۳ نویسندهٔ دیگر» را به شمارهٔ صفحهٔ واقعی تبدیل می‌کند (حل نمادهای خارجی). بارکننده کتاب را در جای خالی قفسه می‌گذارد و چون شمارهٔ صفحه‌ها نسبت به ابتدای کتاب بودند، آن‌ها را با **نشانی شروع** جمع می‌کند (جابه‌جایی یا relocation). برای همین فایل شیء باید بگوید کدام کلمه‌ها نشانی هستند و باید اصلاح شوند.

۳-۵ آدرس‌دهی بی‌واسطه (Immediate)

در این روش **خود عملوند در قالب دستور (IF) قرار دارد**؛ بنابراین برای به دست آوردن آن هیچ دسترسی به ثبات یا حافظه لازم نیست (در شکل مسیرهای داخلی، مسیر ۴).

مثال‌ها

- `sub cx,33` (در 8086/88): عدد 33 دادهٔ بی‌واسطه است که از محتوای ثبات cx کم می‌شود.
- `addi $s0,255` (در پردازندهٔ MIPS): عدد 255 به عنوان دادهٔ بی‌واسطه به ثبات `$s0` اضافه می‌شود.

همهٔ ماشین‌های موجود آدرس‌دهی بی‌واسطه دارند. در نماد اسمبلی ماشین‌های نمونه، دادهٔ بی‌واسطه را با `#` نشان می‌دهیم (مثل `mov r,#data`).

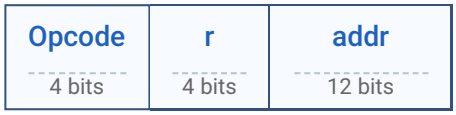
> 💡 **شهود: عدد داخل خود دستور**
>
> «۵ را اضافه کن» در برابر «عدد داخل قفسهٔ ۵ را اضافه کن». در بی‌واسطه عملوند همراه دستور واکشی شده، پس دسترسی دیگری به حافظه لازم نیست و سریع است. محدودیت‌ها: مقدار هنگام نوشتن برنامه ثابت است و اندازه‌اش به طول فیلد محدود است (مثلاً LA در IBM 360 فقط 0 تا 4095).

۳-۶ ماشین نمونهٔ ۴

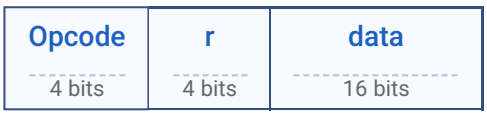
- ماشین دوآدرسه؛ حافظهٔ اصلی 2^{12} واحد آدرس‌پذیر **(هر کدام ۴ بیت)**؛
- کلمه ۱۶ بیت (یعنی ۴ واحد آدرس‌پذیر)، ناهم‌تراز، Big Endian؛
- ۱۶ ثبات همه‌منظوره R0 تا R15؛
- روش‌های آدرس‌دهی: حافظه‌ای مستقیم، حافظه‌ای غیرمستقیم، **ثباتی مستقیم، ثباتی غیرمستقیم و بی‌واسطه**؛
- سه قالب دستور مختلف:



قالب I (۱۲ بیت = ۳ واحد)



قالب II (۲۰ بیت = ۵ واحد)



قالب III (۲۴ بیت = ۶ واحد)

قالب	Opcode	Mnemonic	Operation
I	0000	`mov r1,r2`	r1 ← (r2)
	0001	`mov r1,(r2)`	r1 ← ($M_{(r2)}$)
	0010	`mov (r1),r2`	$M_{(r1)}$ ← (r2)
	0011	`add r1,r2`	r1 ← (r1) + (r2)
	0100	`sub r1,r2`	r1 ← (r1) - (r2)
II	0101	`mov r,addr`	r ← (M_{addr})
	0110	`mov addr,r`	M_{addr} ← (r)
	0111	`mov r,(addr)`	r ← ($M_{(M_{addr})}$)
	1000	`mov (addr),r`	$M_{(M_{addr})}$ ← (r)
	1001	`jz r,addr`	if (r) = 0 then PC ← addr
	1010	`jnz r,addr`	if (r) ≠ 0 then PC ← addr
	1011	`jneg r,addr`	if (r) < 0 then PC ← addr
	1100	`jpos r,addr`	if (r) > 0 then PC ← addr

قالب	Opcode	Mnemonic	Operation
III	1101	`mov r,#data`	r ← data
	1110	`add r,#data`	r ← (r) + data
	1111	`sub r,#data`	r ← (r) - data

کد عمل ۴ بیتی همزمان **قالب و طول دستور** را هم مشخص می‌کند (کدهای 0000 تا 0100 قالب I، 0101 تا 1100 قالب II و 1101 تا 1111 قالب III).

پرسش اسلاید: طول ثبات‌های ماشین ۴

پاسخ

ثبات	طول	دلیل
MAR	12 bits	حافظه 2^{12} واحد آدرس‌پذیر دارد
MBR	16 bits	طول کلمه
IR	24 bits	بلندترین قالب (III)
PC	12 bits	مثل MAR
R0…R15	16 bits	طول کلمه

تمرین اسلاید: جمع دو آرایهٔ ۱۰ عنصری (C ← A + B) و ترجمهٔ آن

برنامه‌ای بنویسید که دو آرایهٔ ۱۰ عنصری A و B را جمع کند و بردار حاصل را در آرایهٔ ۱۰ عنصری C ذخیره کند؛ سپس آن را به کد ماشین ترجمه کنید.

ایدهٔ حل

سه ثبات R0، R1 و R2 اشاره‌گرهای A، B و C هستند (با دستور بی‌واسطه `mov R0,#A` **آدرس** A در R0 قرار می‌گیرد). R5 شمارندهٔ حلقه است. در هر دور با آدرس‌دهی ثباتی غیرمستقیم یک عنصر از A و B خوانده، جمع و در C نوشته می‌شود. سپس هر اشاره‌گر **۴ واحد** جلو می‌رود، چون هر کلمهٔ ۱۶ بیتی ۴ واحد آدرس‌پذیر ۴ بیتی را اشغال می‌کند.

Address	Machine Code	Label	Instr.	Operands	توضیح
			`org`	`0`	
`000`	`D00041`		**`mov`**	`R0,#A`	R0 به آرایهٔ A اشاره کند (A = 041h)
`006`	`D10069`		**`mov`**	`R1,#B`	R1 به آرایهٔ B اشاره کند (B = 069h)

Address	Machine Code	Label	Instr.	Operands	توضیح
00C	D20091		mov	R2,#C	R2 به آرایهٔ C اشاره کند (C = 091h)
012	D5000A		mov	R5,#10	R5 شمارندهٔ حلقه
018	1A0	loop:	mov	R10,(R0)	یک عنصر A در R10
01B	1B1		mov	R11,(R1)	یک عنصر B در R11
01E	3AB		add	R10,R11	جمع آن دو
021	22A		mov	(R2),R10	ذخیره در عنصر متناظر C
024	E00004		add	R0,#4	اشاره‌گر به عنصر بعدی A
02A	E10004		add	R1,#4	اشاره‌گر به عنصر بعدی B
030	E20004		add	R2,#4	اشاره‌گر به عنصر بعدی C
036	F50001		sub	R5,#1	کاهش شمارنده
03C	A5018		jnz	R5,loop	اگر صفر نشد به loop
041	???? ×10	A:	dw	10 dup(?)	۱۰ کلمه = ۴۰ واحد = 28h
069	???? ×10	B:	dw	10 dup(?)	
091	???? ×10	C:	dw	10 dup(?)	
			end		

نحوهٔ محاسبهٔ آدرس‌ها (بر حسب واحد ۴ بیتی): هر دستور قالب III شش واحد، قالب I سه واحد و قالب II پنج واحد طول دارد. چهار دستور اول در 000، 006، 00C و 012؛ `loop` در 018؛ سه دستور قالب I در 018، 01E، 01B و 021؛ سه دستور add و یک sub (قالب III) در 024، 030، 02A و 036؛ و jnz (قالب II) در 03C که تا 040 ادامه دارد. پس A از `041h` شروع می‌شود؛ هر آرایه 10 × 4 = 40 = 28h واحد است، پس B = 041h + 28h = 069h و C = 069h + 28h = 091h.

نحوهٔ ساختن کد ماشین: هر رقم شانزده‌شانزدهی یک واحد ۴ بیتی است. `mov R0,#A` ⇐ کد عمل D = 1101، ثبات 0، داده = 0041 ⇐ `D00041` . `mov R10,(R0)` ⇐ کد 0001 = 1، r1 = ثبات ۱۰ = 0 = r2، A ⇐ `1A0` ؛ به همین ترتیب `mov R11,(R1)` ⇐ `1B1` و `add R10,R11` ⇐ `3AB` . `jnz R5,loop` ⇐ کد 1010 = A، ثبات 5، آدرس 018 ⇐ `A5018` .

جدول نمادها	
Symbol	**Address**
loop	018h
A	041h
B	069h
C	091h

توجه

این برنامه هم (مانند برنامهٔ ۲ ماشین ۳) پس از پایان حلقه به داده‌ها (آرایهٔ A) می‌رسد؛ در عمل باید پس از jnz کنترل را به سیستم‌عامل برگرداند.

۳-۷ آدرس‌دهی شاخص‌دار (Indexed)

آدرس عملوند از **جمع محتوای یک ثبات شاخص (index register) با یک آدرس داده‌شده** ساخته می‌شود (هم ثبات شاخص و هم آدرس پایه در IF آمده‌اند).

مثال

`sub ax,array[bx]` (در 8086/88): array آدرسی نمادین است که با (bx) جمع می‌شود تا آدرس خانهٔ حافظه ساخته شود. این دستور انجام می‌دهد: $(ax) \leftarrow (ax) - (M_{array+(bx)})$.

بیشتر ماشین‌های موجود آدرس‌دهی شاخص‌دار دارند. این روش برای پیمایش آرایه‌ها بسیار مناسب است: آدرس پایه ثابت می‌ماند و فقط ثبات شاخص تغییر می‌کند.

💡 **شهود: «ردیف nام جدولی که از قفسهٔ ۱۰۰ شروع می‌شود»**

در شاخص‌دار، دستور نشانی **شروع آرایه** را ثابت نگه می‌دارد و ثبات شاخص می‌گوید چندمین خانه را می‌خواهیم. با هر دور حلقه فقط ثبات شاخص زیاد می‌شود و خود دستور دست‌نخورده می‌ماند. همین ایده در IBM 360 به شکل فیلد X2 در دستورهای RX آمده است (فصل ۸: `A 3,ARR(4)`).

۳-۸ ماشین نمونهٔ ۵

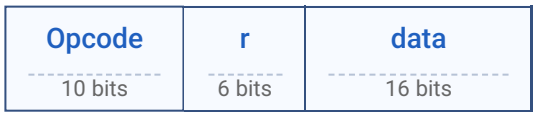
- ماشین ۲/۳ آدرسه (بعضی دستورها سه عملوند دارند)؛ حافظهٔ اصلی 2^{16} واحد آدرس‌پذیر ۸ بیتی؛
- کلمه ۱۶ بیت، ناهم‌تراز، Big Endian؛ **۶۴ ثبات همه‌منظوره** R0 تا R63؛
- روش‌های آدرس‌دهی: ثباتی مستقیم، حافظه‌ای مستقیم، بی‌واسطه و **شاخص‌دار**؛ سه قالب دستور:



قالب I (۱۶ بیت)



قالب II (۳۲ بیت)



قالب III (۳۲ بیت)

قالب III کد عمل ۱۰ بیتی دارد که همیشه با 1111 شروع می‌شود؛ چهار بیت اول همهٔ قالب‌ها کد عمل اصلی است و اگر 1111 باشد، ۶ بیت بعدی هم جزو کد عمل‌اند (به این روش **کد عمل گسترش‌پذیر**، expanding opcode، می‌گویند).

قالب	Opcode	Mnemonic	Operation
I	0000	`mov r1,r2`	r1 ← (r2)
	0001	`add r1,r2`	r1 ← (r1) + (r2)
	0010	`sub r1,r2`	r1 ← (r1) - (r2)
	0011	`and r1,r2`	r1 ← (r1) ∧ (r2)
	0100	`or r1,r2`	r1 ← (r1) ∨ (r2)
	0101	`xor r1,r2`	r1 ← (r1) ⊕ (r2)
II	0110	`mov r1,addr(r2)`	r1 ← $(M_{addr+(r2)})$
	0111	`mov addr(r2),r1`	$M_{addr+(r2)}$ ← (r1)
	1000	`mov r1,r2,addr`	r1 ← (r2); r2 ← (M_{addr})
	1001	`mov addr,r1,r2`	M_{addr} ← (r1); r1 ← (r2)
	1010	`je r1,r2,addr`	if (r1) = (r2) then PC ← addr
	1011	`jne r1,r2,addr`	if (r1) ≠ (r2) then PC ← addr
	1100	`jl r1,r2,addr`	if (r1) < (r2) then PC ← addr
	1101	`jh r1,r2,addr`	if (r1) > (r2) then PC ← addr
	1110	`call r1,addr(r2)`	r1 ← (PC); PC ← addr + (r2)
III	1111000000	`mov r1,#data`	r1 ← data
	1111000001	`add r1,#data`	r1 ← (r1) + data
	1111000010	`sub r1,#data`	r1 ← (r1) - data
	1111000011	`and r1,#data`	r1 ← (r1) ∧ data
	1111000100	`or r1,#data`	r1 ← (r1) ∨ data
	1111000101	`xor r1,#data`	r1 ← (r1) ⊕ data

نکته: دستورهای ویژهٔ 1000، 1001 و 1110

- `mov r1,r2,addr` دو کار می‌کند: ابتدا (r2) ← r1 و سپس r2 را از حافظه بار می‌کند. اگر r1 = r2 انتخاب شود، کار اول بی‌اثر است و دستور عملاً «بار کردن ثبات از حافظه» می‌شود.
- `mov addr,r1,r2` ابتدا (r1) را در حافظه می‌نویسد و سپس (r2) ← r1. با r1 = r2 عملاً «ذخیرهٔ ثبات در حافظه» است.
- `call r1,addr(r2)` آدرس بازگشت (PC فعلی) را در r1 ذخیره می‌کند و به (r2) + addr می‌پرد. با addr = 0 و r2 = ثباتی که آدرس بازگشت را دارد، می‌توان به سیستم‌عامل بازگشت.

پرسش ۱ اسلاید: طول ثبات‌های ماشین ۵

پاسخ

$L_{MAR} = 16$، $L_{MBR} = 16$، $L_{PC} = 16$، $L_{IR} = 32$ (قالب‌های II و III هر دو ۳۲ بیتی‌اند) و $L_{R0..R63} = 16$ بیت.

پرسش ۲ اسلاید: جمع عناصر یک آرایهٔ ۱۰۰ عنصری و ترجمه به کد ماشین

ایدهٔ حل

R0 انباره (مجموع) است و با `xor R0,R0` صفر می‌شود (هر عدد XOR خودش = 0). R1 ثبات شاخص است که از 0 شروع می‌شود و هر دور ۲ واحد (یک کلمه) زیاد می‌شود. R2 مقدار 200 (شاخص بعد از آخرین عنصر) را برای کنترل حلقه نگه می‌دارد. با آدرس‌دهی شاخص‌دار `mov R3,A(R1)` عنصر i ام خوانده می‌شود. در پایان با `mov sum,R0,R0` نتیجه در sum ذخیره و با `call R0,0(R63)` به سیستم‌عامل بازگشت می‌شود (فرض: R63 آدرس بازگشت را دارد).

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	
0100	5000		xor	R0,R0	0101 000000 000000 انباره = 0
0102	0040		mov	R1,R0	0000 000001 000000 ثبات شاخص = 0
0104	F00200C8		mov	R2,#200	1111000000 000010 [00C8h] شاخص پایان
0108	60C10120	loop:	mov	R3,A(R1)	0110 000011 000001 [0120h] بار کردن عنصر i ام A
010C	1003		add	R0,R3	0001 000000 000011 افزودن به انباره
010E	F0410002		add	R1,#2	1111000001 000001 [0002h] شاخص عنصر بعد
0112	B0420108		jne	R1,R2,loop	1011 000001 000010 [0108h] اگر شاخص ≠ 200 ادامه

Address	Machine Code	Label	Instr.	Operands	توضیح
0116	9000011E		mov	sum,R0,R0	ذخیرهٔ نتیجه 1001 000000 000000 [011Eh] در sum
011A	E03F0000		call	R0,0(R63)	بازگشت به 1110 000000 111111 [0000h] سیستم‌عامل
011E	0000	sum:	dw	0	متغیر مجموع
0120	??????????	A:	dw	100 dup(?)	آرایهٔ ۱۰۰ عنصری
			end		

نحوهٔ ساختن کد: بیت‌های هر فیلد را پشت‌سرهم می‌نویسیم و چهارتاچهارتا به شانزده‌شانزدهی تبدیل می‌کنیم. مثلاً `mov R3,A(R1)` : `60C1` = 0110 0000 1100 0001 = 0110|000011|000001 و سپس آدرس A یعنی `0120` . دستورهای قالب I دو بایت و قالب‌های II و III چهار بایت‌اند؛ بنابراین آدرس‌ها: 0100، 0102، 0104، 0108، 010C، 010E، 0112، 0116، 011A و sum در 011E و A در 0120.

تکلیف ۱ (پرسش ۳ اسلاید): جمع محتوای R62 ،... ،R1 ،R0 در R62 و ترجمه به کد ماشین

تحلیل

در ماشین ۵ شمارهٔ ثبات‌ها **داخل خود دستور** (فیلدهای r1 و r2) است و هیچ روش آدرس‌دهی‌ای وجود ندارد که شمارهٔ ثبات را از یک ثبات یا حافظه بگیرد. همچنین همهٔ ۶۳ ثبات R0 تا R62 داده دارند و فقط R63 (که آدرس بازگشت را دارد) باقی می‌ماند. پس دو راه داریم: (الف) نوشتن ۶۲ دستور جمع پشت‌سرهم، یا (ب) یک حلقه با **کد خودتغییرده** که فیلد r2 دستور جمع را در هر دور یکی زیاد کند.

راه‌حل الف: کد خطی (ساده و سریع)

دستور `add R62,Rk` در قالب I برابر 0001 111110 kkkkkk است؛ یعنی 1F80h + k. پس:

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	
0100	1F80		add	R62,R0	0001 111110 000000
0102	1F81		add	R62,R1	0001 111110 000001
0104	1F82		add	R62,R2	
...	...		...	...	در مجموع ۶۲ دستور؛ دستور k ام در آدرس 100h+2k و کد 1F80h+k
0178	1FBC		add	R62,R60	
017A	1FBD		add	R62,R61	0001 111110 111101
017C	E03F0000		call	R0,0(R63)	بازگشت به سیستم‌عامل (R0 دیگر لازم نیست)
			end		

اندازهٔ برنامه: 128 = 4 + 2×62 بایت. چون R62 از ابتدا مقدار خودش را دارد، در پایان R62 = R0 + R1 + … + R62 است.

راه‌حل ب: حلقه با کد خودتغییرده (کوتاه‌تر)

ترفند: ابتدا R61 را به R62 اضافه می‌کنیم تا R61 آزاد شود؛ R63 را هم در حافظه ذخیره می‌کنیم تا آزاد شود. حالا دو ثبات کمکی داریم. دستور `add R62,R0` را در حلقه اجرا می‌کنیم و هر بار **کلمهٔ این دستور** را از حافظه به R61 می‌آوریم، یکی زیاد می‌کنیم (شمارهٔ r2 یکی زیاد می‌شود) و برمی‌گردانیم؛ تا وقتی که دستور به `add R62,R61` (`1FBDh`) برسد که دیگر لازم نیست (R61 قبلاً جمع شده است).

Address	Machine Code	Label	Instr.	Operands	توضیح
			`org`	`100h`	
0100	1FBD		**add**	R62,R61	R61 جمع شد و اکنون آزاد است
0102	9FFF0124		**mov**	ret,R63,R63	ذخیرهٔ آدرس بازگشت (R63) در حافظه
0106	F03F1FBD		**mov**	R63,#1FBDh	R63 = کد «add R62,R61» (شرط پایان)
010A	1F80	**loop:**	**add**	R62,R0	این دستور هر دور تغییر می‌کند (R0، R1، …)
010C	8F7D010A		**mov**	R61,R61,loop	R61 ← کلمهٔ دستور loop
0110	F07D0001		**add**	R61,#1	شمارهٔ ثبات بعدی
0114	9F7D010A		**mov**	loop,R61,R61	نوشتن دستور تغییریافته در حافظه
0118	BF7F010A		**jne**	R61,R63,loop	تا رسیدن به add R62,R61 ادامه
011C	8FFF0124		**mov**	R63,R63,ret	بازیابی آدرس بازگشت
0120	E03F0000		**call**	R0,0(R63)	بازگشت به سیستم‌عامل
0124	0000	**ret:**	**dw**	0	محل ذخیرهٔ R63
			`end`		

ردگیری: دور اول `add R62,R0` اجرا می‌شود و دستور به `1F81` تغییر می‌کند؛ … ؛ در دور آخر `add R62,R60` (`1FBC`) اجرا و دستور به `1FBD` تبدیل می‌شود که با R63 برابر است و حلقه تمام می‌شود. اندازه: ۴۰ بایت در برابر ۱۲۸ بایت؛ اما کندتر است (۵ دستور در هر دور) و همهٔ معایب کد خودتغییرده را دارد؛ مثلاً اگر دوباره اجرا شود دستور loop دیگر `1F80` نیست (باید در ابتدای برنامه دوباره مقداردهی شود). هر دو برنامه با شبیه‌ساز آزموده شده‌اند.

۳-۹ آدرس‌دهی ضمنی (Implied / Inherent)

در این روش **هیچ چیزی دربارهٔ عملوند در IF نیامده است، اما CPU می‌داند عملوند کجاست**؛ محل عملوند جزو تعریف خود دستور است.

مثال‌ها

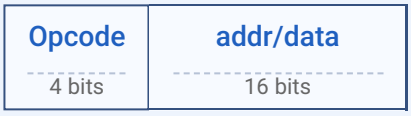
- دستور `loop address` در پردازندهٔ 8086/88: cx ← (cx) - 1; if (cx) ≠ 0 then PC ← address. ثبات cx در دستور نیامده ولی ضمناً استفاده می‌شود.
- دستور ضرب ثباتی `MR 2,4` در خانوادهٔ IBM 360: R2:R3 ← (R3) × (R4). حاصل ۶۴ بیتی در جفت ثبات R2:R3 قرار می‌گیرد؛ R3 (عملوند اول) در دستور نیامده و ضمنی است.

۳-۱۰ ماشین تک‌آدرسه (ماشین انباره‌ای)

در ماشین تک‌آدرسه **یکی از عملوندها همیشه در یک ثبات خاص‌منظوره به نام ACC (انباره، Accumulator) نگه داشته می‌شود و به صورت ضمنی آدرس‌دهی می‌شود**؛ بنابراین هر دستور فقط یک فیلد آدرس دارد.

مشخصات ماشین نمونهٔ تک‌آدرسه

- حافظهٔ اصلی: 2^{16} واحد آدرس‌پذیر (**هر کدام ۴ بیت**)؛ کلمه ۱۶ بیت (ناهم‌تراز، Big Endian)؛
- روش‌های آدرس‌دهی: ضمنی، مستقیم، غیرمستقیم و بی‌واسطه؛ دستورها در یک قالب کد می‌شوند:



قالب دستور (۲۰ بیت = ۵ واحد ۴ بیتی)

Opcode	Mnemonic	Operation	Opcode	Mnemonic	Operation
0000	`load #data`	ACC ← data	1000	`xor addr`	ACC ← (ACC) ⊕ (M_{addr})
0001	`add #data`	ACC ← (ACC) + data	1001	`load (addr)`	ACC ← ($M_{(M_{addr})}$)
0010	`load addr`	ACC ← (M_{addr})	1010	`store (addr)`	$M_{(M_{addr})}$ ← (ACC)
0011	`store addr`	M_{addr} ← (ACC)	1011	`j addr`	PC ← addr
0100	`add addr`	ACC ← (ACC) + (M_{addr})	1100	`jz addr`	if (ACC) = 0 then PC ← addr
0101	`sub addr`	ACC ← (ACC) - (M_{addr})	1101	`jnz addr`	if (ACC) ≠ 0 then PC ← addr
0110	`and addr`	ACC ← (ACC) ∧ (M_{addr})	1110	`jn addr`	if (ACC) < 0 then PC ← addr
0111	`or addr`	ACC ← (ACC) ∨ (M_{addr})	1111	`jp addr`	if (ACC) > 0 then PC ← addr

طول ثبات‌ها در این ماشین: $L_{PC} = L_{MAR} = 16$، $L_{ACC} = L_{MBR} = 16$ و $L_{IR} = 20$ بیت.

مثال اسلاید: تولید ۲۰ جملهٔ اول دنبالهٔ فیبوناچی و ذخیرهٔ آن‌ها در یک آرایه

ایدهٔ حل

آرایه با دو جملهٔ اول (0 و 1) مقداردهی شده است. سه اشاره‌گر داریم: `pointer_1` به جملهٔ i ام، `pointer_2` به جملهٔ i+1 ام و `pointer_3` به جملهٔ i+2 ام. در هر دور: A[i+2] ← A[i] + A[i+1] با آدرس‌دهی غیرمستقیم؛ سپس هر سه اشاره‌گر یک کلمه (۴ واحد ۴ بیتی) جلو می‌روند. چون ماشین فقط یک انباره دارد، از متغیر `sum` برای نگه‌داشتن موقت A[i] استفاده می‌شود. ۱۸ دور حلقه، ۱۸ جملهٔ بعدی را می‌سازد (در مجموع ۲۰ جمله).

```
        org   0
loop:   load  (pointer_1)    ; ACC = A[i]
        store sum            ; sum = A[i]       (متغیر موقت)
        load  (pointer_2)    ; ACC = A[i+1]
        add   sum            ; ACC = A[i] + A[i+1]
        store (pointer_3)    ; A[i+2] = ACC
        load  pointer_1      ; ACC = pointer_1
        add   #4
        store pointer_1      ; pointer_1 += 4  (یک کلمه = ۴ واحد)
        add   #4
        store pointer_2      ; pointer_2 = pointer_1 + 4
        add   #4
        store pointer_3      ; pointer_3 = pointer_1 + 8
        load  counter
        add   #-1
        store counter        ; counter--
        jnz   loop           ; ACC هنوز مقدار counter را دارد
        j     OS_return_addr ; بازگشت به سیستم‌عامل

pointer_1: dw  array
pointer_2: dw  array+4
pointer_3: dw  array+8
counter:   dw  18
sum:       dw  ?
array:     dw  0,1,18 dup(?)
           end
```

ردگیری دور اول: ACC = A[0] = 0 ⇐ sum = 0 ⇐ ACC = A[1] = 1 ⇐ ACC = 1 + 0 = 1 ⇐ A[2] = 1؛ سپس اشاره‌گرها به A[1]، A[2] و A[3] و counter = 17. دور دوم A[3] = 1 + 1 = 2، دور سوم A[4] = 3 و … تا دور ۱۸ که A[19] = 4181.

نکته: دستور `jnz loop` شرط را روی ACC بررسی می‌کند و چون آخرین مقدار ACC همان counter است، حلقه وقتی counter صفر شود متوقف می‌شود.

تکلیف ۲: ترجمهٔ برنامهٔ بالا به کد ماشین

حل

هر دستور ۲۰ بیت یعنی **۵ واحد ۴ بیتی** است (یک رقم شانزده‌شانزدهی برای کد عمل و چهار رقم برای آدرس/داده). ۱۷ دستور داریم، پس دستورها آدرس‌های 0000 تا 0054 را می‌گیرند و داده‌ها از 0055h = 85 = 5×17 شروع می‌شوند. هر کلمهٔ داده ۴ واحد است:

برچسب	آدرس	مقدار
pointer_1	0055	0069 (array)
pointer_2	0059	006D (array+4)
pointer_3	005D	0071 (array+8)
counter	0061	0012 (18)
sum	0065	????
array	0069	0000, 0001, ???? ×18

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	0	
0000	9 0055	loop:	load	(pointer_1)	load (addr) = 1001
0005	3 0065		store	sum	store addr = 0011
000A	9 0059		load	(pointer_2)	
000F	4 0065		add	sum	add addr = 0100
0014	A 005D		store	(pointer_3)	store (addr) = 1010
0019	2 0055		load	pointer_1	load addr = 0010
001E	1 0004		add	#4	add #data = 0001
0023	3 0055		store	pointer_1	
0028	1 0004		add	#4	
002D	3 0059		store	pointer_2	
0032	1 0004		add	#4	
0037	3 005D		store	pointer_3	
003C	2 0061		load	counter	
0041	1 FFFF		add	#-1	-1 در مکمل دو ۱۶ بیتی = FFFFh
0046	3 0061		store	counter	
004B	D 0000		jnz	loop	jnz = 1101
0050	B ????		j	OS_return_addr	j = 1011 ؛ آدرس را سیستم‌عامل تعیین می‌کند

Address	Machine Code	Label	Instr.	Operands	توضیح
0055	0069	pointer_1:	dw	array	
0059	006D	pointer_2:	dw	array+4	
005D	0071	pointer_3:	dw	array+8	
0061	0012	counter:	dw	18	
0065	????	sum:	dw	?	
0069	0000 0001 ???? …	array:	dw	0,1,18 dup(?)	۲۰ کلمه = ۸۰ واحد
			end		

برنامه با شبیه‌ساز اجرا شد و آرایه دقیقاً ۲۰ جملهٔ اول فیبوناچی (0، 1، 1، 2، …، 4181) را دارد.

💡 شهود: ماشین‌حساب با یک صفحه‌نمایش

ماشین‌حساب جیبی را به یاد بیاورید: همیشه یک عدد روی صفحه است و هر عمل «عدد صفحه ⊙ عدد جدید» را حساب می‌کند و نتیجه را دوباره روی صفحه می‌گذارد. انباره (AC) همان صفحه است؛ چون یکی از عملوندها و مقصد همیشه AC است، دستور فقط **یک نشانی** لازم دارد. هزینه‌اش: نتیجه‌های میانی باید مدام با STORE در حافظه ذخیره و دوباره LOAD شوند.

۳-۱۱ ماشین صفرآدرسه (ماشین پشته‌ای)

در ماشین صفرآدرسه **همهٔ عملوندها در یک پشتهٔ سخت‌افزاری داخل CPU نگه داشته می‌شوند.**

پشتهٔ سخت‌افزاری

یک حافظهٔ داخلی با S کلمه که برای نوشتن (**push**) و خواندن (**pop**) داده از راهبرد **LIFO** (آخرین ورودی، اولین خروجی) پیروی می‌کند. ثبات **SP** (Stack Pointer) به **بالای پشته، یعنی جایی که آمادهٔ گرفتن دادهٔ جدید است**، اشاره می‌کند.

- **push x**: $\text{Stack}_{(SP)} \leftarrow x$; SP ← (SP) + 1
- **pop**: SP ← (SP) - 1 و کلمه‌ای که SP به آن اشاره می‌کند برگردانده می‌شود.

توجه کنید که در این قرارداد، SP به اولین خانهٔ **خالی** اشاره می‌کند؛ SP = 0 یعنی پشته خالی است. pop داده را پاک نمی‌کند؛ فقط SP را پایین می‌آورد (دادهٔ قبلی تا وقتی روی آن نوشته نشود باقی می‌ماند — این نکته در ماشین ۶ مهم است).

مثال اسلاید: عملیات push و pop

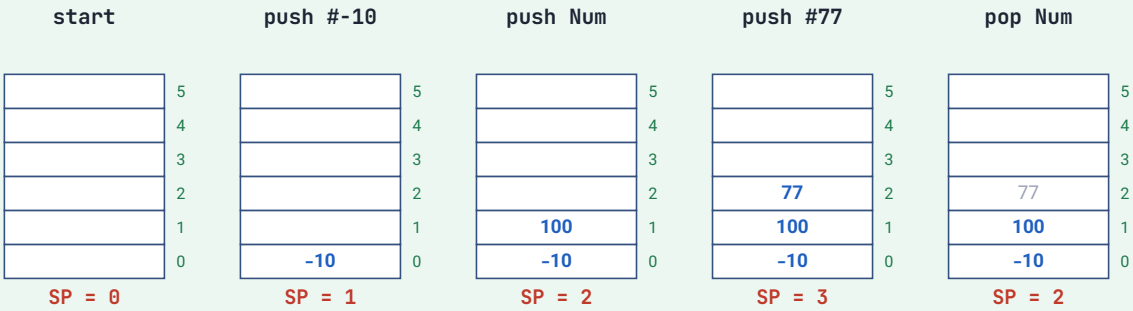
یک پشتهٔ سخت‌افزاری ۱۶ کلمه‌ای داریم و SP صفر است (پشته خالی). Num متغیری در حافظه با مقدار اولیهٔ 100

```
push #-10   ==  Stack(SP) ← -10;      SP ← (SP)+1
push Num    ==  Stack(SP) ← (M_Num); SP ← (SP)+1
push #77    ==  Stack(SP) ← 77;       SP ← (SP)+1
pop  Num    ==  SP ← (SP)-1;          M_Num ← (Stack(SP))
```

است. عملیات زیر را انجام دهید:

حل

بعد از	Stack[0]	Stack[1]	Stack[2]	SP	Num
(ابتدا)				0	100
push #-10	-10			1	100
push Num	-10	100		2	100
push #77	-10	100	77	3	100
pop Num	-10	100	77 (بالای SP)	2	**77**



وضعیت پشته پس از هر عمل (خاکستری: دادهٔ بالای SP که دیگر جزو پشته نیست)

💡 شهود: دستهٔ بشقاب

در یک دستهٔ بشقاب فقط به بشقاب رویی دسترسی داریم. عمل‌ها دو بشقاب بالایی را برمی‌دارند و یک بشقاب (نتیجه) را می‌گذارند. چون عملوندها **همیشه** بالای پشته‌اند، دستورهای حسابی هیچ نشانی‌ای لازم ندارند (صفرآدرسه) و کوتاه می‌شوند؛ فقط PUSH و POP نشانی دارند. هزینه: برای رسیدن به داده‌ای که پایین‌تر است باید ترتیب کارها را از قبل درست چید.

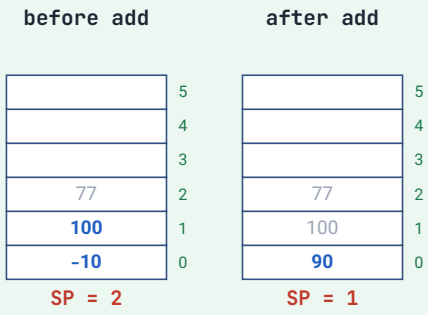
۳-۱۱-۱ اجرای یک تابع در ماشین پشته‌ای

برای اجرای یک تابع مثل func(a,b)، عملوندهای a و b از بالای پشته برداشته می‌شوند و نتیجهٔ func(a,b) در پشته قرار می‌گیرد:

```
func   ==   pop   temp1
            pop   temp2
            temp3 ← func((temp1),(temp2))
            push  temp3
```

مثال: add

اگر پس از مثال بالا (پشته: -10، 100 و SP = 2) دستور `add` اجرا شود، دو عدد 100 و -10 برداشته و حاصل 90 در خانهٔ 0 قرار می‌گیرد و SP = 1 می‌شود.



اجرای add در ماشین پشته‌ای

برای اعمال غیرجابه‌جایی (تفریق و تقسیم) ترتیب مهم است: در این جزوه (و ماشین ۶) **عملوند پایینی منهای/تقسیم بر عملوند بالایی** حساب می‌شود؛ یعنی اگر اول a و سپس b در پشته گذاشته شود، `sub` مقدار a - b را می‌دهد.

💡 شهود: چرا نشانی بازگشت در پشته؟

هر فراخوانی، نشانی برگشت را روی پشته می‌گذارد؛ مثل گذاشتن یک نشانک روی دستهٔ بشقاب. اگر تابع، تابع دیگری را صدا بزند، نشانک دوم روی اولی قرار می‌گیرد. چون همیشه **آخرین** تابع صدا زده‌شده **اول** تمام می‌شود، ترتیب LIFO پشته دقیقاً همان ترتیب درست برگشت است؛ برای همین بازگشتی (recursion) خودبه‌خود کار می‌کند. (مقایسه کنید با IBM 360 در فصل ۸ که نشانی برگشت را در ثبات نگه می‌دارد.)

۳-۱۲ نمادگذاری لهستانی و ماشین‌های پشته‌ای

نمادگذاری معمولی ریاضی **میانوندی (infix)** است: عملگر بین عملوندها می‌آید، مثل x + y یا z * t. اما نمادگذاری‌های دیگری هم وجود دارد:

- **پیشوندی (prefix)**: عملگر قبل از عملوندها: + x y یا * z t.
- **پسوندی (postfix)** که **نمادگذاری لهستانی** (Polish notation) هم نامیده می‌شود: عملگر بعد از عملوندها: x y + یا z t *.

مثال اسلاید

عبارت زیر را به نمادگذاری لهستانی بنویسید:

(x + y - 10 - t / d + 10*s - z) / (20*t - z / t + 100*w)

پاسخ

`xy+10-td/-10s*+z-20t*zt/-100w*+/`

نکات مثبت و منفی

مثبت: (۱) نیازی به نگرانی دربارهٔ اولویت عملگرها نیست و پرانتز لازم نیست؛ (۲) برای محاسبه روی ماشین پشته‌ای بسیار مناسب است.

منفی: برای درک انسان مناسب نیست.

💡 **شهود: «اول مواد، بعد دستور»**

در نماد پسوندی اول عملوندها می‌آیند و بعد عملگر: + 3 2 یعنی «۲ را بیاور، ۳ را بیاور، جمع کن». ترتیب انجام کارها همان ترتیب نوشتن است، پس به پرانتز و قواعد اولویت نیازی نیست. این دقیقاً زبان ماشین پشته‌ای است: هر عدد ⇐ PUSH و هر عملگر ⇐ یک دستور حسابی روی دو عنصر بالای پشته.

۳-۱۲-۱ محاسبهٔ عبارت لهستانی روی ماشین پشته‌ای

روش: عبارت را از چپ به راست پیمایش کنید. اگر به عملوند رسیدید، آن را در پشته push کنید. اگر به عملگر رسیدید، آن را اجرا کنید (دو عنصر بالای پشته را بردارید و حاصل را push کنید).

مثال اسلاید: محاسبهٔ `xy+10-td/-10s*+z-20t*zt/-100w*+/`

فرض کنید ماشین پشته‌ای ۱۶ کلمه‌ای داریم و مقدار فعلی SP برابر 2 است (خانه‌های 0 و 1 داده‌های 90 و 100 از قبل را دارند).

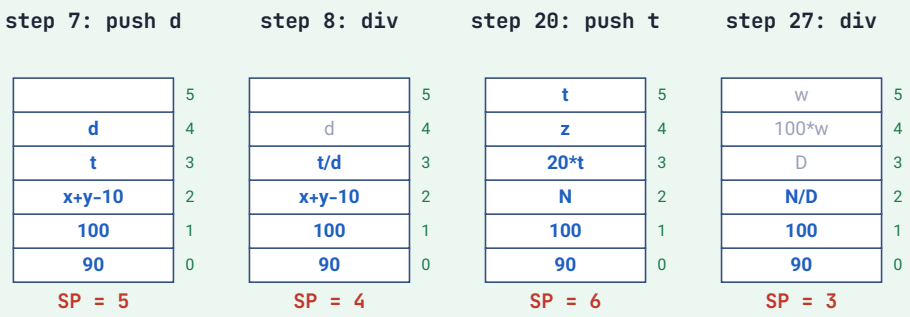
ردگیری کامل

#	نماد	عمل	بعد SP	محتوای پشته (از خانهٔ 2 به بالا)
0		—	2	[90, 100 |
1	x	push x	3	x
2	y	push y	4	x, y
3	+	add	3	(x+y)

#	نماد	عمل	بعد SP	محتوای پشته (از خانهٔ 2 به بالا)
4	10	push 10	4	(x+y), 10
5	-	sub	3	(x+y-10)
6	t	push t	4	(x+y-10), t
7	d	push d	5	(x+y-10), t, d
8	/	div	4	(x+y-10), t/d
9	-	sub	3	(x+y-10-t/d)
10	10	push 10	4	(…), 10
11	s	push s	5	(…), 10, s
12	*	mult	4	(…), 10*s
13	+	add	3	(x+y-10-t/d+10*s)
14	z	push z	4	(…), z
15	-	sub	3	N = (x+y-10-t/d+10*s-z)
16	20	push 20	4	N, 20
17	t	push t	5	N, 20, t
18	*	mult	4	N, 20*t
19	z	push z	5	N, 20*t, z
20	t	push t	6	N, 20*t, z, t
21	/	div	5	N, 20*t, z/t
22	-	sub	4	N, (20*t-z/t)
23	100	push 100	5	N, (20*t-z/t), 100
24	w	push w	6	N, (20*t-z/t), 100, w
25	*	mult	5	N, (20*t-z/t), 100*w
26	+	add	4	N, D = (20*t-z/t+100*w)
27	/	div	3	N / D
28		pop Result	2	[90, 100 | → Result = N/D

در پایان، نتیجهٔ [(x+y-10-t/d+10*s-z)] / [(20*t-z/t+100*w)] با یک pop در متغیر Result در حافظه نوشته می‌شود و SP دوباره 2 می‌شود؛ یعنی دو دادهٔ قبلی پشته (90 و 100) دست‌نخورده باقی می‌مانند.

بیشینهٔ SP در طول محاسبه 6 بود؛ پس این عبارت به ۴ خانهٔ آزاد پشته نیاز داشت.

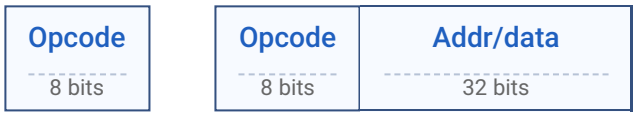


چند لحظه از محاسبه (خاکستری: خانه‌های بالای SP)

۳-۱۳ ماشین نمونهٔ ۶ (ماشین پشته‌ای)

در ماشین پشته‌ای، همهٔ دستورها (به‌جز push و pop) **هیچ فیلد عملوندی در قالب دستور ندارند**؛ بنابراین «تقریباً» همهٔ دستورها بدون فیلد عملوندند.

- حافظهٔ اصلی 2^{32} واحد آدرس‌پذیر ۸ بیتی؛ کلمه ۳۲ بیت؛
- روش‌های آدرس‌دهی: ضمنی، مستقیم و بی‌واسطه؛ دستورها در دو قالب کد می‌شوند:



قالب II (۱ بایت) — قالب I (۵ بایت)

قالب	Opcode	Mnemonic	Operation
I	`00000000`	`push #data`	$Stack_{(SP)} \leftarrow data$; $SP \leftarrow (SP)+1$
	`00000001`	`push addr`	$Stack_{(SP)} \leftarrow (M_{addr})$; $SP \leftarrow (SP)+1$
	`00000010`	`pop addr`	$SP \leftarrow (SP)-1$; $M_{addr} \leftarrow (Stack_{(SP)})$

قالب	Opcode	Mnemonic	Operation
II	`00000011`	`add`	$Stack_{(SP)-2} \leftarrow (Stack_{(SP)-2}) + (Stack_{(SP)-1})$; $SP \leftarrow (SP)-1$
	`00000100`	`sub`	$Stack_{(SP)-2} \leftarrow (Stack_{(SP)-2}) - (Stack_{(SP)-1})$; $SP \leftarrow (SP)-1$
	`00000101`	`swap`	$Stack_{(SP)-1} \leftrightarrow Stack_{(SP)-2}$
	`00000110`	`sp++`	$SP \leftarrow (SP)+1$
	`00000111`	`sp--`	$SP \leftarrow (SP)-1$
	`00001000`	`j`	$PC \leftarrow (Stack_{(SP)-1})$; $SP \leftarrow (SP)-1$
	`00001001`	`jz`	if $(Stack_{(SP)-2}) = 0$ then $PC \leftarrow (Stack_{(SP)-1})$; $SP \leftarrow (SP)-2$
	`00001010`	`jnz`	if $(Stack_{(SP)-2}) \neq 0$ then $PC \leftarrow (Stack_{(SP)-1})$; $SP \leftarrow (SP)-2$
	`00001011`	`jn`	if $(Stack_{(SP)-2}) < 0$ then $PC \leftarrow (Stack_{(SP)-1})$; $SP \leftarrow (SP)-2$

توجه (خطای تایپی اسلاید)

در اسلاید، کد عمل `jn` هم `00001000` نوشته شده که همان کد `j` است. کد عمل‌ها باید یکتا باشند؛ کد منطقی بعدی `00001011` است که در جدول بالا آمده است.

در دستورهای پرش، ابتدا شرط (مثلاً شمارنده) و سپس **آدرس مقصد** با `push #label` در پشته قرار می‌گیرد؛ دستور پرش هر دو را برمی‌دارد.

ترفند `pop x` و سپس `sp++`

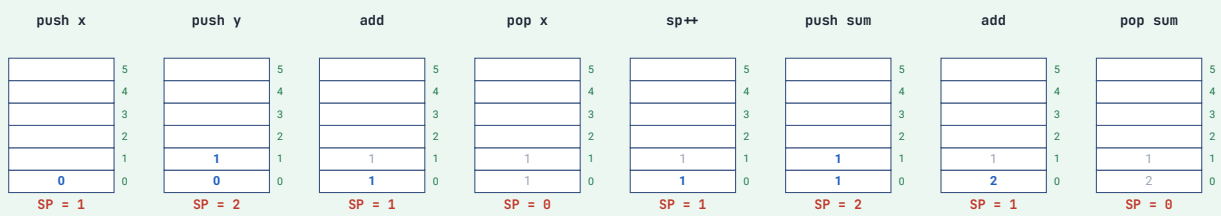
ماشین دستور «کپی بالای پشته» (dup) ندارد. اما چون pop داده را پاک نمی‌کند، اگر بعد از `pop x` دستور `sp++` بیاید، همان مقدار دوباره روی پشته برمی‌گردد. پس جفت `pop x / sp++` یعنی «بالای پشته را در x ذخیره کن ولی آن را از پشته برندار».

مثال اسلاید: مجموع ۲۰ جملهٔ اول فیبوناچی در متغیر sum

```
        org   0
loop:   push  x          ; [x]
        push  y          ; [x, y]
        add              ; [x+y]
        pop   x          ; x = x+y          []
        sp++             ; [x]   (دوباره روی پشته x مقدار جدید)
        push  sum        ; [x, sum]
        add              ; [x+sum]
        pop   sum        ; sum = sum + x    []
        push  x
        push  y
        add              ; [x+y]
        pop   y          ; y = x+y
        sp++             ; [y]
        push  sum
        add
        pop   sum        ; sum = sum + y
        push  counter
        push  #1
        sub              ; [counter-1]
        pop   counter    ; counter = counter-1
        sp++             ; [counter]
        push  #loop      ; [counter, loop]
        jnz              ; if counter≠0 goto loop
        push  #OS_return_addr
        j                ; بازگشت به سیستم‌عامل

sum:     dw  1
counter: dw  10
x:       dw  0
y:       dw  1
        end
```

توضیح: x و y دو جملهٔ آخر هستند (ابتدا 0 و 1) و sum با 1 = 1 + 0 شروع می‌شود. در هر دور، دو جملهٔ جدید ساخته می‌شوند (اول در x و سپس در y) و هر کدام بلافاصله به sum اضافه می‌شود. پشته در پایان هر بخش خالی می‌شود (برنامه پشته را متعادل نگه می‌دارد).



ردگیری پشته در نیمهٔ اول دور اول (x = 1 و سپس sum = 2)

تکلیف اسلاید: کد ماشین برنامهٔ بالا را تولید کنید. آیا می‌توانید برنامهٔ بهتری بنویسید؟

بخش اول: ترجمه به کد ماشین

دستورهای push و pop پنج بایتی (قالب I) و بقیه یک بایتی (قالب II) هستند. آدرس‌ها و داده‌ها ۳۲ بیتی‌اند. محاسبهٔ آدرس‌ها نشان می‌دهد کد تا `54h` ادامه دارد، پس داده‌ها از `55h` شروع می‌شوند: sum = 55h، x = 5Dh، counter = 59h و y = 61h (هر کدام ۴ بایت).

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	0	
00000000	01 0000005D	loop:	push	x	
00000005	01 00000061		push	y	
0000000A	03		add		
0000000B	02 0000005D		pop	x	
00000010	06		sp++		
00000011	01 00000055		push	sum	
00000016	03		add		
00000017	02 00000055		pop	sum	
0000001C	01 0000005D		push	x	
00000021	01 00000061		push	y	
00000026	03		add		
00000027	02 00000061		pop	y	
0000002C	06		sp++		
0000002D	01 00000055		push	sum	
00000032	03		add		
00000033	02 00000055		pop	sum	
00000038	01 00000059		push	counter	
0000003D	00 00000001		push	#1	
00000042	04		sub		
00000043	02 00000059		pop	counter	
00000048	06		sp++		
00000049	00 00000000		push	#loop	آدرس loop = 0
0000004E	0A		jnz		
0000004F	00 ????????		push	#OS_return_addr	
00000054	08		j		
00000055	00000001	sum:	dw	1	
00000059	0000000A	counter:	dw	10	

Address	Machine Code	Label	Instr.	Operands	توضیح
0000005D	00000000	x:	dw	0	
00000061	00000001	y:	dw	1	
			end		

توجه: تعداد جمله‌ها

sum از ابتدا دو جملهٔ 0 و 1 را دارد و هر دور حلقه دو جملهٔ جدید اضافه می‌کند. با `counter = 10` حلقه ۱۰ بار اجرا می‌شود؛ پس در مجموع 2 + 20 = 22 جمله (F_0 تا F_{21}) جمع می‌شود و حاصل 28656 است (با شبیه‌سازی تأیید شده). برای دقیقاً ۲۰ جمله (F_0 تا F_{19}) باید `counter: dw 9` باشد که حاصل **10945** می‌دهد.

بخش دوم: یک برنامهٔ بهتر

ایده: از اتحاد $F_0 + F_1 + \dots + F_{n-1} = F_{n+1} - 1$ استفاده می‌کنیم. پس لازم نیست در هر دور sum را به‌روز کنیم؛ کافی است جمله‌ها را بسازیم و در پایان یک تفریق انجام دهیم. پس از k دور (هر دور دو جمله)، $x = F_{2k}$ و $y = F_{2k+1}$ است. برای ۲۰ جمله به F_{21} نیاز داریم، یعنی ۱۰ دور و سپس sum = y - 1. علاوه بر این، y جدید را مستقیماً از x جدید (که با sp++ روی پشته مانده) می‌سازیم.

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	0	
00000000	01 00000051	loop:	push	x	[x]
00000005	01 00000055		push	y	[x, y]
0000000A	03		add		[x+y]
0000000B	02 00000051		pop	x	x ← x+y
00000010	06		sp++		[x]
00000011	01 00000055		push	y	[x, y]
00000016	03		add		[x+y]
00000017	02 00000055		pop	y	y ← x+y
0000001C	01 0000004D		push	counter	
00000021	00 00000001		push	#1	
00000026	04		sub		
00000027	02 0000004D		pop	counter	counter ← counter-1
0000002C	06		sp++		[counter]
0000002D	00 00000000		push	#loop	
00000032	0A		jnz		

Address	Machine Code	Label	Instr.	Operands	توضیح
00000033	01 00000055		push	y	y = F_{21}
00000038	00 00000001		push	#1	
0000003D	04		sub		
0000003E	02 00000049		pop	sum	sum = F_{21} - 1
00000043	00 ????????		push	#OS_return_addr	
00000048	08		j		
00000049	00000000	sum:	dw	0	
0000004D	0000000A	counter:	dw	10	
00000051	00000000	x:	dw	0	
00000055	00000001	y:	dw	1	
			end		

مقایسه: در هر دور ۱۵ دستور به‌جای ۲۳ دستور اجرا می‌شود و اندازهٔ کد از ۸۵ بایت به ۷۳ بایت کاهش می‌یابد. نتیجه: $x = F_{20} = 6765$، $y = F_{21} = 10946$ و sum = 10945 = مجموع دقیق ۲۰ جملهٔ اول (با شبیه‌سازی تأیید شده).

۳-۱۴ تبدیل عبارت میانوندی به پسوندی

الگوریتم اسلاید دو مرحله دارد:

1. عبارت ورودی را **کامل پرانتزگذاری** کنید (هر عملگر با دو عملوندش یک جفت پرانتز بگیرد).
2. از درونی‌ترین جفت پرانتزها شروع کنید و **پرانتز راست را با عملگر آن جایگزین کنید و پرانتز چپ را حذف کنید**.

مثال اسلاید: تبدیل عبارت (x + y - 10 - t/d + 10*s - z) / (20*t - z/t + 100*w)

مرحلهٔ ۱: پرانتزگذاری کامل

```
(x+y-10-t/d+10*s-z) / (20*t-z/t+100*w)
((x+y)-10-(t/d)+(10*s)-z) / ((20*t)-(z/t)+(100*w))
(((x+y)-10)-(t/d)+((10*s)-z)) / (((20*t)-(z/t))+(100*w))
((((x+y)-10)-(t/d))+((10*s)-z)) / (((20*t)-(z/t))+(100*w))
(((((x+y)-10)-(t/d))+((10*s)-z)) / (((20*t)-(z/t))+(100*w)))
```

مرحلهٔ ۲: جایگزینی پرانتز راست با عملگر

```
(((((x+y)-10)-(t/d))+((10*s)-z)) / (((20*t)-(z/t))+(100*w)))
(((( xy+ -10)- td/ )+( 10s* -z)) / (( 20t* - zt/ )+ 100w*))
((( xy+ 10- - td/ )+ 10s*z- ) / ( 20t* zt/- + 100w*))
(( xy+ 10- td/- + 10s*z- ) / 20t* zt/- 100w*+ )
( xy+ 10- td/- 10s*z- + / 20t* zt/- 100w*+ )
xy+ 10- td/- 10s*z- + 20t* zt/- 100w*+ /
```

نتیجه:

`xy+10-td/-10s*z-+20t*zt/-100w*+/`

نکته: چرا این جواب با جواب قبلی کمی فرق دارد؟

در پرانتزگذاری بالا، z - 10*s + به شکل (z - (10*s))+ گروه شده است، در حالی که پرانتزگذاری استاندارد چپ‌به‌راست (z - ((10*s) + …)) است و جواب `xy+10-td/-10s*+z-20t*zt/-100w*+/` (مثال قبل) را می‌دهد. چون a + (b - c) = (a + b) - c، هر دو عبارت لهستانی **مقدار یکسانی** دارند و هر دو درست‌اند. اگر بخواهید جواب یکتا باشد، همیشه عملگرهای هم‌اولویت را از چپ به راست گروه کنید.

۱-۱۴-۳ روش دوم (تکمیلی): الگوریتم پشته‌ای

کامپایلرها معمولاً با یک پشتهٔ عملگر تبدیل را در یک پیمایش انجام می‌دهند (الگوریتم «ایستگاه قطار» دایکسترا): عبارت را از چپ به راست بخوانید؛ (۱) عملوند را مستقیم به خروجی بفرستید؛ (۲) برای عملگر، تا وقتی بالای پشته عملگری با اولویت **بیشتر یا مساوی** است آن را pop و به خروجی بفرستید، سپس عملگر جدید را push کنید؛ (۳) «(» را push کنید؛ (۴) با رسیدن به «)» تا رسیدن به «(» pop کنید؛ (۵) در پایان همهٔ عملگرهای پشته را pop کنید.

مثال ۳-۵

عبارت a + b * c - d را به پسوندی تبدیل کنید.

حل

نماد	پشتهٔ عملگر	خروجی
a		a
+	+	a
b	+	a b
*	+ *	a b
c	+ *	a b c
-	-	a b c * +
d	-	a b c * + d
(پایان)		a b c * + d -

جواب: `abc*+d-`

تمرین‌های حل‌شدهٔ نمادگذاری لهستانی

۱. عبارت (a + b) * (c - d) / e را به پسوندی و پیشوندی تبدیل کنید و آن را با a=2, b=3, c=7, d=1, e=5 روی ماشین پشته‌ای محاسبه کنید.

حل

پسوندی: `ab+cd-*e/` پیشوندی: `/*+ab-cde`

محاسبه (پشته پس از هر نماد): [6] → [30,5] → [30] → [5,6] → [5,7,1] → [5,7] → [5] → [2,3] → [2]؛ پس حاصل 6 است و بیشینهٔ عمق پشته ۳.

۲. عبارت پسوندی `ab*cd*+e-` را به میانوندی برگردانید.

حل

با همان روش پشته‌ای ولی با ساختن رشته: a, b → a*b؛ c, d → c*d؛ + → (a*b + c*d)؛ e, - → a*b + c*d - e.

۳. برای محاسبهٔ `ab+cd-*e/` روی ماشین ۶ (که ضرب و تقسیم ندارد) فقط بخش (a+b) - (c-d) را بنویسید.

حل

```
push a
push b
add          ; [a+b]
push c
push d
sub          ; [a+b, c-d]
sub          ; [(a+b)-(c-d)]
pop  r
```

خلاصهٔ فصل ۳

- ثبات = n فلیپ‌فلاپ با کنترل مشترک؛ PIPO/SIPO/PISO/SISO؛ فایل ثبات (یک دسترسی در هر لحظه) یا مجزا (موازی)؛ آدرس‌پذیر (AX، R0،…) یا غیرآدرس‌پذیر (PC، IR، MBR، MAR).
- طول ثبات‌ها: MAR و PC به اندازهٔ آدرس، MBR به اندازهٔ پهنای دسترسی، IR به اندازهٔ بلندترین دستور.
- روش‌های آدرس‌دهی این فصل: مستقیم، غیرمستقیم (یک/چند سطحی)، بی‌واسطه، شاخص‌دار و ضمنی.
- بدون آدرس‌دهی غیرمستقیم می‌توان با کد خودتغییرده آرایه را پیمایش کرد (اما از دید مهندسی نرم‌افزار بد است).
- اسمبلر دوگذره است (گذر ۱: جدول نمادها، گذر ۲: کد)؛ پیونددهنده ماژول‌ها و کتابخانه‌ها را ترکیب و جابه‌جا می‌کند؛ بارکننده برنامه را در حافظه می‌گذارد و اجرا می‌کند.
- ماشین‌های ۳، ۲، ۱ و ۰ آدرسه؛ در ماشین پشته‌ای، عبارت‌ها به شکل لهستانی (پسوندی) و بدون پرانتز محاسبه می‌شوند.

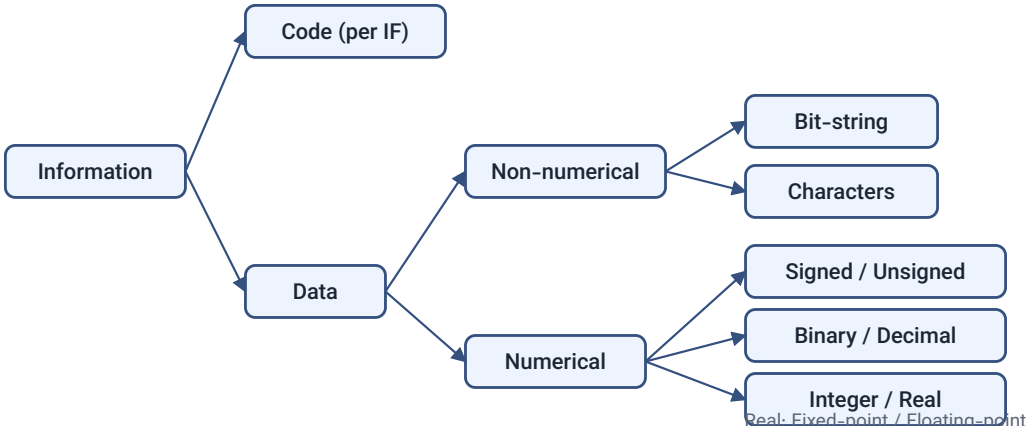
فصل ۴

نمایش داده‌ها (Data Representation)

در این فصل (جلسهٔ چهارم) می‌بینیم اطلاعات چگونه در کامپیوتر به صورت بیت نمایش داده می‌شوند: **سیستم‌های عددی موضعی و تبدیل مبناها**، **نمایش اعداد علامت‌دار** (علامت-مقدار، مکمل یک، مکمل دو و افزونی)، **اعداد حقیقی** در قالب ممیز ثابت و **ممیز شناور** (از جمله استاندارد **IEEE 754**)، و در پایان **کدهای دهدهی** (BCD، افزونی ۳، گری و ...) و **کدهای کاراکتری** (ASCII، EBCDIC و یونی‌کد). تکلیف‌های اثبات جمع در مکمل یک و مکمل دو هم به‌طور کامل حل شده‌اند.

۴-۱ دسته‌بندی اطلاعات

اطلاعاتی که در کامپیوتر نگه داشته می‌شود دو نوع است: **کد** (دستورها، که طبق قالب دستور ماشین کد می‌شوند) و **داده**. داده‌ها می‌توانند **غیرعددی** (رشتهٔ بیتی یا کاراکتر) یا **عددی** باشند. اعداد هم از چند جهت دسته‌بندی می‌شوند: علامت‌دار یا بدون علامت؛ دودویی یا دهدهی؛ صحیح یا حقیقی (و اعداد حقیقی به صورت ممیز ثابت یا ممیز شناور).



شکل: دسته‌بندی اطلاعات در کامپیوتر

۴-۲ سیستم اعداد دودویی

- داده‌ها در کامپیوتر به صورت دنباله‌ای از **بیت‌ها** (**bi**nary digits) نمایش داده می‌شوند.
- واحدهای دیگر: **بایت** = ۸ بیت؛ **نیبل** = ۴ بیت (امروزه کم‌کاربرد)؛ **کلمه** = مضربی از بایت (مثلاً ۲ یا ۴ بایت) بسته به معماری.
- در سیستم دودویی، **n بیت می‌تواند تا 2^n مقدار** را نشان دهد؛ مثلاً ۲ بیت ۴ مقدار (00، 01، 10، 11) و ۴ بیت ۱۶ مقدار (0000 تا 1111).

- برای نمایش **M مقدار**، $\lceil \log_2 M \rceil$ بیت لازم است؛ مثلاً ۳۲ مقدار ۵ بیت و ۱۰۰۰ مقدار ۱۰ بیت ($2^{10} = 1024 \geq 1000$).

تمرین ۴-۱ (حل‌شده)

برای نمایش هر یک از موارد زیر حداقل چند بیت لازم است؟ (الف) ۲۶ حرف انگلیسی (ب) یک میلیون مقدار مختلف (ج) شمارهٔ دانشجویی ۱۵۰ دانشجوی یک کلاس.

حل

(الف) $\lceil \log_2 26 \rceil = 5$ (چون $2^4 = 16 < 26 \leq 32 = 2^5$). (ب) $2^{19} = 524288 < 10^6 \leq 1048576 = 2^{20} \Rightarrow$ ۲۰ بیت. (ج) $2^7 = 128 < 150 \leq 256 \Rightarrow$ ۸ بیت.

۴-۳ سیستم‌های عددی موضعی و تبدیل مبنا

💡 **شهود: کیلومترشمار**

یک کیلومترشمار مبنای R را تصور کنید: هر چرخ رقم‌های 0 تا R-1 دارد و وقتی از R-1 رد شود به 0 برمی‌گردد و چرخ سمت چپش یکی جلو می‌رود. پس هر واحد در جایگاه i معادل R^i واحد از جایگاه صفر است؛ این همان «ارزش مکانی» است و همهٔ روش‌های تبدیل مبنا از آن نتیجه می‌شوند.

۴-۳-۱ سیستم دهدهی (مبنای ۱۰)

یک سیستم **موضعی-وزن‌دار** (weighted-positional) است: **مبنا** (base/radix) ۱۰ است، ارقام {0, 1, …, 9} هستند و هر موقعیت وزنی برابر توانی از ۱۰ دارد. مثال:

$$(7594.36)_{10} = 7\times10^3 + 5\times10^2 + 9\times10^1 + 4\times10^0 + 3\times10^{-1} + 6\times10^{-2}$$

به‌طور کلی برای یک عدد دهدهی با n + m رقم (n رقم صحیح و m رقم کسری):

$$(a_{n-1}a_{n-2}\ldots a_0.f_1f_2\ldots f_m)_{10} = a_{n-1}\times10^{n-1} + \ldots + a_0\times10^0 + f_1\times10^{-1} + \ldots + f_m\times10^{-m}$$

۴-۳-۲ سیستم‌های عددی دیگر

- **هشت‌هشتی (مبنای ۸)**: وزن‌ها توان‌های ۸؛ ارقام 0 تا 7.
- **شانزده‌شانزدهی (مبنای ۱۶)**: وزن‌ها توان‌های ۱۶؛ ارقام 0 تا 9 و A تا F (A = 10، …، F = 15).
- **مبنای R**: وزن‌ها توان‌های R؛ ارقام 0، 1، …، R-1.
- در بعضی زبان‌ها نمادهای خاصی به کار می‌رود: در زبان C پیشوند `0x` برای شانزده‌شانزدهی (`0xF2` یعنی $(F2)_{16}$)؛ در Verilog، `8'b11110000` ، `8'hF0` و `8'd240` همگی عدد دودویی ۸ بیتی 11110000 را نشان می‌دهند.

۴-۳-۳ تبدیل مبنای R به دهدهی

ساده است! برای عدد $a_{n-1}\ldots a_1a_0.a_{-1}a_{-2}\ldots a_{-m}$ در مبنای R کافی است $\Sigma_{k=-m}^{n-1} R^k\cdot a_k$ را حساب کنیم.

مثال‌های اسلاید

- $1101.101_2 = 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 + 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} = 8+4+1+0.5+0.125 = 13.625_{10}$
- $572.6_8 = 5\times8^2 + 7\times8^1 + 2\times8^0 + 6\times8^{-1} = 320+56+2+0.75 = 378.75$
- $2A.8_{16} = 2\times16^1 + 10\times16^0 + 8\times16^{-1} = 32+10+0.5 = 42.5$
- $341.24_5 = 3\times5^2 + 4\times5^1 + 1\times5^0 + 2\times5^{-1} + 4\times5^{-2} = 75+20+1+0.4+0.16 = 96.56$

۴-۳-۴ تبدیل دهدهی به مبنای R

- **برای بخش صحیح:** روش **تقسیم‌های متوالی بر R**.
- **برای بخش کسری:** روش **ضرب‌های متوالی در R**.

تقسیم‌های متوالی بر ۲

برای تبدیل یک عدد صحیح به دودویی، آن را پشت‌سرهم بر ۲ تقسیم می‌کنیم تا خارج‌قسمت 0 شود. **باقی‌مانده‌ها جواب را می‌سازند: اولین باقی‌مانده کم‌ارزش‌ترین بیت (LSB) و آخرین باقی‌مانده پرارزش‌ترین بیت (MSB) است.**

تقسیم	خارج‌قسمت	باقی‌مانده
43 ÷ 2	21	1 ← LSB
21 ÷ 2	10	1
10 ÷ 2	5	0
5 ÷ 2	2	1
2 ÷ 2	1	0
1 ÷ 2	0	1 ← MSB

با خواندن باقی‌مانده‌ها از پایین به بالا:

$$(43)_{10} = (101011)_2$$

بررسی: $32+8+2+1 = 43$ ✓

ضرب‌های متوالی در ۲

برای تبدیل یک عدد کسری به دودویی، آن را پشت‌سرهم در ۲ ضرب می‌کنیم تا بخش کسری حاصل‌ضرب 0 شود (یا به تعداد رقم دلخواه برسیم). **رقم‌های صحیح تولیدشده (Carry) جواب را می‌سازند: اولین آن‌ها MSB و آخرین LSB است.**

ضرب	حاصل	Carry
0.3125 × 2	0.625	0 ← MSB
0.625 × 2	1.25	1
0.25 × 2	0.50	0
0.5 × 2	1.00	1 ← LSB

$$(0.3125)_{10} = (0.0101)_2$$

بررسی: $0.25 + 0.0625 = 0.3125$ ✓

تمرین ۴-۲ (حل‌شده)

(الف) 43.3125_{10} را به دودویی، (ب) 156_{10} را به شانزده‌شانزدهی، (ج) 0.8125_{10} را به هشت‌هشتی و (د) 0.1_{10} را به دودویی تبدیل کنید.

حل

(الف) بخش صحیح و کسری جداگانه تبدیل می‌شوند: 101011.0101_2.

(ب) $156 \div 16 = 9$ باقی‌ماندهٔ 12 (C)، $9 \div 16 = 0$ باقی‌ماندهٔ $9 \Leftarrow 9C_{16}$ (بررسی: $9\times16 + 12 = 156$).

(ج) $0.8125\times8 = 6.5 \Rightarrow 6$؛ $0.5\times8 = 4.0 \Rightarrow 4 \Leftarrow 0.64_8$ (بررسی: $6/8 + 4/64 = 0.8125$).

(د) $0.1 \to 0.2$ (0)، $0.2 \to 0.4$ (0)، $0.4 \to 0.8$ (0)، $0.8 \to 1.6$ (1)، $0.6 \to 1.2$ (1)، $0.4 \to 0.2$ (0) و از اینجا الگو تکرار می‌شود: $0.1_{10} = 0.0001100110011\ldots_2$. پس **0.1 در دودویی با تعداد محدود بیت دقیقاً قابل نمایش نیست**؛ این همان دلیل خطای گرد کردن در محاسبات ممیز شناور است.

💡 **شهود: تقسیم یعنی «چند بستهٔ کامل R تایی؟»**

وقتی N را بر R تقسیم می‌کنیم، باقی‌مانده تعداد واحدهایی است که در هیچ بستهٔ R تایی جا نشدند (رقم یکان) و خارج‌قسمت تعداد بسته‌هاست که دوباره باید بسته‌بندی شود. برای همین رقم‌ها **از کم‌ارزش به پرارزش** بیرون می‌آیند. برای بخش کسری برعکس است: ضرب در R رقم بعد از ممیز را به بخش صحیح «هل می‌دهد»، پس رقم‌ها از پرارزش به کم‌ارزش بیرون می‌آیند.

۴-۳-۵ تبدیل میان مبناها

در حالت کلی تبدیل از مبنای i به مبنای j از راه دهدهی انجام می‌شود: **مبنای i ← دهدهی ← مبنای j**. اما برای مبناهایی که توان ۲ هستند (2، 4، 8، 16) میان‌بر وجود دارد: $2^i \to 2^j$:

1. هر رقم عدد ورودی را با i بیت بنویسید.
2. از سمت راست (از ممیز به چپ برای بخش صحیح، و از ممیز به راست برای بخش کسری) بیت‌ها را در گروه‌های j بیتی دسته کنید.
3. معادل هر گروه را در مبنای 2^j بنویسید.

مثال اسلاید: $(7452)_8$ را به شانزده‌شانزدهی تبدیل کنید.

حل

$$7452_8 \to 111\ 100\ 101\ 010 \to 1111\ 0010\ 1010 \to F2A_{16}$$

تمرین ۴-۳ (حل‌شده)

(الف) $3AF.C_{16}$ را به هشت‌هشتی و (ب) 10110111_2 را به مبنای ۴ تبدیل کنید.

حل

(الف) `1100 . 1111 1010 0011` ⇐ گروه‌های ۳ تایی: `110 . 111 101 110 001` ⇐ 1657.6_8.
بررسی: هر دو برابر 943.75 هستند.

(ب) گروه‌های ۲ تایی: `10 11 01 11` ⇐ 2313_4 (هر دو = 183).

💡 **شهود: هر رقم هگز یعنی دقیقاً ۴ بیت**

چون $2^4 = 16$، هر چهار بیت (از سمت ممیز) دقیقاً یک رقم هگز می‌سازد و هیچ تأثیری روی گروه کناری ندارد؛ مثل سکه‌هایی که در بسته‌های ۱۶ تایی گذاشته شوند. برای اکتال گروه‌ها ۳ بیتی‌اند ($2^3 = 8$). برای همین برنامه‌نویسان سیستم برای نوشتن الگوهای بیتی از هگز استفاده می‌کنند: کوتاه است و تبدیلش ذهنی انجام می‌شود.

۴-۴ اعداد منفی

اعداد بدون علامت فقط مقدارهای نامنفی را دارند و **اعداد علامت‌دار** همهٔ مقدارها (مثبت و منفی). چهار روش برای نمایش اعداد دودویی علامت‌دار وجود دارد:

- علامت-مقدار (Sign-Magnitude)
- مکمل یک (1s-Complement)
- مکمل دو (2s-Complement)
- افزونی (Excess-Number) — کم‌کاربردتر ولی در بعضی جاها (مثل نمای اعداد ممیز شناور) استفاده می‌شود.

۴-۴-۱ علامت-مقدار (Sign-Magnitude)

علامت با یک **بیت علامت** نشان داده می‌شود (0 برای + و 1 برای -) و بقیهٔ بیت‌ها **اندازه** (قدر مطلق) را نشان می‌دهند. مثلاً قالب ۸ بیتی: ۱ بیت علامت و ۷ بیت اندازه.

مثال‌ها

`00110100` ⇐ $+110100_2 = +52_{10}$ $\quad$ `10010011` ⇐ $-10011_2 = -19_{10}$

- بزرگ‌ترین مقدار (۸ بیت): `01111111` = +127؛ کوچک‌ترین مقدار: `11111111` = -127.
- **دو صفر** داریم: `00000000` = +0 و `10000000` = -0.
- گسترهٔ ۸ بیتی: -127 تا +127.

پرسش اسلاید: گسترهٔ نمایش علامت-مقدار n بیتی چیست؟

پاسخ

$(1 - 2^{n-1})$- تا $2^{n-1} - 1$؛ چون اندازه n-1 بیت دارد و بزرگ‌ترین اندازه $2^{n-1} - 1$ است. (در مجموع $2^n - 1$ مقدار متمایز، چون صفر دو نمایش دارد.)

منفی کردن یک عدد: کافی است **بیت علامت را معکوس کنیم**. مثال: منفی 00100001_{sm} (33) برابر 10100001_{sm} (-33) و منفی 10000101_{sm} (-5) برابر 00000101_{sm} (+5) است.

محاسبه در علامت-مقدار

جمع $S = A + B$ که $A = (A_s|A_m)$ و $B = (B_s|B_m)$ (s علامت و m اندازه):

```
if As = Bs then return S(As | Am+Bm)
else
    if As=1 & AmBm then return S(1 | Am-Bm)
    if As=0 & AmBm then return S(0 | Am-Bm)
```

یعنی اگر هم‌علامت باشند اندازه‌ها جمع می‌شوند و علامت حفظ می‌شود؛ اگر ناهم‌علامت باشند، اندازهٔ کوچک‌تر از بزرگ‌تر کم می‌شود و علامت عدد با اندازهٔ بزرگ‌تر به نتیجه داده می‌شود.

تفریق $S = A - B$ مشابه است با این تفاوت که شرط اول $A_s \neq B_s$ است (تفریق دو عدد ناهم‌علامت یعنی جمع اندازه‌ها) و در غیر این صورت همان چهار حالت بالا بررسی می‌شود.

ضرب و تقسیم ساده‌اند: $S = (A_s \oplus B_s \mid A_m \times B_m)$ و $S = (A_s \oplus B_s \mid A_m / B_m)$.

نکتهٔ مهم اسلاید

محاسبه در علامت-مقدار (به‌ویژه جمع و تفریق) **پیچیده است و نسبت به مکمل یک و مکمل دو سخت‌افزار بیشتری لازم دارد** (مقایسهٔ اندازه‌ها، جمع‌کننده و تفریق‌کننده، و انتخاب علامت). همچنین حالت $A_m = B_m$ در الگوریتم بالا جداگانه باید به +0 نگاشته شود.

مثال ۴-۱

در قالب ۸ بیتی علامت-مقدار: (الف) (-3) + (+5)، (ب) (+3) - (-5) و (ج) (+7) × (-6) را حساب کنید.

حل

(الف) `0|0000101` + `1|0000011`: علامت‌ها متفاوت، $A_s = 0$ و $A_m = 5 > B_m = 3$ ⇐ `0|0000010` = +2.

(ب) تفریق با علامت‌های متفاوت ⇐ $(A_s \mid A_m+B_m) = (1 \mid 8)$ ⇐ `1|0001000` = -8.

(ج) علامت: $1 = 0 \oplus 1$؛ اندازه: $6\times7 = 42 = 0101010$ ⇐ `1|0101010` = -42.

۴-۴-۲ مکمل یک (1s-Complement)

تعریف

اگر x یک عدد n بیتی باشد، مقدار منفی آن در مکمل یک چنین نمایش داده می‌شود: $-x = 2^n - x - 1$.

مثال اسلاید

با ۸ بیت، منفیِ `00001100` (12) در مکمل یک: $11110011_2 = 243 = 1 - 12 - 2^8$. یعنی -12_{10} در مکمل یک ۸ بیتی `11110011` نوشته می‌شود.

- **روش منفی کردن: همهٔ بیت‌ها را معکوس کنید** (چون $2^n - 1$ همهٔ بیت‌هایش ۱ است، کم کردن x از آن یعنی معکوس کردن بیت‌ها).
- بزرگ‌ترین مقدار (۸ بیت): `01111111` = +127؛ کوچک‌ترین: `10000000` = -127.
- دو صفر: `00000000` = +0 و `11111111` = -0.
- گستره: ۸ بیت -127 تا +127؛ n بیت $-(2^{n-1} - 1)$ تا $2^{n-1} - 1$.
- پرارزش‌ترین بیت (MSB) همچنان علامت را نشان می‌دهد: 0 برای + و 1 برای -.

جمع و تفریق در مکمل یک

الگوریتم جمع A + B:

1. دو عدد را با جمع‌کنندهٔ دودویی معمولی جمع کنید.
2. **رقم نقلی خروجی از MSB را به نتیجه اضافه کنید** (نقلی چرخشی، end-around carry).
3. سرریز را بررسی کنید: **سرریز وقتی رخ می‌دهد که علامت نتیجه مخالف علامت A و B (که هم‌علامت‌اند) باشد.**

اگر نتیجهٔ جمع/تفریق از گسترهٔ نمایش بیرون بزند **سرریز (Overflow)** رخ داده است. سرریز به‌سادگی تشخیص داده می‌شود: مثبت + مثبت → منفی یا منفی + منفی → مثبت. (جمع دو عدد ناهم‌علامت هرگز سرریز نمی‌کند.)

الگوریتم تفریق A − B: (B)مکمل‌یکِ + A = (-B) + A = A - B؛ یعنی مکمل یک B را بگیرید و به A اضافه کنید.

مثال‌های اسلاید: جمع در سیستم ۴ بیتی مکمل یک (آیا سرریزی هست؟)

وضعیت	نتیجه	محاسبهٔ دودویی	دهدهی
بدون سرریز	+7	`0011 + 0100 = 0111`	(+3) + (+4)
بدون سرریز (نتیجه -0 است)	-0	`0101 + 1010 = 1111`	(+5) + (-5)
بدون سرریز (با نقلی چرخشی)	-7	`1101 + 1010 = 1 0111 → 0111 + 1 = 1000`	(-2) + (-5)
سرریز! منفی + منفی = مثبت (10- در گسترهٔ 7+..7- نیست)	+5 ✗	`1100 + 1000 = 1 0100 → 0100 + 1 = 0101`	(-3) + (-7)

۳-۴-۴ مکمل دو (2s-Complement)

تعریف

اگر x یک عدد n بیتی باشد، مقدار منفی آن در مکمل دو: **$-x = 2^n - x = $ مکمل‌یکِ(x) + 1**.

مثال اسلاید

با ۸ بیت، منفیِ `00001100` (12): $11110100_2 = 244 = 2^8 - 12$. یعنی -12_{10} در مکمل دو ۸ بیتی `11110100` است.

- **روش منفی کردن:** همهٔ بیت‌ها را معکوس و سپس **۱ اضافه کنید**. یا از راست به چپ، **بیت‌ها را تا اولین ۱ (خود آن ۱ هم) همان‌طور بنویسید و بقیه را معکوس کنید.** مثلاً `1100  0000 ⇐ 0100  1111` (بیت‌های 100 سمت راست ثابت ماندند).
- بزرگ‌ترین مقدار (۸ بیت): `01111111` = +127؛ کوچک‌ترین: `10000000` = **-128**.
- **فقط یک صفر**: `00000000` = +0.
- گستره: ۸ بیت -128 تا +127؛ n بیت -2^{n-1} تا $2^{n-1} - 1$.
- MSB همچنان علامت است.

جمع و تفریق در مکمل دو

الگوریتم جمع (۱) **A + B:** دو عدد را جمع کنید؛ (۲) **رقم نقلی خروجی از MSB را دور بریزید**؛ (۳) سرریز را بررسی کنید: **سرریز وقتی رخ می‌دهد که نقلی ورودی به MSB و نقلی خروجی از MSB متفاوت باشند**، یا (به‌طور معادل) نتیجه علامتی مخالف هر دو عملوند داشته باشد.

الگوریتم تفریق A − B: (B)مکمل‌دوِ + A = A + (−B) = A − B.

مثال‌های اسلاید: جمع ۴ بیتی مکمل دو (کدام سرریز است؟)

وضعیت	نتیجه	Cin/Cout MSB	محاسبهٔ دودویی	دهدهی
بدون سرریز	+7	0 / 0	`0011 + 0100 = 0111`	(+3) + (+4)
بدون سرریز (نقلی دور ریخته می‌شود)	-8	1 / 1	`1110 + 1010 = 11000`	(-2) + (-6)
بدون سرریز	+3	1 / 1	`0110 + 1101 = 10011`	(+6) + (-3)
بدون سرریز	-3	0 / 0	`0100 + 1001 = 1101`	(+4) + (-7)
سرریز! (9- خارج از 7+..8-)	+7 ✗	**0 / 1**	`1101 + 1010 = 10111`	(-3) + (-6)
سرریز! (+11 خارج از گستره)	-5 ✗	**1 / 0**	`0101 + 0110 = 1011`	(+5) + (+6)

مثال‌های اسلاید: تفریق ۴ بیتی مکمل دو

وضعیت	نتیجه	محاسبه	تبدیل	عبارت
بدون سرریز	-3	`0100 + 1001 = 1101`	4 + (-7)	4 - 7
بدون سرریز	+5	`0110 + 1111 = 10101`	6 + (-1)	6 - 1
سرریز! (نقلی ورودی MSB = 0، خروجی = 1)	+7 ✗	`1011 + 1100 = 10111`	-5 + (-4)	-5 - 4

💡 **شهود: دایرهٔ اعداد**

یک کیلومترشمار ۴ بیتی را روی 0000 تصور کنید و یک واحد **عقب** بکشید: 1111 می‌شود. پس طبیعی است که 1111 را 1- بدانیم. همهٔ 2^n الگو روی یک دایره‌اند و x- همان 2^n - x است (فاصله تا دور زدن). نتیجه: **همان جمع‌کنندهٔ معمولی** اعداد منفی را هم درست جمع می‌کند، فقط یک صفر داریم و تفریق = جمع با قرینه. **سرریز** یعنی عبور از مرز بین بزرگ‌ترین مثبت (0111…) و کوچک‌ترین منفی (1000…) روی این دایره.

۴-۴-۴ مقایسهٔ روش‌ها

مکمل ۲	مکمل ۱	علامت-مقدار	مقدار
0111	0111	0111	+7
0110	0110	0110	+6
0101	0101	0101	+5

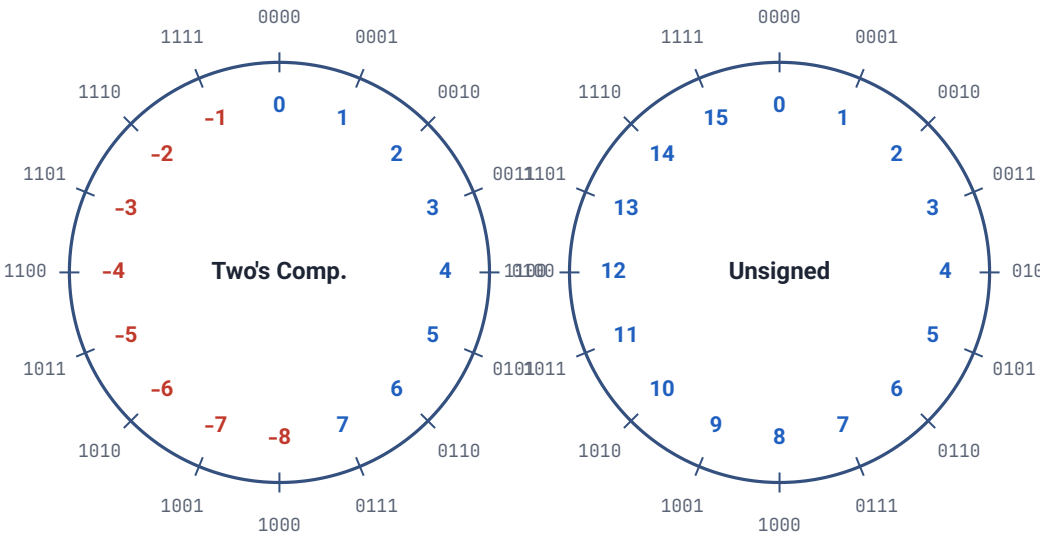
مکمل ۲	مکمل ۱	علامت-مقدار	مقدار
—	1111	1000	-0
1111	1110	1001	-1
1110	1101	1010	-2

مقدار	علامت-مقدار	مکمل ۱	مکمل ۲
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000

مقدار	علامت-مقدار	مکمل ۱	مکمل ۲
-3	1011	1100	1101
-4	1100	1011	1100
-5	1101	1010	1011
-6	1110	1001	1010
-7	1111	1000	1001
-8	—	—	1000

جدول: مقایسهٔ سه روش برای اعداد ۴ بیتی (چپ: مقدارهای مثبت، راست: منفی)

نمایش پیمانه‌ای: اگر الگوهای n بیتی را روی یک دایره بچینیم، هر روش مقدارهای متفاوتی به نقاط دایره نسبت می‌دهد. در مکمل دو، حرکت در جهت ساعت (جمع با ۱) همیشه مقدار را یکی زیاد می‌کند، جز در گذر از `0111` (+7) به `1000` (-8) که همان نقطهٔ سرریز است. همین خاصیت پیمانه‌ای است که اجازه می‌دهد جمع‌کنندهٔ معمولی (بدون علامت) برای اعداد مکمل دو هم درست کار کند.



شکل: نمایش پیمانه‌ای (دایره‌ای) اعداد ۴ بیتی — مکمل دو (چپ) و بدون علامت (راست)

تکلیف ۱: اثبات درستی جمع در مکمل یک

ثابت کنید جمع دو عدد A و B در سیستم مکمل یک با یک جمع‌کنندهٔ دودویی و سپس افزودن نقلی تولیدشده (نقلی چرخشی) به نتیجه، حاصل A+B را در سیستم مکمل یک می‌دهد. (همهٔ حالت‌ها را بررسی کنید؛ فرض: سرریز رخ نمی‌دهد.)

اثبات

نمایش n بیتی: عدد نامنفی x با خودش و عدد منفی |x|- با |x| - 2^n - 1 نمایش داده می‌شود. جمع‌کننده مقدار

خام $R = a + b$ را حساب می‌کند (a و b الگوهای بیتی‌اند). اگر $R \geq 2^n$ باشد نقلی 1 تولید می‌شود و n بیت خروجی $R - 2^n$ است؛ با افزودن نقلی، نتیجه $R - 2^n + 1$ می‌شود. یعنی افزودن نقلی چرخشی معادل کم کردن $2^n - 1$ است. فرض عدم سرریز: $|A + B| \leq 2^{n-1} - 1$.

حالت ۱: $A \geq 0, B \geq 0$. $R = A + B \leq 2^{n-1} - 1 < 2^n$ ⇐ نقلی ندارد و نتیجه $A + B$ است که نمایش درست عدد مثبت A+B است. ✓

حالت ۲: $A \geq 0, B < 0$. $R = A + 2^n - 1 - |B| = (2^n - 1) + (A - |B|)$.

- اگر $A > |B|$ (حاصل مثبت): $R \geq 2^n$ ⇐ نقلی 1؛ نتیجه $R - 2^n + 1 = A - |B| = A + B$ که مثبت و درست است. ✓
- اگر $A \leq |B|$ (حاصل صفر یا منفی): $R = 2^n - 1 - (|B| - A) < 2^n$ ⇐ نقلی ندارد؛ نتیجه $2^n - 1 - (|B| - A)$ که دقیقاً نمایش مکمل یک عدد منفی $A + B = -(|B| - A)$ است (اگر $A = |B|$، نتیجه 0- = 1…11). ✓

حالت ۳: $A < 0, B \geq 0$. با جابه‌جایی A و B همان حالت ۲ است (جمع جابه‌جایی است). ✓

حالت ۴: $A < 0, B < 0$. $R = (2^n - 1 - |A|) + (2^n - 1 - |B|) = 2^n + (2^n - 2 - (|A| + |B|))$. چون $|A| + |B| \leq 2^{n-1} - 1$، عبارت داخل پرانتز نامنفی است و $R \geq 2^n$ ⇐ نقلی 1؛ نتیجه $2^n - 1 - (|A| + |B|) + 1 = 2^n - 2 - (|A|+|B|)$ که نمایش درست $A + B = -(|A|+|B|)$ است. ✓ (این همان مثال اسلاید است: $2^n + 2^n-(|A|+|B|) -1 = 2^n + 2^n-|B|-1 + 2^n-|A|-1$ و با افزودن نقلی ۱، $2^n-(|A|+|B|)-1$.)

صفرها: اگر یکی از عملوندها $2^n - 1 = -0$ باشد، همان فرمول‌ها با $|x| = 0$ برقرارند (مثلاً $A + (-0)$ در حالت ۲ با $|B| = 0$ به A می‌رسد). بنابراین در همهٔ حالت‌ها نتیجه درست است. ∎

تکلیف ۲: اثبات درستی جمع در مکمل دو

ثابت کنید جمع A و B در مکمل دو با یک جمع‌کنندهٔ دودویی (با دور ریختن نقلی خروجی) حاصل A+B را در مکمل دو می‌دهد.

اثبات

نمایش: x نامنفی با خودش و $-|x|$ با $2^n - |x|$. جمع‌کننده با دور ریختن نقلی، $(a + b) \bmod 2^n$ را می‌دهد. فرض عدم سرریز: $-2^{n-1} \leq A + B \leq 2^{n-1} - 1$.

حالت ۱: $A, B \geq 0$. $a + b = A + B < 2^n$ ⇐ نتیجه A+B. ✓

حالت ۲: $A \geq 0, B < 0$. $a + b = 2^n + (A - |B|)$.

- اگر $A \geq |B|$: مقدار 2^n است، نقلی دور ریخته می‌شود و نتیجه $A + B = A - |B|$ (نامنفی). ✓
- اگر $A < |B|$: مقدار $2^n - (|B| - A) < 2^n$ است که نمایش مکمل دو $A + B = -(|B| - A)$ است. ✓

حالت ۳: قرینهٔ حالت ۲. ✓

حالت ۴: $A, B < 0$. $a + b = 2^n - |A| + 2^n - |B| = 2^n + (2^n - (|A| + |B|))$. چون $1 \leq |A|+|B| \leq 2^{n-1}$، داریم

$2^n - (|A|+|B|) < 2^n$؛ $2^{n-1} \le 2^n - (|A| + |B|)$ پس با دور ریختن نقلی نتیجه $2^n - (|A| + |B|)$ است که نمایش درست $-(|A|+|B|) = A + B$ است. ✓ (مثال اسلاید: $-(|A|+|B|) = 2^n + 2^n - |A| - |B| = 2^n + 2^n - |A| + 2^n - |B|$.) ∎

تکمیل: چرا قاعدهٔ سرریز درست است؟ سرریز فقط در جمع دو عدد هم‌علامت ممکن است. اگر هر دو مثبت باشند، نقلی خروجی MSB صفر است (هر دو MSB صفرند)، پس نتیجه فقط وقتی منفی می‌شود (MSB = 1) که نقلی ورودی به MSB یک باشد ⇐ ورودی ≠ خروجی. اگر هر دو منفی باشند، نقلی خروجی MSB همیشه ۱ است و نتیجه فقط وقتی مثبت می‌شود که نقلی ورودی MSB صفر باشد ⇐ باز هم ورودی ≠ خروجی.

۴-۴-۵ مکمل در اعداد کسری

ایدهٔ مکمل را می‌توان به بخش کسری هم گسترش داد؛ در مکمل دو، «افزودن ۱» یعنی افزودن ۱ به **کم‌ارزش‌ترین موقعیت** (مثلاً 2^{-3}).

مثال‌های اسلاید

توضیح	پاسخ	در روش	منفی کردن
فقط بیت علامت معکوس می‌شود	11011.011	Sign-Magnitude	01011.011
همهٔ بیت‌ها معکوس می‌شوند	00111.101	1s-complement	11000.010
معکوس: 10110.100، به اضافهٔ 0.001	10110.101	2s-complement	01001.011

بررسی آخری: 01001.011 = 9.375 و 10110.101 = 22.625 = 2^5 - 9.375 ✓.

۴-۵ نمایش افزونی (Excess Representation)

علاوه بر علامت-مقدار و روش‌های مکمل، **نمایش افزونی** روش دیگری است. این روش با یک **جابه‌جایی سطح ساده** (جمع یا تفریق یک عدد ثابت K) گسترهٔ مقدارها را **به‌طور یکنواخت** میان مثبت‌ها و منفی‌ها پخش می‌کند: کد = مقدار + K و مقدار = کد - K.

مقدار	Excess-4
-4	000
-3	001
-2	010
-1	011
0	100
1	101

مثال: **Excess-4** روی اعداد ۳ بیتی (جدول کنار). به‌طور کلی برای توزیع یکنواخت کدها از **2^{n-1}-Excess** یا **$(2^{n-1} - 1)$-Excess** استفاده می‌کنیم؛ مثلاً برای اعداد ۴ بیتی Excess-8 یا Excess-7. **مقدارهای دیگر K توزیع ناهمسانی می‌سازند**؛ مثلاً Excess-5 در ۴ بیت گسترهٔ -5 تا +10 را می‌دهد.

نکته: 2^{n-1}-Excess همان مکمل دو با MSB معکوس‌شده است؛ مثلاً -3 در ۴ بیت: مکمل دو 1101 و Excess-8 0101.

مقدار	Excess-4
2	110
3	111

مقدار	Excess-8	Excess-7	Excess-5	مقدار	Excess-8	Excess-7	Excess-5
-8	0000	—	—	2	1010	1001	0111
-7	0001	0000	—	3	1011	1010	1000
-6	0010	0001	—	4	1100	1011	1001
-5	0011	0010	0000	5	1101	1100	1010
-4	0100	0011	0001	6	1110	1101	1011
-3	0101	0100	0010	7	1111	1110	1100
-2	0110	0101	0011	8	—	1111	1101
-1	0111	0110	0100	9	—	—	1110
0	1000	0111	0101	10	—	—	1111
1	1001	1000	0110				

Excess-8 و Excess-7 توزیع یکنواخت دارند (۸ مقدار منفی و ۷ مثبت، یا برعکس)؛ Excess-5 ناهمسان است.

تمرین ۴-۴ (حل‌شده)

(الف) عدد -20 را در Excess-127 هشت‌بیتی بنویسید. (ب) کد `10000101` در Excess-127 چه مقداری است؟

حل

(الف) 01101011 = 107 = 127 + -20. (ب) 6 = 127 - 133. (این همان روش نمایش نما در IEEE 754 تک‌دقتی است.)

💡 شهود: جابه‌جا کردن خط‌کش

در افزونی به همهٔ اعداد یک مقدار ثابت (bias) اضافه می‌کنیم تا کوچک‌ترین عدد به 000…0 برسد. مثل اینکه صفر خط‌کش را جابه‌جا کنیم تا هیچ عددی منفی نباشد. نتیجهٔ مهم: ترتیب الگوهای بیتی (اگر بدون علامت مقایسه شوند) همان ترتیب اعداد است؛ برای همین توان در ممیز شناور افزونی ذخیره می‌شود تا مقایسهٔ دو عدد شناور ساده باشد.

۴-۶ اعداد حقیقی: ممیز ثابت

بسیاری از کاربردها علاوه بر اعداد صحیح، روی **اعداد حقیقی** هم محاسبه دارند. **به دلیل محدود بودن تعداد بیت‌ها، اعداد حقیقی اغلب با مقدار تقریبی‌شان نمایش داده می‌شوند.** دو شکل نمایش داریم: **ممیز ثابت** و **ممیز شناور**.

در **نمایش ممیز ثابت**، تعداد بیت‌های بخش صحیح و بخش کسری ثابت است و ممیز در جای مشخصی «فرض» می‌شود (ذخیره نمی‌شود).

مثال اسلاید: قالب ۸ بیتی با ۶ بیت صحیح و ۲ بیت کسری

اگر از مکمل دو استفاده شود:

- `011010.11` = 16+8+2+0.5+0.25 = 26.75_{10}
- `111110.11` منفی است؛ قرینهٔ آن (معکوس و افزودن 0.01): `000001.01` = 1.25 ⇐ مقدار -1.25_{10}.

تمرین ۴-۵ (حل‌شده)

در همین قالب، (الف) گسترهٔ اعداد و دقت (فاصلهٔ دو عدد متوالی) چیست؟ (ب) -5.5 را نمایش دهید.

حل

(الف) مانند عدد صحیح ۸ بیتی مکمل دو تقسیم بر ۴: از -128/4 = -32 تا 127/4 = 31.75 با گام $0.25 = 2^{-2}$.

(ب) 000101.10 = 5.5 ⇐ معکوس `111010.10` + `0.01` = `111010.01`.

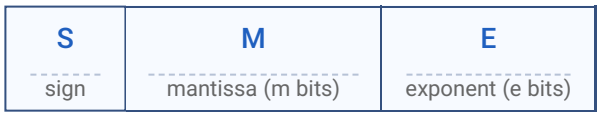
۴-۷ اعداد حقیقی: ممیز شناور

نمایش ممیز ثابت **گسترهٔ محدودی** دارد. جایگزین آن **اعداد ممیز شناور** است که امکان نمایش اعداد بسیار بزرگ و بسیار کوچک را می‌دهد؛ مثلاً $+0.23\times10^{23}$ (عدد مثبت بسیار بزرگ)، $+0.5\times10^{-37}$ (عدد مثبت بسیار کوچک) و -0.2397×10^{-18} (عدد منفی بسیار کوچک).

یک عدد ممیز شناور دو بخش دارد: **مانتیس** (کسر، mantissa/fraction) و **نما** (exponent):

$$\text{value} = \text{mantissa} \times b^{\text{exponent}}$$

- در بیشتر سیستم‌ها مبنا b = 2 فرض می‌شود (IBM 360/370 از b = 16 استفاده می‌کند).
- هرچند برای مانتیس و نما هر روش نمایشی را می‌توان به کار برد، قالب رایج و پذیرفته‌شده این است: **مانتیس m بیتی به صورت علامت-مقدار** و **نمای e بیتی به صورت Excess-2^{e-1} (بایاس‌دار)**:

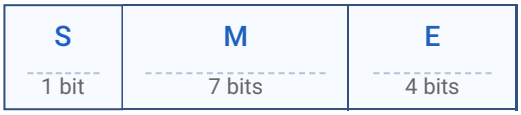


$$\text{Value} = (-1)^{S} \times 0.M \times 2^{E\ -\ \text{Bias}}$$

💡 شهود: نماد علمی در دودویی

عدد آووگادرو را می‌نویسیم 6.02×10^{23}: «رقم‌های مهم» (مانتیس) و «جای ممیز» (توان) جدا نگه داشته می‌شوند. ممیز شناور همین است در مبنای ۲. با تعداد بیت ثابت، بیت بیشتر برای توان یعنی **بازهٔ** بزرگ‌تر و بیت بیشتر برای مانتیس یعنی **دقت** بیشتر. **نرمال‌سازی** (اولین رقم مانتیس ۱ باشد) هم نمایش را یکتا می‌کند و هم هیچ بیتی را با صفرهای بی‌فایده هدر نمی‌دهد؛ و چون آن ۱ همیشه هست، IEEE آن را ذخیره نمی‌کند (بیت پنهان).

۱-۷-۴ یک قالب ۱۲ بیتی نمونه



$$\text{Value} = (-1)^S \times 0.M \times 2^{E-8} \quad \text{(نما در Excess-8)}$$

مانتیس را **نرمال** می‌کنیم: اولین بیت بعد از ممیز باید 1 باشد (0.1xxxxxx)، مگر برای عدد صفر.

مقدار	محاسبه	الگو (S M E)	مورد
127	$+0.1111111 \times 2^{15-8} = 1111111_2$	`0 1111111 1111`	بزرگ‌ترین عدد
-127	$-0.1111111 \times 2^{15-8}$	`1 1111111 1111`	کوچک‌ترین عدد
-0.0000305176	$-2^{-7} \times 2^{0-8} = -2^{-15}$	`1 0000001 0000`	بزرگ‌ترین عدد منفی
0.0000305176	$2^{-7} \times 2^{0-8} = 2^{-15}$	`0 0000001 0000`	کوچک‌ترین عدد مثبت

(دو سطر آخر بدون شرط نرمال بودن است؛ کوچک‌ترین عدد مثبت **نرمال**، $2^{-9} = 2^{-8} \times 0.1$ است.)

مثال اسلاید: نمایش -3.625

حل

$-3.625 = -11.101_2 = -0.11101 \times 2^2$ ⇐ $S = 1$، $M = 1110100$، $E = 2 + 8 = 10 = 1010$ ⇐ **`1 1110100 1010`** (نرمال).

شکل‌های غیرنرمال هم ممکن‌اند: -0.011101×2^3 ⇐ `1 0111010 1011` یا -0.0011101×2^4 ⇐ `1 0011101 1100`. نرمال‌سازی نمایش را یکتا می‌کند.

مثال اسلاید: نمایش 25.375

حل

$25.375 = 11001.011_2 = 0.11001011 \times 2^5$. مانتیس ۸ بیت لازم دارد ولی فقط ۷ بیت داریم؛ پس بیت آخر **بریده (truncate)** می‌شود: M = 1100101، E = 13 = 1101 ⇐ `0 1100101 1101` که مقدار 11001.01 = 25.25 را نشان می‌دهد!!! LSB مانتیس (به ارزش 0.125 در این عدد) از دست رفت.

۴-۷-۲ جمع و تفریق ممیز شناور

برای جمع دو عدد ممیز شناور: (۱) **هم‌ترازی**: مانتیس عدد با نمای کوچک‌تر را به اندازهٔ اختلاف نماها به راست شیفت می‌دهیم و نمایش را برابر نمای بزرگ‌تر می‌کنیم؛ (۲) مانتیس‌ها را جمع/تفریق می‌کنیم؛ (۳) نتیجه را نرمال می‌کنیم (اگر سرریز شد یک شیفت به راست و افزایش نما؛ اگر صفرهای پیشرو داشت شیفت به چپ و کاهش نما) و بیت‌های اضافه بریده می‌شوند.

مثال اسلاید: جمع 23.25 + 11.5

حل

$11.5 = 1011.1 = 0.10111 \times 2^4$ ⇐ `0 1011100 1100`؛ $23.25 = 10111.01 = 0.1011101 \times 2^5$ ⇐ `0 1011101 1101`.

1. هم‌ترازی: برای حفظ MSB عدد 23.25، مانتیس 11.5 را یک بیت به راست شیفت می‌دهیم (یعنی تقسیم بر ۲) و یکی به نمای آن اضافه می‌کنیم (ضرب در ۲) تا مقدار 11.5 تغییر نکند: `0 0101110 1101`.
2. جمع مانتیس‌ها: `1011101 + 0101110 = 10001011` (۸ بیت؛ سرریز مانتیس).
3. برای جا شدن در ۷ بیت، LSB (اینجا ۱) **بریده** می‌شود: `1000101` و یکی به نمای `1101` اضافه می‌شود: `1110`.

نتیجه: **+34.5** = $100010.1 = 0.1000101 \times 2^6$ = `0 1000101 1110` و نه +34.75!

مثال اسلاید: جمع 12.375 + 0.6015625

با قلم و کاغذ: 12.375 + 0.6015625 = 12.9765625.

حل در سیستم ۱۲ بیتی

$0.6015625 = 0.1001101_2 \times 2^0$ ⇐ E = 0 + 8 = 1000 ⇐ `0 1001101 1000`.

$12.375 = 1100.011 = 0.1100011 \times 2^4$ ⇐ `0 1100011 1100`.

هم‌ترازی: اختلاف نماها ۴ است؛ مانتیس عدد کوچک‌تر ۴ بیت به راست می‌رود: 0.1001101 → 0.0000100|1101 ⇐ `0 0000100 1100` (چهار بیت 1101 از دست رفت).

جمع: `0000100 + 1100011 = 1100111` ⇐

12.875 = 1100.111 = 0.1100111×2^4 = `0 1100111 1100` !!! (خطا ≈ 0.1)

توجه (تفاوت با اسلاید)

در اسلاید، 0.6015625 به صورت `0 1001101 0111` (یعنی با نمای 0111) نوشته شده که در واقع $0.1001101 \times 2^{-1} = 0.30078125$ است؛ با آن نما شیفت ۵ بیتی و نتیجهٔ 12.625 به دست می‌آید. نمای درست 1000 است و نتیجه 12.875 می‌شود. **پیام اصلی مثال تغییر نمی‌کند: در جمع/تفریق ممیز شناور، عدد کوچک‌تر فدای عدد بزرگ‌تر می‌شود (و هرچه اختلاف نماها بیشتر باشد، جریمه بزرگ‌تر است).**

مثال اسلاید: مشکل نمایش صفر

چهار ثبات R0 = 10.75، R1 = -12.25، R2 = -23.0 و R3 = 0.248046875 داریم و کد زیر اجرا می‌شود:

```
FSUB R0,R1
FADD R2,R0
FADD R3,R2
```

حل

نمایش‌ها: `R0 = 0 1010110 1100` ($2^4 \times 0.101011 = 10.75$)، `R1 = 1 1100010 1100` (-12.25)، `R2 = 1 1011100 1101` (-23.0 = $2^5 \times -0.10111$)، `R3 = 0 1111111 0110` (0.248046875 = $2^{-2} \times 0.1111111$).

- `FSUB R0,R1` ⇐ 10.75 - (-12.25) = 23.0 ⇐ `R0 = 0 1011100 1101`.
- `FADD R2,R0` ⇐ -23 + 23 = 0 ⇐ مانتیس صفر ولی نما همان 1101 می‌ماند: 0.0×2^5 = `R2 = 0 0000000 1101`.
- `FADD R3,R2` : برای هم‌ترازی، عدد با نمای کوچک‌تر (R3 با نمای 0110) باید **۷ بیت** به راست شیفت داده شود تا به نمای 1101 برسد؛ همهٔ بیت‌های مانتیس آن بیرون می‌ریزد و حاصل `0 0000000 1101` = **0.0** **!!!** می‌شود، در حالی که جواب درست 0.248046875 است.

راه‌حل: به صفر ممیز شناور **کوچک‌ترین نمای ممکن** را بدهیم: 0.0×2^{-8} = `0 0000000 0000`. در این صورت در جمع با هر عددی، مانتیس **صفر** است که شیفت داده می‌شود ⇐ **بدون خطا** ☺. یعنی هر گاه مانتیس نتیجه صفر شد، سخت‌افزار باید نما را هم صفر (کوچک‌ترین) کند.

۴-۷-۳ پرسش‌های مفهومی اسلاید

پرسش ۱: نرمال‌سازی چه فایده‌ای دارد؟

(۱) **بهترین استفاده از بیت‌های مانتیس:** صفرهای پیشرو بیت‌ها را هدر می‌دهند؛ در نرمال، همهٔ بیت‌ها رقم بامعنا هستند و دقت بیشینه است. (۲) **یکتا شدن نمایش هر عدد:** مثلاً 3.625- سه نمایش داشت ولی فقط یک نمایش نرمال دارد (مقایسهٔ برابری ساده می‌شود).

پرسش ۲: چرا برای صفر کوچک‌ترین نمای ممکن را در نظر بگیریم؟

برای **کمینه کردن خطای محاسبات**؛ همان‌طور که در مثال ثبات‌ها دیدیم، صفر با نمای بزرگ هنگام هم‌ترازی عددهای دیگر را از بین می‌برد.

پرسش ۳: نمای بایاس‌دار (Excess) چه مزایا و معایبی دارد؟

مزایا:

1. عدد ممیز شناور 0.0 با الگوی تمام‌صفر نمایش داده می‌شود (کوچک‌ترین نما = 0000).
2. مقایسهٔ نماها ساده‌تر است؛ چون نمای بایاس‌دار عددی بدون علامت است و ترتیب کدها همان ترتیب مقدارهاست.
3. بررسی صفر یا ناصفر بودن یک عدد ممیز شناور با همان مدار اعداد صحیح انجام می‌شود (همهٔ بیت‌ها صفر؟).
4. برای پرش‌های شرطی صفر/ناصفر روی اعداد ممیز شناور به کد عمل جداگانه نیازی نیست.

معایب: (۱) در **ضرب** ممیز شناور، نماها جمع می‌شوند و بایاس دو بار می‌آید: $(e_1+B) + (e_2+B) = e_1+e_2+2B$ ⇐ باید یک B کم شود (بایاس اضافه). (۲) در **تقسیم**، نماها کم می‌شوند و بایاس حذف می‌شود: $(e_1+B) - (e_2+B) = e_1-e_2$ ⇐ باید B اضافه شود (بایاس حذف‌شده).

تمرین ۴-۶ (حل‌شده): ضرب در قالب ۱۲ بیتی

2.5 × 3.0 را در قالب ۱۲ بیتی بالا حساب کنید.

حل

$3 = 0.11\times2^2$ ⇐ 1010 1100000 ؛ $2.5 = 0.101\times2^2$ ⇐ 1010 1010000 0. علامت: $0 \oplus 0 = 0$. مانتیس: $0.11 \times 0.101 = 0.01111$. نما: 1010 + 1010 - 1000 = 1100 (حذف بایاس اضافه). نرمال‌سازی: $0.01111 = 0.1111 \times 2^{-1}$ ⇐ نما 1011 = 1 - 1100 = $0.1111 \times 2^3 = 7.5$ 1011 1111000 0 ✓.

۴-۸ استاندارد IEEE 754

این استاندارد سه فیلد دارد: **علامت**، **نما** و **کسر** (اندازهٔ مانتیس):



$$\text{Value} = (-1)^S \times 1.F \times 2^{E - \text{Bias}}$$

- **دقت ساده (Single, 32 بیت)**: ۱ بیت علامت، ۸ بیت نما (بایاس = ۱۲۷) و ۲۳ بیت کسر.
- **دقت مضاعف (Double, 64 بیت)**: ۱ بیت علامت، ۱۱ بیت نما (بایاس = ۱۰۲۳) و ۵۲ بیت کسر.
- مانتیس با یک **بیت ۱ ضمنی (پنهان)** نرمال می‌شود: 1.xxxx؛ چون بیت اول همیشه ۱ است ذخیره نمی‌شود و یک بیت دقت مجانی به دست می‌آید. مثال: $110.1_2 \rightarrow 1.101_2 \times 2^2$ ⇐ فقط `101` در فیلد کسر ذخیره می‌شود؛ $0.00101101_2 \rightarrow 1.01101_2 \times 2^{-3}$ ⇐ فقط `01101` ذخیره می‌شود.

مثال اسلاید: -6.5 در IEEE 754 دقت ساده

حل

$-6.5_{10} = -110.1_2 = -1.101_2 \times 2^2$ ⇐ $S = 1$؛ نما $2 + 127 = 129 = 10000001$؛ کسر `10100000000000000000000`.

$$1\ 10000001\ 10100000000000000000000_2 = \text{C0D00000}_{16}$$

۴-۸-۱ گستره و مقدارهای ویژه

	Denormalized	Normalized	Approx. decimal
Single	$\pm 2^{-149}$ to $(1-2^{-23})\times2^{-126}$	$\pm 2^{-126}$ to $(2-2^{-23})\times2^{127}$	$\pm \approx 10^{-44.85}$ to $\approx 10^{38.53}$
Double	$\pm 2^{-1074}$ to $(1-2^{-52})\times2^{-1022}$	$\pm 2^{-1022}$ to $(2-2^{-52})\times2^{1023}$	$\pm \approx 10^{-323.3}$ to $\approx 10^{308.3}$

مقدارهای ویژه وقتی رخ می‌دهند که بیت‌های فیلد نما همه 0 یا همه 1 باشند:

1. **عدد غیرنرمال (Denormalized)**: وقتی نما تمام‌صفر است، عدد غیرنرمال است و مقدار آن $(-1)^s \times 0.f \times 2^{-126}$ (دقت ساده) یا $(-1)^s \times 0.f \times 2^{-1022}$ (دقت مضاعف) است. این اعداد فاصلهٔ میان صفر و کوچک‌ترین عدد نرمال را پر می‌کنند (سرریز تدریجی به پایین).
2. **صفر**: عدد غیرنرمالی که f آن تمام‌صفر است. +0 و -0 داریم که در مقایسه برابرند.
3. **بی‌نهایت**: ∞+ و ∞- با نمای تمام‌یک و کسر تمام‌صفر.
4. **NaN (عدد نیست)**: برای مقداری که عدد حقیقی نیست؛ نمای تمام‌یک و کسر ناصفر.

Sign	Exponent (e)	Fraction (f)	Value
0	00…00	00…00	+0
0	00…00	00…01 … 11…11	+Denormalized: $0.f \times 2^{(-b+1)}$
0	00…01 … 11…10	XX…XX	+Normalized: $1.f \times 2^{(e-b)}$

Operation	Result
n ÷ ±∞	0
±∞ × ±∞	±∞
±nonZero ÷ ±0	±∞

Operation	Result
±finite × ±∞	±∞
∞ + ∞ ، ∞ - (-∞)	+∞
-∞ - ∞ ، -∞ + (-∞)	-∞
∞ - ∞ ، -∞ + ∞	NaN
±0 ÷ ±0	NaN
±∞ ÷ ±∞	NaN
±∞ × 0	NaN
NaN == NaN	false

Sign	Exponent (e)	Fraction (f)	Value
0	11…11	00…00	+∞
0	11…11	00…01 … 11…11	NaN
1	00…00	00…00	-0
1	00…00	00…01 … 11…11	-Denormalized
1	00…01 … 11…10	XX…XX	-Normalized
1	11…11	00…00	-∞
1	11…11	00…01 … 11…11	NaN

چپ: عملیات با مقدارهای ویژه — راست: خلاصهٔ قالب IEEE 754 (b = بایاس). برای اطلاعات بیشتر: steve.hollasch.net/cgindex/coding/ieeefloat.html

تمرین ۴-۷ (حل‌شده): IEEE 754 دقت ساده

(الف) 0.15625، (ب) 85.125 و (ج) 0.1 را به IEEE 754 دقت ساده (هگز) تبدیل کنید. (د) `C2F00000h` چه عددی است؟

حل

(الف) $0.15625 = 0.00101_2 = 1.01 \times 2^{-3}$؛ نما $-3 + 127 = 124 = 01111100$؛ کسر $0100\ldots0 \Leftarrow$ `3E200000h` = 0 01111100 0100000….

(ب) $85.125 = 1010101.001_2 = 1.010101001 \times 2^{6}$؛ نما $133 = 10000101 \Leftarrow$ `42AA4000h` = 0 10000101 01010100100….

(ج) $0.1 = 1.100110011001\ldots_2 \times 2^{-4}$؛ نما $123 = 01111011$؛ کسر ۲۳ بیتی با گرد کردن به نزدیک‌ترین: $10011001100110011001101 \Leftarrow$ `3DCCCCCDh` (که دقیقاً 0.1 نیست: ≈ 0.100000001490116).

(د) 1 10000101 1110000… $\Leftarrow$ منفی؛ نما $133 - 127 = 6$؛ مانتیس $1.111_2 \Leftarrow -1.111 \times 2^{6} = -1111000_2 = -120$.

۴-۹ اعداد دهدهی و کدهای دهدهی

قالب مشهور نمایش اعداد دهدهی **BCD** (Binary Coded Decimal) است که در آن معادل دودویی **هر رقم** (۴ بیت) جداگانه نوشته می‌شود.

مثال اسلاید: BCD عدد 7089652

```
7 0 8 9 6 5 2 → 0111 0000 1000 1001 0110 0101 0010
```

(برای مقایسه، همین عدد در دودویی خالص $\lceil \log_2 7089652 \rceil = 23$ بیت لازم دارد ولی در BCD ۲۸ بیت.)

دسته‌بندی کدهای دهدهی: از نظر وزن: **وزن‌دار** (با وزن‌های همه‌مثبت مثل 8421 یا وزن‌های مختلط/منفی مثل 84-2-1) و **بی‌وزن** (مثل افزونی ۳ و گری)؛ از نظر طول: **۴ بیتی**، **۵ بیتی** و **بیشتر از ۵ بیت**.

رقم	8421 (BCD)	Excess-3	5421	84-2-1	7421	5311	631-1	2421
0	0000	0011	0000	0000	0000	0000	0011	0000
1	0001	0100	0001	0111	0001	0001	0010	0001
2	0010	0101	0010	0110	0010	0011	0101	0010
3	0011	0110	0011	0101	0011	0100	0111	0011
4	0100	0111	0100	0100	0100	0101	0110	0100
5	0101	1000	0101	1011	0101	1000	1001	1011
6	0110	1001	0110	1010	0110	1001	1000	1100
7	0111	1010	0111	1001	1000	1011	1010	1101
8	1000	1011	1011	1000	1001	1100	1101	1110
9	1001	1100	1100	1111	1010	1101	1100	1111

چند کد دهدهی ۴ بیتی. در کدهای وزن‌دار، مقدار رقم = جمع وزن بیت‌های ۱ (مثلاً در 84-2-1: 5 = 1 - 2 - 8 = 1011). Excess-3 = BCD + 3.

کد خودمکمل (Self-complementing)

کدی که در آن **مکمل ۹ هر رقم با NOT بیتی کد آن به دست می‌آید**؛ مثلاً در Excess-3: کد 3 = `0110` و NOT آن `1001` = کد 6 = 3 - 9. طبق اسلاید 631-1، Excess-3 و 2421 خودمکمل‌اند. (کد 84-2-1 هم خودمکمل است: `0111` (1) ↔ `1000` (8). شرط لازم برای خودمکمل بودن یک کد وزن‌دار این است که جمع وزن‌هایش ۹ باشد.) این ویژگی تفریق دهدهی را با سخت‌افزار ساده ممکن می‌کند.

رقم	2-out-of-5	63210	Shift-counter	86421	51111
0	00011	00001	00000	00000	00000
1	00101	00011	00001	00001	00001
2	00110	00101	00011	00010	00011

رقم	2-out-of-5	63210	Shift-counter	86421	51111
3	01001	01001	00111	00011	00111
4	01010	01010	01111	00100	01111
5	01100	01100	11111	00101	10000
6	10001	10001	11110	01000	11000
7	10010	10010	11100	01001	11100
8	10100	10100	11000	10000	11110
9	11000	11000	10000	10001	11111

چند کد دهدهی ۵ بیتی. کد 51111 خودمکمل است. در 2-out-of-5 هر کد دقیقاً دو بیت ۱ دارد؛ Shift-counter همان شمارندهٔ جانسون است.

رقم	50 43210 (biquinary)	543210	9876543210 (one-hot)
0	01 00001	000001	0000000001
1	01 00010	000010	0000000010
2	01 00100	000100	0000000100
3	01 01000	001000	0000001000
4	01 10000	010000	0000010000
5	10 00001	100001	0000100000
6	10 00010	100010	0001000000
7	10 00100	100100	0010000000
8	10 01000	101000	0100000000
9	10 10000	110000	1000000000

چند کد دهدهی با بیش از ۵ بیت

💡 شهود: هر رقم دهدهی جداگانه

در BCD هر رقم دهدهی جداگانه در ۴ بیت نگه داشته می‌شود. مزیت: کسرهای دهدهی مثل 0.1 (که در دودویی دقیق نمایش داده نمی‌شوند) دقیق می‌مانند و نمایش روی صفحه آسان است؛ برای همین در حساب پولی استفاده می‌شود (دستور AP در IBM 360). هزینه: ۶ الگو از ۱۶ الگوی هر رقم هدر می‌رود و جمع نیاز به اصلاح (+6) دارد.

۱-۹-۴ کد گری (Gray code)

رقم	BCD 8421	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101

ویژگی: کدهای دو عدد متوالی فقط در **یک بیت** اختلاف دارند؛ برای همین در رمزگذارهای موقعیت (encoder) و انتقال داده میان حوزه‌های کلاک استفاده می‌شود تا در لحظهٔ تغییر، مقدار نادرست خوانده نشود.

تبدیل دودویی به گری: $g_{n-1} = b_{n-1}$ و $g_i = b_{i+1} \oplus b_i$. مثال: 1001 (9) → 1, 1⊕0, 0⊕0, 0⊕1 = 1101 ✓.

تبدیل گری به دودویی: $b_{n-1} = g_{n-1}$ و $b_i = b_{i+1} \oplus g_i$.

تمرین ۴-۸ (حل‌شده): کدهای دهدهی

(الف) عدد 395 را در Excess-3 بنویسید. (ب) جمع BCD 15 + 27 را انجام دهید. (ج) چرا کد out-of-5-2 خطای تک‌بیتی را آشکار می‌کند؟

حل

(الف) هر رقم + 3: 5→1000، 9→1100، 3→0110 ⇐ 1000 1100 0110.

(ب) 0011 1100 = 0001 0101 + 0010 0111. رقم کم‌ارزش 1100 (12) بزرگ‌تر از ۹ است، پس 6 (0110) به آن اضافه می‌کنیم: 1100 + 0110 = 1 0010 و نقلی به رقم بعد: 0011 + 1 = 0100 ⇐ 0100 0010 = 42 ✓. (قاعده: اگر رقمی بیش از ۹ شد یا نقلی تولید کرد، ۶ اضافه کن.)

(ج) هر کد معتبر دقیقاً دو بیت ۱ دارد؛ تغییر هر یک بیت، تعداد ۱ها را به ۱ یا ۳ می‌رساند که کد نامعتبر است و آشکار می‌شود.

💡 شهود: فقط یک بیت در هر قدم

یک چرخ رمزگذار را تصور کنید که موقعیتش با چند حسگر خوانده می‌شود. اگر از 0111 به 1000 برویم، چهار بیت باید **دقیقاً هم‌زمان** عوض شوند؛ اگر حسگرها لحظه‌ای ناهم‌زمان بخوانند، ممکن است عدد کاملاً غلطی مثل 1111 دیده شود. در کد گری بین دو عدد متوالی فقط یک بیت عوض می‌شود، پس خطای لحظهٔ گذار حداکثر یک واحد است.

۴-۱۰ کدهای کاراکتری

برای نمایش و ذخیرهٔ داده‌های الفبایی از کدهای کاراکتری استفاده می‌شود.

LSBs \ MSBs	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

جدول کد ASCII هفت‌بیتی: ستون = ۳ بیت پرارزش، سطر = ۴ بیت کم‌ارزش. مثلاً 'A' = 100 0001 = 65 = 41h

- **ASCII** (American Standard Code for Information Interchange): کدی ۸ بیتی (۷ بیت کد + ۱ بیت توازن) که در کامپیوترهای شخصی رایج است.
- **EBCDIC** (Extended Binary Coded Decimal Interchange Code): کد کاراکتری ۸ بیتی استاندارد IBM (در خانوادهٔ IBM 360/370 استفاده می‌شود).
- **یونی‌کد (Unicode)**: برای نمایش همهٔ کاراکترهای همهٔ زبان‌ها؛ با قالب‌های UTF-8 (واحدهای ۸ بیتی)، UTF-16 و UTF-32 کدگذاری می‌شود. (در اسلاید به اشتباه UFT نوشته شده است.)

تمرین ۴-۹ (حل‌شده): کدهای کاراکتری

(الف) رشتهٔ «CSL» را در ASCII (هگز) بنویسید. (ب) تفاوت کد 'a' و 'A' چیست و چگونه حروف کوچک و بزرگ را تبدیل کنیم؟ (ج) کد کاراکتر '7' را به مقدار عددی 7 تبدیل کنید. (د) بیت توازن زوج را برای 'A' و 'C' اضافه کنید. (هـ)

حرف فارسی «س» (U+0633) در UTF-8 چه بایت‌هایی است؟

حل

(الف) C = 43h، S = 53h، L = 4Ch ⇐ `43 53 4C`.

(ب) 'a' = 61h = 97 و 'A' = 41h = 65؛ اختلاف ۳۲ یعنی فقط **بیت ۵**. با OR 20h حرف کوچک، با AND DFh حرف بزرگ و با XOR 20h تغییر حالت به دست می‌آید.

(ج) '7' = 37h؛ مقدار = 37h - 30h = 7 (یا AND 0Fh).

(د) 'A' = 1000001 دو بیت ۱ دارد ⇐ توازن 0 ⇐ `01000001`؛ 'C' = 1000011 سه بیت ۱ دارد ⇐ توازن 1 ⇐ `11000011`.

(هـ) 0633h = 000 0110 0011 0011 (۱۱ بیت معنادار) ⇐ قالب دوبایتی `110xxxxx 10xxxxxx`: **110**11000 **10**110011 = `D8 B3`.

خلاصهٔ فصل ۴

- n بیت ⇐ 2^n مقدار؛ M مقدار ⇐ $\lceil \log_2 M \rceil$ بیت. تبدیل به دهدهی با جمع وزن‌ها؛ از دهدهی با تقسیم متوالی (صحیح) و ضرب متوالی (کسری)؛ میان مبناهای توان ۲ با گروه‌بندی بیت‌ها.
- علامت-مقدار و مکمل یک: دو صفر و گسترهٔ $\pm(2^{n-1}-1)$؛ مکمل دو: یک صفر و گسترهٔ $-2^{n-1}\dots 2^{n-1}-1$؛ جمع مکمل یک با نقلی چرخشی و مکمل دو با دور ریختن نقلی؛ سرریز: هم‌علامت‌ها با نتیجهٔ ناهم‌علامت (یا Cin ≠ Cout در MSB).
- افزونی: کد = مقدار + K؛ برای توزیع یکنواخت $K = 2^{n-1}$ یا $2^{n-1}-1$.
- ممیز شناور: $(-1)^S \times 0.M \times 2^{E-Bias}$ با مانتیس نرمال؛ در جمع، عدد کوچک‌تر قربانی می‌شود؛ صفر باید کوچک‌ترین نما را داشته باشد.
- IEEE 754: $1.F \times 2^{E-127}$ (ساده) و 2^{E-1023} (مضاعف)؛ مقدارهای ویژه: غیرنرمال، ±0، ±∞ و NaN.
- کدهای دهدهی: Excess-3، BCD، کدهای وزن‌دار، خودمکمل، out-of-5-2، گری؛ کدهای کاراکتری: ASCII، EBCDIC و یونی‌کد.

فصل ۵

آدرس‌دهی نسبی، قطعه‌ای و صفحه‌ای

در این فصل (جلسهٔ پنجم) سه روش آدرس‌دهی دیگر را می‌بینیم: **نسبی** (نسبت به PC)، **قطعه‌ای** (با ثبات قطعه و جمع) و **صفحه‌ای** (با ثبات صفحه و الحاق بیت‌ها). با **ماشین نمونهٔ ۷** (تک‌آدرسه با پرش‌های نسبی) یک برنامهٔ **مرتب‌سازی** کامل را به کد ماشین ترجمه می‌کنیم و تکلیف **جمع ۱۰۰ عدد اول** را حل می‌کنیم. در پایان، مسئلهٔ **جابه‌جایی برنامه در حافظه** و **پشتیبانی چندکاربره** را بررسی می‌کنیم.

۵-۱ آدرس‌دهی نسبی (Relative)

تاکنون پنج روش دیدیم: مستقیم، غیرمستقیم، بی‌واسطه، شاخص‌دار و ضمنی. روش ششم:

۶. آدرس‌دهی نسبی

آدرس با **جمع جابه‌جایی (displacement) موجود در IF با محتوای PC** محاسبه می‌شود: EA = (PC) + displacement. دلیل: **بیشتر پرش‌ها به محلی نزدیک به خود دستور پرش انجام می‌شوند** (حلقه‌ها و if‌ها)؛ پس به‌جای نگه‌داشتن کل آدرس مقصد در IF، فقط جابه‌جایی نگه داشته می‌شود که **بیت‌های کمتری** لازم دارد.

مثال

`jcxz loop_addr` در پردازندهٔ 8086/88: اگر (cx) = 0 باشد، یک عدد ۸ بیتی موجود در IF با PC جمع می‌شود.

نکته: جابه‌جایی ۸ بیتی برچسب `loop_addr` را **اسمبلر** با استفاده از آدرس مقصد پرش و آدرس محل فعلی تولید می‌کند.

محاسبهٔ جابه‌جایی

هنگام اجرای دستور، PC قبلاً به **دستور بعدی** اشاره می‌کند (چون واکشی تمام شده است). بنابراین:

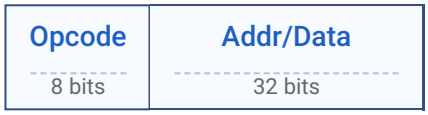
$$\text{displacement} = \text{آدرس مقصد} - \text{آدرس دستور بعد از پرش}$$

جابه‌جایی یک عدد علامت‌دار (مکمل دو) است؛ با ۸ بیت، پرش‌ها فقط در بازهٔ 128- تا 127+ بایت نسبت به دستور بعدی ممکن‌اند. برای پرش‌های دورتر باید از پرش مطلق استفاده کرد. مزیت دیگر: کدی که فقط پرش نسبی دارد **مستقل از مکان (position independent)** است و اگر جای دیگری از حافظه بار شود، پرش‌ها نیازی به اصلاح ندارند.

> **💡 شهود: «سه خانه جلوتر» در برابر «پلاک ۱۲۰»**
>
> اگر به کسی بگویید «به پلاک ۱۲۰ برو»، با جابه‌جا شدن محله دستور غلط می‌شود؛ ولی «سه خانه جلوتر برو» هرجا که باشید درست است. نشانی نسبی (PC + جابه‌جایی) همین است: برنامه را هر جای حافظه بگذاریم، پرش‌ها درست می‌مانند (کد مستقل از مکان). چون بیشتر پرش‌ها به نزدیکی خود دستورند، جابه‌جایی کوچک کافی است و دستور کوتاه می‌شود.

۵-۲ ماشین نمونهٔ ۷

- ماشین تک‌آدرسه؛ حافظهٔ اصلی 2^{32} واحد آدرس‌پذیر ۸ بیتی؛
- کلمه ۳۲ بیت، ناهم‌تراز، Big Endian؛ نمایش اعداد: **مکمل دو**؛
- روش‌های آدرس‌دهی: حافظه‌ای مستقیم و غیرمستقیم، بی‌واسطه و **نسبی**؛ دو قالب دستور:



قالب I (۵ بایت)



قالب II (۲ بایت)

قالب	Opcode	Mnemonic	Operation
I	`00000000`	`load addr`	ACC ← (M_{addr})
	`00000001`	`load (addr)`	ACC ← $(M_{(M_{addr})})$
	`00000010`	`store addr`	M_{addr} ← (ACC)
	`00000011`	`store (addr)`	$M_{(M_{addr})}$ ← (ACC)
	`00000100`	`add addr`	ACC ← (ACC) + (M_{addr})
	`00000101`	`sub addr`	ACC ← (ACC) - (M_{addr})
	`00000110`	`addi data`	ACC ← (ACC) + data
	`00000111`	`subi data`	ACC ← (ACC) - data
	`00001000`	`jmpl addr`	PC ← addr (پرش مطلق/دور)
II	`10000000`	`jnz addr`	if (ACC) ≠ 0 then PC ← (PC) + displ.
	`10000001`	`jz addr`	if (ACC) = 0 then PC ← (PC) + displ.
	`10000010`	`jneg addr`	if (ACC) < 0 then PC ← (PC) + displ.
	`10000011`	`jpos addr`	if (ACC) ≥ 0 then PC ← (PC) + displ.
	`10000100`	`jmp addr`	PC ← (PC) + displ.

در قالب اسمبلی، برنامه‌نویس آدرس مقصد (برچسب) را می‌نویسد و اسمبلر آن را به جابه‌جایی تبدیل می‌کند. توجه کنید که `jpos` در این ماشین برای ACC ≥ 0 (شامل صفر) پرش می‌کند.

پرسش اسلاید: طول ثبات‌های ماشین ۷

پاسخ

$L_{MAR} = 32$، $L_{MBR} = 32$، $L_{ACC} = 32$، $L_{IR} = 40$ (قالب I) و $L_{PC} = 32$ بیت.

تمرین‌های اسلاید برای این ماشین: (۱) برنامه‌ای برای مرتب کردن یک آرایهٔ ۱۰۰ عنصری و ترجمهٔ آن به کد ماشین؛ (۲) برنامه‌ای برای جمع ۱۰۰ عدد اول و ترجمهٔ آن (تکلیف).

۵-۲-۱ مرتب‌سازی Gnome

از مرتب‌سازی **Gnome** (که در اصل «مرتب‌سازی احمقانه»، Stupid sort، نام داشت) استفاده می‌کنیم چون کدنویسی آن کار کمتری لازم دارد: **تنها الگوریتم مرتب‌سازی غیربازگشتی است که فقط یک حلقه دارد!** در نماد زبان Pascal:

```
program my_sort;
var  i: integer;  A: Array [1..100] of integer;
i := 1;
while i < n do
    if A[i] > A[i+1] then
        begin swap(A[i], A[i+1]);
              if i > 1 then i := i-1;
        end
    else i := i+1;
end.
```

ایده: مثل یک کوتوله (gnome) که گلدان‌ها را مرتب می‌کند: اگر دو عنصر کنار هم مرتب‌اند یک قدم جلو برو؛ اگر نه، جایشان را عوض کن و یک قدم عقب برگرد (تا عنصر جابه‌جاشده را با عنصر قبلی هم مقایسه کنی). در عمل مثل مرتب‌سازی درجی است.

مثال ۵-۱: ردگیری روی A = [3, 1, 2] (n = 3)

i	مقایسه	عمل	پس از آن A
1	3 > 1 بله؟	swap؛ i = 1 می‌ماند	[1, 3, 2]
1	1 > 3 خیر؟	i = 2	[1, 3, 2]
2	3 > 2 بله؟	swap؛ i = 1	[1, 2, 3]
1	1 > 2 خیر؟	i = 2	[1, 2, 3]
2	2 > 3 خیر؟	پایان ⇐ i = 3 = n	[1, 2, 3]

💡 شهود: کوتولهٔ باغبان

کوتوله‌ای جلوی ردیف گلدان‌ها ایستاده است: اگر گلدان جلویی و گلدان کنارش مرتب باشند یک قدم جلو می‌رود؛ وگرنه آن دو را جابه‌جا می‌کند و یک قدم عقب می‌رود (تا ببیند جابه‌جایی ترتیب قبلی را خراب نکرده باشد). وقتی به انتهای ردیف برسد، همه مرتب‌اند. چون فقط یک اندیس (جای کوتوله) دارد، تنها یک حلقه لازم است؛ همین آن را برای اسمبلی مناسب می‌کند (در فصل ۸ نسخهٔ IBM 360 آن را هم می‌بینیم).

۵-۲-۲ برنامهٔ اسمبلی مرتب‌سازی (صعودی) در ماشین ۷

ایدهٔ پیاده‌سازی

- به‌جای اندیس i، اشاره‌گر `p` را نگه می‌داریم که به A[i] اشاره می‌کند (ابتدا `p = array`).
- در هر دور: temp1 ← A[i]؛ p یک کلمه (۴ بایت) جلو می‌رود؛ temp2 ← A[i+1]؛ سپس ACC = A[i+1] - A[i]. اگر 0 ≥ بود (یعنی مرتب‌اند) به `continue` می‌رویم؛ چون p از قبل به A[i+1] اشاره می‌کند، این همان i := i + 1 است.
- در غیر این صورت جابه‌جایی: temp1 در A[i+1] و temp2 در A[i] نوشته می‌شود؛ سپس p دو بار ۴ تا کم می‌شود که یعنی i := i - 1.
- **ترفند نگهبان (sentinel):** به‌جای بررسی شرط i > 1، درست قبل از آرایه یک کلمه با مقدار `80000000h` (کوچک‌ترین عدد ۳۲ بیتی مکمل دو) قرار داده شده است. اگر i به 0 برسد، مقایسهٔ این نگهبان با A[1] باید همیشه «مرتب» باشد و i دوباره ۱ می‌شود.
- پایان: در `continue` بررسی می‌شود که آیا p به `array+396` (آدرس آخرین عنصر، 99×4 = 396) رسیده است یا نه؛ اگر رسیده باشد، i = n و کار تمام است.

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	

Address	Machine Code	Label	Instr.	Operands	توضیح
00000100	01 0000016F	loop:	load	(p)	ACC = A[i]
00000105	02 00000173		store	temp1	temp1 = A[i]
0000010A	00 0000016F		load	p	
0000010F	06 00000004		addi	4	
00000114	02 0000016F		store	p	p → A[i+1]
00000119	01 0000016F		load	(p)	ACC = A[i+1]
0000011E	02 00000177		store	temp2	temp2 = A[i+1]
00000123	05 00000173		sub	temp1	ACC = A[i+1] - A[i]
00000128	83 34		jpos	continue	مرتب (ACC ≥ 0) ⇐ i = i+1 ؛ displ = 15E - 12A = 34h
0000012A	00 00000173		load	temp1	؛ جابه‌جایی عناصر
0000012F	03 0000016F		store	(p)	A[i+1] = temp1
00000134	00 0000016F		load	p	
00000139	07 00000004		subi	4	
0000013E	02 0000016F		store	p	p → A[i]
00000143	00 00000177		load	temp2	
00000148	03 0000016F		store	(p)	A[i] = temp2
0000014D	00 0000016F		load	p	
00000152	07 00000004		subi	4	
00000157	02 0000016F		store	p	A[i-1] ← p یعنی i = i-1
0000015C	84 A2		jmp	loop	displ = 100 - 15E = -5Eh = A2h
0000015E	00 0000016F	continue:	load	p	
00000163	07 0000030B		subi	array+396	ACC = p - (array+396) ؛ 17F + 18C = 30B
00000168	80 96		jnz	loop	displ = 100 - 16A = -6Ah = 96h
0000016A	08 ????????		jmpl	OS_return_point	بازگشت به سیستم‌عامل
0000016F	0000017F	p:	dw	array	
00000173	????????	temp1:	dw	?	
00000177	????????	temp2:	dw	?	
0000017B	80000000		dw	80000000h	نگهبان: کوچک‌ترین عدد
0000017F	????????…	array:	dw	100 dup(?)	۴۰۰ بایت
			end		

Symbol Table	
loop	00000100h
continue	0000015Eh
p	0000016Fh
temp1	00000173h
temp2	00000177h
array	0000017Fh

محاسبهٔ جابه‌جایی‌ها (با مکمل دو ۸ بیتی)

- `jpos continue` در 128h و دستور بعدی در 12Ah: 15Eh - 12Ah = +34h ⇐ `83 34`.
- `jmp loop` در 15Ch، دستور بعدی 15Eh: 100h - 15Eh = -5Eh؛ مکمل دو ۸ بیتی: 100h - 5Eh = A2h ⇐ `84 A2`.
- `jnz loop` در 168h، دستور بعدی 16Ah: 100h - 16Ah = -6Ah ⇐ 96h ⇐ `80 96`.
- همهٔ این جابه‌جایی‌ها در بازهٔ +127 … -128 هستند (بیشترین فاصله -106).

توجه ۱ (خطای تایپی اسلاید)

در اسلاید، کد ماشین دستور `load temp1` در آدرس 12Ah به صورت `0100000173` نوشته شده است؛ `01` کد عمل `load (addr)` (غیرمستقیم) است. چون `load temp1` مستقیم است، کد درست **`00`**`0000000173` است (مانند `load temp2` در 143h). همچنین کد ماشین سطرهای 14D تا 16A در اسلاید ناقص بود که در جدول بالا کامل شده است.

توجه ۲ (نکتهٔ تکمیلی): سرریز در مقایسه با تفریق

مقایسه با تفریق A[i] - A[i+1] فقط وقتی درست است که تفریق سرریز نکند. وقتی p به نگهبان برمی‌گردد، A[1] - 80000000h محاسبه می‌شود؛ اگر A[1] ≥ 0 باشد این تفریق در ۳۲ بیت **سرریز** می‌کند و نتیجه منفی به نظر می‌رسد ⇐ نگهبان با A[1] جابه‌جا می‌شود و آرایه خراب می‌شود (این را با شبیه‌سازی آزمودیم: برای آرایهٔ تمام‌منفی درست کار می‌کند ولی برای آرایهٔ تمام‌مثبت نه).

اصلاح: (الف) اگر بدانیم $|A[i]| < 2^{30}$، نگهبان را `C0000000h` (-2^{30}) بگذاریم؛ آن‌گاه هیچ تفریقی سرریز نمی‌کند (آزموده شد). (ب) یا نگهبان را حذف کنیم و پیش از کم کردن p بررسی کنیم که p برابر array نباشد:

```
        load  p          ; (پس از نوشتن A[i] = temp2)
        subi  array      ; آیا i = 1 ؟
        jz    loop       ; بله: i بماند همان 1
        load  p
        subi  4
        store p          ; i = i - 1
        jmp   loop
```

تکلیف: برنامه‌ای برای جمع ۱۰۰ عدد اول بنویسید و آن را به کد ماشین ترجمه کنید.

تحلیل و انتخاب الگوریتم

ماشین ۷ ضرب و تقسیم ندارد؛ پس آزمون اول بودن با تقسیم‌های آزمایشی (که به باقی‌مانده نیاز دارد) پرهزینه است. **غربال اراتستن** فقط به جمع نیاز دارد: مضرب‌های هر عدد اول را با جمع‌های متوالی علامت می‌زنیم. می‌دانیم صدمین عدد اول **541** است، پس آرایهٔ `flags` با ۵۴۲ کلمه (اندیس ۰ تا ۵۴۱) کافی است. flags[k] = 0 یعنی k هنوز علامت نخورده است.

- `i` عدد جاری (از ۲) و `ip` اشاره‌گر به flags[i] (آدرس flags + 4i) است.
- اگر flags[i] = 0: i اول است ⇐ sum += i؛ count -= 1 (از ۱۰۰ به پایین)؛ اگر count صفر شد کار تمام است. سپس step = 4i (با سه جمع: i+i، و دو برابر آن) و از jp = ip + step (یعنی flags[2i]) تا انتهای آرایه هر خانه را علامت می‌زنیم و jp را step تا جلو می‌بریم.
- سپس i و ip را یکی (۴ بایت) جلو می‌بریم و تکرار می‌کنیم.

Address	Machine Code	Label	Instr.	Operands	توضیح
			org	100h	
00000100	01 00000194	next:	load	(ip)	ACC = flags[i]
00000105	80 65		jnz	notprime	علامت خورده ⇐ اول نیست ؛ displ = 16C - 107
00000107	00 000001A4		load	sum	
0000010C	04 00000198		add	i	
00000111	02 000001A4		store	sum	sum = sum + i
00000116	00 000001A8		load	count	
0000011B	07 00000001		subi	1	
00000120	02 000001A8		store	count	count = count - 1
00000125	81 68		jz	done	۱۰۰ عدد اول جمع شد ؛ displ = 18F - 127
00000127	00 00000198		load	i	

Address	Machine Code	Label	Instr.	Operands	توضیح
0000012C	04 00000198		add	i	ACC = 2i
00000131	02 0000019C		store	step	
00000136	04 0000019C		add	step	ACC = 4i
0000013B	02 0000019C		store	step	step = 4i (فاصلهٔ دو مضرب متوالی بر حسب بایت)
00000140	00 00000194		load	ip	
00000145	04 0000019C		add	step	ACC = آدرس flags[2i]
0000014A	02 000001A0	mark:	store	jp	
0000014F	07 00000A28		subi	flags+2168	jp - (انتهای آرایه)
00000154	83 16		jpos	notprime	از آرایه گذشت ⇐ پایان علامت‌زنی
00000156	00 000001AC		load	one	
0000015B	03 000001A0		store	(jp)	علامت زدن مضرب i ؛ flags[j] = 1
00000160	00 000001A0		load	jp	
00000165	04 0000019C		add	step	مضرب بعدی
0000016A	84 DE		jmp	mark	displ = 14A - 16C = -22h
0000016C	00 00000194	notprime:	load	ip	
00000171	06 00000004		addi	4	
00000176	02 00000194		store	ip	ip → flags[i+1]
0000017B	00 00000198		load	i	
00000180	06 00000001		addi	1	
00000185	02 00000198		store	i	i = i + 1
0000018A	08 00000100		jmpl	next	پرش مطلق (فاصله 140- از بازهٔ پرش نسبی بیرون است)
0000018F	08 ????????	done:	jmpl	OS_return_point	
00000194	000001B8	ip:	dw	flags+8	اشاره‌گر به flags[2]
00000198	00000002	i:	dw	2	
0000019C	????????	step:	dw	?	
000001A0	????????	jp:	dw	?	
000001A4	00000000	sum:	dw	0	
000001A8	00000064	count:	dw	100	
000001AC	00000001	one:	dw	1	
000001B0	00000000…	flags:	dw	542 dup(0)	۵۴۲ کلمه (۲۱۶۸ بایت = 878h)
			end		

نکته‌های ترجمه

- آدرس‌ها: flags = 1B0h، ip_0 = flags + 8 = 1B8h، انتهای آرایه flags + 2168 = 1B0h + 878h = A28h.
- جابه‌جایی‌ها: `jnz notprime` : 16Ch − 107h = 65h؛ `jz done` : 18Fh − 127h = 68h؛ `jpos notprime` : 16h = 16Ch − 156h؛ `jmp mark` : DEh = −22h = 16Ch − 14Ah.
- پرش از انتهای حلقه به `next` فاصلهٔ 100h − 18Ch = −140 دارد که با ۸ بیت قابل نمایش نیست؛ پس از `jmpl` (قالب ا، پرش مطلق) استفاده شد. این محدودیت آدرس‌دهی نسبی را نشان می‌دهد.
- شرط `jpos` برای ACC ≥ 0 برقرار است؛ پس وقتی jp ≥ flags + 2168 (یعنی بیرون از آرایه) شد، علامت‌زنی متوقف می‌شود.

نتیجه: برنامه با شبیه‌ساز ماشین ۷ اجرا شد؛ پس از ۱۴۵۲۰ دستور متوقف شد و sum = 24133 (مجموع ۱۰۰ عدد اول ۲، ۳، ۵، ...، ۵۴۱) و i = 541 بود.

۵-۳ آدرس‌دهی قطعه‌ای (Segment)

۷. آدرس‌دهی قطعه‌ای

آدرس نهایی (که گاهی **آدرس فیزیکی** نامیده می‌شود) با **جمع** آدرس محاسبه‌شده از روی فیلدهای IF (که گاهی **آدرس منطقی** نامیده می‌شود) و **محتوای ثبات قطعه (Segment Register)** به دست می‌آید.

۵-۳-۱ مسئلهٔ جابه‌جایی برنامه در حافظه

فرض کنید برنامهٔ اسمبلی (بخشی از برنامهٔ مرتب‌سازی، در نسخهٔ اسلاید) با `org 100h` اسمبل و پیوند شده و فایل اجرایی آن آماده است. **اگر بتوان کد دودویی را از آدرس 00000100h بار کرد** (این خانه آزاد باشد)، بارکننده آن را عیناً از همان آدرس در حافظه قرار می‌دهد و همه چیز درست است.

اما **اگر خانهٔ 00000100h قبلاً اشغال شده باشد** و برنامه را فقط بتوان از آدرس 00050100h بار کرد چه؟ آن‌گاه همهٔ آدرس‌های داخل برنامه (مثل آدرس p، temp1 و ...) اشتباه خواهند بود. **بارکنندهٔ جابه‌جاکننده (Relocating loader)** این مشکل را حل می‌کند: **همهٔ آدرس‌های برنامه را به اندازهٔ 50000h = 50100h − 100h افزایش می‌دهد.**

دستور	کد اصلی (org 100h)	پس از جابه‌جایی (+50000h)
load (p)	`01 00000179`	`01 00050179`
store temp1	`02 0000017D`	`02 0005017D`
load p	`00 00000179`	`00 00050179`
addi 4	`06 00000004`	`06 00000004` (بدون تغییر!)
store p	`02 00000179`	`02 00050179`

دستور	کد اصلی (org 100h)		پس از جابه‌جایی (+50000h)
store temp2	02 00000181	02 00050181	
jpos continue	83 0000015E	83 0005015E	
subi 4	07 00000004	07 00000004 (بدون تغییر!)	
...	...		...

(در این نسخهٔ اسلاید، پرش jpos با آدرس مطلق ۳۲ بیتی کد شده است و متغیرها در 17Dh، 179h و 181h هستند.)

مراقبت ویژه برای داده‌های بی‌واسطه!

عدد 4 در `addi 4` و `subi 4` آدرس نیست و **نباید** تغییر کند. اما بارکننده از روی الگوی بیتی نمی‌تواند تشخیص دهد کدام فیلد آدرس است و کدام عدد؛ این اطلاعات را باید اسمبلر در **اطلاعات جابه‌جایی** فایل شیء/اجرایی ثبت کند.

پرسش‌های اسلاید

Q1. تکلیف آدرس‌هایی که به عنوان دادهٔ بی‌واسطه استفاده می‌شوند چیست؟

پاسخ

گاهی دادهٔ بی‌واسطه خودش یک آدرس است؛ مثلاً `subi array+396` در برنامهٔ مرتب‌سازی یا `mov R0,#A` در ماشین ۴. این مقدارها **باید** جابه‌جا شوند (وگرنه شرط پایان حلقه یا اشاره‌گرها اشتباه می‌شوند)، در حالی که `addi 4` نباید تغییر کند. در هر دو حالت کد عمل یکی است؛ پس فقط اسمبلر (که می‌داند عملوند از یک برچسب ساخته شده است) می‌تواند در رکوردهای جابه‌جایی مشخص کند کدام فیلد بی‌واسطه «آدرس» است.

Q2. تکلیف آدرس‌های غیرمستقیم چیست؟

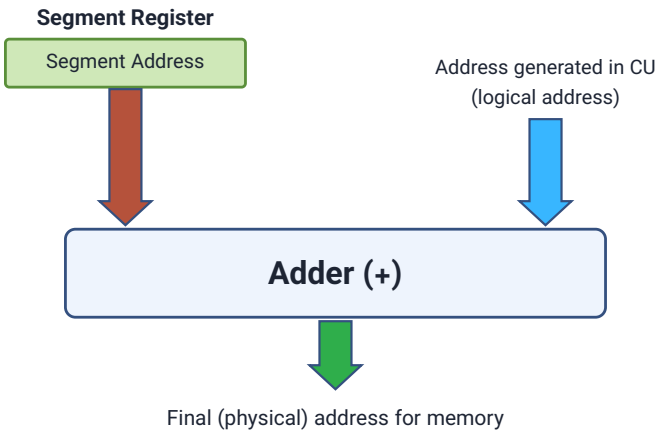
پاسخ

در آدرس‌دهی غیرمستقیم، آدرس واقعی عملوند **داخل داده‌ها** ذخیره شده است؛ مثلاً `p: dw array` مقدار 17Fh را دارد. اگر فقط فیلدهای آدرس دستورها جابه‌جا شوند، `load (p)` به خانهٔ درست p می‌رود ولی محتوای p هنوز آدرس قدیمی آرایه است! پس **کلمه‌های داده‌ای که آدرس نگه می‌دارند (اشاره‌گرها) هم باید جابه‌جا شوند** و این هم به اطلاعات اسمبلر نیاز دارد. بدتر از آن، آدرس‌هایی که در زمان اجرا ساخته می‌شوند (مثلاً از ورودی خوانده می‌شوند یا از عدد ثابت ساخته می‌شوند) اصلاً قابل جابه‌جایی توسط بارکننده نیستند.

۵-۳-۲ راه‌حل: ثبات قطعه

با آدرس‌دهی قطعه‌ای، کد تغییر نمی‌کند و بارکنندهٔ جابه‌جاکننده فقط **ثبات قطعه** داخل CU را با مقدار مناسب (اینجا `00050000h`) بار می‌کند. هر آدرسی که CU تولید می‌کند (آدرس منطقی، از هر روشی: مستقیم، غیرمستقیم، اشاره‌گر یا

محاسبه‌شده در زمان اجرا) پیش از رفتن به حافظه با ثبات قطعه جمع می‌شود. بنابراین هر دو مشکل Q1 و Q2 خودبه‌خود حل می‌شوند.



شکل: آدرس‌دهی قطعه‌ای — آدرس فیزیکی = محتوای ثبات قطعه + آدرس منطقی

مثبت

سیستم‌عامل (شامل پیونددهنده و بارکننده) برای مدیریت برنامه‌های کاربران دردسر کمتری دارد؛ برای برنامه‌نویسان/گردانندگان سیستم خوب است ☺.

منفی

برای هر آدرس حافظه‌ای که تولید می‌شود باید یک عمل جمع انجام شود! این باعث طولانی‌تر شدن زمان اجرای برنامه می‌شود ☹.

مثال ۵-۲

ثبات قطعه `00050000h` است. دستور `load (p)` با p = 0000016Fh اجرا می‌شود و محتوای p برابر `0000017Fh` است. چه آدرس‌های فیزیکی‌ای دسترسی می‌شوند؟

حل

ابتدا خود p: 0000016Fh + 00050000h = 0005016Fh. محتوای آن (آدرس منطقی 17Fh) دوباره با ثبات قطعه جمع می‌شود: 0005017Fh؛ پس ACC محتوای خانهٔ فیزیکی `0005017Fh` را می‌گیرد. (بدون هیچ تغییری در کد یا داده.)

پیش‌نمایش 8086: در پردازندهٔ 8086 آدرس فیزیکی = segment × 16 + offset است؛ مثلاً 1234h:0010h → 12340h + 0010h = 12350h. در فصل‌های بعدی درس با آن آشنا می‌شویم.

💡 شهود: نشانی ساختمان و شمارهٔ واحد

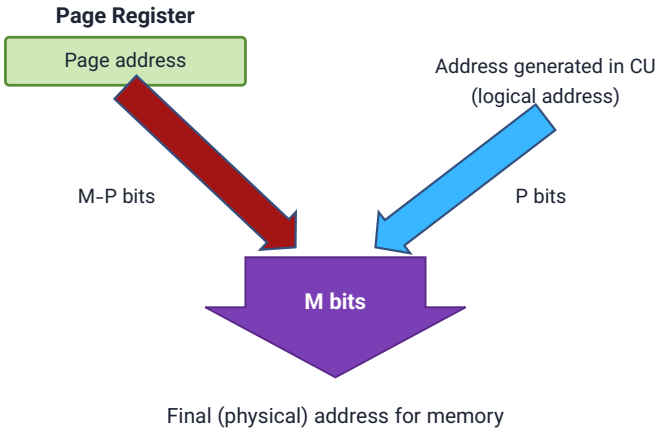
نشانی‌های داخل برنامه مثل «شمارهٔ واحد» در یک ساختمان‌اند و ثبات قطعه «نشانی خود ساختمان». اگر برنامه به ساختمان دیگری منتقل شود، فقط نشانی ساختمان (ثبات قطعه) عوض می‌شود و همهٔ شماره‌واحدها درست می‌مانند. همین ایده در IBM 360 به شکل **ثبات پایه** آمده است (فصل ۶).

۵-۴ آدرس‌دهی صفحه‌ای (Page)

۸. آدرس‌دهی صفحه‌ای

آدرس نهایی (فیزیکی) با **الحاق (concatenation)** آدرس داده‌شده در IF (آدرس منطقی) و محتوای **ثبات صفحه (Page Register)** ساخته می‌شود.

برای صفحه‌هایی با اندازهٔ 2^P کلمه در یک حافظهٔ اصلی با 2^M کلمه: ثبات صفحه M - P بیت (شمارهٔ صفحه) دارد و آدرس تولیدشده در CU، P بیت (شمارهٔ کلمه در صفحه). کنار هم گذاشتن آن‌ها آدرس M بیتی می‌سازد. حافظه به 2^{M-P} صفحه (Page 0 تا Page 2^{M-P}-1) تقسیم می‌شود.



شکل: آدرس‌دهی صفحه‌ای — آدرس فیزیکی = (ثبات صفحه) ‖ (آدرس منطقی)؛ الحاق بیت‌ها، بدون جمع

مزیت

نیازی به جمع آدرس نیست (فقط سیم‌کشی کنار هم). پس همهٔ مزایای آدرس‌دهی قطعه‌ای را بدون عیب آن داریم ☺.

عیب

قطعه‌ها فقط می‌توانند از آدرس‌های مضرب 2^P شروع شوند و کمترین طول قطعه 2^P کلمه است ☹. اگر برنامه به بیش از 2^P کلمه نیاز داشته باشد چه؟ چند صفحه... مدیریتش آسان نیست ☹.

تمرین‌های حل‌شدهٔ آدرس‌دهی قطعه‌ای و صفحه‌ای

۱. در حافظه‌ای با 2^{16} کلمه و صفحه‌های ۱۰۲۴ کلمه‌ای، ثبات صفحه `000101` و آدرس منطقی `0A3h` است. آدرس فیزیکی چیست؟

حل

P = 10، M - P = 6. آدرس منطقی ۱۰ بیتی: `00 1010 0011`. الحاق: `14A3h` = `0001 0100 1010 0011` = `0010100011` | `000101` (معادل 163 + 1024 × 5).

۲. حافظه 2^{20} کلمه و صفحه‌ها ۴K کلمه‌ای هستند. طول ثبات صفحه، طول فیلد آدرس منطقی و تعداد صفحه‌ها چقدر است؟ برنامه‌ای با ۱۰K کلمه چند صفحه لازم دارد و چقدر هدر می‌رود؟

حل

P = 12 ⇐ آدرس منطقی ۱۲ بیت؛ ثبات صفحه 8 = 12 - 20 بیت؛ 2^8 = 256 صفحه. برنامهٔ ۱۰K کلمه‌ای ۳ صفحه (۱۲K) می‌گیرد و ۲K کلمه هدر می‌رود. همچنین چون آدرس منطقی فقط ۱۲ بیت است، برنامه هنگام عبور از یک صفحه به صفحهٔ بعد باید ثبات صفحه را عوض کند (همان «مدیریت دشوار»).

۳. در آدرس‌دهی قطعه‌ای ۳۲ بیتی، ثبات قطعه `0003A000h` و آدرس منطقی `00001234h` است. آدرس فیزیکی؟ اگر به‌جای آن صفحه‌بندی با صفحهٔ ۶۴K بایتی داشتیم و ثبات صفحه `0003h` بود چطور؟

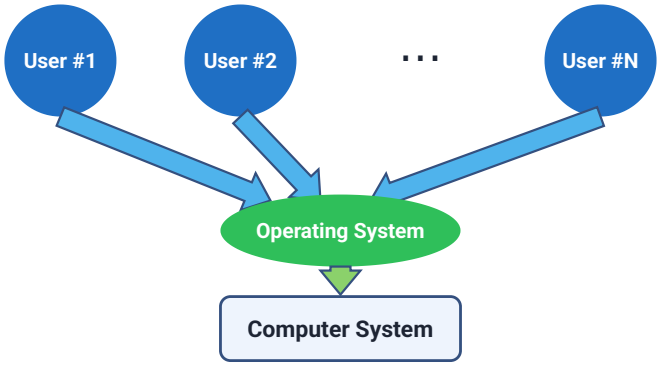
حل

قطعه‌ای: 0003A000h + 00001234h = 0003B234h. صفحه‌ای: P = 16؛ الحاق 0003h ‖ 1234h = 00031234h.

> **💡 شهود: کتابی با صفحه‌های جدا**
>
> به‌جای اینکه کل برنامه پشت سر هم در حافظه باشد، آن را به صفحه‌های هم‌اندازه می‌بُریم. نشانی = (شمارهٔ صفحه، سطر داخل صفحه). جدول صفحه مثل فهرستی است که می‌گوید صفحهٔ ۳ کتاب الان در کدام قاب حافظه است؛ پس صفحه‌ها می‌توانند پراکنده و در هر قاب خالی باشند و دیگر به یک فضای خالی پیوستهٔ بزرگ نیاز نداریم.

۵-۵ پشتیبانی چندکاربره با آدرس‌دهی قطعه‌ای

در یک ماشین چندکاربره که کاربران هر زمان بخواهند می‌آیند و می‌روند، سیستم‌عامل باید محیطی بسازد که هر کاربر فکر کند به‌راحتی با کامپیوتر کار می‌کند و برنامه‌اش را اجرا می‌کند.



شکل: سیستم چندکاربره — سیستم‌عامل منابع را میان کاربران فعال تقسیم می‌کند

- هر کاربر هنگام ورود (login) بخشی از حافظه را می‌گیرد؛ اندازهٔ این بخش به دسته/سطح کاربر بستگی دارد.
- هنگام خروج (logout) آن بخش آزاد می‌شود.

- میان کاربران فعال، سیستم‌عامل CPU را به صورت **تسهیم زمانی** (time multiplexing) برای اجرای کد آن‌ها تقسیم می‌کند.
- سیستم‌عامل جدولی از اطلاعات کاربران فعال نگه می‌دارد (نمونه در جدول زیر).
- وقتی یک کاربر کنترل CPU را به دست می‌گیرد، داده‌های حیاتی کاربر قبلی (محتوای ثبات‌ها از جمله ثبات قطعه یا صفحه، PC، ثبات وضعیت و ...) که **متن (Context)** نامیده می‌شود در حافظه ذخیره و متن کاربر بعدی از حافظه بار می‌شود. به این کار **تعویض متن (Context Switching)** می‌گویند.
- تعویض متن زمان‌بر است ⇐ **برش زمانی (time slice) طولانی‌تر بهرهٔ بهتری از CPU می‌دهد** (هرچند پاسخ‌دهی به کاربران کندتر می‌شود).
- بعضی طراحی‌های مدرن از **فایل‌های ثبات بزرگ** برای تعویض متن بدون تأخیر استفاده می‌کنند (مثلاً GPUها که متن همهٔ نخ‌های فعال را هم‌زمان در ثبات‌ها نگه می‌دارند).

User ID	Segment Start Address	Segment End Address	Access Rights / Priorities	Pointer to User's Context	...
User #1	0107FD5D	01FFFFFF	Write/Read/Supervisor	00010200	...
User #2	000F0000	001FFFFF	Write/Read	00010300	...
...	...	...	...	...	...
User #N	F00FDDC0	FF000000	Write/Read/Supervisor	00010900	...

نکته: نقش ثبات قطعه در حفاظت

با داشتن آدرس شروع و پایان قطعهٔ هر کاربر، سخت‌افزار می‌تواند هر آدرس فیزیکی تولیدشده را بررسی کند و اگر خارج از بازهٔ قطعهٔ کاربر فعلی بود (یا با حق دسترسی او نمی‌خواند) خطا اعلام کند؛ بنابراین برنامهٔ یک کاربر نمی‌تواند حافظهٔ کاربر دیگر یا سیستم‌عامل را خراب کند. در تعویض متن کافی است ثبات قطعه (و بقیهٔ متن) عوض شود.

خلاصهٔ فصل ۵ و جمع‌بندی روش‌های آدرس‌دهی

#	روش	محل عملوند / آدرس مؤثر	مثال
۱	مستقیم	آدرس (ثبات یا حافظه) در IF	mov ax,array
۲	غیرمستقیم	IF به خانه/ثباتی اشاره می‌کند که آدرس عملوند را دارد	L 2,0(3)
۳	بی‌واسطه	خود عملوند در IF	sub cx,33
۴	شاخص‌دار	آدرس IF + محتوای ثبات شاخص	sub ax,array[bx]
۵	ضمنی	در تعریف دستور (مثل ACC یا cx)	loop address
۶	نسبی	(PC) + جابه‌جایی	jcxz loop_addr
۷	قطعه‌ای	(ثبات قطعه) + آدرس منطقی	جابه‌جایی برنامه بدون تغییر کد
۸	صفحه‌ای	(ثبات صفحه) ǁ آدرس منطقی	بدون جمع، ولی صفحه‌های هم‌اندازه

فصل ۶

آشنایی با IBM System/360: ساختار، آدرس‌دهی پایه-جابه‌جایی و قالب دستورها

تا اینجا با ماشین‌های فرضی کار کردیم تا اصول را یاد بگیریم. از این فصل (جلسهٔ ششم) سراغ یک **ماشین واقعی و تاریخی** می‌رویم: خانوادهٔ **IBM System/360** و جانشینش **System/370**. در این فصل یاد می‌گیریم این ماشین چه حافظه و چه ثبات‌هایی دارد، داده‌ها چطور در حافظه «هم‌تراز» می‌شوند، نشانی‌ها چطور با روش **پایه-جابه‌جایی** ساخته می‌شوند، شش **قالب دستور** آن چیست و هر کدام چطور به کد ماشین تبدیل می‌شود، و سرانجام یک برنامهٔ کامل 360 را خط‌به‌خط اسمبل و اجرا می‌کنیم. همهٔ مثال‌های اسلاید با اسمبلر و شبیه‌ساز 360 بازسازی و بررسی شده‌اند.

۶-۱ چرا IBM 360؟ تاریخچه و جایگاه

شرکت IBM در سال **۱۹۶۴** خانوادهٔ **System/360** را معرفی کرد و در سال **۱۹۷۰** نسل بعدی آن یعنی **System/370** را عرضه کرد. 370 همهٔ دستورهای 360 را دارد و **۱۳ دستور بیشتر** هم به آن اضافه شده است؛ برای همین در این درس هر دو را با هم «360/370» می‌نامیم.

- 360 یک ماشین **CISC** بود که حدود **۳۵ KIPS** سرعت داشت.
- اتحاد جماهیر شوروی بعدها از روی آن نسخه‌ای به نام **Ryad** (ریاد) ساخت.
- در سال ۱۹۸۹، ماشین‌های 360/370 بیش از **نیمی** از ارزش ۲۶۰ میلیارد دلاری بازار کامپیوترهای بزرگ (Mainframe) جهان را در اختیار داشتند.

تعریف: KIPS

KIPS مخفف Kilo Instructions Per Second یعنی «هزار دستور در ثانیه». پس ۳۵ KIPS یعنی حدود 35,000 دستور در ثانیه. برای مقایسه، یک پردازندهٔ امروزی چند **میلیارد** دستور در ثانیه اجرا می‌کند؛ یعنی حدود صد هزار برابر سریع‌تر. (دقت کنید KIPS با FLOPS فصل ۱ فرق دارد: KIPS همهٔ دستورها را می‌شمارد ولی FLOPS فقط عملیات ممیز شناور را.)

تعریف: CISC در برابر RISC

- **CISC** (Complex Instruction Set Computer، کامپیوتر با مجموعه‌دستور پیچیده): تعداد زیادی دستور دارد، طول دستورها **متغیر** است (در 360: ۲، ۴ یا ۶ بایت)، و دستورها می‌توانند **مستقیماً روی حافظه** کار کنند؛ حتی دستوری مثل MVC هر دو عملوندش در حافظه است.
- **RISC** (Reduced Instruction Set Computer): دستورهای کم و ساده، طول **ثابت**، و فقط دستورهای Load و Store با حافظه کار می‌کنند و بقیه فقط با ثبات‌ها. (در ادامهٔ درس RISC-V را می‌بینیم.)

💡 **شهود: چرا هنوز یک ماشین ۶۰ ساله را می‌خوانیم؟**

360 نخستین «**خانواده**» از کامپیوترها بود: مدل‌های ارزان و کند و مدل‌های گران و سریع همه **یک مجموعه‌دستور** داشتند، پس برنامه‌ای که برای یکی نوشته می‌شد روی همه اجرا می‌شد. این ایده که «مجموعه‌دستور یک **قرارداد** بین نرم‌افزار و سخت‌افزار است» از 360 آمد. بسیاری از چیزهایی که امروز بدیهی می‌دانیم هم از همین ماشین شروع شد: بایت ۸ بیتی، حافظهٔ بایت‌آدرس‌پذیر، کلمهٔ ۳۲ بیتی، ثبات‌های همه‌منظوره و کد شرط. کامپیوترهای بزرگ امروزی IBM (سری z) هنوز برنامه‌های 360 را اجرا می‌کنند. پس یاد گرفتن 360 یعنی دیدن ریشهٔ ایده‌هایی که در همهٔ پردازنده‌ها هست.

۶-۲ ساختار کلی ماشین

ویژگی	مقدار در 360	توضیح
اندازهٔ حافظه	2^{24} bytes	2^{24} = 16,777,216 بایت یعنی ۱۶ مگابایت؛ پس هر نشانی **۲۴ بیت** است.
واحد آدرس‌پذیر	۸ بیت (بایت)	هر بایت نشانی خودش را دارد (مثل ماشین ۲ و ۵ و ۷).
طول کلمه	۳۲ بیت	**Big Endian** و **هم‌تراز (Aligned)**؛ هر دو را در ادامه کامل توضیح می‌دهیم.
انواع معماری مجموعه‌دستور	RR، RM، MM	ثبات-ثبات، ثبات-حافظه و حافظه-حافظه.
روش‌های آدرس‌دهی	قطعه‌ای، شاخص‌دار، ثباتی (مستقیم/غیرمستقیم)، ضمنی، بی‌واسطه	همان روش‌های فصل‌های ۳ و ۵.
انواع داده	صحیح علامت‌دار (مکمل دو)، دودویی بدون علامت، کاراکتر (بایت)، دهدهی، ممیز شناور	همان نمایش‌های فصل ۴.

۶-۲-۱ اندازه‌های داده: بایت، نیم‌کلمه، کلمه، کلمهٔ مضاعف

در 360 چهار اندازهٔ پایه برای داده وجود دارد و هر کدام نام دارد:

نام	انگلیسی	طول	نوع در اسمبلر
بایت	byte	۸ بیت = ۱ بایت	C، X، B
نیم‌کلمه	halfword	۱۶ بیت = ۲ بایت	H
کلمه	fullword (word)	۳۲ بیت = ۴ بایت	F
کلمهٔ مضاعف	doubleword	۶۴ بیت = ۸ بایت	D

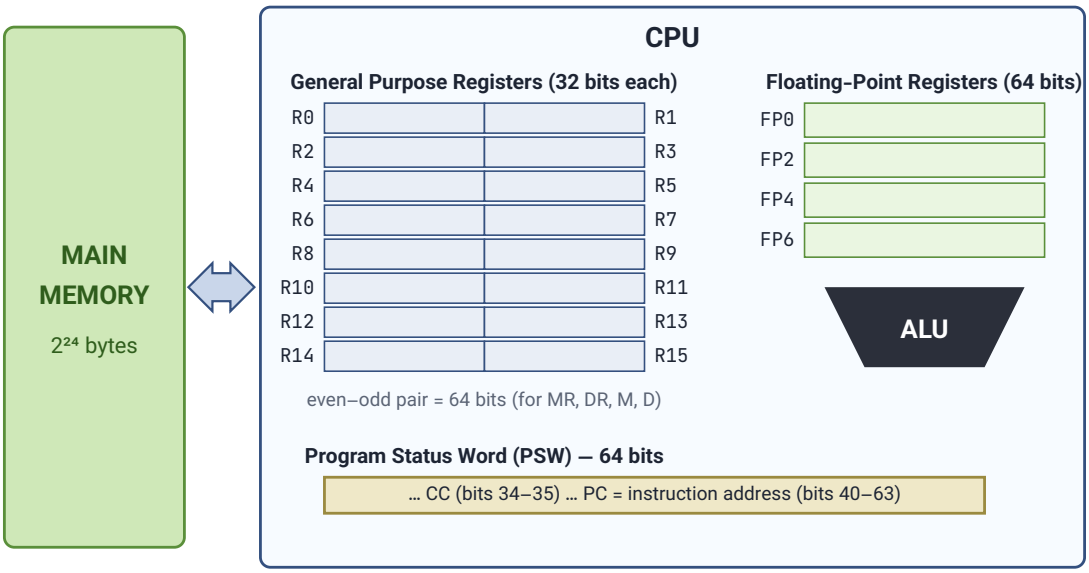
۶-۲-۲ سه نوع معماری مجموعه‌دستور: RR، RM و MM

یک دستور دوعملوندی را در نظر بگیرید (مثلاً «جمع کن»). عملوندها یا در ثبات‌اند یا در حافظه. پس سه حالت داریم:

- **ثبات–ثبات (Register-Register)**: هر دو عملوند در ثبات‌اند؛ مثل `AR 5,6` (R5 ← R5 + R6). کوتاه‌ترین و سریع‌ترین دستورها هستند.
- **ثبات–حافظه (Register-Memory)**: یک عملوند در ثبات و یکی در حافظه؛ مثل `A 5,X` (R5 ← R5 + محتوای X).
- **حافظه–حافظه (Memory-Memory)**: هر دو عملوند در حافظه؛ مثل `MVC A(10),B` که ۱۰ بایت را از B به A کپی می‌کند.

> **💡 شهود: چرا هر سه را دارد؟**
>
> یادتان هست ماشین ۲ و ۳ فصل‌های قبل «حافظه-حافظه» بودند و ماشین ۵ ثبات داشت؟ 360 هر سه را دارد و برای هر کار مناسب‌ترین را انتخاب می‌کند: وقتی داده‌ای را بارها استفاده می‌کنیم آن را در ثبات نگه می‌داریم (RR، سریع)؛ وقتی فقط یک بار با یک عدد حافظه کار داریم از RM استفاده می‌کنیم تا لازم نباشد اول آن را جداگانه به ثبات بیاوریم؛ و وقتی رشته‌ای از بایت‌ها (مثلاً یک متن) را جابه‌جا یا مقایسه می‌کنیم MM بهتر است چون اصلاً در ثبات جا نمی‌شود.

۶-۳ ثبات‌های داخلی



شکل: ثبات‌های ۱۶ — IBM 360 ثبات همه‌منظورهٔ ۳۲ بیتی (جفت زوج-فرد)، ۴ ثبات ممیز شناور ۶۴ بیتی و PSW

۶-۳-۱ ۱۶ ثبات همه‌منظوره (GPR)

- شانزده ثبات **R0 تا R15** داریم و هر کدام **۳۲ بیت** است. چون ۱۶ ثبات داریم، شمارهٔ هر ثبات در دستور با $\log_2 16 = 4$ بیت یعنی **یک رقم هگز** نوشته می‌شود (R10 = A، R15 = F).
- از این ثبات‌ها برای سه کار استفاده می‌شود: **نگه‌داشتن داده** (مثل انباره)، **ثبات پایه** (base) و **ثبات شاخص** (index) برای ساختن نشانی‌ها (در ادامهٔ همین فصل).
- در شکل، ثبات‌ها **جفت‌جفت** کشیده شده‌اند: R0-R1، R2-R3، …، R14-R15. علت این است که چند دستور (ضرب و تقسیم: M، D، DR، MR) یک **جفت زوج-فرد** را مثل یک ثبات ۶۴ بیتی به کار می‌برند. در این دستورها شمارهٔ ثبات اول باید **زوج** باشد (فصل ۷).
- **R0 حالت ویژه دارد:** اگر R0 به‌عنوان ثبات پایه یا شاخص نام برده شود، سخت‌افزار به‌جای محتوای آن **صفر** می‌گذارد. ولی به‌عنوان یک ثبات دادهٔ معمولی (مثلاً در `AR 0,5`) کاملاً عادی است.

۶-۳-۲ ۴ ثبات ممیز شناور (FPR)

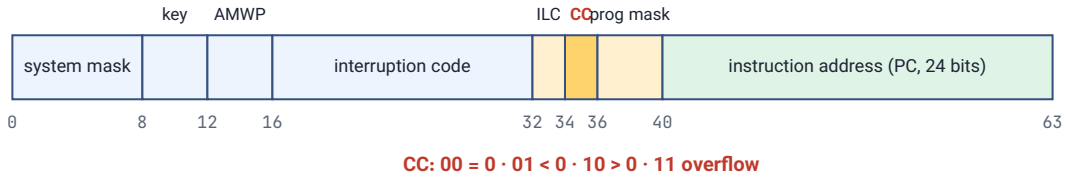
چهار ثبات **FP0، FP2، FP4 و FP6** داریم و هر کدام **۶۴ بیت** است. یک عدد ممیز شناور دقت مضاعف (۶۴ بیتی) کل ثبات را می‌گیرد و یک عدد دقت ساده (۳۲ بیتی) فقط نیمهٔ چپ آن را. (در شکل اسلاید هر ثبات به دو نیمهٔ ۳۲ بیتی تقسیم شده است.) عملیات ممیز شناور 360 در این بخش از درس استفاده نمی‌شود.

۳-۳-۶ کلمهٔ وضعیت برنامه (PSW)

تعریف: PSW

PSW (Program Status Word) یک ثبات **۶۴ بیتی** است که «وضعیت فعلی برنامهٔ در حال اجرا» را نگه می‌دارد. دو بخش آن برای ما مهم‌ترند:

- **بیت‌های ۳۴ و ۳۵: کد شرط (CC)**؛ نتیجهٔ آخرین دستور حسابی/منطقی را خلاصه می‌کند (صفر، منفی، مثبت یا سرریز). پرش‌های شرطی از روی آن تصمیم می‌گیرند (فصل‌های ۷ و ۸).
- **بیت‌های ۴۰ تا ۶۳: نشانی دستور بعد**؛ یعنی همان **PC** فصل‌های قبل. این بخش 24 = 1 + 40 - 63 بیت است، دقیقاً به اندازهٔ یک نشانی حافظه.



شکل: کلمهٔ وضعیت برنامه (PSW) در 360 — کد شرط در بیت‌های ۳۴ و ۳۵ و نشانی دستور بعد (PC) در بیت‌های ۴۰ تا ۶۳

بقیهٔ بیت‌ها برای کارهای سیستم‌عامل‌اند: ماسک‌های وقفه (کدام وقفه‌ها پذیرفته شوند)، کلید حفاظت حافظه، کد وقفه، **ILC** (طول دستور آخر) و ماسک برنامه (مثلاً اینکه سرریز باعث وقفه شود یا نه). در این درس فقط CC و PC را لازم داریم.

💡 شهود: PSW = عکس فوری از برنامه

اگر در وسط اجرای برنامه از آن «عکس» بگیریم، دو چیز مهم‌ترند: **کجا هستیم** (PC) و **نتیجهٔ آخرین مقایسه چه بود** (CC). PSW همین عکس است. یادتان هست در فصل ۵ گفتیم سیستم‌عامل برای «تعویض متن» بین کاربران، ثبات‌های یک کاربر را ذخیره و ثبات‌های کاربر دیگر را بار می‌کند؟ در 360 بخش اصلی این کار با ذخیره و بارکردن PSW انجام می‌شود: سخت‌افزار هنگام وقفه، PSW فعلی را کنار می‌گذارد و PSW دیگری را بار می‌کند و برنامهٔ دیگری از همان نقطه‌ای که بود ادامه می‌یابد.

مثال ۶-۱: طول ثبات‌های 360 (سؤال ۱ امتحان‌ها)

حل گام‌به‌گام

1. **MAR و PC:** حافظه 2^{24} بایت است و واحد آدرس‌پذیر بایت است ⇐ **۲۴ بیت**. (PC همان بیت‌های ۴۰ تا ۶۳ PSW است.)
2. **IR:** بلندترین دستور (قالب‌های SS) ۶ بایت است ⇐ $6 \times 8 =$ **۴۸ بیت**.
3. **ثبات‌های همه‌منظوره:** هر کدام **۳۲ بیت** (۱۶ عدد).
4. **ثبات‌های ممیز شناور:** هر کدام **۶۴ بیت** (۴ عدد).
5. **PSW: ۶۴ بیت**؛ داخل آن CC دو بیتی است.
6. **MBR:** به پهنای دسترسی حافظه بستگی دارد (در مدل‌های مختلف 360 فرق داشت)؛ اگر یک کلمه فرض شود ⇐ **۳۲ بیت**.

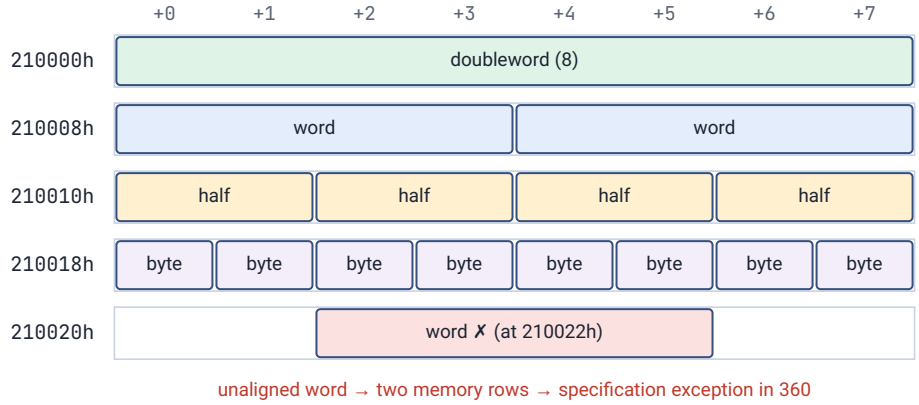
۶-۴ هم‌ترازی داده‌ها در حافظه (Alignment)

در فصل ۲ دیدیم اگر یک کلمه از وسط یک سطر حافظه شروع شود، ممکن است دو بار دسترسی لازم شود. 360 این مشکل را با یک قانون ساده حل کرده است:

قانون هم‌ترازی در 360

هر داده باید از نشانی‌ای شروع شود که **مضرب طول خودش** باشد:

- نیم‌کلمه (۲ بایت): نشانی **زوج**؛
- کلمه (۴ بایت): نشانی **مضرب ۴**؛
- کلمهٔ مضاعف (۸ بایت): نشانی **مضرب ۸**؛
- بایت: هر نشانی‌ای.



شکل: هم‌ترازی در 360 — نیم‌کلمه در نشانی زوج، کلمه در مضرب ۴ و کلمهٔ مضاعف در مضرب ۸ (هر سطر ۸ بایت)

۶-۴-۱ چطور سریع بفهمیم یک نشانی هگز هم‌تراز است؟

کافی است به **آخرین رقم هگز** نگاه کنیم، چون رقم آخر، چهار بیت پایین نشانی است و مضرب ۲، ۴ یا ۸ بودن فقط به همین بیت‌ها بستگی دارد:

نوع	شرط	رقم آخر هگز باید یکی از این‌ها باشد
نیم‌کلمه	بیت ۰ صفر	0 2 4 6 8 A C E
کلمه	دو بیت پایین صفر	0 4 8 C
کلمهٔ مضاعف	سه بیت پایین صفر	0 8

مثال ۶-۲

کدام نشانی‌ها برای یک **کلمه** مجازند؟ 210040h، 21001Eh، 210028h، 210014h، 21000Ah.

حل

رقم آخر: A (۱۰ = 1010) ⇐ دو بیت پایین 10 ⇐ **نامجاز** (فقط برای نیم‌کلمه مجاز است)؛ 4 ⇐ مجاز؛ 8 ⇐ مجاز (و برای کلمهٔ مضاعف هم مجاز است)؛ E (1110) ⇐ نامجاز برای کلمه، مجاز برای نیم‌کلمه؛ 0 ⇐ مجاز برای همه.

اگر قانون رعایت نشود؟

در 360 اگر یک دستور بخواهد کلمه‌ای را از نشانی غیرمضرب ۴ بخواند یا بنویسد، اجرا متوقف می‌شود و یک وقفهٔ خطا به نام **specification exception** رخ می‌دهد. از 370 به بعد بیشتر دستورها دسترسی ناهم‌تراز را هم انجام می‌دادند، ولی کندتر. خوشبختانه **اسمبلر خودش متغیرهای کلمه‌ای را هم‌تراز می‌کند** (بخش رهنمودها را ببینید).

نکته دربارهٔ اسلایدها: در چند مثال اسلاید (مثلاً `S 10,NUM` با جابه‌جایی 01Ah یا `L 15,N10` با 11Bh) نشانی کلمه مضرب ۴ نیست. این مثال‌ها فقط برای تمرین **کدگذاری** آمده‌اند؛ در یک برنامهٔ واقعی برچسب‌های کلمه‌ای همیشه هم‌ترازند.

💡 **شهود: چرا هم‌ترازی سرعت می‌دهد؟**

حافظه را مثل قفسه‌ای با ردیف‌های ۴ یا ۸ خانه‌ای تصور کنید (همان بانک‌های فصل ۲). اگر یک کلمه از اول یک ردیف شروع شود، با یک بار «برداشتن ردیف» کامل خوانده می‌شود. اگر از وسط ردیف شروع شود، نیمی از آن در ردیف بعد است و باید دو ردیف را برداشت و تکه‌ها را کنار هم چید. قانون «مضرب طول» تضمین می‌کند که هیچ داده‌ای مرز ردیف را رد نکند.

۶-۵ آدرس‌دهی پایه-جابه‌جایی (Base-Displacement)

۶-۵-۱ مسئله: نشانی ۲۴ بیتی در هر دستور؟

هر نشانی حافظه در 360 ۲۴ بیت است. اگر قرار بود هر دستوری که به حافظه اشاره می‌کند نشانی کامل ۲۴ بیتی را در خودش داشته باشد، دو مشکل پیش می‌آمد:

1. **دستورها بلند می‌شدند:** یک دستور RX با کد عمل ۸ بیتی، یک ثبات ۴ بیتی و نشانی ۲۴ بیتی دست‌کم ۳۶ بیت (۵ بایت) می‌شد؛ و دستورهای SS که دو نشانی دارند دست‌کم ۸ بایت. برنامه‌ها بزرگ‌تر می‌شدند و حافظه در دهه ۶۰ بسیار گران بود.
2. **برنامه به یک جای ثابت حافظه وابسته می‌شد:** همان «مسئلهٔ جابه‌جایی» فصل ۵؛ اگر بارکننده برنامه را جای دیگری بگذارد، همهٔ نشانی‌های داخل دستورها باید اصلاح شوند.

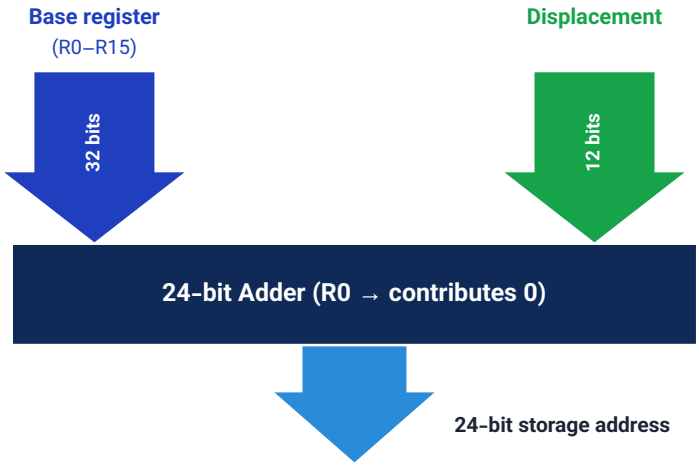
۲-۵-۶ راه‌حل: نشانی = پایه + جابه‌جایی

تعریف

نشانی هر عملوند حافظه از دو بخش ساخته می‌شود:

نشانی حافظه = (ثبات پایه) + جابه‌جایی Storage address = (B) + D

- **ثبات پایه (B)**: یکی از ثبات‌های R1 تا R15 که نشانی شروع یک ناحیه از حافظه را دارد. در دستور فقط **شمارهٔ** آن (۴ بیت) می‌آید.
- **جابه‌جایی (D، Displacement)**: یک عدد **۱۲ بیتی بدون علامت** داخل خود دستور، یعنی از 0 تا 4095 = $2^{12} - 1$.
- این دو با یک **جمع‌کنندهٔ ۲۴ بیتی** جمع می‌شوند؛ از ۳۲ بیت ثبات پایه فقط ۲۴ بیت پایین مهم است و اگر حاصل از 2^{24} بزرگ‌تر شود، بیت‌های اضافه دور ریخته می‌شوند (حساب به پیمانهٔ 2^{24}).



شکل: آدرس‌دهی پایه-جابه‌جایی — نشانی = (ثبات پایه) + جابه‌جایی ۱۲ بیتی

مثال ۶-۳: محاسبهٔ نشانی گام‌به‌گام

فرض کنید (R12) = 00012340h و در دستور B = 12 (C) و D = 1A5h آمده است.

حل

۱. ۲۴ بیت پایین پایه: 012340h.

۲. جمع با جابه‌جایی: 012340h + 1A5h = 0124E5h.

۳. نشانی عملوند = **0124E5h**.

اگر B = 0 بود، مقدار پایه **صفر** حساب می‌شد (نه محتوای R0) و نشانی همان 1A5h می‌شد.

مثال ۶-۴: سرریز ۲۴ بیتی

(R5) = FFFFFFF0h، D = 20h. نشانی چیست؟

حل

۲۴ بیت پایین پایه FFFFF0h است؛ FFFFF0h + 020h = 1000010h؛ این عدد ۲۵ بیت است، پس بیت اضافه دور ریخته می‌شود و نشانی **000010h** است (حافظه مثل یک دایره دور می‌زند).

۶-۵-۳ پنجرهٔ ۴ کیلوبایتی

چون جابه‌جایی از 0 تا 4095 است، با یک ثبات پایه فقط به ۴۰۹۶ بایت پشت‌سرهم دسترسی داریم: از (B) تا (B) + 4095. به این ناحیه می‌گوییم «پنجرهٔ» آن ثبات پایه. برنامه‌ای که کد و داده‌اش بیش از ۴ کیلوبایت است، باید **چند ثبات پایه** داشته باشد که هر کدام یک پنجرهٔ ۴ کیلوبایتی را پوشش دهد.

مثال ۶-۵

برنامه‌ای ۱۰٬۰۰۰ بایتی دست‌کم چند ثبات پایه لازم دارد؟

حل

هر ثبات ۴۰۹۶ بایت را پوشش می‌دهد ⇐ ⌈10000 / 4096⌉ = ⌈2.44⌉ = 3 ثبات پایه.

۶-۵-۴ ثبات شاخص (Index)

در دستورهای قالب RX (پرکاربردترین قالب) یک ثبات سوم هم می‌تواند به نشانی اضافه شود: **ثبات شاخص X**.

$$\text{نشانی مؤثر} = D + (B) + (X)$$

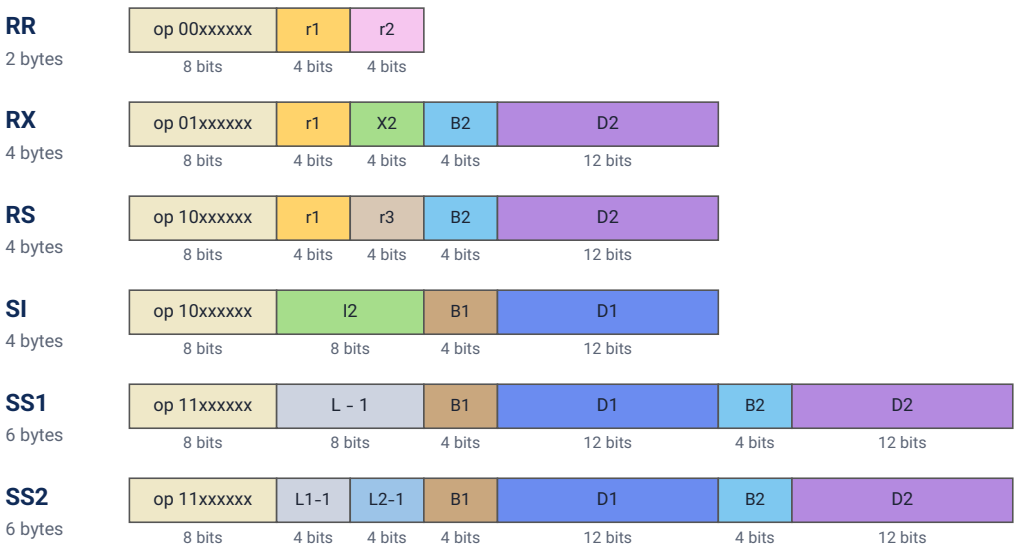
اینجا هم اگر X = 0 باشد، سهم آن **صفر** است. ثبات شاخص برای آرایه‌ها عالی است: B و D به ابتدای آرایه اشاره می‌کنند و X فاصلهٔ عنصر iام از ابتدا را بر حسب بایت نگه می‌دارد (برای آرایهٔ کلمه‌ای 4i). جزئیات کامل را در فصل ۸ می‌بینیم.

💡 شهود: آدرس دادن با «نشانه»

وقتی به کسی آدرس می‌دهیم معمولاً نمی‌گوییم «مختصات جغرافیایی فلان»؛ می‌گوییم «از میدان آزادی ۱۲۰ متر به سمت شمال». میدان آزادی همان **ثبات پایه** است (یک نقطهٔ شناخته‌شده) و ۱۲۰ متر همان **جابه‌جایی** (یک عدد کوچک). اگر کل شهر را جابه‌جا کنیم (برنامه را جای دیگری بار کنیم)، فاصله‌ها تغییر نمی‌کنند؛ فقط کافی است مکان میدان را بدانیم.

این همان **آدرس‌دهی قطعه‌ای فصل ۵** است با یک تفاوت مهم: آنجا یک ثبات قطعهٔ سخت‌افزاری داشتیم؛ اینجا هر کدام از ۱۵ ثبات می‌تواند نقش «ثبات قطعه» را بازی کند و برنامه‌نویس (به کمک اسمبلر) انتخاب می‌کند. یعنی 360 آدرس‌دهی قطعه‌ای را «نرم‌افزاری» و انعطاف‌پذیر کرده است.

۶-۶ شش قالب دستور



شکل: شش قالب دستور 360؛ دو بیت اول کد عمل طول دستور را مشخص می‌کند (00 ⇒ ۲، 01 و 10 ⇒ ۴، 11 ⇒ ۶ بایت)

۶-۶-۱ دو بیت اول کد عمل = طول دستور

همهٔ دستورها با یک **کد عمل ۸ بیتی** شروع می‌شوند و **دو بیت پرارزش** آن قالب و طول دستور را مشخص می‌کنند:

دو بیت اول	رقم اول هگز کد عمل	قالب	طول
`00`	0 تا 3	RR	۲ بایت
`01`	4 تا 7	RX	۴ بایت
`10`	8 تا B	RS یا SI	۴ بایت
`11`	C تا F	SS	۶ بایت

> 💡 **شهود: پردازنده از کجا می‌فهمد دستور چند بایت است؟**
>
> در ماشینی با طول دستور متغیر، واحد کنترل باید قبل از هر کاری بداند چند بایت دیگر را واکشی کند. 360 این اطلاعات را در همان **دو بیت اول** گذاشته است؛ پس با خواندن بایت اول، طول دستور معلوم است. (به همین دلیل یک برنامه‌نویس هم با یک نگاه به رقم اول هگز می‌فهمد کد ماشین را کجا باید ببُرد: `1A..` دو بایتی است، `58....` چهار بایتی و `D2........` شش بایتی.)

۶-۶-۲ قالب ۲ بایت — RR (Register-Register)



هر دو عملوند ثبات‌اند. نیمونیک همهٔ دستورهای این قالب با حرف **R** تمام می‌شود (LR، SR، AR، …). کد ماشین ۴ رقم هگز است: دو رقم کد عمل، یک رقم r1 و یک رقم r2.

۶-۶-۳ قالب ۴ بایت — RX (Register-Indexed)

OPCODE	r1	X2	B2	D2
8 bits (01xxxxxx)	4	4	4	12 bits

یک ثبات (r1) و یک عملوند حافظه با نشانی (X2) + (B2) + D2. **بیشتر دستورهای ماشین در همین قالب‌اند** (L، ST، A، S، …).

۶-۶-۴ قالب ۴ بایت — RS (Register-Storage)

OPCODE	r1	r3	B2	D2
8 bits (10xxxxxx)	4	4	4	12 bits

مثل RX است ولی به‌جای ثبات شاخص، یک ثبات سوم (r3) دارد؛ پس **بعضی دستورهای سه‌عملوندی** در این قالب‌اند. مثلاً `LM r1,r3,S2` ثبات‌های r1 تا r3 را از حافظه بار می‌کند. در دستورهای شیفت (مثل SLL) فیلد r3 استفاده نمی‌شود (صفر) و «نشانی» (B2) + D2 اصلاً به حافظه نمی‌رود؛ خودِ این عدد **تعداد بیت‌های شیفت** است.

۶-۶-۵ قالب ۴ بایت — SI (Storage-Immediate)

OPCODE	I2	B1	D1
8 bits (10xxxxxx)	8 bits	4	12 bits

یک عملوند حافظه (نشانی (B1) + D1) و یک دادهٔ بی‌واسطهٔ **یک بایتی** (I2). در این قالب **فقط دادهٔ بی‌واسطهٔ یک بایتی** مجاز است؛ برای ثابت‌های بزرگ‌تر از DC و دستورهای RX استفاده می‌کنیم.

۶-۶-۶ قالب ۶ بایت — SS1 (Storage-Storage، یک طول)

OPCODE	L-1	B1	D1	B2	D2
8 bits (11xxxxxx)	8 bits	4	12	4	12

دو عملوند حافظه با طول مشترک L بایت (مثل MVC: کپی L بایت).

۶-۶-۷ قالب ۶ بایت — SS2 (Storage-Storage، دو طول)

OPCODE	L1-1	L2-1	B1	D1	B2	D2
8 bits (11xxxxxx)	4	4	4	12	4	12

دو عملوند حافظه که هر کدام طول خودش را دارد (مثل جمع دهدهی AP که دو عدد با طول‌های متفاوت را جمع می‌کند).

💡 شهود: چرا «طول منهای یک»؟

یک فیلد ۸ بیتی ۲۵۶ مقدار دارد (0 تا 255). طول صفر بی‌معناست (کپی صفر بایت؟)، پس اگر خودِ L را ذخیره کنیم یک حالت هدر می‌رود و بیشترین طول ۲۵۵ می‌شود. با ذخیرهٔ L - 1، مقدار 0 یعنی ۱ بایت و 255 یعنی ۲۵۶ بایت؛ پس همهٔ ۲۵۶ حالت مفیدند. در SS2 هم فیلدهای ۴ بیتی طول‌های ۱ تا ۱۶ را نشان می‌دهند. نتیجه: **اسمبلر همیشه یکی کم می‌کند** و ما در کدگذاری دستی نباید فراموش کنیم.

تلهٔ مهم: معنای پرانتز در قالب‌های مختلف

- در **RX**: `D(X,B)` ؛ یعنی اولین عدد داخل پرانتز **ثبات شاخص** است و دومی **ثبات پایه**. `33(7)` یعنی X = 7 و B = 0.
- در **RS و SI**: `D(B)` ؛ فیلد شاخص وجود ندارد و عدد داخل پرانتز **ثبات پایه** است.
- در **SS**: `D(L,B)` ؛ اولین عدد داخل پرانتز **طول** است! `MVC DST(100),SRC` یعنی طول ۱۰۰ و `OC AREA(10),10(4)` یعنی عملوند اول با طول ۱۰ و عملوند دوم با D = 10 و B = 4.

۸-۶-۶ جدول جمع‌بندی قالب‌ها

قالب	طول	عملوندها	نشانی(ها)	نمونه
RR	۲	دو ثبات	—	AR، SR، LR، CR، BCR
RX	۴	ثبات + حافظه	D2+(B2)+(X2)	L، ST، A، S، C، LA، BC
RS	۴	دو ثبات + حافظه / شیفت	D2+(B2)	LM، STM، SLL
SI	۴	حافظه + بایت بی‌واسطه	D1+(B1)	MVI، OI، NI، CLI
SS1	۶	دو حافظه، یک طول	D1+(B1)، D2+(B2)	MVC، OC، CLC
SS2	۶	دو حافظه، دو طول	D1+(B1)، D2+(B2)	AP

۶-۷ مثال‌های کدگذاری هر قالب (همهٔ مثال‌های اسلاید)

روش کلی کدگذاری یک دستور 360

1. از روی نیمونیک، **کد عمل** (دو رقم هگز) و **قالب** را پیدا کن.
2. عملوندها را به فیلدهای قالب نسبت بده (شمارهٔ ثبات‌ها، I، L، D، B، X).
3. اگر عملوند حافظه یک **برچسب** است، نشانی آن را به «ثبات پایه + جابه‌جایی» تبدیل کن: نشانی برچسب − محتوای ثبات پایه = D (بخش USING را ببینید).
4. هر فیلد ۴ بیتی = یک رقم هگز، هر فیلد ۸ بیتی = دو رقم، هر فیلد ۱۲ بیتی = سه رقم. ارقام را به ترتیب قالب کنار هم بگذار.

۶-۷-۱ RR

مثال ۶-۶: `SR 12,5` و `NR 6,10`

- `SR 12,5` : عمل (R5) − (R12) ← R12. کد عمل SR = 1B؛ r1 = 12 = C؛ r2 = 5 ⇐ **1BC5**.
- `NR 6,10` : عمل (R6) ∧ (R10) ← R6. کد عمل NR = 14؛ r1 = 6؛ r2 = 10 = A ⇐ **146A**.

۶-۷-۲ RX

مثال ۶-۷: `S 10,NUM` و `L 15,N10`

- `S 10,NUM` : (R10) − (M_{NUM}) ← R10. فرض: نشانی NUM = (R12) + 01Ah. کد عمل S = 5B؛ r1 = A؛ X2 = 0 (شاخص نداریم)؛ B2 = C؛ D2 = 01A ⇐ **5BA0C01A**.
- `L 15,N10` : (M_{N10}) ← R15. نشانی N10 = (R11) + 11Bh. کد عمل L = 58؛ r1 = F؛ X2 = 0؛ B2 = B؛ D2 = 11B ⇐ **58F0B11B**.

۶-۷-۳ RS

مثال ۶-۸: `SLL 5,3` و `LM 2,5,A`

- `SLL 5,3` (Shift Left Logical): (R5) ← (R5) << 3؛ یعنی بیت‌های R5 سه خانه به چپ می‌روند و از راست صفر وارد می‌شود (معادل ضرب در $2^3 = 8$ اگر سرریز نشود). عدد 3 در جای «نشانی» می‌آید: B2 = 0 و D2 = 003؛ r3 استفاده نمی‌شود (0). کد عمل **89500003** ⇐ SLL = 89.
- `LM 2,5,A` (Load Multiple): چهار ثبات R2 تا R5 از چهار کلمهٔ پشت‌سرهم حافظه بار می‌شوند: $R2 \leftarrow M_{A}$، $R3 \leftarrow M_{A+4}$، $R4 \leftarrow M_{A+8}$، $R5 \leftarrow M_{A+12}$. نشانی A = (R12) + 01Bh. کد عمل LM = 98؛ r1 = 2؛ r3 = 5؛ B2 = C؛ **9825C01B** ⇐ D2 = 01B.

تکمیلی: بازهٔ ثبات‌ها در LM/STM دور می‌زند: `STM 14,12,12(13)` ثبات‌های R12، …، R0، R15، R14 را ذخیره می‌کند (کد 90ECD00C). این دستور در برنامه‌های واقعی برای ذخیرهٔ ثبات‌ها هنگام شروع زیربرنامه رایج است.

۶-۷-۴ SI

مثال ۶-۹: `MVI T1,X'5F'` و `OI 100(3),C'*'`

- `MVI T1,X'5F'` (Move Immediate): $M_{T1} \leftarrow$ 5F (یک بایت). نشانی T1 = (R12) + 0BAh. کد عمل MVI = 92؛ I2 = 5F؛ B1 = C؛ **925FC0BA** ⇐ D1 = 0BA.
- `OI 100(3),C'*'` (Or Immediate): $M_{(R3)+100} \leftarrow (M_{(R3)+100})$ ∨ 5C. کاراکتر `'*'` در کد **EBCDIC** برابر 5Ch است (بخش اعداد و ثابت‌ها). B1 = 3 ⇐ 100 = 64h، D1 = 064. کد عمل **965C3064** ⇐ OI = 96.

۶-۷-۵ SS1

مثال ۶-۱۰: `MVC DST(100),SRC` و `OC AREA(10),10(4)`

- `MVC DST(100),SRC` (Move Characters): ۱۰۰ بایت از نشانی SRC به نشانی DST کپی می‌شود. نشانی DST = (R12) + 0Ah و SRC = (R12) + 110h. کد عمل MVC = D2؛ L - 1 = 99 = 63h؛ B1 = C، D1 = 00A؛ B2 = C، D2 = 110 ⇐ **D263C00AC110**.
- `OC AREA(10),10(4)` (Or Characters): ۱۰ بایت از نشانی AREA با ۱۰ بایت از نشانی (R4) + 10 بیت‌به‌بیت OR می‌شوند و نتیجه در AREA می‌ماند. نشانی AREA = (R9) + 11Bh. کد عمل OC = D6؛ L - 1 = 9 ⇐ 09؛ B1 = 9، D1 = 11B؛ **D609911B400A** ⇐ D2 = 00A، B2 = 4.

۶-۷-۶ SS2

مثال ۶-۱۱: `AP NUM1(6),NUM2(5)`

`AP` (Add Packed) یک عدد **دهدهی** ۵ بایتی در NUM2 را به یک عدد دهدهی ۶ بایتی در NUM1 اضافه می‌کند (اعداد دهدهی «فشرده» در هر بایت دو رقم BCD دارند؛ فصل ۴). نشانی NUM1 = (R12) + 10Ah و NUM2 = (R12) + F10h. کد عمل AP = FA؛ L1 - 1 = 5، L2 - 1 = 4 ⇐ بایت دوم 54؛ B1 = C، D1 = 10A؛ B2 = C، D2 = F10 ⇐ **FA54C10ACF10**.

۶-۸ ساختار برنامهٔ اسمبلی 360 و نمادگذاری

۶-۸-۱ اسکلت یک برنامه

```
PROG-NAME  START  Initial-Address     ← default 0
START-LABEL ...
           (directives/instructions that define the base register and initialize it)
           (instructions)
           (instructions that return control to the OS)
           (directives that define the variables)
           END    START-LABEL
```

- **START**: نام برنامه و نشانی شروع شمارندهٔ مکان (LC). اگر عددی نیاید، LC از **صفر** شروع می‌شود. `START 256` یعنی LC = 100h.
- **END**: پایان متن برنامه. برچسبی که جلوی END می‌آید **نقطهٔ شروع اجرا** (entry point) است؛ بارکننده پس از بارکردن برنامه به آنجا می‌پرد.
- بین این دو: اول دستورهایی که **ثبات پایه** را مقداردهی می‌کنند (BALR و USING، بخش بعد)، سپس دستورهای اصلی، سپس دستورهایی که **کنترل را به سیستم‌عامل برمی‌گردانند** (مثل EOJ) و در آخر تعریف متغیرها.

۶-۸-۲ قالب یک خط

Label Mnemonic Opr1,Opr2,Opr3 توضیح درون‌خطی

- **برچسب** باید از **ستون اول** شروع شود (حداکثر ۸ نویسه و اولین نویسه حرف).
- اگر خط برچسب ندارد، نیمونیک از **ستون دوم به بعد** شروع می‌شود (یعنی ستون اول خالی است).
- اگر ستون اول * باشد، کل خط توضیح است.
- برچسب، نیمونیک، عملوندها و توضیح با **یک یا چند فاصله** از هم جدا می‌شوند. داخل فیلد عملوندها **فاصله مجاز نیست** (چون اولین فاصله یعنی شروع توضیح).

```
LOOP     AR    5,6           Add the content of R6 into R5
*        "LOOP" is a label, "AR" the mnemonic, "5,6" the operands
         L     6,ADAD1
```

خط اول: LOOP برچسب، AR نیمونیک، 5,6 عملوندها و بقیه توضیح. خط دوم کل‌اش توضیح است (* در ستون اول). خط سوم برچسب ندارد و توضیح هم ندارد.

> 💡 **شهود: چرا قاعدهٔ ستون‌ها؟**
>
> برنامه‌های 360 روی **کارت‌های پانچ ۸۰ ستونی** نوشته می‌شدند و اسمبلر باید بدون هیچ علامت اضافه (مثل `:` بعد از برچسب در ماشین‌های فرضی‌مان) بفهمد هر بخش چیست. قاعده ساده است: «اگر ستون اول پر است، برچسب است». به همین دلیل یک فاصلهٔ اضافه در اول خط یا یک فاصله در وسط عملوندها، معنای خط را کاملاً عوض می‌کند.

۶-۹ اعداد، ثابت‌ها و رهنمودهای DS، DC و EQU

۶-۹-۱ نوشتن اعداد در مبناهای مختلف

اعداد به‌طور پیش‌فرض **دهدهی**‌اند. برای مبناهای دیگر از این پیشوندها استفاده می‌کنیم: `B'…'` دودویی، `O'…'` هشت‌هشتی، `X'…'` شانزده‌شانزدهی و `C'…'` کاراکتر (کد EBCDIC).

> **مثال ۶-۱۲:** `B'000001010111' = O'0127' = X'057' = 87`
>
> **بررسی گام‌به‌گام**
>
> - دودویی: $1010111_2 = 64 + 16 + 4 + 2 + 1 = 87$.
> - هشت‌هشتی: بیت‌ها را از راست سه‌تاسه‌تا گروه می‌کنیم: `000 001 010 111` = 0 1 2 7 ⇐ $0127_8 = 1\times64 + 2\times8 + 7 = 87$.
> - شانزده‌شانزدهی: چهارتاچهارتا: `0000 0101 0111` = 0 5 7 ⇐ $057_{16} = 5\times16 + 7 = 87$.
>
> این شکل‌ها هر جایی که عدد لازم است کار می‌کنند، حتی برای شمارهٔ ثبات: `LR B'101',X'D'` همان `LR 5,13` است.

۶-۹-۲ کد کاراکتری EBCDIC

360 برای کاراکترها از **EBCDIC** (فصل ۴) استفاده می‌کند، نه ASCII. چند کد پرکاربرد:

char	blank	.	(	+	*	)	-	,	=	A–I	J–R	S–Z	0–9
EBCDIC	40	4B	4D	4E	5C	5D	60	6B	7E	C1–C9	D1–D9	E2–E9	F0–F9

برای مثال `C'AB'` برابر C1C2h است و رقم '7' برابر F7h (در ASCII برابر 37h بود).

۶-۹-۳ DS: رزرو فضا (Define Storage)

`DS` فقط **جا رزرو می‌کند** و مقدار اولیه نمی‌دهد. مثال: `ARRAY DS 20F` یعنی ۲۰ کلمه (۸۰ بایت) برای ARRAY کنار گذاشته شود.

۶-۹-۴ DC: تعریف ثابت (Define Constant)

`DC` جا رزرو می‌کند **و** مقدار اولیه می‌دهد. مثال: `NUM1 DC F'-5'` یک کلمه با مقدار 5- (یعنی FFFFFFFBh در مکمل دو) تعریف می‌کند.

شکل کلی یک عملوند DS/DC: نوع ['مقدار'] [تکرار]؛ مثلاً `2F'100'` یعنی دو کلمه با مقدار ۱۰۰. چند عملوند را می‌توان با ویرگول پشت‌سرهم آورد: `DC F'1',F'-2',2F'100'`.

نوع	معنا	طول پیش‌فرض	هم‌ترازی خودکار	مثال	بایت‌ها
F	کلمهٔ صحیح	۴	مضرب ۴	F'-5'	FFFFFFFB
H	نیم‌کلمهٔ صحیح	۲	زوج	H'-1'	FFFF
D	کلمهٔ مضاعف	۸	مضرب ۸	DS D	—
C	کاراکتر	تعداد نویسه‌ها	ندارد	C'AB'	C1C2
X	هگز	نصف تعداد رقم‌ها	ندارد	X'FF'	FF
B	دودویی	تعداد بیت‌ها ÷ ۸	ندارد	B'1010'	0A

۶-۹-۵ EQU: تعریف نماد ثابت (Equate)

`EQU` هیچ جایی در حافظه نمی‌گیرد؛ فقط به یک نام، یک مقدار می‌دهد. مثال: `R5 EQU 5`؛ از این به بعد `AR R5,R5` دقیقاً همان `AR 5,5` است. فایدهٔ آن خوانایی است: نام‌گذاری ثبات‌ها یا ثابت‌ها.

> 💡 **شهود: DS، DC و EQU سه کار متفاوت**
>
> DS مثل «رزرو یک صندلی خالی» است، DC مثل «رزرو صندلی و نشاندن کسی روی آن»، و EQU مثل «لقب دادن به یک عدد»: هیچ صندلی‌ای رزرو نمی‌شود و فقط هر جا آن لقب را دیدیم، عدد را جایش می‌گذاریم.

۶-۹-۶ هم‌ترازی خودکار و شمارندهٔ مکان

وقتی اسمبلر به DS یا DC از نوع F (یا H یا D) می‌رسد، اگر LC مضرب ۴ (یا ۲ یا ۸) نباشد، آن را **به اولین مضرب بعدی گرد می‌کند** و بایت‌های وسط را خالی رها می‌کند. سپس برچسب را برابر LC جدید قرار می‌دهد.

> **مثال ۶-۱۳ (اسلاید): تعریف متغیرها با LC = 0FEh**

```
VAR1    DS    3F
VAR2    DC    F'1'
VAR3    DC    F'1',F'-2',2F'100'
```

حل گام‌به‌گام

۱. LC = 0FEh. خط VAR1 از نوع F است و 0FEh مضرب ۴ نیست (رقم آخر E) ⇐ LC به **100h** گرد می‌شود (بایت‌های 0FE و 0FF خالی می‌مانند). VAR1 = 100h. سه کلمه = ۱۲ بایت ⇐ LC = 100h + 0Ch = **10Ch**.

۲. VAR2: 10Ch مضرب ۴ است (رقم آخر C) ⇐ VAR2 = 10Ch؛ مقدار 00000001؛ LC = 110h.

۳. VAR3 = 110h؛ چهار کلمه: 1، 100 (64h)، 100 (64h)، -2 (FFFFFFFE) ⇐ بایت‌های 110h تا 11Fh؛ LC = 120h.

جدول نمادها: VAR1 = 000100h، VAR2 = 00010Ch، VAR3 = 000110h. محتوای حافظه:

address	+0	+1	+2	+3	توضیح
0000FC	--	--	--	--	0FE و 0FF برای هم‌ترازی رها شدند
000100	--	--	--	--	VAR1 (DS: مقدار ندارد)
000104	--	--	--	--	VAR1 + 4
000108	--	--	--	--	VAR1 + 8
00010C	00	00	00	01	VAR2
000110	00	00	00	01	VAR3
000114	FF	FF	FF	FE	VAR3 + 4 = -2
000118	00	00	00	64	100
00011C	00	00	00	64	100

فهرست تولیدشده با اسمبلر:

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
0000FE			VARS	START	X'FE'	
000100			VAR1	DS	3F	۳ کلمه، هم‌تراز
00010C	00000001		VAR2	DC	F'1'	
000110	00000001…		VAR3	DC	F'1',F'-2',2F'100'	
				END		

تمرین ۶-۱ (حل‌شده): هم‌ترازی با انواع مختلف

LC از 1001h شروع می‌شود. نشانی هر برچسب را پیدا کنید.

حل

1. `A DC C'AB'` : کاراکتر هم‌ترازی ندارد ⇐ A = 1001h؛ ۲ بایت (C1 C2) ⇐ LC = 1003h.
2. `B DC H'5'` : نیم‌کلمه باید زوج باشد ⇐ 1003h به 1004h گرد می‌شود ⇐ B = 1004h؛ LC = 1006h.
3. `C DS F` : مضرب ۴ ⇐ 1008h؛ LC = 100Ch.
4. `D DC 2F'7'` : 100Ch مضرب ۴ است ⇐ D = 100Ch؛ ۸ بایت ⇐ LC = 1014h.
5. `E DC X'FF'` : بدون هم‌ترازی ⇐ E = 1014h؛ LC = 1015h.
6. `F DC H'-1'` : زوج ⇐ 1016h؛ LC = 1018h.
7. `G DS D` : مضرب ۸ ⇐ 1018h (رقم آخر 8) ⇐ G = 1018h.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
001001			V2	START	X'1001'	
001001	C1C2		A	DC	C'AB'	۲ بایت، بدون هم‌ترازی
001004	0005		B	DC	H'5'	نیم‌کلمه: نشانی زوج
001008			C	DS	F	کلمه: مضرب ۴
00100C	00000007…		D	DC	2F'7'	دو کلمه
001014	FF		E	DC	X'FF'	۱ بایت
001016	FFFF		F	DC	H'-1'	نیم‌کلمه
001018			G	DS	D	کلمهٔ مضاعف: مضرب ۸
				END		

۶-۱۰ BALR و USING: تعیین ثبات پایه

۶-۱۰-۱ مسئله

اسمبلر برای کدگذاری `L 3,OLDOH` باید بداند از چه ثبات پایه‌ای و با چه جابه‌جایی‌ای به OLDOH برسد: محتوای ثبات پایه - OLDOH نشانی = D. ولی محتوای ثبات پایه در **زمان اجرا** معلوم می‌شود (وقتی بارکننده برنامه را جایی از حافظه گذاشت)، نه در زمان اسمبل! راه‌حل دو قدم دارد: یک **دستور** که در زمان اجرا ثبات پایه را پر می‌کند، و یک **رهنمود** که به اسمبلر می‌گوید آن ثبات چه مقداری خواهد داشت.

۶-۱۰-۲ قدم اول: BALR در زمان اجرا

BALR r1,r2 (Branch And Link Register) — قالب RR، کد عمل 05

1. نشانی دستور بعد ← r1 (یعنی PC بعد از واکشی همین دستور)؛ به این کار «link» می‌گویند.
2. اگر r2 ≠ 0: (r2) ← PC (پرش). اگر r2 = 0: **هیچ پرشی انجام نمی‌شود**.

پس `BALR 11,0` (کد 05B0) فقط یک کار می‌کند: نشانی دستور بعدی را در R11 می‌گذارد. (در سخت‌افزار واقعی، بایت سمت چپ r1 هم اطلاعاتی مثل ILC و CC می‌گیرد؛ ولی نشانی ۲۴ بیتی در سه بایت سمت راست است و برای ما فقط همین مهم است.)

۶-۱۰-۳ قدم دوم: USING در زمان اسمبل

USING value,reg (رهنمود؛ کد ماشین ندارد)

به اسمبلر می‌گوید: «فرض کن ثبات reg در زمان اجرا مقدار value را دارد و از آن برای ساختن نشانی‌ها استفاده کن.» `*` یعنی مقدار فعلی LC. پس `USING *,11` درست بعد از `BALR 11,0` یعنی «R11 نشانی همین نقطه را دارد» که دقیقاً همان کاری است که BALR انجام داد. از این به بعد اسمبلر برای هر برچسب X در پنجرهٔ ۴ کیلوبایتی، جابه‌جایی (USING مقدار) - X را حساب می‌کند. `DROP reg` این فرض را لغو می‌کند.

💡 شهود: چرا این ترفند همیشه درست کار می‌کند؟

اسمبلر نمی‌داند برنامه کجا بار می‌شود، ولی **فاصله‌ها** را دقیق می‌داند: OLDOH همیشه 12h بایت بعد از دستورِ بعد از BALR است، هر جای حافظه که برنامه بار شود. BALR در زمان اجرا «نشانی واقعی» همان نقطه را در R11 می‌گذارد و اسمبلر از قبل «فاصله» را در دستور گذاشته است؛ جمع این دو، نشانی واقعی OLDOH است. این دقیقاً راه‌حل فصل ۵ برای جابه‌جایی برنامه است: **کد عوض نمی‌شود، فقط محتوای ثبات پایه عوض می‌شود.**

تله‌ها

- USING باید **بلافاصله بعد از BALR** بیاید؛ چون `*` باید همان نشانی‌ای باشد که BALR در ثبات گذاشته (نشانی دستورِ بعد از BALR).
- USING هیچ کاری در زمان اجرا نمی‌کند؛ اگر BALR را فراموش کنیم، ثبات پایه زباله دارد و همهٔ نشانی‌ها غلط می‌شوند.
- اگر برچسبی بیش از 4095 بایت از مقدار USING فاصله داشته باشد، اسمبلر خطای «addressability» می‌دهد و باید ثبات پایهٔ دیگری تعریف کرد.

۶-۱۱ برنامهٔ نمونهٔ اسلاید: موجودی انبار (STOCK) — اسمبل و اجرای گام‌به‌گام

برنامه، موجودی جدید انبار را حساب می‌کند: NEWOH = OLDOH + RECPT - ISSUE (موجودی قبلی + دریافتی - صادره).

```
STOCK    START 256
BEGIN    BALR  11,0
         USING *,11
         L     3,OLDOH
         A     3,RECPT
         S     3,ISSUE
         ST    3,NEWOH
         EOJ
OLDOH    DC    F'9'
RECPT    DC    F'4'
ISSUE    DC    F'6'
NEWOH    DS    F
         END   BEGIN
```

۶-۱۱-۱ گذر ۱: شمارندهٔ مکان و جدول نمادها

LC	line	size	توضیح
000100	STOCK START 256	0	STOCK = 100h ؛256 = 100h
000100	BEGIN BALR 11,0	2	BEGIN = 100h ؛RR
000102	USING *,11	0	رهنمود: R11 = 102h
000102	L 3,OLDOH	4	RX
000106	A 3,RECPT	4	
00010A	S 3,ISSUE	4	
00010E	ST 3,NEWOH	4	
000112	EOJ	2	ماکرو: پایان کار
000114	OLDOH DC F'9'	4	114h مضرب ۴ است
000118	RECPT DC F'4'	4	
00011C	ISSUE DC F'6'	4	
000120	NEWOH DS F	4	

EOJ (End Of Job) یک **ماکرو**ی سیستم‌عامل است، یعنی نامی برای یک یا چند دستور آماده. در سیستم‌عامل DOS/360 به دستور دوبایتی `SVC 14` (کد 0A0E) ترجمه می‌شود که از سیستم‌عامل می‌خواهد کار برنامه را تمام کند و کنترل را پس بگیرد. (حتی اگر EOJ را صفر بایت فرض کنیم، OLDOH به‌خاطر هم‌ترازی باز هم 114h می‌شد.)

۶-۱۱-۲ گذر ۲: ساختن کد ماشین هر دستور

- `BALR 11,0` ⇐ 05 | B | 0 ⇐ **05B0**.
- `L 3,OLDOH` : OLDOH = 114h؛ ثبات پایه R11 با مقدار 012 | B | 0 | 3 | 58 ⇐ 012h = 114h - 102h ⇐ D = 102h ⇐ **5830B012**.
- `A 3,RECPT` : 118h - 102h = 016h ⇐ **5A30B016**.
- `S 3,ISSUE` : 11Ch - 102h = 01Ah ⇐ **5B30B01A**.
- `ST 3,NEWOH` : 120h - 102h = 01Eh ⇐ **5030B01E**.
- داده‌ها: 00000009، 00000004، 00000006؛ NEWOH فقط جا دارد.

فهرست کامل (ساخته‌شده با اسمبلر و دقیقاً مطابق فهرست اسلاید):

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000100			STOCK	START	256	
000100	05B0	op r1 r2 05 B 0	BEGIN	BALR	11,0	R11 = نشانی دستور بعد
				USING	*,11	به اسمبلر: R11 = همین نقطه
000102	5830 B012	op r1 X2 B2 D2 58 3 0 B 012		L	3,OLDOH	R3 = موجودی قبلی
000106	5A30 B016	op r1 X2 B2 D2 5A 3 0 B 016		A	3,RECPT	+ دریافتی
00010A	5B30 B01A	op r1 X2 B2 D2 5B 3 0 B 01A		S	3,ISSUE	- صادره
00010E	5030 B01E	op r1 X2 B2 D2 50 3 0 B 01E		ST	3,NEWOH	موجودی جدید
000112	0A0E	SVC I 0A 0E		EOJ		پایان کار و بازگشت به OS
000114	00000009		OLDOH	DC	F'9'	
000118	00000004		RECPT	DC	F'4'	
00011C	00000006		ISSUE	DC	F'6'	
000120			NEWOH	DS	F	
				END	BEGIN	

۶-۱۱-۳ اجرای گام‌به‌گام

PC	instruction	R11	R3	توضیح
100	BALR 11,0	102	?	نشانی دستور بعد در R11
102	L 3,OLDOH	102	9	نشانی = 102 + 012 = 114
106	A 3,RECPT	102	13	4 + 9
10A	S 3,ISSUE	102	7	6 - 13
10E	ST 3,NEWOH	102	7	NEWOH = 7

PC	instruction	R11	R3	توضیح
112	EOJ			پایان

مثال ۶-۱۴: اگر برنامه در 20000h بار شود؟

حل

بارکننده بایت‌ها را بدون هیچ تغییری از 20000h می‌گذارد. BALR در 20000h اجرا می‌شود و R11 = 20002h. دستور L (کد 5830B012، تغییرنکرده) نشانی 20014h = 20002h + 012h را می‌خواند که دقیقاً جای جدید OLDOH است؛ چون کل برنامه به اندازهٔ 20000h - 100h = 1FF00h جابه‌جا شده و 20014h = 1FF00h + 114h. **هیچ کدی اصلاح نشد**؛ این همان مزیت آدرس‌دهی پایه-جابه‌جایی است.

۶-۱۲ تمرین‌های حل‌شدهٔ فصل ۶

تمرین ۶-۲: کدگذاری

این دستورها را به کد ماشین تبدیل کنید (هر جا برچسب نیست، عددها مستقیم‌اند): `AR 3,4` · `L 7,0(2,9)` · `ST 1,X'FF'(,8)` · `MVI 0(5),C'A'` · `MVC 10(5,7),0(8)` · `LA 9,800(5)` · `CLI 0(3),C' '` · `MVC 0(80,10),0(11)`

حل

instruction	fields	code
AR 3,4	1A | 3 | 4	1A34
L 7,0(2,9)	58 | r1=7 | X2=2 | B2=9 | D2=000	58729000
ST 1,X'FF'(,8)	50 | 1 | X2=0 | B2=8 | 0FF	501080FF
MVI 0(5),C'A'	92 | I2='A'=C1 | B1=5 | 000	92C15000
MVC 10(5,7),0(8)	D2 | L-1=04 | 7 | 00A | 8 | 000	D204700A8000
LA 9,800(5)	41 | 9 | X2=5 | B2=0 | 320 (800=320h)	41950320
CLI 0(3),C' '	95 | I2=blank=40 | 3 | 000	95403000
MVC 0(80,10),0(11)	D2 | L-1=79=4F | A | 000 | B | 000	D24FA000B000

تمرین ۶-۳: رمزگشایی

این کدهای ماشین چه دستوری هستند؟ `5A3C4010` · `1B77` · `4150000A` · `98ECD00C` · `89600008`

حل

1. `5A3C4010`: رقم اول 5 ⇐ دو بیت اول 01 ⇐ RX (۴ بایت). کد A = 5A؛ r1 = 3؛ X2 = C؛ B2 = 4؛ D2 = 010 ⇐ `A 3,16(12,4)`: $R3 \leftarrow R3 + M_{16+(R4)+(R12)}$.
2. `1B77`: RR؛ 0 = R7 - R7 → R7: `SR 7,7` ⇐ SR = 1B (روش رایج صفر کردن یک ثبات).
3. `4150000A`: RX؛ LA = 41؛ 10 → R5: `LA 5,10` ⇐ D2 = 00A، B2 = 0، X2 = 0، r1 = 5.
4. `98ECD00C`: LM (RS) = 98؛ D2 = 00C، B2 = D، r3 = C، r1 = E ⇐ `LM 14,12,12(13)`: ثبات‌های R12، ...، R0، R15، R14 از نشانی (R13) + 12 بار می‌شوند (۱۵ ثبات).
5. `89600008`: R6 = 89: `SLL 6,8` ⇐ SLL هشت بیت به چپ.

تمرین ۶-۴: محاسبهٔ نشانی

(R12) = 012340h، (R4) = 8، (R5) = 00FFFF00h. نشانی مؤثر را حساب کنید: (الف) X = 4، B = 12، D = 1F4 (ب) X = 0، B = 5، D = 200 (ج) X = 0، B = 0، D = 7FF.

حل

(الف) 1F4h + 012340h + 8 = 01253Ch. (ب) 200h + FFFF00h = 1000100h ⇐ ۲۴ بیت پایین: **000100h** (دور زدن). (ج) B و X صفرند ⇐ **0007FFh** (نشانی مطلق در ۴ کیلوبایت اول حافظه).

تمرین ۶-۵: پرسش‌های مفهومی

(الف) چرا جابه‌جایی ۱۲ بیتی است و نه ۲۴ بیتی؟ (ب) چرا در SS فیلد طول L-1 است؟ (ج) فرق `33(7)` در دستور L با `33(7)` در دستور MVI چیست؟ (د) USING چه چیزی را عوض می‌کند و BALR چه چیزی را؟

پاسخ

(الف) برای کوتاه ماندن دستورها (هر نشانی فقط ۱۶ بیت: ۴ بیت B و ۱۲ بیت D) و قابل‌جابه‌جا بودن برنامه؛ هزینه‌اش پنجرهٔ ۴ کیلوبایتی و نیاز به ثبات پایه است. (ب) تا از هر ۲۵۶ حالت ۸ بیت استفاده شود (طول ۱ تا ۲۵۶). (ج) در L (قالب RX) عدد داخل پرانتز **شاخص** است (X2 = 7، B2 = 0)؛ در MVI (قالب SI) **پایه** است (B1 = 7). نشانی در هر دو (R7) + 33 است ولی فیلدها فرق دارند. (د) USING فقط محاسبات اسمبلر در زمان اسمبل را عوض می‌کند (کد ماشین ندارد)؛ BALR در زمان اجرا محتوای ثبات را عوض می‌کند.

خلاصهٔ فصل ۶

- 360 (۱۹۶۴)، ماشین CISC با حافظهٔ 2^{24} بایت، بایت‌آدرس‌پذیر، کلمهٔ ۳۲ بیتی Big Endian و هم‌تراز؛ ۱۶ ثبات ۳۲ بیتی (جفت زوج-فرد برای ضرب/تقسیم)، ۴ ثبات ممیز شناور ۶۴ بیتی و ۶۴ PSW بیتی (CC در بیت‌های ۳۴–۳۵ و PC در ۴۰–۶۳).
- هم‌ترازی: نیم‌کلمه زوج، کلمه مضرب ۴، کلمهٔ مضاعف مضرب ۸؛ اسمبلر F/H/D را خودکار هم‌تراز می‌کند.
- نشانی = [(X) +] (B) + D با D ۱۲ بیتی (۰ تا ۴۰۹۵) و جمع ۲۴ بیتی؛ R0 در نقش B یا X یعنی صفر.
- شش قالب: SI (۴)، RS، RX، RR (۲)، SS1 ،SS2 (۶ بایت)؛ دو بیت اول کد عمل طول را تعیین می‌کند؛ در SS طول به‌صورت L-1 ذخیره می‌شود.
- برنامه: START، BALR و USING برای ثبات پایه، دستورها، DS/DC/EQU، EOJ و END؛ برچسب از ستون اول.

فصل ۷

دستورهای RR (ثبات-ثبات) و کد شرط

در این فصل (جلسهٔ هفتم) **همهٔ دستورهای قالب RR** را یکی‌یکی یاد می‌گیریم: انتقال (LR، LCR، LNR، LPR)، جمع و تفریق علامت‌دار (AR، SR)، ضرب و تقسیم با **جفت ثبات** (MR، DR)، جمع و تفریق منطقی (ALR، SLR)، عملیات بیتی (NR، OR، XR)، مقایسه (CR، CLR) و حلقهٔ شمارشی (BCTR). قبل از آن، مهم‌ترین مفهوم این فصل یعنی **کد شرط (CC)** را کامل توضیح می‌دهیم. برای هر دستور: کد عمل، معنا، اثر روی CC، مثال‌های اسلاید با حل گام‌به‌گام و نکته‌های ظریف. همهٔ مثال‌ها با شبیه‌ساز 360 اجرا و بررسی شده‌اند.

۷-۱ مرور قالب RR و روش کدگذاری



قالب RR: ۲ بایت = ۴ رقم هگز

- دستور ۲ بایت است: **دو رقم هگز کد عمل**، **یک رقم r1** و **یک رقم r2**. چون ۱۶ ثبات داریم، هر شمارهٔ ثبات دقیقاً یک رقم هگز است (R10 = A، R15 = F).
- نیمونیک همهٔ این دستورها به **R** ختم می‌شود.
- عملوندها را می‌توان با هر شکل عددی نوشت: `LR B'101',X'D'` همان `LR 5,13` است و اگر `REG5 EQU 5` تعریف شده باشد، `LCR REG5,1` همان `LCR 5,1` .
- هنگام اجرا: CPU دو بایت را واکشی می‌کند، PC ← PC + 2 و سپس عمل را روی ثبات‌ها انجام می‌دهد. هیچ دسترسی‌ای به حافظه برای داده لازم نیست؛ برای همین RRها سریع‌ترین دستورها هستند.

مثال ۷-۱: LR (Load Register) — کد عمل 18

عمل: r1 ← (r2)؛ یعنی محتوای r2 در r1 کپی می‌شود و r2 تغییر نمی‌کند. **LR کد شرط را تغییر نمی‌دهد.**

- `LR 3,5` ⇐ R3 ← (R5) ⇐ کد **1835**.
- `LR B'101',X'D'` ⇐ B'101' = 5 و X'D' = 13 ⇐ R5 ← (R13) ⇐ کد **185D**.

۷-۲ کد شرط (Condition Code — CC)

تعریف

کد شرط دو بیت از PSW (بیت‌های **۳۴ و ۳۵**) است که **همهٔ دستورهای حسابی و منطقی** پس از اجرا آن را بر اساس نتیجه به‌روز می‌کنند. دستورهای پرش شرطی (فصل ۸) از روی آن تصمیم می‌گیرند. دو بیت چهار حالت دارد:

CC	دستورهای حسابی و بیتی	دستورهای مقایسه	جمع/تفریق منطقی (بدون علامت)
00	نتیجه = 0	دو عملوند برابرند	نتیجه = 0 و بدون نقلی
01	نتیجه < 0	اولی کوچک‌تر است	—
10	نتیجه > 0	اولی بزرگ‌تر است	نتیجه ≠ 0 و بدون نقلی
11	سرریز (overflow)	—	نقلی (یا قرض) رخ داده

۷-۲-۱ کدام دستورها CC را تغییر می‌دهند؟

CC را تغییر می‌دهند	CC را تغییر نمی‌دهند
حسابی علامت‌دار: AR، SR، MR، DR (و A، S، M، D) LCR، LNR، LPR، منطقی: NR، OR، XR (و N، O، X) ALR، SLR (و AL، SL) بیتی: CR، CLR (و C، CL) مقایسه:	انتقال: LR، L، ST محاسبهٔ نشانی: LA پرش و شمارش: BCTR، BCT، BC، BCR، BALR

💡 شهود: CC یک «حافظهٔ دوبیتی» از شکل آخرین نتیجه است

فرض کنید می‌خواهید بگویید «اگر R5 منفی شد، به فلان جا برو». بدون CC، دستور پرش باید خودش R5 را بخواند و با صفر مقایسه کند؛ یعنی دستور پرش باید شمارهٔ ثبات را هم داشته باشد. با CC، دستور حسابی هنگام محاسبه «شکل» نتیجه (صفر/منفی/مثبت/سرریز) را در دو بیت یادداشت می‌کند و پرش فقط به همان دو بیت نگاه می‌کند. این همان ایدهٔ پرچم‌های **z، s، v** در ماشین‌های میان‌ترم ۱۴۰۱ و ۱۴۰۲ است؛ با این تفاوت که 360 چهار حالت را در دو بیت فشرده کرده است.

نتیجهٔ مهم: چون LR، L و LA کد شرط را تغییر نمی‌دهند، می‌توانیم بین یک مقایسه و پرش مربوطه چند دستور انتقال بگذاریم و CC دست‌نخورده بماند.

۷-۲-۲ سرریز چیست و کی رخ می‌دهد؟

عددهای علامت‌دار ۳۲ بیتی در مکمل دو از -2^{31} = -2,147,483,648 تا $2^{31} - 1$ = 2,147,483,647 هستند. اگر نتیجهٔ واقعی یک عمل از این بازه بیرون بزند، ۳۲ بیت نتیجه عدد غلطی را نشان می‌دهد و CC = 11 می‌شود. همان‌طور که در فصل ۴ دیدیم، در جمع، سرریز یعنی **دو عدد هم‌علامت، نتیجه‌ای با علامت مخالف** بدهند.

نکتهٔ تکمیلی: قرارداد درس در برابر سخت‌افزار واقعی

جدول بالا قرارداد **اسلایدهای درس** است و همهٔ مثال‌های اسلاید با آن سازگارند. در سخت‌افزار واقعی 360 چند جزئیات فرق دارد (مثلاً ضرب و تقسیم CC را تغییر نمی‌دهند و CC دستورهای منطقی معنای «صفر/ناصفر و نقلی» دارد). **در این درس و امتحان، ملاک قرارداد اسلایدهاست.**

۷-۳ دستورهای بارگذاری ثبات: LCR، LNR، LPR

این سه دستور مثل LR محتوای r2 را در r1 می‌گذارند، اما با یک «تغییر» و **CC را هم به‌روز می‌کنند**:

دستور	کد	عمل	معنا
LR	18	r1 ← (r2)	کپی (بدون CC)
LCR	13	r1 ← -(r2)	قرینه (Complement)
LNR	11	r1 ← -|(r2)|	منفی‌شده (Negative)
LPR	10	r1 ← |(r2)|	قدر مطلق (Positive)

مثال ۷-۲: LCR (Load Complement Register) — کد 13

- `LCR 2,2` با (R2) = -33 = 33: R2 ← -(-33) = 33؛ نتیجه مثبت ⇐ **CC = 10**؛ کد **1322**.
- `REG5 EQU 5` و `LCR REG5,1` با (R1) = 0: R5 ← -0 = 0 ⇐ **CC = 00**؛ کد **1351**.

قرینه در مکمل دو یعنی همهٔ بیت‌ها را معکوس کن و یکی اضافه کن (فصل ۴): FFFFFFDFh = -33 ⇐ معکوس: 33 = 00000021h ⇐ +1: 00000020h.

مثال ۷-۳: LNR (Load Negative Register) — کد 11

- `LNR 2,4` با (R4) = -123: عدد از قبل منفی است ⇐ R2 ← -123 ⇐ **CC = 01**؛ کد **1124**.
- `LNR 5,5` با (R5) = 13: R5 ← -13 ⇐ **CC = 01**؛ کد **1155**.

نتیجهٔ LNR همیشه صفر یا منفی است، پس CC فقط 00 یا 01 می‌شود.

مثال ۷-۴: LPR (Load Positive Register) — کد 10

- `LPR B'010',7` با (R7) = -123: R2 ← |-123| = 123 ⇐ **CC = 10**؛ کد **1027**.
- `LPR 5,B'101'` با (R5) = 13: R5 ← 13 ⇐ **CC = 10**؛ کد **1055**.

حالت مرزی: کوچک‌ترین عدد منفی

در ۳۲ بیت، 80000000h = -2^{31} قرینهٔ مثبت ندارد (بزرگ‌ترین مثبت $2^{31}-1$ است). پس:

- `LCR` و `LPR` روی 80000000h ⇐ نتیجه همان 80000000h و **CC = 11 (سرریز)**.
- `LNR` هرگز سرریز نمی‌کند، چون هر عدد مثبت یک قرینهٔ منفی دارد.

💡 **شهود: چهار «کپی» با چهار عینک**

LR مثل کپی ساده است. LCR کپی را «در آینه» نگاه می‌کند (علامت عوض می‌شود). LPR فقط «فاصله تا صفر» را نگه می‌دارد و LNR همان فاصله را منفی می‌کند. چون سه تای آخر علامت را عوض می‌کنند، CC را هم تنظیم می‌کنند تا بتوانیم بلافاصله تصمیم بگیریم.

۷-۴ جمع و تفریق علامت‌دار: AR و SR

مثال ۷-۵: AR (Add Register) — کد 1A — r1 ← (r1) + (r2)

- `AR 2,7` با (R7) = 5، (R2) = -123: -118 = 5 + -123 ⇐ **CC = 01**؛ کد **1A27**.
- `AR 5,6` با (R6) = 20، (R5) = -13: 7 ⇐ **CC = 10**؛ کد **1A56**.

مثال ۷-۶: SR (Subtract Register) — کد 1B — r1 ← (r1) - (r2)

- `SR 8,8` با (R8) = -50000: هر عددی منهای خودش صفر است ⇐ R8 ← 0، **CC = 00**؛ کد **1B88**.
- `SR 5,6` با (R6) = -22، (R5) = -13: 9 = (-22) - -13 ⇐ **CC = 10**؛ کد **1B56**.

ترفند: `SR r,r` رایج‌ترین راه **صفر کردن یک ثبات** است (۲ بایت، بدون دسترسی به حافظه).

مثال ۷-۷: سرریز

(R1) = 7FFFFFFFh (بزرگ‌ترین عدد مثبت). `AR 1,1` چه می‌کند؟

حل

حاصل واقعی $2^{32} - 2 = 2 \times (2^{31} - 1)$ است که در ۳۲ بیت علامت‌دار جا نمی‌شود. بیت‌های نتیجه FFFFFFFEh (یعنی -2) می‌شوند: دو عدد مثبت، نتیجهٔ منفی ⇐ **CC = 11**. به همین ترتیب با (R7) = 80000000h و (R8) = 1، `SR 7,8` نتیجهٔ 7FFFFFFFh و **CC = 11** می‌دهد (منفی منهای مثبت، نتیجهٔ مثبت).

۷-۵ ضرب: MR و جفت ثبات زوج-فرد

۷-۵-۱ چرا جفت ثبات؟

حاصل‌ضرب دو عدد ۳۲ بیتی ممکن است تا **۶۴ بیت** لازم داشته باشد؛ مثلاً $(2^{31}-1)^2 \approx 2^{62}$. پس نتیجه در یک ثبات جا نمی‌شود و 360 از **دو ثبات پشت‌سرهم** استفاده می‌کند: یک ثبات **زوج** (نیمهٔ بالایی، بیت‌های ۶۳ تا ۳۲) و ثبات **فرد** بعدی (نیمهٔ پایینی، بیت‌های ۳۱ تا ۰).

کد 1C — MR r1,r2 (Multiply Register)

- **r1 باید زوج باشد** (R0، R2، …، R14)؛ وگرنه خطای specification رخ می‌دهد.
- **مضروب** از ثبات **فرد r1+1** خوانده می‌شود (محتوای قبلی r1 اهمیتی ندارد و پاک می‌شود).
- **مضروب‌فیه** محتوای r2 است.
- حاصل ۶۴ بیتی در جفت r1:r1+1 قرار می‌گیرد: (r1+1) × (r2) ← r1:r1+1 و CC طبق علامت حاصل.

مثال ۷-۸: `MR 8,4` با (R4) = -7، (R9) = 5، (R8) = -5

حل گام‌به‌گام

۱. r1 = 8 زوج است ⇐ جفت R8:R9. مضروب = (R9) = 5 (مقدار R8 یعنی -5 نادیده گرفته می‌شود).

۲. 5 × (-7) = -35.

۳. -35 در ۶۴ بیت مکمل دو: FFFFFFFF FFFFFFDDh.

۴. نیمهٔ بالا در R8: FFFFFFFFh = -1؛ نیمهٔ پایین در R9: FFFFFFDDh = -35. حاصل منفی ⇐ **CC = 01**.

کد ماشین: **1C84**.

مثال ۷-۹: `MR 4,5` با (R5) = -2، (R4) = -10

جفت R4:R5؛ مضروب (R5) = -2 و مضروب‌فیه هم (R5) = -2 (r2 همان ثبات فرد است!) ⇐ (-2) × (-2) = 4 ⇐ R4 = 0 و **CC = 10**، R5 = 4؛ کد **1C45**. (پس `MR 4,5` مربع R5 را حساب می‌کند.)

مثال ۷-۱۰: حاصل‌ضربی که واقعاً ۶۴ بیت لازم دارد

(R5) = 100000. `MR 4,5`:

حل

$100000^2 = 10^{10} = 2540BE400h$ که ۳۴ بیت است. پس R4 = 00000002h و R5 = 540BE400h؛ CC = 10. اگر فقط R5 را نگاه کنیم عدد غلطی می‌بینیم؛ بخش «۲» در ثبات زوج است.

💡 شهود: نیمهٔ بالایی معمولاً فقط «گسترش علامت» است

اگر حاصل‌ضرب در ۳۲ بیت جا شود (که در بیشتر برنامه‌ها همین‌طور است)، جواب کامل در ثبات **فرد** است و ثبات زوج فقط تکرار بیت علامت است: همه‌اش 0 (برای مثبت) یا همه‌اش 1 یعنی 1- (برای منفی). در مثال ۷-۸ دقیقاً همین را دیدیم: R8 = -1 و R9 = -35.

۷-۶ تقسیم: DR

DR r1,r2 (Divide Register) — کد 1D

- **r1 باید زوج باشد. مقسوم** یک عدد ۶۴ بیتی در جفت r1:r1+1 است.
- **مقسوم‌علیه** محتوای r2 (۳۲ بیتی) است.
- **خارج‌قسمت** در ثبات **فرد r1+1** و **باقی‌مانده** در ثبات **زوج r1** قرار می‌گیرد؛ CC طبق علامت خارج‌قسمت.

قاعدهٔ علامت: خارج‌قسمت به سمت صفر بریده می‌شود و **علامت باقی‌مانده همیشه برابر علامت مقسوم است**. همیشه باید رابطهٔ زیر برقرار باشد:

مقسوم = خارج‌قسمت × مقسوم‌علیه + باقی‌مانده

مثال ۷-۱۱: `DR 8,4` با (R4) = 5، (R8):(R9) = 503

100 = 5 ÷ 503 با باقی‌ماندهٔ 3 (بررسی: 503 = 3 + 5 × 100) ⇐ R9 ← 100، R8 ← 3؛ **CC = 10**؛ کد **1D84**.

مثال ۷-۱۲: `DR 4,6` با (R6) = 3، (R4):(R5) = -11

حل گام‌به‌گام

1. 11- در ۶۴ بیت: R4 = FFFFFFFFh و R5 = FFFFFFF5h.
2. 3 = 3 ÷ 11 با باقی‌ماندهٔ 2. علامت‌ها مختلف‌اند ⇐ خارج‌قسمت 3-. باقی‌مانده علامت مقسوم (منفی) را می‌گیرد ⇐ 2-.
3. بررسی: 11- = (2-) + 3 × (3-) ✓.
4. R5 ← 3-، R4 ← 2-، خارج‌قسمت منفی ⇐ **CC = 01**؛ کد **1D46**.

۷-۶-۱ آماده کردن مقسوم: گسترش علامت

معمولاً مقسوم یک عدد ۳۲ بیتی است (مثلاً در R3). برای DR باید آن را به ۶۴ بیت در جفت R2:R3 تبدیل کنیم؛ یعنی R2 باید **گسترش علامت** R3 باشد: اگر R3 ≥ 0 آنگاه R2 = 0، و اگر R3 < 0 آنگاه R2 = -1 (FFFFFFFFh). یک راه بدون هیچ پرش: **ضرب در ۱!**

```
LA    4,1          R4 = 1
MR    2,4          R2:R3 = R3 × 1  → R2 = sign extension of R3
DR    2,6          R3 = quotient, R2 = remainder
```

مثال ۷-۱۳

(R3) = -47، (R6) = 5 و R2 مقدار تصادفی 12345678h دارد. سه دستور بالا را اجرا کنید.

حل

بعد از `MR 2,4`: R2:R3 = -47 ⇐ R2 = -1 و R3 = -47 (مقدار تصادفی R2 پاک شد). بعد از `DR 2,6`: 5 ÷ -47 ⇐ R2 = -2، R3 = -9 (بررسی: -47 = 2 - 5 × (-9))؛ CC = 01. اگر MR را فراموش کرده بودیم، مقسوم 12345678 FFFFFFD1h می‌شد که یک عدد بسیار بزرگ و غلط است.

خطاهای تقسیم

- r1 فرد (مثلاً `DR 3,5` با کد 1D35) ⇐ **specification exception**.
- تقسیم بر صفر، یا خارج‌قسمتی که در ۳۲ بیت جا نشود ⇐ **fixed-point divide exception**؛ برنامه متوقف می‌شود و ثبات‌ها تغییر نمی‌کنند.

💡 **شهود: تقسیم، برعکسِ ضرب**

ضرب «دو عدد ۳۲ بیتی ⇐ یک عدد ۶۴ بیتی» است و تقسیم «یک عدد ۶۴ بیتی ⇐ دو عدد ۳۲ بیتی (خارج‌قسمت و باقی‌مانده)». برای همین هر دو از همان جفت زوج-فرد استفاده می‌کنند و می‌توان نتیجهٔ یک ضرب را مستقیم تقسیم کرد. برای به خاطر سپردن جای جواب‌ها: «**خارج‌قسمتِ مهم‌تر در فرد**، همان جایی که حاصل‌ضرب کوچک قرار می‌گرفت».

۷-۷ جمع و تفریق منطقی (بدون علامت): ALR و SLR

در دستورهای «منطقی»، ۳۲ بیت به‌عنوان یک عدد **بدون علامت** (۰ تا $2^{32} - 1$) دیده می‌شوند. خودِ جمع بیت‌به‌بیت دقیقاً همان AR است؛ فقط **تفسیر نتیجه و CC** فرق می‌کند: به‌جای سرریز علامت‌دار، به **نقلی خروجی** از بیت ۳۱ (در جمع) یا **قرض** (در تفریق) نگاه می‌کنیم.

مثال ۷-۱۴: ALR (Add Logical Register) — کد 1E

- `ALR 2,7` با (R7) = 5، (R2) = FFFFFFFEh: 1 00000003h = FFFFFFFEh + 5؛ نتیجهٔ ۳۲ بیتی ۳ و یک **نقلی** بیرون رفت ⇐ **CC = 11**؛ کد **1E27**.
- `ALR 5,6` با 13 و 20: نتیجه 33، بدون نقلی و ناصفر ⇐ **CC = 10**؛ کد **1E56**.

مثال ۷-۱۵: SLR (Subtract Logical Register) — کد 1F

- `SLR 8,8` با (R8) = FFF78955h: نتیجه 0، بدون قرض ⇐ **CC = 00**؛ کد **1F88**.
- `SLR 5,6` با 20 و 32: چون 32 > 20 (بدون علامت)، باید از بیت ۳۲ «قرض» گرفت ⇐ FFFFFFF4h = 32 − 20 + 2^{32} و **CC = 11**؛ کد **1F56**.

مثال ۷-۱۶: AR و ALR روی یک جفت عدد

(R5) = 7FFFFFFFh، (R6) = 1: `AR 5,6` ⇐ 80000000h و CC = 11 (سرریز علامت‌دار). ولی `ALR 5,6` ⇐ همان 80000000h و **10** = CC؛ چون بدون علامت 2^{31} = 1 + 1 − 2^{31} کاملاً درست است و هیچ نقلی بیرون نرفته است.

💡 شهود: یک جمع‌کننده، دو خط‌کش

دایرهٔ پیمانه‌ای فصل ۴ را به یاد بیاورید: همان الگوی بیتی را می‌توان روی خط‌کش «علامت‌دار» یا «بدون علامت» خواند. جمع‌کننده فرقی نمی‌گذارد؛ فقط جایی که «از مرز رد شدیم» فرق دارد: در علامت‌دار مرز بین 7FFFFFFF و 80000000 است (سرریز) و در بدون علامت بین FFFFFFFF و 00000000 (نقلی). کاربرد: نشانی‌ها و شمارنده‌هایی که منفی نمی‌شوند، و **حساب چندکلمه‌ای** (مثلاً جمع اعداد ۶۴ بیتی: نیمه‌های پایین را با ALR جمع کن و اگر CC = 11 بود یک واحد به جمع نیمه‌های بالا اضافه کن؛ برنامهٔ کامل در فصل ۸).

۷-۸ دستورهای منطقی بیتی: NR، OR، XR

این سه دستور روی هر **بیت** جداگانه کار می‌کنند (بیت iام نتیجه فقط به بیت iام دو عملوند بستگی دارد) و CC را طبق علامت نتیجه تنظیم می‌کنند (00 صفر، 01 منفی، 10 مثبت).

a	b	AND (NR)	OR (OR)	XOR (XR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

مثال ۷-۱۷: NR (And Register) — کد 14

- `NR 4,9` با (R9) = 1 و (R4) = FFFFFFFEh: 00…000 = 00…001 ∧ 11…110 ⇐ R4 ← 0، **CC = 00**؛ کد **1449**.
- `NR 5,6` با 20 و 36: 000100 = 100100 ∧ 010100 ⇐ R5 ← 4، **CC = 10**؛ کد **1456**.

مثال ۷-۱۸: OR (Or Register) — کد 16

- `OR 4,6` با FFFFFFFEh و 1: 11…111 = 00…001 ∨ 11…110 ⇐ R4 ← -1، **CC = 01**؛ کد **1646**.
- `OR 5,6` با 20 و 36: 110100 = 100100 ∨ 010100 ⇐ R5 ← 52، **CC = 10**؛ کد **1656**. (در اسلاید به‌جای ∨ اشتباهاً علامت ۸ چاپ شده است.)

مثال ۷-۱۹: XR (Exclusive-Or Register) — کد 17

- `XR 4,6` با FFFFFFFEh و 11: 11…10101 = 00…01011 ⊕ 11…11110 = **CC = 01** ⇐ 11-؛ کد **1746**.
- `XR 5,6` با 1- و **CC = 10** ⇐ 9 = 00…01001 = 11…10110 ⊕ 11…11111 :10-؛ کد **1756**.

💡 شهود: صافی، رنگ و کلید

AND مثل یک صافی است: بیت‌هایی که در «ماسک» ۱ هستند عبور می‌کنند و بقیه صفر می‌شوند (برای جدا کردن یا آزمودن بیت‌ها). **OR** مثل رنگ زدن است: هر جا ماسک ۱ است، نتیجه حتماً ۱ می‌شود (برای روشن کردن بیت‌ها). **XOR** مثل کلید برق است: هر جا ماسک ۱ است، بیت برعکس می‌شود (برای معکوس کردن). نتیجهٔ جالب: $x \oplus x = 0$ پس `XR r,r` هم ثبات را صفر می‌کند، و با سه XR می‌توان دو ثبات را **بدون ثبات کمکی** جابه‌جا کرد (تمرین ۷-۴).

۷-۹ مقایسه: CR و CLR

CR (کد 19) و CLR (کد 15)

هر دو «در ذهن» (r2) - (r1) را حساب می‌کنند ولی **نتیجه را جایی ذخیره نمی‌کنند**؛ فقط CC را تنظیم می‌کنند: **00** اگر (r1) = (r2)، **01** اگر (r1) < (r2)، **10** اگر (r1) > (r2). CR عددها را **علامت‌دار** مقایسه می‌کند و CLR **بدون علامت** (دودویی خالص). مقایسه هیچ‌وقت CC = 11 نمی‌دهد.

مثال ۷-۲۰: مثال‌های اسلاید

- `CR 8,8` با 50780-: برابر ⇐ **CC = 00**؛ کد **1988**.
- `CR 5,6` با 13- و **CC = 10** ⇐ 22- < 13- :22-؛ کد **1956**.
- `CLR 8,8` با FFF78901h: برابر ⇐ **CC = 00**؛ کد **1588**.
- `CLR 5,6` با 1- و 31: بدون علامت 31 < FFFFFFFFh = 4,294,967,295 ⇐ **CC = 10**؛ کد **1556**.

💡 **شهود: یک الگوی بیتی، دو ترتیب**

با 1- = (R1) و 1 = (R2): `CR 1,2` می‌گوید «R1 کوچک‌تر است» (CC = 01) ولی `CLR 1,2` می‌گوید «R1 بزرگ‌تر است» (CC = 10)، چون همان بیت‌های FFFFFFFF روی خط‌کش بدون علامت بزرگ‌ترین عدد است. انتخاب بین CR و CLR یعنی انتخاب خط‌کش: برای مقدارهای علامت‌دار CR، برای نشانی‌ها و مقدارهای همیشه نامنفی CLR.

۷-۱۰ BCTR: کم کردن و پرش (Branch on Count Register)

BCTR r1,r2 — کد 06

1. (r1) - 1 ← r1 (کد شرط تغییر نمی‌کند).
2. اگر (r1) ≠ 0: (r2) ← PC؛ یعنی به نشانی‌ای که در r2 است می‌پرد.
3. اگر **r2 = 0** باشد هرگز پرش نمی‌کند و فقط r1 یکی کم می‌شود.

مثال ۷-۲۱

- `BCTR 8,7` با (R7) = 011FFFF0h، (R8) = 2: 1 ← R8؛ چون ≠ 0، (R7) ← PC. فقط ۲۴ بیت پایین به‌عنوان نشانی استفاده می‌شود ⇐ PC = 1FFFF0h. کد **0687**.
- `BCTR 5,0` با (R5) = 23: 22 ← R5 و هیچ پرشی رخ نمی‌دهد (چون r2 = 0؛ مهم نیست R0 چه دارد). کد **0650**. به گفتهٔ اسلاید این **ارزان‌ترین راه کم کردن یک واحد از یک ثبات** است (۲ بایت، بدون دسترسی به حافظه، بدون تغییر CC).

💡 **شهود: «یکی کم کن و اگر هنوز کار داریم برگرد»**

بیشتر حلقه‌ها این شکل‌اند: یک شمارنده با N شروع می‌شود، بدنه اجرا می‌شود، شمارنده یکی کم می‌شود و اگر صفر نشده دوباره. BCTR (و نسخهٔ RX آن یعنی BCT در فصل ۸) دو کار «کم کردن» و «پرش شرطی» را در یک دستور انجام می‌دهد. همان دستور `loop` در 8086 و ماشین ۱۳۹۹.

۷-۱۱ زیبایی طراحی کدهای عمل

اسلاید تأکید می‌کند که کدهای عمل «با دقت و نه تصادفی» انتخاب شده‌اند؛ برای ساده کردن سخت‌افزار و زیبایی طراحی. به جدول زیر نگاه کنید:

عمل	RR	RX	SI	SS
AND	NR 14	N 54	NI 94	NC D4
مقایسهٔ منطقی	CLR 15	CL 55	CLI 95	CLC D5
OR	OR 16	O 56	OI 96	OC D6
XOR	XR 17	X 57	XI 97	XC D7
بارگذاری / انتقال	LR 18	L 58	MVI 92	MVC D2
مقایسه	CR 19	C 59	—	—
جمع / تفریق	AR 1A / SR 1B	A 5A / S 5B	—	—
ضرب / تقسیم	MR 1C / DR 1D	M 5C / D 5D	—	—
جمع/تفریق منطقی	ALR 1E / SLR 1F	AL 5E / SL 5F	—	—
شمارش و پرش	BCTR 06	BCT 46	—	—
پرش شرطی	BCR 07	BC 47	—	—

- **رقم دوم (چهار بیت پایین) نوع عمل را می‌گوید:** 4 = AND، 5 = مقایسهٔ منطقی، 6 = XOR، 7 = OR؛ در هر چهار قالب! (همان «۴، ۶، ۷» دست‌نویس اسلاید کنار NR، OR و XR.)
- **رقم اول (چهار بیت بالا) قالب و منبع عملوند دوم را می‌گوید:** 0 یا 1 ⇐ RR (ثبات)؛ 4 یا 5 ⇐ RX (حافظهٔ کلمه‌ای)؛ 9 ⇐ SI (بایت بی‌واسطه)؛ SS ⇐ D (رشتهٔ بایت).
- در نتیجه **کد RX هر دستور = کد RR آن +** **40h**: AR 1A ⇐ A 5A، CR 19 ⇐ C 59، BCTR 06 ⇐ BCT 46.

> 💡 **شهود: چرا این نظم سخت‌افزار را ساده می‌کند؟**
>
> واحد کنترل می‌تواند کد عمل را به دو نیمه بشکند: نیمهٔ بالا به بخش «آوردن عملوند» می‌گوید عملوند دوم را از کجا بیاورد (ثبات، حافظه، خود دستور) و نیمهٔ پایین مستقیم به ALU می‌گوید چه عملی انجام دهد. به‌جای یک جدول بزرگ برای ۲۵۶ کد، دو مدار کوچک و مستقل کافی است. برای ما هم حفظ کردن آسان می‌شود: اگر کد RR را بدانیم، کد RX را هم می‌دانیم.

۷-۱۲ جدول خلاصهٔ دستورهای RR

نیمونیک	کد	عمل	CC	مثال	کد مثال
LR	18	r1 ← (r2)	—	LR 3,5	1835
LCR	13	r1 ← -(r2)	حسابی	LCR 2,2	1322
LNR	11	r1 ← -|(r2)|	حسابی	LNR 2,4	1124
LPR	10	r1 ← |(r2)|	حسابی	LPR 2,7	1027
AR	1A	r1 ← (r1)+(r2)	حسابی	AR 2,7	1A27
SR	1B	r1 ← (r1)-(r2)	حسابی	SR 8,8	1B88
MR	1C	r1:r1+1 ← (r1+1)×(r2)	حسابی	MR 8,4	1C84
DR	1D	r1+1 ← q ; r1 ← rem	حسابی	DR 8,4	1D84
ALR	1E	r1 ← (r1)+(r2) unsigned	منطقی	ALR 2,7	1E27
SLR	1F	r1 ← (r1)-(r2) unsigned	منطقی	SLR 5,6	1F56
NR	14	r1 ← (r1) ∧ (r2)	علامت نتیجه	NR 4,9	1449
OR	16	r1 ← (r1) ∨ (r2)	علامت نتیجه	OR 4,6	1646
XR	17	r1 ← (r1) ⊕ (r2)	علامت نتیجه	XR 4,6	1746
CR	19	(r1) ? (r2) signed	مقایسه	CR 5,6	1956
CLR	15	(r1) ? (r2) unsigned	مقایسه	CLR 5,6	1556
BCTR	06	r1--; if ≠0 PC ← (r2)	—	BCTR 8,7	0687
BCR	07	if mask: PC ← (r2)	—	BCR 8,7	0787
BALR	05	r1 ← next ; if r2≠0 PC ← (r2)	—	BALR 11,0	05B0

۷-۱۳ تمرین‌های حل‌شدهٔ فصل ۷

تمرین ۷-۱: ردگیری یک دنباله

در ابتدا (R4) = 10، (R3) = -3، (R2) = 7 و بقیه صفرند. پس از هر دستور، ثبات‌ها و CC را بنویسید.

حل (با شبیه‌ساز بررسی شده)

instruction	code	R2	R3	R5	R6	R7	CC	توضیح
LR 5,2	1852	7	-3	7	0	0	—	کپی؛ CC بدون تغییر

instruction	code	R2	R3	R5	R6	R7	CC	توضیح
AR 5,3	1A53	7	-3	4	0	0	10	7 + (-3)
SR 5,4	1B54	7	-3	-6	0	0	01	4 - 10
LPR 6,5	1065	7	-3	-6	6	0	10	|-6|
LCR 7,6	1376	7	-3	-6	6	-6	01	-6
XR 7,5	1775	7	-3	-6	6	0	00	x ⊕ x = 0
MR 2,4	1C24	-1	-30	-6	6	0	01	R3 × R4 = -30
DR 2,6	1D26	0	-5	-6	6	0	01	-30 / 6 = -5 r 0

تمرین ۷-۲: حالت‌های سرریز

با (R1) = 7FFFFFFFh، (R3) = 80000000h، (R5) = 7FFFFFFFh، (R6) = 1، (R7) = 80000000h، (R8) = 1 نتیجه و CC هر دستور چیست؟ `AR 1,1` · `LCR 2,3` · `LPR 4,3` · `ALR 5,6` · `SR 7,8`

حل

instr	result	CC	دلیل
AR 1,1	FFFFFFFE	11	مثبت + مثبت = منفی
LCR 2,3	80000000	11	قرینهٔ -2^{31} وجود ندارد
LPR 4,3	80000000	11	همان دلیل
ALR 5,6	80000000	10	بدون علامت درست است و نقلی ندارد
SR 7,8	7FFFFFFF	11	منفی - مثبت = مثبت

تمرین ۷-۳: رمزگشایی

`0600` · `07FE` · `1D35` · `1C67` · `1A3F`

حل

1. `1A3F` = `AR 3,15`.
2. `1C67` = `MR 6,7` : (R7) × (R7) ← R6:R7 (مربع R7).
3. `1D35` = `DR 3,5` : **نامعتبر**؛ r1 فرد است ⇐ specification exception.
4. `07FE` = `BCR 15,14` یعنی `BR 14` : پرش بی‌شرط به نشانی داخل R14 (فصل ۸).
5. `0600` = `BCTR 0,0` : R0 یکی کم می‌شود؛ پرشی نیست.

تمرین ۷-۴: جابه‌جایی دو ثبات با سه XR

$(R2) = 35h = 00110101_2$، $(R1) = 0Fh = 00001111_2$. دستورهای `XR 1,2` ، `XR 2,1` ، `XR 1,2` را اجرا کنید.

حل

after	R1 (bin)	R1	R2 (bin)	R2
start	00001111	0F	00110101	35
XR 1,2	00111010	3A	00110101	35
XR 2,1	00111010	3A	00001111	0F
XR 1,2	00110101	35	00001111	0F

دلیل: بعد از قدم اول $R1 = a \oplus b$؛ قدم دوم $R2 = b \oplus (a \oplus b) = a$؛ قدم سوم $R1 = (a \oplus b) \oplus a = b$.
کدها: 1712، 1721، 1712.

تمرین ۷-۵: ضرب و تقسیم

(الف) (R5) = 100000؛ بعد از `MR 4,5` محتوای R4 و R5؟ (ب) (R6) = 2، (R5) = -7:(R4)؛ بعد از `DR 4,6` ؟ (ج) چرا `DR 2,6` با (R6) = 0 اجرا نمی‌شود؟

حل

(الف) 540BE400h ⇐ 10^{10} = 2 CC = 10، R5 = 540BE400h، R4 = 00000002h. (ب) 2 ÷ -7 ⇐ R5 = -3، R4 = -1 (بررسی: -7 = 1 - 2 × (-3))، CC = 01. (ج) تقسیم بر صفر ⇐ fixed-point divide exception.

تمرین ۷-۶: ترفندهای یک‌دستوری با RR

برای هر کار یک دستور RR بنویسید.

حل

صفر کردن R7	XR 7,7 یا SR 7,7
کپی R2 در R9 (بدون تغییر CC)	LR 9,2
قرینه / قدر مطلق / منفی R4	LCR 4,4 / LPR 4,4 / LNR 4,4
دو برابر کردن R3	AR 3,3
یکی کم کردن R5 (بدون تغییر CC)	BCTR 5,0
تنظیم CC بر اساس علامت R6 بدون تغییر مقدارش	NR 6,6 (x ∨ x = x) یا OR 6,6
مربع R7 در جفت R6:R7	MR 6,7

خلاصهٔ فصل ۷

- قالب RR: ۲ بایت = کد عمل + r2 + r1؛ نیمونیک‌ها به R ختم می‌شوند.
- CC (بیت‌های ۳۴–۳۵ PSW): 00 صفر/برابر، 01 منفی/کوچک‌تر، 10 مثبت/بزرگ‌تر، 11 سرریز (یا نقلی در دستورهای منطقی). LA، ST، L، LR و پرش‌ها CC را تغییر نمی‌دهند.
- LR (کپی)، LCR (قرینه)، LPR (|x|)، LNR (-|x|)؛ -2^{31} در LCR/LPR سرریز می‌دهد.
- MR و DR با جفت زوج-فرد: ضرب ⇐ حاصل ۶۴ بیتی در r1:r1+1 (مضروب از r1+1)؛ تقسیم ⇐ خارج‌قسمت در فرد، باقی‌مانده (با علامت مقسوم) در زوج؛ مقسوم را با MR در ۱ گسترش علامت بده.
- ALR/SLR بدون علامت با CC «نقلی = 11»؛ NR/OR/XR بیتی؛ CR علامت‌دار و CLR بدون علامت؛ BCTR = کم کن و اگر r2 ≠ 0 بپر (r2 = 0 یعنی فقط کم کن).
- رقم پایین کد عمل نوع عمل و رقم بالا قالب را می‌گوید؛ کد RX = کد RR + 40h.

فصل ۸

دستورهای RX (حافظه)، پرش‌ها و برنامه‌نویسی 360

در این فصل (جلسهٔ هشتم) دستورهایی را یاد می‌گیریم که عملوند دومشان یک **کلمه در حافظه** است (قالب RX): بارگذاری و ذخیره (L، ST)، حساب و منطق با حافظه (A، S، M، D، AL، SL، C، CL، N، O، X)، محاسبهٔ نشانی (LA)، حلقهٔ شمارشی (BCT) و مهم‌تر از همه **پرش شرطی با ماسک** (BC و BCR). بعد یاد می‌گیریم ساختارهای if، while و for و **زیربرنامه** را به اسمبلی 360 ترجمه کنیم و در پایان ۱۸ برنامهٔ کامل را خط‌به‌خط حل می‌کنیم. همهٔ کدها با اسمبلر و شبیه‌ساز 360 اجرا و بررسی شده‌اند.

۸-۱ چرا دستورهای RX؟

در فصل ۷ همهٔ عملوندها در ثبات بودند. اما ۱۶ ثبات برای نگه داشتن داده‌های یک برنامهٔ واقعی (آرایه‌ها، جدول‌ها، متغیرها) بسیار کم است؛ داده‌ها در **حافظه** هستند. دستورهای RX یک عملوند را از ثبات (r1) و عملوند دیگر را از **یک کلمهٔ حافظه** می‌گیرند. یادآوری قالب از فصل ۶:

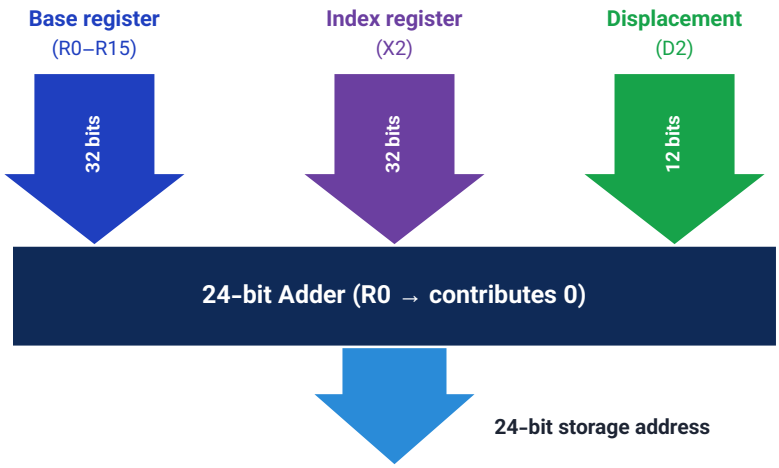


قالب ۴ بایتی RX: ۸ رقم هگز

نشانی مؤثر (Effective Address) در RX

EA = D2 + (B2) + (X2) (فقط ۲۴ بیت پایین جمع نگه داشته می‌شود)

- **D2** (جابه‌جایی): عددی ۱۲ بیتی و بدون علامت (0 تا 4095) که در خود دستور نوشته شده است.
- **B2** (ثبات پایه): شمارهٔ ثباتی که «نقطهٔ شروع» در آن است.
- **X2** (ثبات اندیس): شمارهٔ ثباتی که یک فاصلهٔ اضافی (معمولاً اندیس آرایه بر حسب بایت) در آن است.
- **قاعدهٔ R0:** اگر B2 یا X2 برابر 0 باشد، به‌جای محتوای R0 عدد **صفر** در جمع شرکت می‌کند. یعنی «0 در جای پایه یا اندیس» = «پایه یا اندیس نداریم».



شکل: نشانی مؤثر در RX = D2 + (B2) + (X2) با جمع‌کنندهٔ ۲۴ بیتی

> 💡 **شهود: سه تکهٔ نشانی، سه سؤال**
>
> نشانی `ARR[i]` را در نظر بگیرید. سه سؤال داریم: (۱) «برنامهٔ من الآن کجای حافظه بارگذاری شده؟» ⇐ جوابش در **ثبات پایه** است (با BALR و USING پر می‌شود). (۲) «آرایه از ابتدای برنامه چقدر فاصله دارد؟» ⇐ این را اسمبلر هنگام ترجمه می‌داند، پس در **جابه‌جایی** نوشته می‌شود. (۳) «الان عنصر چندم را می‌خواهم؟» ⇐ این هنگام اجرا عوض می‌شود، پس در **ثبات اندیس** است. یک دستور RX هر سه را با هم جمع می‌کند؛ پس پیمایش آرایه فقط با عوض کردن یک ثبات انجام می‌شود.

۸-۲ شش شکل نوشتن عملوند دوم

در زبان اسمبلی، عملوند دوم یک دستور RX را به شش شکل می‌توان نوشت. **S2** یعنی یک نشانی نمادین (مثل نام متغیر) که اسمبلر خودش آن را به «پایه + جابه‌جایی» تبدیل می‌کند (با کمک USING) و **D2** یعنی یک عدد صریح.

کد	مثال (ARR = (R12)+123h)	D2	B2	X2	شکل
5824C123	L 2,ARR(4)	از USING	از USING	X2	S2(X2)
5820C123	L 2,ARR	از USING	از USING	0	S2
58243014	L 2,20(4,3)	D2	B2	X2	D2(X2,B2)
58250000	L 2,0(5)	D2	0	X2	D2(X2)
5820500A	L 2,10(,5)	D2	B2	0	D2(,B2)
41500008	LA 5,8	D2	0	0	D2

دام پرانتز

در RX اگر داخل پرانتز **فقط یک عدد** باشد، آن عدد **اندیس** است نه پایه: در `L 2,0(5)` ثبات R5 در خانهٔ X2 قرار می‌گیرد (کد 58**25**0000) و B2 = 0. برای اینکه فقط پایه بدهیم باید ویرگول بگذاریم: `L 2,10(,5)` ⇐ X2 = 0 و B2 = 5. البته چون هر دو با هم جمع می‌شوند، **نشانی مؤثر یکی است**؛ فقط کد ماشین فرق می‌کند (و در امتحان کد ماشین خواسته می‌شود!). یادآوری فصل ۶: در RS و SI داخل پرانتز «پایه» است و در SS «طول و پایه».

روش گام‌به‌گام کدگذاری یک دستور RX

1. **کد عمل** را از جدول بنویس (دو رقم هگز).
2. **r1** را یک رقم هگز بنویس (برای BC به‌جای r1 ماسک M1 می‌آید).
3. شکل عملوند دوم را از جدول بالا تشخیص بده.
4. اگر نمادین (S2) است: نشانی نماد منهای محتوای ثبات پایه (طبق USING) = D2، و B2 = شمارهٔ ثبات پایه. اگر صریح است، X2، D2 و B2 را مستقیم بخوان.
5. X2 و B2 را هر کدام یک رقم و D2 را **سه رقم** هگز بنویس (با صفرهای جلو: 8 ⇐ 008).
6. بررسی کن: ۸ رقم هگز؟ D2 ≤ FFFh؟

هم‌ترازی در RX

همهٔ دستورهای RX این فصل (به‌جز LA و پرش‌ها که فقط نشانی می‌سازند) یک **کلمه** را می‌خوانند یا می‌نویسند؛ پس **نشانی مؤثر** باید مضرب ۴ باشد، وگرنه specification exception رخ می‌دهد. در مثال‌های اسلاید جابه‌جایی‌هایی مثل 123h دیده می‌شود؛ این فقط وقتی درست است که جمع آن با محتوای پایه و اندیس مضرب ۴ شود. برای همین متغیرها را با DC/DS از نوع F تعریف می‌کنیم تا اسمبلر خودش آن‌ها را هم‌تراز کند (فصل ۶).

۸-۳ بارگذاری و ذخیره: L و ST

دستور	کد	عمل	CC
L r1,S2	58	r1 ← M[EA]	تغییر نمی‌کند
ST r1,S2	50	M[EA] ← (r1)	تغییر نمی‌کند

مثال ۸-۱: L (Load) — کد 58 — ARR = (R12) + 123h

- `L 2,ARR(4)` ⇐ R2 ← M[(R12) + (R4) + 123h] ⇐ **5824C123**.
- `L 2,ARR` ⇐ بدون اندیس، **5820C123** ⇐ X2 = 0.
- `L 2,20(4,3)` ⇐ R2 ← M[(R3) + (R4) + 20]؛ 20 دهدهی = **58243014** ⇐ 014h.
- `L 2,0(5)` ⇐ R2 ← M[(R5)]؛ R5 اندیس است ⇐ **58250000**.
- `L 2,10(,5)` ⇐ R2 ← M[(R5) + 10]؛ R5 پایه است، 10 = **5820500A** ⇐ 00Ah.

مثال ۸-۲: ST (Store) — کد 50 — ARR = (R12) + 100h

- `ST 2,ARR(4)` ⇐ (R2) ⇐ M[(R12)+(R4)+100h] ⇐ **5024C100**؛ و `ST 2,ARR` ⇐ **5020C100**.
- `ST 2,20(4,3)` ⇐ **50243014**؛ `ST 6,0(5)` ⇐ **50650000**؛ `ST 3,10(,5)` ⇐ **5030500A**.

جهت داده در ST

در همهٔ دستورهای RX ثبات اول نوشته می‌شود و حافظه دوم. در L داده از راست به چپ می‌رود (حافظه ⇐ ثبات) ولی در **ST از چپ به راست** (ثبات ⇐ حافظه). ST تنها دستور این فصل است که حافظه را تغییر می‌دهد؛ بقیه فقط از حافظه می‌خوانند.

💡 **شهود: L و ST مثل برداشتن و گذاشتن کتاب روی میز**

ثبات‌ها مثل میز کار هستند (کوچک ولی سریع) و حافظه مثل قفسهٔ کتابخانه (بزرگ ولی دور). ALU فقط روی میز کار می‌کند؛ پس الگوی همهٔ برنامه‌ها این است: **L** (از قفسه بیاور) ⇐ حساب کن ⇐ **ST** (برگردان). دستورهای A، S، C و ... با حافظه فقط یک میان‌بُرند: عملوند دوم را مستقیم از قفسه می‌خوانند، ولی نتیجه باز هم روی میز (در r1) می‌ماند.

۸-۴ حساب با حافظه: A، S، M، D

این‌ها دقیقاً همان MR، SR، AR و DR فصل ۷ هستند، با این فرق که عملوند دوم M[EA] است (کد = کد RR + 40h). قاعده‌های CC، جفت زوج-فرد، علامت باقی‌مانده و سرریز همه همان است.

مثال ۸-۳: A (Add) — کد 5A

- `A 7,ABS(6)` با ABS = (R11)+234h، M = -123 و (R7) = 200 ⇐ 77 ← R7، **5A76B234** ⇐ **CC = 10**.
- `A 4,5(6,7)` با M[(R6)+(R7)+5] = 333 و (R4) = -400 ⇐ -67 ← R4، **5A467005** ⇐ **CC = 01**.
- `A 6,0(10)` با M[(R10)] = 25 و (R6) = -25 ⇐ 0 ← R6، **5A6A0000** ⇐ **CC = 00**.

مثال ۸-۴: S (Subtract) — کد 5B

- `S 3,AAA(6)` با AAA = (R10)+200 (= C8h)، M = -1، (R3) = 7FFFFFFFh: $7FFFFFFFh - (-1) = 2^{31}$ که در ۳۲ بیت علامت‌دار جا نمی‌شود ⇐ R3 ← 80000000h، **5B36A0C8** ⇐ **CC = 11**.
- `S 2,33(7)` با M = 20، (R2) = -10 ⇐ R2 ← -30، (33 = 021h) **5B270021** ⇐ **CC = 01**.
- `S 0,0(1)` با M[(R1)] = 0، (R0) = 5 ⇐ R0 ← 5، **5B010000** ⇐ **CC = 10**. (R0 به‌عنوان **ثبات داده** کاملاً مجاز است؛ فقط در جای پایه و اندیس معنی «صفر» دارد.)

مثال ۸-۵: M (Multiply) — کد 5C — r1 زوج

r1 : r1+1 ← (r1+1) × M[EA]

- `M 2,Hamid(6)` با (R3) = 7FFFFFFFh: M = -1، (R12)+27Fh = Hamid، ۶۴ ⇐ -7FFFFFFFh = 7FFFFFFFh × (-1) بیتی FFFFFFFF 80000001h ⇐ R2 ← -1، R3 ← 80000001h، **5C26C27F** ⇐ **CC = 01**.
- `M 2,33(7)` با M = 20، (R3) = -10 و (R2) = 45F2h (زباله): -200 ⇐ R2 ← FFFFFFFFh، R3 ← -200، **5C270021** ⇐ **CC = 01**. مقدار قبلی R2 اهمیتی نداشت.
- `M 0,0(10)` با M[(R10)] = 21، (R1) = 2 ⇐ R0:R1 ← 42 یعنی R0 = 0 و ⇐ **CC = 10**، R1 = 42، **5C0A0000**.

مثال ۸-۶: D (Divide) — کد 5D — r1 زوج

r1 ← باقی‌مانده و r1+1 ← (r1:r1+1) ÷ M[EA]

- `D 2,Hokm(2)` با Hokm = (R12)+15، (R2) = 0، (R3) = 173 و M[(R2)+(R12)+15] = -17:
  1. اول نشانی ساخته می‌شود؛ R2 در این لحظه 0 است، پس EA = (R12) + 15.
  2. مقسوم ۶۴ بیتی R2:R3 = 173 و مقسوم‌علیه -17.
  3. 3 + (-17) × (-10) = 173 ⇐ R3 ← -10، R2 ← 3؛ خارج‌قسمت منفی ⇐ **CC = 01**؛ کد **5D22C00F**.

اینکه R2 هم اندیس است و هم نیمهٔ بالای مقسوم، مشکلی ندارد: **نشانی همیشه قبل از اجرای عمل** و با مقدار قدیمی ثبات‌ها حساب می‌شود.

- `D 2,33(7)` با M = 10 و R2:R3 = -20 (R2 = -1) ⇐ R3 ← -2، R2 ← 0، **5D270021** ⇐ **CC = 01**.
- `D 0,0(10)` با M = 21 و R0:R1 = 2 ⇐ 2 = 0 × 21 + 2 ⇐ R1 ← 0، R0 ← 2، خارج‌قسمت صفر ⇐ **CC = 00**، **5D0A0000** ⇐.

۸-۵ حساب منطقی با حافظه: AL و SL

مثال ۸-۷: AL (Add Logical) — کد 5E

- `AL 8,SOS+2(7)` با SOS: نماد به‌اضافهٔ عدد هم مجاز است ⇐ SOS = (R12)+200h، D2 = 202h. M = 123 ⇐ **5E87C202** ⇐ **CC = 10**، 323 ← R8 ⇐ 200 = (R8).
- `AL 4,X'555'(6,7)` با M = 333، (R4) = 1400 ⇐ 1733، **5E467555** ⇐ **CC = 10**.
- `AL 2,0(1)` با M = 25، FFFFFFE7h = -25 = (R2): 00000000h 1 + 19h = FFFFFFE7h ⇐ 0 ← R2 با **نقلی** ⇐ **5E210000** ⇐ **CC = 11**. (با A همین جمع CC = 00 می‌داد.)

مثال ۸-۸: SL (Subtract Logical) — کد 5F

- `SL 8,SIX-16(7)` با **CC** ⇐ 77، (R8) = 200 ⇐ D2 = 200h - 10h = 1F0h. M = 123، SIX = (R12)+200h ⇐ **5F87C1F0** ⇐ **10** =.
- `SL 4,B'111'(6,7)` با M = 333، (R4) = 1400 ⇐ 1067، **5F467007** ⇐ **CC = 10**.
- `SL 2,0(1)` با M = 35، (R2) = 35 ⇐ 0، **5F210000** ⇐ **CC = 00**.

۸-۶ مقایسه با حافظه: C و CL

مثال ۸-۹: C (Compare) — کد 59 — و CL (Compare Logical) — کد 55

HEY = (R12) + 70h.

- `C 2,HEY(11)` با M = -100، (R2) = -120: -100 < -120 ⇐ **592BC070** ⇐ **CC = 01**.
- `C 4,B'11'(B'10',X'A')` : D2 = 3، B2 = 10 = A، X2 = 2 ⇐ M[(R2)+(R10)+3] = 33 با (R4) = 140 ⇐ **5942A003** ⇐ **CC = 10**.
- `C 2,0(1)` با 35 و 35 ⇐ **59210000** ⇐ **CC = 00**.
- `CL 2,HEY(11)` با M = 100، FFFFFF88h = -120 = (R2): بدون علامت FFFFFF88h خیلی بزرگ است ⇐ **552BC070** ⇐ **CC = 10**.
- `CL 4,B'11'(B'10',X'A')` با M = 3، (R4) = 1 ⇐ **5542A003** ⇐ **CC = 01**؛ `CL 2,0(1)` با 5- و 5- ⇐ **55210000** ⇐ **CC = 00**.

۸-۷ منطق بیتی با حافظه: N، O، X

مثال ۸-۱۰: N (کد 54)، O (کد 56)، X (کد 57)

70h + (R12) = ME و در هر سه مورد اول M = 77h = 0…0111 0111 و (R0) = -4 = FFFFFFFCh = 1…1111 1100:

instr	code	bits	result	CC
N 0,ME(11)	540BC070	1…11111100 ∧ 0…01110111	00000074h	10
O 0,ME(11)	560BC070	1…11111100 ∨ 0…01110111	FFFFFFFFh = -1	01
X 0,ME(11)	570BC070	1…11111100 ⊕ 0…01110111	FFFFFF8Bh	01

با M[(R5)+(R6)+4] = -13 = FFFFFFF3h و (R3) = 1: `N 3,4(5,6)` ⇐ 1، CC = 10 (54356004)؛ `O 3,4(5,6)` ⇐ CC = 01 (56356004)؛ -13، `X 3,4(5,6)` ⇐ CC = 01 (57356004)، -14 = FFFFFFF2h. و با M = (R2) = -5: `N 2,0(1)` ⇐ -5 (CC = 01، 56210000)، `O 2,0(1)` ⇐ -5 (CC = 01، 54210000)، `X 2,0(1)` ⇐ 0 (CC = 00، 57210000).

💡 شهود: آزمودن یک بیت با N

برای اینکه بفهمیم عددی زوج است یا فرد، فقط بیت شمارهٔ ۰ آن مهم است. اگر آن را با کلمهٔ ONE (= 1) AND کنیم، همهٔ بیت‌ها جز بیت ۰ صفر می‌شوند؛ نتیجه 0 یعنی زوج (CC = 00) و نتیجه 1 یعنی فرد (CC = 10). این ایده در برنامهٔ «شمارش زوج‌ها» در ادامه به کار رفته است. به همین ترتیب با ماسک 80000000h بیت علامت آزموده می‌شود.

۸-۸ LA: محاسبهٔ نشانی (Load Address) — کد 41

LA r1,S2

r1 ← EA = D2 + (B2) + (X2). **هیچ دسترسی‌ای به حافظه انجام نمی‌شود**؛ خودِ نشانی (عدد) در r1 قرار می‌گیرد. CC تغییر نمی‌کند و چون نشانی ۲۴ بیتی است، **۸ بیت بالای r1 صفر می‌شوند**.

مثال ۸-۱۱: کاربردهای LA

کاربرد	معنا	instr
نشانی یک متغیر (مثلاً برای زیربرنامه)	R2 ← address of PUT	LA 2,PUT
بارگذاری ثابت 0 تا 4095 (بدون نیاز به DC)	R5 ← 8	LA 5,8
افزایش یک ثبات با ثابت (شمارنده، اندیس)	R1 ← (R1) + 1	LA 1,1(1)
جمع دو ثبات و یک ثابت در یک دستور	R9 ← (R5)+(R7)+77	LA 9,77(5,7)
جمع دو ثبات در ثبات سوم	R9 ← (R5)+(R7)	LA 9,0(5,7)
جمع با ثابت	R9 ← (R5)+800	LA 9,800(5)

مثال‌های اسلاید: `LA 3,4(5,6)` با (R5)+(R6)+4 = 001AF234h ⇐ R3 ← 001AF234h ⇐ **41356004**. و `LA 1,1(1)` با (R1) = 77 ⇐ R1 ← 78 ⇐ **41110001**.

دام‌های LA

- **LA در برابر L:** `LA 5,8` عدد 8 را در R5 می‌گذارد، ولی `L 5,8` **محتوای خانهٔ 8 حافظه** را می‌آورد. (در شبیه‌ساز: اگر M[8] = 1234 باشد، L 5,8 ⇐ R5 = 1234.)
- جابه‌جایی منفی ندارد: `LA 4,-1` غلط است. برای کم کردن از `BCTR r,0` یا S استفاده کنید.
- نتیجه ۲۴ بیتی است: اگر (R5) = -1 = FFFFFFFFh باشد، `LA 5,1(5)` نتیجهٔ **00000000h** می‌دهد (درست به نظر می‌رسد، ولی فقط تصادفاً!) و اگر (R5) = -5 باشد نتیجه 00FFFFFCh است، نه -4. پس LA فقط برای عددهای نامنفی کوچک‌تر از 2^{24} مناسب است.
- CC را تغییر نمی‌دهد؛ پس `LA 3,1(3)` بین یک مقایسه و پرش آن خرابی به بار نمی‌آورد.

💡 شهود: LA یعنی «از ماشین‌حساب نشانی، به‌جای جمع‌کننده استفاده کن»

CPU برای هر دستور RX یک جمع‌کنندهٔ سه‌ورودی (D + پایه + اندیس) دارد. LA همین مدار را به کار می‌گیرد و جوابش را به‌جای فرستادن به حافظه، در ثبات می‌ریزد. نتیجه: یک جمع سریع، بدون دسترسی به حافظه و بدون خراب کردن CC. برای همین برنامه‌نویسان 360 برای افزایش شمارنده‌ها تقریباً همیشه `LA r,1(r)` و برای پیمایش آرایه `LA r,4(r)` می‌نویسند.

۸-۹ BCT: حلقهٔ شمارشی (Branch on Count) — کد 46

BCT r1,S2

r1 ← (r1) − 1؛ اگر (r1) ≠ 0 آنگاه PC ← EA. همان BCTR فصل ۷ است ولی مقصد پرش یک نشانی RX است (نه محتوای ثبات). CC تغییر نمی‌کند.

مثال ۸-۱۲

- `BCT 3,LOOP` با LOOP = (R12)+1FFh و (R3) = 2 ⇐ ⇐ PC ← (R12) + 1FFh ⇐ 0 ≠ 1، ← R3 **4630C1FF**.
- `BCT 1,2(3,4)` با (R1) = 1 ⇐ R1 ← 0 ⇐ پرش نمی‌شود و اجرا با دستور بعدی ادامه می‌یابد ⇐ **46134002**.

الگوی حلقهٔ N باره:

```
        LA    5,N           counter = N   (N ≥ 1)
LOOP    ...                 loop body
        BCT   5,LOOP        counter--; repeat while counter ≠ 0
```

حلقه با شمارندهٔ صفر

بدنه قبل از اولین آزمون اجرا می‌شود (حلقهٔ do-while). اگر شمارنده با **0** شروع شود، BCT آن را 1- می‌کند که ≠ 0 است و حلقه 2^{32} بار تکرار می‌شود (تا شمارنده دور بزند و دوباره به 0 برسد). اگر N ممکن است صفر باشد، قبل از حلقه آن را بیازمایید.

۸-۱۰ پرش شرطی با ماسک: BC — کد 47



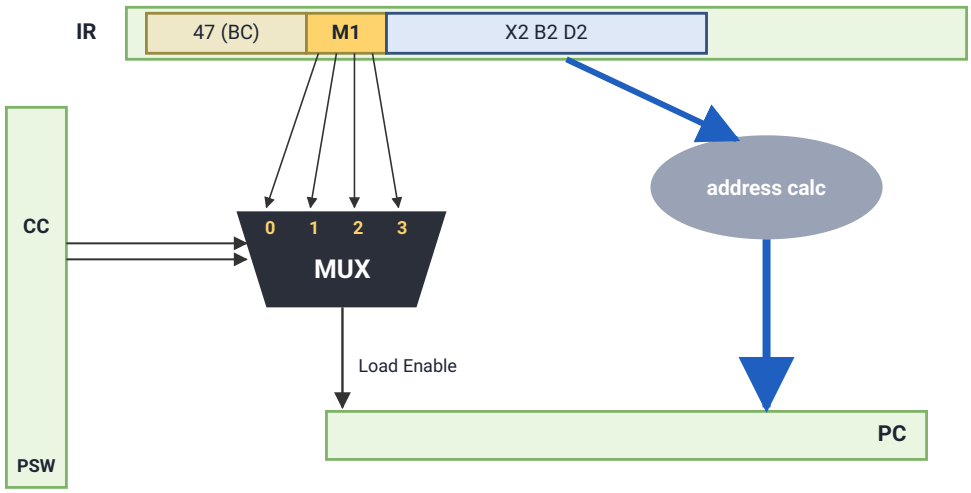
BC: به‌جای r1، یک ماسک ۴ بیتی M1

ماسک M1

هر یک از ۴ بیت ماسک به **یکی از چهار مقدار ممکن CC** مربوط است:

بیت ماسک (از چپ)	0	1	2	3
ارزش عددی	8	4	2	1
مربوط به CC =	00	01	10	11
بعد از حسابی	صفر	منفی	مثبت	سرریز
بعد از مقایسه	برابر	اولی کوچک‌تر (low)	اولی بزرگ‌تر (high)	—

قاعده: اگر بیتِ مربوط به CC فعلی در ماسک 1 باشد ⇐ EA ← PC (پرش)؛ وگرنه دستور بعدی اجرا می‌شود. BC کد شرط را تغییر نمی‌دهد.



شکل: سخت‌افزار CC — BC یکی از چهار بیت ماسک را انتخاب می‌کند؛ اگر ۱ باشد PC با نشانی مؤثر بار می‌شود

💡 شهود: ماسک یعنی «فهرست حالت‌هایی که در آن‌ها می‌پرم»

تصور کنید CC چهار چراغ دارد و در هر لحظه دقیقاً یکی روشن است. ماسک یک برگهٔ چهارخانه است که روی خانه‌های دلخواه تیک زده‌اید. اگر چراغ روشن روی خانهٔ تیک‌دار بیفتد، می‌پرید. سخت‌افزار این را با یک **مالتی‌پلکسر ۴ به ۱** می‌سازد (شکل بالا): ۴ بیت ماسک ورودی‌های MUX هستند، CC خط انتخاب است و خروجی MUX تعیین می‌کند PC بار شود یا نه. نتیجه: **یک کد عمل برای ۱۶ نوع پرش**؛ در حالی که ماشین‌های فرضی درس برای هر شرط (jz، jn، ...) یک دستور جدا داشتند.

روش گام‌به‌گام ساختن ماسک

1. بپرس: «در کدام مقدارهای CC باید پرش کنم؟»
2. ارزش هر کدام را بنویس: CC=11 ⇐ 1، CC=10 ⇐ 2، CC=01 ⇐ 4، CC=00 ⇐ 8.
3. جمع کن. مثال: «بعد از C، پرش اگر اولی ≤ دومی» ⇐ برابر (8) یا بزرگ‌تر (2) ⇐ 8 + 2 = 10؟ **خیر!** با قرارداد اسلاید BNL = 11 است (8 + 2 + 1)؛ چون مقایسه هرگز CC = 11 نمی‌دهد، اضافه کردن بیت 1 بی‌اثر است و این‌طوری «نه کوچک‌تر» = متمم دقیق «کوچک‌تر» (4) یعنی 11 = 4 - 15 می‌شود.
4. قانون متمم: ماسک «= 1 - 15 ⇐ BNO، BNH = 15 - 2 = 13، BNE = 15 - 8 = 7. (ماسک X - 15 = «not X» 14.)

۱-۱۰-۸ جدول کامل ۱۶ ماسک (اسلاید)

M1	bits	CC	Meaning	Extended
0	0000	—	No operation	NOP
1	0001	11	Branch on overflow	BO
2	0010	10	Branch on plus / high	BP / BH
3	0011	10, 11	plus/high or overflow	—
4	0100	01	Branch on minus / low	BM / BL
5	0101	01, 11	minus/low or overflow	—
6	0110	01, 10	plus or minus (high or low)	—
7	0111	01, 10, 11	not zero / not equal	BNZ / BNE
8	1000	00	zero / equal	BZ / BE
9	1001	00, 11	zero/equal or overflow	—
10	1010	00, 10	zero/equal or plus/high	—
11	1011	00, 10, 11	not minus / not low	BNM / BNL
12	1100	00, 01	zero/equal or minus/low	—
13	1101	00, 01, 11	not plus / not high	BNP / BNH
14	1110	00, 01, 10	not overflow	BNO
15	1111	all	Branch always	B

نیمونیک‌های گسترش‌یافته (Extended Mnemonics)

اسمبلر برای ماسک‌های پرکاربرد نام آماده دارد تا مجبور نباشیم عدد ماسک را حفظ کنیم. `BE LOOP` دقیقاً همان `BC 8,LOOP` است و همان کد ماشین را تولید می‌کند: 4780.... نام‌های (high، low، equal) H/L/E برای **بعد از مقایسه** و نام‌های (plus، minus، zero، overflow) P/M/Z/O برای **بعد از حساب** هستند، ولی ماسکشان یکی است (BH = BP = 2). **B** پرش بی‌شرط (15) و **NOP** «هیچ کار» (0) است.

مثال ۸-۱۳: مثال‌های اسلاید BC

- `BC 3,LOOP` (LOOP = (R12)+1FFh) با **CC = 11**: ماسک 0011؛ بیت مربوط به CC=11 (ارزش 1) روشن است ⇐ پرش: PC ← (R12)+1FFh ⇐ **4730C1FF**.
- `BC B'1011',2(3,4)` با **CC = 10**: ماسک 1011 = 11؛ بیت ارزش 2 روشن است ⇐ PC ← (R3)+(R4)+2 ⇐ **47B34002**.
- `BNE LOP1` (LOP1 = (R12)+100h) با BNE = 7 = 0111: **CC = 00**؛ بیت ارزش 8 خاموش ⇐ **پرش نمی‌شود** ⇐ **4770C100**.
- `BNO 12(3,4)` با BNO = 14 = 1110: **CC = 10**؛ بیت ارزش 2 روشن ⇐ PC ← (R3)+(R4)+12 ⇐ **47E3400C**.

دام: BNE 8(14)

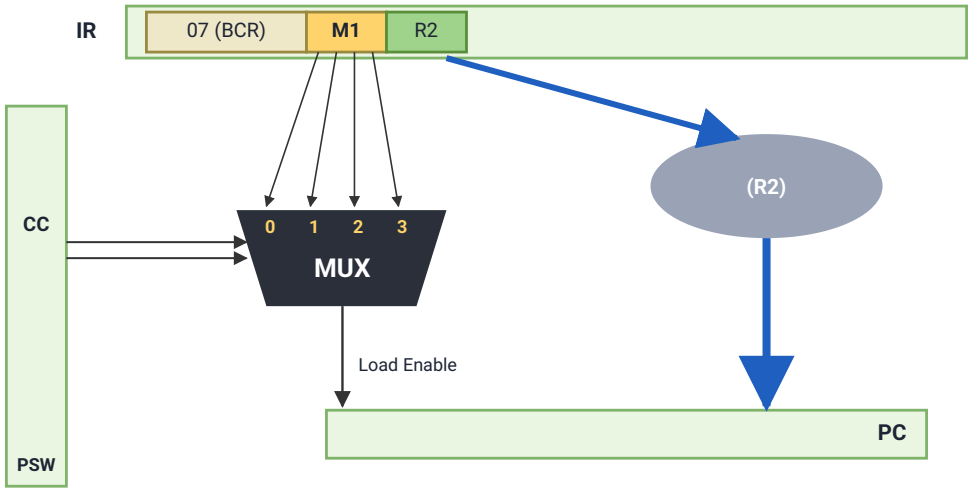
برای نیمونیک‌های گسترش‌یافته، اسمبلر خودش ماسک را در جای r1 می‌گذارد و شما فقط عملوند دوم را می‌نویسید. پس `BNE 8(14)` یعنی **D2 = 8 و X2 = 14** (یک عدد داخل پرانتز = اندیس!) ⇐ کد **477E0008**، نه 4770E008. نشانی مقصد در هر دو یکی است: 8 + (R14).

۸-۱۱ BCR: پرش شرطی به نشانی داخل ثبات — کد 07



BCR M1,r2 (قالب RR)

همان ماسک و همان قاعده، ولی مقصد پرش **محتوای ثبات r2** است: اگر بیت CC در M1 روشن باشد PC ← (r2). اگر **r2 = 0** باشد هرگز پرش نمی‌کند (مثل BCTR).



شکل: سخت‌افزار BCR — بیت‌های ماسک M1 ورودی‌های MUXاند و CC انتخاب‌کننده؛ خروجی، اجازهٔ بارشدن PC با (R2) است

M1	RR extended	Meaning
1	BOR r2	overflow
2	BPR / BHR r2	plus / high
4	BMR / BLR r2	minus / low
7	BNZR / BNER r2	not zero / not equal
8	BZR / BER r2	zero / equal
11	BNMR / BNLR r2	not minus / not low
13	BNPR / BNHR r2	not plus / not high
14	BNOR r2	not overflow
15	BR r2	always
0	NOPR r2	never

(در جدول اسلاید جلوی ماسک 11 «not minus/not plus» نوشته شده که باید «not minus/not low» باشد.)

مثال ۸-۱۴

- `BCR 8,7` با **0787** ⇐ PC ← (R7) ⇐ **CC = 00**.
- `BOR 5` (= `BCR 1,5`) با **CC = 01** ⇐ بیت ارزش 4 در ماسک 0001 خاموش ⇐ هیچ کاری ⇐ **0715**.
- `BR 14` = `BCR 15,14` ⇐ **07FE**: پرش بی‌شرط به نشانی داخل R14؛ **بازگشت از زیربرنامه** (پایین‌تر).

۸-۱۲ ترجمهٔ ساختارهای سطح بالا به اسمبلی 360

هر ساختار کنترلی زبان‌های سطح بالا با «مقایسه + پرش شرطی + پرش بی‌شرط» ساخته می‌شود. این الگوها را یک بار یاد بگیرید، در همهٔ برنامه‌ها تکرار می‌شوند.

الگوی if-else: «با شرط برعکس، از روی then بپر»

```
        L     3,A
        C     3,B
        BNH   ELSE        NOT (a > b)
        ST    3,MAX       then
        B     ENDIF
ELSE    L     3,B         else
        ST    3,MAX
ENDIF   ...
```

```
if (a > b)
    max = a;
else
    max = b;
```

چرا شرط برعکس؟ چون پرش یعنی «رد شدن از کد». اگر شرط درست باشد باید then اجرا شود، یعنی **نپریم**؛ پس وقتی شرط **نادرست** است می‌پریم (به ELSE). و آخر then یک `B ENDIF` لازم است تا وارد else نشویم.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			IFMAX	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	5830 C01E	op r1 X2 B2 D2 58 3 0 C 01E		L	3,A	R3 = a
000006	5930 C022	op r1 X2 B2 D2 59 3 0 C 022		C	3,B	مقایسهٔ a با b
00000A	47D0 C014	op M1 X2 B2 D2 47 D 0 C 014		BNH	ELSE	شرط برعکس: a ≤ b ⇒ برو به else
00000E	5030 C026	op r1 X2 B2 D2 50 3 0 C 026		ST	3,MAX	then: max = a
000012	47F0 C01C	op M1 X2 B2 D2 47 F 0 C 01C		B	ENDIF	از روی else بپر
000016	5830 C022	op r1 X2 B2 D2 58 3 0 C 022	ELSE	L	3,B	else: max = b
00001A	5030 C026	op r1 X2 B2 D2 50 3 0 C 026		ST	3,MAX	
00001E	0A0E	SVC I 0A 0E	ENDIF	EOJ		
000020	FFFFFFF8		A	DC	F'-8'	
000024	0000000D		B	DC	F'13'	
000028			MAX	DS	F	
				END	BEGIN	

نتیجه با شبیه‌ساز: a = -8، b = 13 ⇒ MAX = 13 (و برای (13, 13)، (20, 13) و (-5, -9) هم درست است). به کد `47D0C014` نگاه کنید: **D** = ماسک BNH.

الگوهای حلقه

ساختار	C	IBM 360

for با شمارنده (N ≥ 1)	for (k = N; k > 0; k--) body;	`L 5,N`
		`LOOP ... body`
		`BCT 5,LOOP`

while (آزمون در ابتدا)	while (i < j) body;	`LOOP CR 4,5`
		`BNL DONE NOT (i < j)`
		`... body`
		`B LOOP`
		`DONE ...`

do-while (آزمون در انتها)	do body; while (x != 0);	`LOOP ... body (sets CC on x)` `BNZ LOOP`

پیمایش آرایهٔ کلمه‌ای	for (i = 0; i < 10; i++) … A[i] …	`SR 4,4 byte offset = 0`
		`LA 5,10`
		`LOOP L 6,A(4) A[i]`
		`...`
		`LA 4,4(4) next word`
		`BCT 5,LOOP`

> 💡 **شهود: اندیس بر حسب بایت، نه بر حسب عنصر**
>
> در C می‌نویسیم `A[i]` و کامپایلر خودش i را در ۴ ضرب می‌کند. در اسمبلی خودمان باید این کار را بکنیم: ثبات اندیس **فاصله بر حسب بایت** را نگه می‌دارد، پس به‌جای i++ می‌نویسیم offset += 4 (یعنی `LA 4,4(4)`). اگر شمارهٔ عنصر را هم لازم داریم (مثلاً برای خروجی)، یک ثبات جدا با گام ۱ نگه می‌داریم.

۸-۱۳ زیربرنامه: BALR و BR

در فصل ۳ دیدیم که CALL نشانی بازگشت را در **پشته** ذخیره می‌کند. 360 پشتهٔ سخت‌افزاری ندارد؛ نشانی بازگشت در یک **ثبات** (طبق قرارداد R14) ذخیره می‌شود:

1. `LA 15,SUB` ⇐ نشانی زیربرنامه در R15.
2. `BALR 14,15` ⇐ BALR نشانی دستور بعد از ← R14 و سپس (R15) ← PC. (Branch And Link: «ببر و پیوند را نگه دار».)

3. آخر زیربرنامه: `BR 14` ⇐ (R14) ← PC؛ یعنی برگرد به همان‌جا که صدا زده شدی.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			CALLSUB	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	41F0 C01A	op r1 X2 B2 D2 41 F 0 C 01A		LA	15,SQR	R15 = نشانی زیربرنامه
000006	5810 C01E	op r1 X2 B2 D2 58 1 0 C 01E		L	1,X	
00000A	05EF	op r1 r2 05 E F		BALR	14,15	R14 = نشانی بازگشت؛ پرش به SQR
00000C	5010 C026	op r1 X2 B2 D2 50 1 0 C 026		ST	1,X2	X2 = x²
000010	5810 C022	op r1 X2 B2 D2 58 1 0 C 022		L	1,Y	
000014	05EF	op r1 r2 05 E F		BALR	14,15	فراخوانی دوباره
000016	5010 C02A	op r1 X2 B2 D2 50 1 0 C 02A		ST	1,Y2	
00001A	0A0E	SVC I 0A 0E		EOJ		
00001C	1C01	op r1 r2 1C 0 1	SQR	MR	0,1	R0:R1 = R1 × R1
00001E	07FE	op M1 r2 07 F E		BR	14	بازگشت به نشانی داخل R14
000020	0000000C		X	DC	F'12'	
000024	FFFFFFF9		Y	DC	F'-7'	
000028			X2	DS	F	
00002C			Y2	DS	F	
				END	BEGIN	

نتیجه با شبیه‌ساز: X2 = 144 و Y2 = 49. زیربرنامهٔ SQR فقط دو دستور است: `MR 0,1` (مربع R1 در جفت R0:R1) و `BR 14`.

> 💡 **شهود: BALR همان BALR 11,0 فصل ۶ است!**
>
> در ابتدای هر برنامه نوشتیم `BALR 12,0`: «نشانی بعدی را در R12 بگذار و (چون r2 = 0) نپر». حالا با `BALR 14,15` هم نشانی بعدی را نگه می‌داریم و هم می‌پریم. «پیوند» (link) همان نشانی بازگشت است. چون فقط یک ثبات برای آن داریم، اگر زیربرنامه خودش زیربرنامهٔ دیگری صدا بزند، باید R14 را قبلش در حافظه ذخیره کند (با ST) وگرنه راه برگشت گم می‌شود؛ کاری که پشته در فصل ۳ خودکار انجام می‌داد.

۸-۱۴ برنامه‌های کامل حل‌شده

در همهٔ برنامه‌های زیر الگوی ابتدایی یکی است: `BALR 12,0` و `USING *,12` (R12 = 2) چون برنامه از نشانی 0 شروع می‌شود و BALR دو بایت است)؛ پس جابه‌جایی هر متغیر = نشانی آن - 2. مثلاً در برنامهٔ اول ARR در نشانی 1Ch است ⇐ `A 3,ARR(4)` = 5A34C01A. D2 = 1Ah ⇐ آرایهٔ نمونه در همهٔ برنامه‌ها: ARR = 5, -2, 7, 10, 0, 3, -8, 1, 4, 6.

۸-۱۴-۱ برنامهٔ ۸-۱: جمع عناصر آرایه

ایده: R3 جمع، R4 فاصلهٔ بایتی عنصر جاری و R5 شمارندهٔ BCT. در هر دور `A 3,ARR(4)` عنصر جاری را اضافه می‌کند و `LA 4,4(4)` به کلمهٔ بعد می‌رود.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			SUMARR	START	0	
000000	05C0	op=05 r1=C r2=0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B33	op=1B r1=3 r2=3		SR	3,3	sum = 0
000004	1B44	op=1B r1=4 r2=4		SR	4,4	اندیس = 0
000006	4150 000A	op=41 r1=5 X2=0 B2=0 D2=00A		LA	5,10	شمارنده = 10
00000A	5A34 C01A	op=5A r1=3 X2=4 B2=C D2=01A	LOOP	A	3,ARR(4)	sum += ARR[i]
00000E	4144 0004	op=41 r1=4 X2=4 B2=0 D2=004		LA	4,4(4)	اندیس += 4
000012	4650 C008	op=46 r1=5 X2=0 B2=C D2=008		BCT	5,LOOP	۱۰ بار
000016	5030 C042	op=50 r1=3 X2=0 B2=C D2=042		ST	3,SUM	
00001A	0A0E	SVC=0A I=0E		EOJ		
00001C	00000005…		ARR	DC	F'5',F'-2',F'7', F'10',F'0'	
000030	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	
000044			SUM	DS	F	
				END	BEGIN	

ردگیری اجرای برنامهٔ ۸-۱ (شبیه‌ساز)

PC	instr	R3 (sum)	R4 (offset)	R5 (count)	PC بعدی
0006	LA 5,10	0	0	10	000A
000A	A 3,ARR(4)	5	0	10	000E
000E	LA 4,4(4)	5	4	10	0012
0012	BCT 5,LOOP	5	4	9	000A (پرش)
000A	A 3,ARR(4)	3	4	9	000E
000E	LA 4,4(4)	3	8	9	0012
0012	BCT 5,LOOP	3	8	8	000A (پرش)
…	…	…	…	…	…
0012	BCT 5,LOOP	26	40	0	0016 (بدون پرش)
0016	ST 3,SUM	26	40	0	001A

جمع = 26 و در کل 36 = 2 + 10 × 3 + 4 دستور اجرا می‌شود.

۸-۱۴-۲ برنامهٔ ۸-۲: بیشینهٔ آرایه

ایده: فرض کن ARR[0] بیشینه است؛ ۹ عنصر بعدی را یکی‌یکی با آن مقایسه کن و اگر بزرگ‌تر بود جایگزین کن. `C 3,ARR(4)` و سپس `BNL NEXT`: «اگر max کوچک‌تر نیست، کاری نکن».

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			MAXARR	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	5830 C026	op r1 X2 B2 D2 58 3 0 C 026		L	3,ARR	max = ARR[0]
000006	4140 0004	op r1 X2 B2 D2 41 4 0 0 004		LA	4,4	اندیس ARR[1]
00000A	4150 0009	op r1 X2 B2 D2 41 5 0 0 009		LA	5,9	۹ عنصر باقی
00000E	5934 C026	op r1 X2 B2 D2 59 3 4 C 026	LOOP	C	3,ARR(4)	مقایسهٔ max با ARR[i]
000012	47B0 C018	op M1 X2 B2 D2 47 B 0 C 018		BNL	NEXT	max ≥ ARR[i]: بماند
000016	5834 C026	op r1 X2 B2 D2 58 3 4 C 026		L	3,ARR(4)	بیشینهٔ جدید
00001A	4144 0004	op r1 X2 B2 D2 41 4 4 0 004	NEXT	LA	4,4(4)	
00001E	4650 C00C	op r1 X2 B2 D2 46 5 0 C 00C		BCT	5,LOOP	

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000022	5030 C04E	op r1 X2 B2 D2 50 3 0 C 04E		ST	3,MAX	
000026	0A0E	SVC I 0A 0E		EOJ		
000028	00000005…		ARR	DC	F'5',F'-2',F'7', F'10',F'0'	
00003C	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	
000050			MAX	DS	F	
				END	BEGIN	

نتیجه: MAX = 10. (برای کمینه فقط BNL را به BNH تبدیل کنید.)

۳-۸-۱۴ برنامهٔ ۸-۳: شمارش عناصر منفی

ایده: R7 = 0 را با هر عنصر مقایسه کن. اگر ARR[i] < 0 (CC = 10) عنصر منفی است؛ پس با BNH (یعنی «ARR[i] ≥ 0») از شمارش رد شو. شمارش با `LA 3,1(3)`.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			CNTNEG	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B33	op r1 r2 1B 3 3		SR	3,3	count = 0
000004	1B77	op r1 r2 1B 7 7		SR	7,7	R7 = 0 برای مقایسه
000006	1B44	op r1 r2 1B 4 4		SR	4,4	اندیس = 0
000008	4150 000A	op r1 X2 B2 D2 41 5 0 0 00A		LA	5,10	
00000C	5974 C026	op r1 X2 B2 D2 59 7 4 C 026	LOOP	C	7,ARR(4)	مقایسهٔ ARR[i] با 0
000010	47D0 C016	op M1 X2 B2 D2 47 D 0 C 016		BNH	NEXT	ARR[i] ≤ 0: منفی نیست
000014	4133 0001	op r1 X2 B2 D2 41 3 3 0 001		LA	3,1(3)	count += 1
000018	4144 0004	op r1 X2 B2 D2 41 4 4 0 004	NEXT	LA	4,4(4)	
00001C	4650 C00A	op r1 X2 B2 D2 46 5 0 C 00A		BCT	5,LOOP	
000020	5030 C04E	op r1 X2 B2 D2 50 3 0 C 04E		ST	3,COUNT	
000024	0A0E	SVC I 0A 0E		EOJ		
000028	00000005…		ARR	DC	F'5',F'-2',F'7', F'10',F'0'	
00003C	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000050			COUNT	DS	F	
				END	BEGIN	

نتیجه: COUNT = 2 (-2 و -8).

۸-۱۴-۴ برنامهٔ ۸-۴: شمارش عددهای زوج با N

ایده: هر عنصر را در R6 بیاور، با ONE (=1) AND کن؛ نتیجه ناصفر یعنی فرد ⇐ BNZ از شمارش رد می‌شود.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			EVENS	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B33	op r1 r2 1B 3 3		SR	3,3	count = 0
000004	1B44	op r1 r2 1B 4 4		SR	4,4	
000006	4150 000A	op r1 X2 B2 D2 41 5 0 0 00A		LA	5,10	
00000A	5864 C02A	op r1 X2 B2 D2 58 6 4 C 02A	LOOP	L	6,ARR(4)	
00000E	5460 C026	op r1 X2 B2 D2 54 6 0 C 026		N	6,ONE	فقط بیت ۰ بماند
000012	4770 C018	op M1 X2 B2 D2 47 7 0 C 018		BNZ	NEXT	بیت ۰ = 1: فرد
000016	4133 0001	op r1 X2 B2 D2 41 3 3 0 001		LA	3,1(3)	زوج: count += 1
00001A	4144 0004	op r1 X2 B2 D2 41 4 4 0 004	NEXT	LA	4,4(4)	
00001E	4650 C008	op r1 X2 B2 D2 46 5 0 C 008		BCT	5,LOOP	
000022	5030 C052	op r1 X2 B2 D2 50 3 0 C 052		ST	3,COUNT	
000026	0A0E	SVC I 0A 0E		EOJ		
000028	00000001		ONE	DC	F'1'	
00002C	00000005…		ARR	DC	F'5',F'-2',F'7', F'10',F'0'	
000040	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	
000054			COUNT	DS	F	
				END	BEGIN	

نتیجه: COUNT = 6 (-2، 10، 0، -8، 4، 6). دقت کنید N مقدار R6 را خراب می‌کند؛ برای همین هر دور دوباره L می‌کنیم.

۸-۱۴-۵ برنامهٔ ۸-۵: میانگین (M برای گسترش علامت، سپس D)

ایده: جمع را در R3 حساب کن. برای تقسیم، مقسوم باید در جفت R2:R3 باشد؛ `M 2,ONE` (ضرب در ۱) R2 را گسترش علامت R3 می‌کند. بعد `D 2,N`.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			AVG	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B33	op r1 r2 1B 3 3		SR	3,3	
000004	1B44	op r1 r2 1B 4 4		SR	4,4	
000006	5850 C02A	op r1 X2 B2 D2 58 5 0 C 02A		L	5,N	N = شمارنده
00000A	5A34 C02E	op r1 X2 B2 D2 5A 3 4 C 02E	LOOP	A	3,ARR(4)	R3 = مجموع
00000E	4144 0004	op r1 X2 B2 D2 41 4 4 0 004		LA	4,4(4)	
000012	4650 C008	op r1 X2 B2 D2 46 5 0 C 008		BCT	5,LOOP	
000016	5C20 C026	op r1 X2 B2 D2 5C 2 0 C 026		M	2,ONE	R2:R3 = sum × 1 (گسترش علامت)
00001A	5D20 C02A	op r1 X2 B2 D2 5D 2 0 C 02A		D	2,N	R3 = خارج‌قسمت، R2 = باقی‌مانده
00001E	5030 C056	op r1 X2 B2 D2 50 3 0 C 056		ST	3,MEAN	
000022	5020 C05A	op r1 X2 B2 D2 50 2 0 C 05A		ST	2,REM	
000026	0A0E	SVC I 0A 0E		EOJ		
000028	00000001		ONE	DC	F'1'	
00002C	0000000A		N	DC	F'10'	
000030	00000005…		ARR	DC	F'5',F'-2',F'7', F'10',F'0'	
000044	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	
000058			MEAN	DS	F	
00005C			REM	DS	F	
				END	BEGIN	

نتیجه: 26 ÷ 10 ⇐ MEAN = 2 و REM = 6.

۸-۱۴-۶ برنامهٔ ۸-۶: تقسیم یک عدد منفی

ایده: همان آماده‌سازی جفت ثبات، این بار با `LA 4,1` و `MR 2,4` (بدون نیاز به ثابت ONE در حافظه). اگر این گام را فراموش کنیم و R2 = 0 باشد، مقسوم یک عدد مثبت بسیار بزرگ تفسیر می‌شود.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			DIVNEG	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	5830 C01A	op r1 X2 B2 D2 58 3 0 C 01A		L	3,X	R3 = x (منفی)
000006	4140 0001	op r1 X2 B2 D2 41 4 0 0 001		LA	4,1	R4 = 1
00000A	1C24	op r1 r2 1C 2 4		MR	2,4	R2:R3 = x × 1 (گسترش علامت)
00000C	5D20 C01E	op r1 X2 B2 D2 5D 2 0 C 01E		D	2,Y	R3 = x / y ، R2 = باقی‌مانده
000010	5030 C022	op r1 X2 B2 D2 50 3 0 C 022		ST	3,Q	
000014	5020 C026	op r1 X2 B2 D2 50 2 0 C 026		ST	2,R	
000018	0A0E	SVC I 0A 0E		EOJ		
00001C	FFFFFFD1		X	DC	F'-47'	
000020	00000005		Y	DC	F'5'	
000024			Q	DS	F	
000028			R	DS	F	
				END	BEGIN	

نتیجه: 5 ÷ -47 ⇐ R = -2، Q = -9 (بررسی: -47 = 2 - 5 × -9).

۸-۱۴-۷ برنامهٔ ۸-۷: جمع ۱ تا N

ایده: خود شمارندهٔ BCT مقدارهای N، N-1، …، 1 را می‌گیرد؛ پس کافی است در هر دور آن را به جمع اضافه کنیم (`AR 3,5`).

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			SUMN	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B33	op r1 r2 1B 3 3		SR	3,3	sum = 0
000004	5850 C012	op r1 X2 B2 D2 58 5 0 C 012		L	5,N	R5 = N
000008	1A35	op r1 r2 1A 3 5	LOOP	AR	3,5	sum += R5
00000A	4650 C006	op r1 X2 B2 D2 46 5 0 C 006		BCT	5,LOOP	R5 = N, N-1, …, 1
00000E	5030 C016	op r1 X2 B2 D2 50 3 0 C 016		ST	3,SUM	
000012	0A0E	SVC I 0A 0E		EOJ		
000014	00000064		N	DC	F'100'	

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000018			SUM	DS	F	
				END	BEGIN	

نتیجه: SUM = 5050 برای N = 100.

۸-۱۴-۸ برنامهٔ ۸-۸: فاکتوریل

ایده: مثل برنامهٔ قبل، ولی با ضرب: `MR 2,5` یعنی R2:R3 ← R3 × R5؛ حاصل‌ضرب جاری در ثبات فرد R3 می‌ماند و R2 (نیمهٔ بالا) تا وقتی جواب در ۳۲ بیت جا شود صفر است.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			FACT	START	0	
000000	05C0	op=05 r1=C r2=0	BEGIN	BALR	12,0	
				USING	*,12	
000002	4130 0001	op=41 r1=3 X2=0 B2=0 D2=001		LA	3,1	R3 = حاصل‌ضرب = 1
000006	5850 C016	op=58 r1=5 X2=0 B2=C D2=016		L	5,N	n = شمارنده
00000A	1C25	op=1C r1=2 r2=5	LOOP	MR	2,5	R2:R3 = R3 × R5
00000C	4650 C008	op=46 r1=5 X2=0 B2=C D2=008		BCT	5,LOOP	R5 = n, n-1, …, 1
000010	5030 C01A	op=50 r1=3 X2=0 B2=C D2=01A		ST	3,F	!F = n
000014	0A0E	SVC=0A I=0E		EOJ		
000018	00000006		N	DC	F'6'	
00001C			F	DS	F	
				END	BEGIN	

نتیجه: 720 = !6. (از 13! به بعد جواب در ۳۲ بیت جا نمی‌شود و نیمهٔ بالا در R2 ناصفر می‌شود.)

۸-۱۴-۹ برنامهٔ ۸-۹: توان‌های ۲

ایده: `AR 3,3` هر بار R3 را دو برابر می‌کند (همان شیفت به چپ).

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			POW2	START	0	
000000	05C0	op=05 r1=C r2=0	BEGIN	BALR	12,0	
				USING	*,12	
000002	4130 0001	op=41 r1=3 X2=0 B2=0 D2=001		LA	3,1	p = 1
000006	5850 C016	op=58 r1=5 X2=0 B2=C D2=016		L	5,EXP	n = شمارنده
00000A	1A33	op=1A r1=3 r2=3	LOOP	AR	3,3	p = 2p

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
00000C	4650 C008	op r1 X2 B2 D2 46 5 0 C 008		BCT	5,LOOP	
000010	5030 C01A	op r1 X2 B2 D2 50 3 0 C 01A		ST	3,P	
000014	0A0E	SVC I 0A 0E		EOJ		
000018	0000000A		EXP	DC	F'10'	
00001C			P	DS	F	
				END	BEGIN	

نتیجه: $2^{10} = 1024$.

۸-۱۴-۱۰ برنامهٔ ۸-۱۰: دنبالهٔ فیبوناچی

ایده: دو جملهٔ آخر را در R3 (a) و R4 (b) نگه دار؛ در هر دور a را ذخیره کن، t = a + b، سپس a ← b و b ← t. ST با اندیس R6 جمله‌ها را پشت سر هم در آرایهٔ F می‌نویسد.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			FIB	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B33	op r1 r2 1B 3 3		SR	3,3	a = F0 = 0
000004	4140 0001	op r1 X2 B2 D2 41 4 0 0 001		LA	4,1	b = F1 = 1
000008	1B66	op r1 r2 1B 6 6		SR	6,6	اندیس = 0
00000A	4170 0014	op r1 X2 B2 D2 41 7 0 0 014		LA	7,20	۲۰ جمله
00000E	5036 C022	op r1 X2 B2 D2 50 3 6 C 022	LOOP	ST	3,F(6)	F[i] = a
000012	1853	op r1 r2 18 5 3		LR	5,3	
000014	1A54	op r1 r2 1A 5 4		AR	5,4	t = a + b
000016	1834	op r1 r2 18 3 4		LR	3,4	a = b
000018	1845	op r1 r2 18 4 5		LR	4,5	b = t
00001A	4166 0004	op r1 X2 B2 D2 41 6 6 0 004		LA	6,4(6)	اندیس += 4
00001E	4670 C00C	op r1 X2 B2 D2 46 7 0 C 00C		BCT	7,LOOP	
000022	0A0E	SVC I 0A 0E		EOJ		
000024			F	DS	20F	
				END	BEGIN	

نتیجه: 0، 1، 1، 2، 3، 5، ...، 4181 (۲۰ جمله).

۸-۱۴-۱۱ برنامهٔ ۸-۱۱: جابه‌جایی دو متغیر با XOR

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			SWAP	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	5830 C01A	op r1 X2 B2 D2 58 3 0 C 01A		L	3,X	
000006	5840 C01E	op r1 X2 B2 D2 58 4 0 C 01E		L	4,Y	
00000A	1734	op r1 r2 17 3 4		XR	3,4	R3 = x ⊕ y
00000C	1743	op r1 r2 17 4 3		XR	4,3	R4 = x
00000E	1734	op r1 r2 17 3 4		XR	3,4	R3 = y
000010	5030 C01A	op r1 X2 B2 D2 50 3 0 C 01A		ST	3,X	حافظه جابه‌جا شد
000014	5040 C01E	op r1 X2 B2 D2 50 4 0 C 01E		ST	4,Y	
000018	0A0E	SVC I 0A 0E		EOJ		
00001C	000004D2		X	DC	F'1234'	
000020	FFFFFFB3		Y	DC	F'-77'	
				END	BEGIN	

نتیجه: X = -77 و Y = 1234؛ بدون هیچ ثبات کمکی (سه XR فصل ۷).

۸-۱۴-۱۲ برنامهٔ ۸-۱۲: جمع دو عدد ۶۴ بیتی

ایده: هر عدد ۶۴ بیتی دو کلمه است (HI و LO). نیمه‌های پایین را با **AL** (بدون علامت) جمع کن؛ اگر نقلی رخ داد (CC = 11) یک واحد به نیمهٔ بالا اضافه کن. BNO یعنی «اگر CC ≠ 11 از اضافه کردن نقلی رد شو». این دقیقاً جمع دستی ستونی است: «یکی می‌رود بالا».

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			ADD64	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	5820 C026	op r1 X2 B2 D2 58 2 0 C 026		L	2,AHI	نیمهٔ بالای A
000006	5830 C02A	op r1 X2 B2 D2 58 3 0 C 02A		L	3,ALO	نیمهٔ پایین A
00000A	5E30 C032	op r1 X2 B2 D2 5E 3 0 C 032		AL	3,BLO	جمع پایین‌ها (بدون علامت)
00000E	47E0 C014	op M1 X2 B2 D2 47 E 0 C 014		BNO	NOCARRY	CC ≠ 11: نقلی نداشت
000012	5A20 C022	op r1 X2 B2 D2 5A 2 0 C 022		A	2,ONE	نقلی: +1 به بالا

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000016	5A20 C02E	op r1 X2 B2 D2 5A 2 0 C 02E	NOCARRY	A	2,BHI	جمع بالاها
00001A	5020 C036	op r1 X2 B2 D2 50 2 0 C 036		ST	2,SHI	
00001E	5030 C03A	op r1 X2 B2 D2 50 3 0 C 03A		ST	3,SLO	
000022	0A0E	SVC I 0A 0E		EOJ		
000024	00000001		ONE	DC	F'1'	
000028	00000001		AHI	DC	X'00000001'	
00002C	FFFFFFFF		ALO	DC	X'FFFFFFFF'	
000030	00000002		BHI	DC	X'00000002'	
000034	00000005		BLO	DC	X'00000005'	
000038			SHI	DS	F	
00003C			SLO	DS	F	
				END	BEGIN	

نتیجه: 00000004 00000004h = (00000002 00000005h) + (00000001 FFFFFFFFh).

۸-۱۴-۱۳ برنامهٔ ۸-۱۳: مرتب‌سازی Gnome

ایده (شهود کوتولهٔ باغبان): کوتوله‌ای جلوی ردیف گلدان‌ها ایستاده است. اگر گلدان جلویی از گلدان کنارش کوچک‌تر یا برابر باشد (مرتب‌اند) یک قدم جلو می‌رود؛ وگرنه آن دو را جابه‌جا می‌کند و یک قدم عقب می‌رود (مگر در ابتدای ردیف باشد). وقتی به انتهای ردیف رسید، همه مرتب‌اند. فقط یک اندیس (R4) لازم است. R8 = 0 برای آزمون «ابتدای ردیف» و R9 = 36 (فاصلهٔ آخرین عنصر) برای آزمون پایان.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			SORT	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B44	op r1 r2 1B 4 4		SR	4,4	i = 0 (فاصله بر حسب بایت)
000004	1B88	op r1 r2 1B 8 8		SR	8,8	R8 = 0
000006	4190 0024	op r1 X2 B2 D2 41 9 0 0 024		LA	9,36	فاصلهٔ آخرین عنصر
00000A	41A0 0004	op r1 X2 B2 D2 41 A 0 0 004		LA	10,4	R10 = 4
00000E	5854 C03A	op r1 X2 B2 D2 58 5 4 C 03A	LOOP	L	5,A(4)	A[i]
000012	5864 C03E	op r1 X2 B2 D2 58 6 4 C 03E		L	6,A+4(4)	A[i+1]
000016	1956	op r1 r2 19 5 6		CR	5,6	
000018	47D0 C02E	op M1 X2 B2 D2 47 D 0 C 02E		BNH	NEXT	مرتب است: A[i] ≤ A[i+1]

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
00001C	5064 C03A	op r1 X2 B2 D2 = 50 6 4 C 03A		ST	6,A(4)	جابه‌جایی
000020	5054 C03E	op r1 X2 B2 D2 = 50 5 4 C 03E		ST	5,A+4(4)	
000024	1948	op r1 r2 = 19 4 8		CR	4,8	آیا i = 0 ؟
000026	4780 C00C	op M1 X2 B2 D2 = 47 8 0 C 00C		BE	LOOP	بله: بماند
00002A	1B4A	op r1 r2 = 1B 4 A		SR	4,10	i = i - 1
00002C	47F0 C00C	op M1 X2 B2 D2 = 47 F 0 C 00C		B	LOOP	
000030	4144 0004	op r1 X2 B2 D2 = 41 4 4 0 004	NEXT	LA	4,4(4)	i = i + 1
000034	1949	op r1 r2 = 19 4 9		CR	4,9	
000036	4740 C00C	op M1 X2 B2 D2 = 47 4 0 C 00C		BL	LOOP	تا i = آخر
00003A	0A0E	SVC I = 0A 0E		EOJ		
00003C	00000005…		A	DC	F'5',F'-2',F'7', F'10',F'0'	
000050	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	
				END	BEGIN	

نتیجه: آرایه صعودی شد (10، 7، 6، 5، 4، 3، 1، 0، -2، -8)؛ با ۲۰ آرایهٔ تصادفی دیگر (با عددهای تکراری) هم آزموده شد.

۸-۱۵ تمرین‌های حل‌شدهٔ فصل ۸

تمرین ۸-۱: کدگذاری

فرض کنید TAB = (R12)+3A0h و LOOP = (R12)+20h. کد ماشین هر دستور را بنویسید.

حل (با اسمبلر بررسی شده)

instr	op	r1/M1	X2	B2	D2	code	نکته
L 5,TAB(6)	58	5	6	C	3A0	5856C3A0	S2(X2)
ST 7,0(,9)	50	7	0	9	000	50709000	ویرگول ⇐ پایه
LA 1,4095(2,3)	41	1	2	3	FFF	41123FFF	بیشترین D2
BC 4,LOOP	47	4	0	C	020	4740C020	BL / BM =
BNE 8(14)	47	7	E	0	008	477E0008	14 اندیس است
BCT 6,LOOP	46	6	0	C	020	4660C020	

نکته	code	D2	B2	X2	r1/M1	op	instr
نماد + عدد	5900C3A8	3A8	C	0	0	59	C 0,TAB+8
X2 = 0	5A30500C	00C	5	0	3	5A	A 3,12(0,5)
RR	07FE	r2 = E			F	07	BR 14
RR	0700	r2 = 0			0	07	NOPR 0

تمرین ۸-۲: رمزگشایی

`4180000C` · `47F0C100` · `5C46B008` · `07F7` · `4720C010` · `0710`

حل

۱. `41 8 0 0 00C` = `LA 8,12` ⇐ 12 ← R8.

۲. `47 F 0 C 100` = `B 256(0,12)` ⇐ پرش بی‌شرط به (R12) + 100h.

۳. `5C 4 6 B 008` = `M 4,8(6,11)` ⇐ (R5) × M[(R6)+(R11)+8] ← R4:R5.

۴. `07 F 7` = `BR 7` ⇐ پرش بی‌شرط به نشانی داخل R7.

۵. `47 2 0 C 010` = `BH` `16(0,12)` (یا BP) ⇐ پرش اگر CC = 10.

۶. `07 1 0` = `BCR 1,0` ⇐ چون r2 = 0 هرگز پرش نمی‌کند.

تمرین ۸-۳: نشانی مؤثر و ترتیب کارها

(الف) (R3) = 00012000h، (R4) = 10h؛ نشانی مؤثر `L 2,20(4,3)`؟ (ب) آیا هم‌تراز است؟ (R9) = 40000h، (R3) = 100، (R2) = 0 و M[40000h] = 7؛ نتیجهٔ `D 2,0(2,9)`؟ (ج) (R9) = 40000h، (R5) = -3، (R4) = 8 و M[40008h] = 11؛ نتیجهٔ `M 4,0(4,9)`؟

حل

(الف) 14h + 12000h + 10h = 12024h؛ رقم آخر 4 ⇐ مضرب ۴ ⇐ هم‌تراز. (ب) R2) EA = 0 + 40000h + 0 هنوز 0 است) ⇐ 100 ÷ 7 ⇐ CC = 10، R2 = 2، R3 = 14. (ج) EA با مقدار قدیمی R4: CC = 01، R5 = -33، R4 = -1 ⇐ 11 × (-3) = -33 ⇐ 40000h + 8 = 40008h. **نکته:** نشانی همیشه قبل از تغییر ثبات‌ها حساب می‌شود.

تمرین ۸-۴: انتخاب ماسک

برای هر حالت ماسک (و نیمونیک اگر هست) را بنویسید: (۱) بعد از `C 3,X` پرش اگر R3 ≥ X؛ (۲) بعد از C پرش اگر R3 ≠ X؛ (۳) بعد از A پرش اگر نتیجه صفر یا منفی ولی بدون سرریز؛ (۴) بعد از AL پرش اگر نقلی رخ داد؛ (۵) اگر CC = 10 باشد، کدام ماسک‌ها پرش می‌کنند؟

حل

(۱) 11 = 15 - 4 ⇐ (۳) **BNE**. (۲) 7 = 15 - 8 ⇐ **BNL**. 12 = 8 + 4 ⇐ `BC 12,…` (نیمونیک ندارد؛ BNP = 13 در سرریز هم می‌پرد!). (۴) **BO** ⇐ 1 = CC ⇐ 11 (و «بدون نقلی» ⇐ 14 ⇐ BNO، مثل برنامهٔ ۸-۱۲). (۵) ماسک‌هایی که بیت ارزش 2 دارند: 2، 3، 6، 7، 10، 11، 14، 15 (شبیه‌ساز همین را تأیید می‌کند).

تمرین ۸-۵: چند بار؟

در حلقهٔ `LA 5,N` / `LOOP …` / `BCT 5,LOOP` ، بدنه برای N = 1، N = 10 و N = 0 چند بار اجرا می‌شود؟

حل

۱۰ ⇐ N = 10 بار؛ ۱ ⇐ N = 1 بار (0 ⇐ BCT ⇐ خروج)؛ N = 0 ⇐ بدنه یک بار اجرا می‌شود، BCT شمارنده را 1- می‌کند و حلقه 2^{32} بار تکرار می‌شود.

تمرین ۸-۶: جست‌وجوی خطی

برنامه‌ای بنویسید که اندیس (شمارهٔ عنصر، از 0) اولین خانهٔ ARR برابر KEY را در POS بنویسد و اگر نبود 1-.

حل

R4 فاصلهٔ بایتی و R6 شمارهٔ عنصر است (دو اندیس با گام‌های ۴ و ۱). به محض برابری (BE) به FOUND می‌پریم؛ اگر حلقه کامل تمام شد، R6 را 1- می‌کنیم. چون 1- را نمی‌توان با LA ساخت، از ثابت MINUS1 استفاده شده است.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			SEARCH	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	5830 C02E	op r1 X2 B2 D2 58 3 0 C 02E		L	3,KEY	R3 = کلید
000006	1B44	op r1 r2 1B 4 4		SR	4,4	فاصله = 0 بایت
000008	1B66	op r1 r2 1B 6 6		SR	6,6	i = 0
00000A	4150 000A	op r1 X2 B2 D2 41 5 0 0 00A		LA	5,10	۱۰ عنصر

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
00000E	5934 C032	op r1 X2 B2 D2 59 3 4 C 032	LOOP	C	3,ARR(4)	؟ KEY = ARR[i]
000012	4780 C024	op M1 X2 B2 D2 47 8 0 C 024		BE	FOUND	
000016	4144 0004	op r1 X2 B2 D2 41 4 4 0 004		LA	4,4(4)	عنصر بعد
00001A	4166 0001	op r1 X2 B2 D2 41 6 6 0 001		LA	6,1(6)	i += 1
00001E	4650 C00C	op r1 X2 B2 D2 46 5 0 C 00C		BCT	5,LOOP	
000022	5860 C02A	op r1 X2 B2 D2 58 6 0 C 02A		L	6,MINUS1	پیدا نشد: -1
000026	5060 C05A	op r1 X2 B2 D2 50 6 0 C 05A	FOUND	ST	6,POS	
00002A	0A0E	SVC I 0A 0E		EOJ		
00002C	FFFFFFFF		MINUS1	DC	F'-1'	
000030	00000001		KEY	DC	F'1'	
000034	00000005…		ARR	DC	F'5',F'-2',F'7', F'10',F'0'	
000048	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	
00005C			POS	DS	F	
				END	BEGIN	

نتیجه: POS = 7 ⇐ KEY = 1؛ آزموده با 6 ⇐ KEY = -8، 9 ⇐ KEY = 6، 0 ⇐ KEY = 5 و -1 ⇐ KEY = 99.

تمرین ۸-۷: وارونه کردن آرایه در جا

حل

دو اندیس: R4 از ابتدا (0) و R5 از انتها (36). تا وقتی i < j (الگوی while با BNL)، دو عنصر را با دو ثبات کمکی جابه‌جا کن، i را ۴ تا جلو و j را ۴ تا عقب ببر. چون LA نمی‌تواند کم کند، از `S 5,FOUR` استفاده شده (`SR 5,10` با R10 = 4 هم درست است).

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			REVERSE	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	1B44	op r1 r2 1B 4 4		SR	4,4	i = 0 (ابتدا)
000004	4150 0024	op r1 X2 B2 D2 41 5 0 0 024		LA	5,36	j = 9 × 4 (انتها)
000008	1945	op r1 r2 19 4 5	LOOP	CR	4,5	تا وقتی i < j

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
00000A	47B0 C028	op M1 X2 B2 D2 47 B 0 C 028		BNL	DONE	
00000E	5864 C02E	op r1 X2 B2 D2 58 6 4 C 02E		L	6,ARR(4)	t1 = ARR[i]
000012	5875 C02E	op r1 X2 B2 D2 58 7 5 C 02E		L	7,ARR(5)	t2 = ARR[j]
000016	5074 C02E	op r1 X2 B2 D2 50 7 4 C 02E		ST	7,ARR(4)	ARR[i] = t2
00001A	5065 C02E	op r1 X2 B2 D2 50 6 5 C 02E		ST	6,ARR(5)	ARR[j] = t1
00001E	4144 0004	op r1 X2 B2 D2 41 4 4 0 004		LA	4,4(4)	i += 1
000022	5B50 C02A	op r1 X2 B2 D2 5B 5 0 C 02A		S	5,FOUR	j -= 1
000026	47F0 C006	op M1 X2 B2 D2 47 F 0 C 006		B	LOOP	
00002A	0A0E	SVC I 0A 0E	DONE	EOJ		
00002C	00000004		FOUR	DC	F'4'	
000030	00000005…		ARR	DC	F'5',F'-2',F'7', F'10',F'0'	
000044	00000003…			DC	F'3',F'-8',F'1', F'4',F'6'	
				END	BEGIN	

نتیجه: ARR = 6، 4، 1، -8، 3، 0، 10، 7، -2، 5.

تمرین ۸-۸: شمارش بیت‌های ۱ (ترفند x AND (x-1))

حل

شهود: در x - 1، پایین‌ترین بیت ۱ صفر می‌شود و همهٔ صفرهای زیر آن ۱ می‌شوند؛ پس x ∧ (x - 1) دقیقاً پایین‌ترین ۱ را پاک می‌کند. مثلاً 10110000 = 10110011 ∧ 10110100. هر دور حلقه یک بیت ۱ پاک می‌شود، پس تعداد دورها = تعداد بیت‌های ۱. `OR 4,4` مقدار را عوض نمی‌کند ولی CC را تنظیم می‌کند (برای حالت x = 0)، و `BCTR 5,0` ارزان‌ترین «منهای یک» است.

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
000000			POPCNT	START	0	
000000	05C0	op r1 r2 05 C 0	BEGIN	BALR	12,0	
				USING	*,12	
000002	5840 C022	op r1 X2 B2 D2 58 4 0 C 022		L	4,X	R4 = x
000006	1B33	op r1 r2 1B 3 3		SR	3,3	count = 0
000008	1644	op r1 r2 16 4 4		OR	4,4	فقط برای تنظیم CC

Loc	Object code	Fields	Label	Instr.	Operands	توضیح
00000A	4780 C01A	op M1 X2 B2 D2 47 8 0 C 01A		BZ	DONE	x = 0: تمام
00000E	4133 0001	op r1 X2 B2 D2 41 3 3 0 001	LOOP	LA	3,1(3)	count += 1
000012	1854	op r1 r2 18 5 4		LR	5,4	
000014	0650	op r1 r2 06 5 0		BCTR	5,0	R5 = x - 1
000016	1445	op r1 r2 14 4 5		NR	4,5	x = x AND (x-1): پاک شدن پایین‌ترین ۱
000018	4770 C00C	op M1 X2 B2 D2 47 7 0 C 00C		BNZ	LOOP	
00001C	5030 C026	op r1 X2 B2 D2 50 3 0 C 026	DONE	ST	3,COUNT	
000020	0A0E	SVC I 0A 0E		EOJ		
000024	0000B6F1		X	DC	F'46833'	0000B6F1h =
000028			COUNT	DS	F	
				END	BEGIN	

نتیجه: x = 0000B6F1h ⇐ COUNT = 10؛ آزموده با 0، 1، (=1) 80000000h، (=32) 1- و (=13) 12345678h.

خلاصهٔ فصل ۸

- RX: ۴ بایت = D2، B2، X2، r1، op؛ EA = D2 + (B2) + (X2) (۲۴ بیت)؛ 0 در جای پایه یا اندیس یعنی «ندارد». یک عدد تنها داخل پرانتز = اندیس.
- کد RX = کد 56، O 54، N 55، CL 59، C 5F، SL 5E، AL 5D، D 5C، M 5B، S 5A، A 50، ST 58، L :RR + 40h. X 57، LA 41، BCT 46، BC 47.
- BC، BCT، LA، ST، L و BCR کد شرط را تغییر نمی‌دهند. نشانی مؤثر L/ST/A/... باید مضرب ۴ باشد.
- LA = جمع‌کنندهٔ نشانی بدون دسترسی به حافظه: ثابت 0..4095، افزایش، جمع دو ثبات؛ نتیجه ۲۴ بیتی و نامنفی.
- BC M1: ارزش بیت‌ها 8 ↔ 11، 1 ↔ 10، 2 ↔ 01، 4 ↔ CC00؛ ماسک «0 = NOP، 15 = B. X - 15 = not X» BE = BZ = 8، BNE = 7، BH = BP = 2، BL = BM = 4، BNH = 13، BNL = 11، BO = 1، BNO = 14.
- BCR M1,r2 همان با مقصد (r2)؛ BR 14 = 07FE بازگشت از زیربرنامه؛ فراخوانی با LA 15,SUB و BALR 14,15.
- if: با شرط برعکس از روی then بپر؛ for: BCT؛ while: آزمون در ابتدا + B به ابتدا؛ آرایه: اندیس بر حسب بایت با LA r,4(r).

پیوست الف

خلاصهٔ ماشین‌های نمونه

جدول زیر مشخصات همهٔ ماشین‌های فرضی درس را کنار هم آورده است؛ برای مرور سریع پیش از امتحان مفید است. (واحد = واحد آدرس‌پذیر)

ماشین	نوع	حافظه / واحد	کلمه	قالب‌های دستور	آدرس‌دهی	طول ثبات‌ها
۱	دوآدرسه	۱۲۸ خانهٔ ۱۶ بیتی	16	Op2|7|7	مستقیم	MAR 7, MBR 16
۲	دوآدرسه	2^{16} × 8 bit	16	Op8|16|16 (5B)	مستقیم	MAR 16, MBR 16, IR 40
۳	دوآدرسه	2^{16} × 8 bit	16 (ناهم‌تراز، BE)	Op8|16|16 (5B)	مستقیم، غیرمستقیم، پرش شرطی	MAR 16, MBR 16, IR 40, PC 16
۴	دوآدرسه، ۱۶ ثبات	2^{12} × 4 bit	16 (۴ واحد)	I: 4|4|4 ، II: 4|4|12 ، III: 4|4|16	مستقیم، غیرمستقیم (حافظه/ثبات)، بی‌واسطه	MAR 12, MBR 16, IR 24, PC 12, R 16
۵	۲/۳ آدرسه، ۶۴ ثبات	2^{16} × 8 bit	16	I: 4|6|6 ، II: 4|6|6|16 ، III: 10|6|16	ثباتی، مستقیم، بی‌واسطه، شاخص‌دار	MAR 16, MBR 16, IR 32, PC 16, R 16
تک‌آدرسه	انباره‌ای	2^{16} × 4 bit	16 (۴ واحد)	4|16 (5 units)	ضمنی، مستقیم، غیرمستقیم، بی‌واسطه	MAR 16, MBR 16, ACC 16, IR 20, PC 16
۶	صفرآدرسه (پشته‌ای)	2^{32} × 8 bit	32	I: 8|32 ، II: 8	ضمنی، مستقیم، بی‌واسطه	IR 40, SP
۷	تک‌آدرسه	2^{32} × 8 bit	32 (مکمل دو)	I: 8|32 ، II: 8|8	مستقیم، غیرمستقیم، بی‌واسطه، نسبی	MAR 32, MBR 32, ACC 32, IR 40, PC 32
IBM 360	RR / RX / RS / SI / SS، ۱۶ ثبات	2^{24} × 8 bit	32 (BE، هم‌تراز)	RR 2B ، RX/RS/SI 4B ، SS 6B	ثباتی، پایه-جابه‌جایی، شاخص‌دار، بی‌واسطه (SI)	GPR 16×32, FPR 4×64, PSW 64, PC 24

الف-۱ فهرست برنامه‌های حل‌شده در جزوه

ماشین	برنامه	وضعیت
۲	مجموع ۷ جملهٔ فیبوناچی (اسلاید)، جابه‌جایی با XOR، محاسبهٔ 2(a+b)-c	کد ماشین کامل، آزموده
۳	مجموع ۱۰۰ جملهٔ فیبوناچی متناوب، جمع آرایه با غیرمستقیم، جمع آرایه با کد خودتغییرده	کد ماشین کامل، آزموده
۴	جمع برداری C = A + B	کد ماشین کامل، آزموده
۵	جمع آرایه با شاخص‌دار؛ تکلیف ۱: جمع R0…R62 (دو راه‌حل)	کد ماشین کامل، آزموده
تک‌آدرسه	تولید ۲۰ جملهٔ فیبوناچی؛ تکلیف ۲: ترجمه	کد ماشین کامل، آزموده
۶	مجموع فیبوناچی (اسلاید) و برنامهٔ بهتر	کد ماشین کامل، آزموده
۷	مرتب‌سازی Gnome؛ تکلیف: جمع ۱۰۰ عدد اول (غربال)	کد ماشین کامل، آزموده
IBM 360 (فصل ۶)	STOCK (اسلاید)، تعریف متغیرها با DS/DC (دو برنامه)	فهرست کامل اسمبلر، آزموده
IBM 360 (فصل ۸)	جمع، بیشینه، شمارش منفی‌ها و زوج‌ها، میانگین، تقسیم منفی، جمع ۱..N، فاکتوریل، توان ۲، فیبوناچی، جابه‌جایی XOR، جمع ۶۴ بیتی، مرتب‌سازی Gnome، if-else، زیربرنامه، جست‌وجوی خطی، وارونه‌سازی، شمارش بیت‌ها	فهرست کامل اسمبلر، آزموده

پیوست ب

واژه‌نامهٔ انگلیسی – فارسی

English	فارسی	English	فارسی
Accumulator (ACC)	انباره	Linker	پیونددهنده
Addressable unit	واحد آدرس‌پذیر	Listing file	فایل فهرست
Addressing mode	روش آدرس‌دهی	Loader	بارکننده
Aligned / Unaligned	هم‌تراز / ناهم‌تراز	Location counter	شمارندهٔ مکان
Assembler	اسمبلر (همگذار)	Mantissa	مانتیس (بخش کسری)
Bank	بانک حافظه	Memory Address Register (MAR)	ثبات آدرس حافظه
Bias / Excess	بایاس / افزونی	Memory Buffer Register (MBR)	ثبات دادهٔ حافظه
Big / Little Endian	ترتیب بایت پرارزش‌اول / کم‌ارزش‌اول	Mnemonic	نیمونیک (نماد یادیار)
Context switching	تعویض متن (زمینه)	Normalization	نرمال‌سازی
Control Unit (CU)	واحد کنترل	Object file	فایل شیء
Data / Instruction stream	جریان داده / دستور	Opcode	کد عمل
Decoder	رمزگشا	Operand	عملوند
Denormalized	غیرنرمال	Overflow	سرریز
Direct / Indirect	مستقیم / غیرمستقیم	Page register	ثبات صفحه
Directive	رهنمود اسمبلر	Pipeline	خط لوله
Displacement	جابه‌جایی	Program Counter (PC)	شمارندهٔ برنامه
Exponent	نما	Register / Register file	ثبات / فایل ثبات
Fixed / Floating point	ممیز ثابت / ممیز شناور	Relative addressing	آدرس‌دهی نسبی
Flip-flop	فلیپ‌فلاپ	Relocation	جابه‌جایی (بازمکان‌یابی)
General / Special purpose	همه‌منظوره / خاص‌منظوره	Segment register	ثبات قطعه
Immediate	بی‌واسطه	Self-complementing code	کد خودمکمل

English	فارسی	English	فارسی
Implied / Inherent	ضمنی	Self-modifying code	کد خودتغییرده
Indexed	شاخص‌دار	Sentinel	نگهبان
Infix / Prefix / Postfix	میانوندی / پیشوندی / پسوندی	Sign-magnitude	علامت-مقدار
Instruction format (IF)	قالب دستور	Stack / Stack Pointer (SP)	پشته / اشاره‌گر پشته
Instruction Register (IR)	ثبات دستور	Symbol table	جدول نمادها
Interrupt service routine	روال خدمت وقفه	Two's / One's complement	مکمل دو / مکمل یک
Label	برچسب	Word / Access width	کلمه / پهنای دسترسی
Base register	ثبات پایه	Condition Code (CC)	کد شرط
Base-displacement addressing	آدرس‌دهی پایه-جابه‌جایی	Effective address (EA)	نشانی مؤثر
Branch and link (BALR)	پرش و پیوند	Even-odd register pair	جفت ثبات زوج-فرد
Branch mask	ماسک پرش	EBCDIC	کد کاراکتری EBCDIC
Extended mnemonic	نیمونیک گسترش‌یافته	Fullword / Halfword / Doubleword	کلمه / نیم‌کلمه / کلمهٔ مضاعف
Index register	ثبات شاخص (اندیس)	Program Status Word (PSW)	کلمهٔ وضعیت برنامه
Logical (unsigned) arithmetic	حساب منطقی (بدون علامت)	Sign extension	گسترش علامت
Specification exception	استثنای مشخصات (خطای هم‌ترازی/ثبات)	Subroutine / Return address	زیربرنامه / نشانی بازگشت

— پایان جزوهٔ جلسه‌های ۱ تا ۸ —